

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 414 — Past Questions & Materials
by Renance

Course: CSC 414
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 2
Levels: 400

This study pack was compiled and restructured by Renance for CSC 414 students at Federal University of Petroleum Resources, Effurun. It brings together 2 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 2 — CSC 414 PQ

Level 400 • First Semester • PastQuestion

a i! a Led

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

Department of Computer Science

} 1" Semester: BSc (Computer Science Examination) 2022/2023 Session _

€SC414 ~ ARTIFICIAL INTELLIGENCE Time Allowed: 2¢Hrs

"Instruction: Attempt Questions 1, 2 AND ANY OTHER 2

Question i:

In FUPRE, the probability that a person has HIV is*0.21. However, candidate decides on further blood

investigation which shows test was positive with a probability of 0, 83. peters.

~ it came with a probability of 0.15. Apply Bayes theorem in evalua

a. Find the value Probability of test being positive Pe) 1

b. If the test is positive, showet!

Cid test showed i negative

; patie av i yirus > PoHiPy "s

e probability t! pa ss the virus > P(H|-P) 5mks

+ the probabili n not have the virus > P(-H|-P) mks

2mks

, e process respons|

\$ above'in (c) if it is possible and when it occurs

1e 8-Q problem "ie on lal diagram

3 ae —

#@. Describe in your words: (1) Mixed Strategy, (2) Minimax, a (d) Pure strategy 4mks

b. Describe the typical prisoner's dilemma

2mks

¢. State the Expected Payoff theorem 2mks

d. Given the table below, describe the saddle-point in the game 2mks

Ce a a 9 ,. -

4 9 2 1 10 4

Ca a a) z o

10 rT 15 Boy % =

Question 4: -m

a, owen P, 2 and R, evaluate the implication for the falsi teuth of the argument 10mks

R | 2-Q |Rvag | PvR [PAR [RAP] oR] QSPI RaQ] Pan Q=Rk |

T = =x, i =

= av T oie kK

a ES E TRS =

ais Lina I — I

Am ae | <r!

Te oa Ws a :

we Pa Ge Isa =

ate Te cl — :

Question 5: Given the roll of two dice, evaluate the following

a. Draw the Probability Matrix table for the roll of two dice

b. P(total of an 11 AND roll of just first die, iryields c 3) Ponititg Orn

€ -Pltotal of 8 OR total of 6) = — 2mks

d. Describe the term: (1) Modus Ponens, (b) Modus Tollens mks

= OJUGO, Arnold Adimabua (PhD) - Examiner

FEDERAL UNIVERSITY op peTROLEUM RESOURCES, EFFURUN

DePartmer of Computer Science

¢ 1" Semester: BSc (Computer Science Examination) 2023/2024 Session

CSC414 - ARTIFICIAL INTELLTGENCE Time Allowed: 2} Hrs

Instruction: Attempt Questions ay

sia: 2 AND ANY OTHER 2

Question 1

In FUPRE, the probability that a person has Hry is 0.21. However, candidate decides on further blood

Investigation which shows test was positive with q Probability of 0.83 However, when test showed negative

~ it came with a probability of 0.15 Apply Bayes theorem in evaluating the probability that

a. Find the value Probability of test being positive $P(p)$ and test negative $P(-P)$ Ammks

b. If the test is positive, show the Probability that patient has the virus $> P(H|P)$ Smks

¢. If the test is positive, the Probability that the Patient does not have the virus $> P(-H|P)$ 5Simks

d._ Tf the test is negative, the probability that the patient has the virus $> P(H|-P)$ Sika

¢. If the test is negative, the probability that the patient does not have the virus $> P(H|>P)$ —_ 5Smks.

Question 2:

&. Describe the Concept of Genetic Algorithm cons

b. Ina problem space of chromosomes that are Properly initialized, the fitness function ranges from (0.65, 0.9]. We discovered 2-candidates with decimal values [107 and 181] which are selected for reproduction Work out the following

Perform the Binary encoding of the dato therein

Show the working using a 1-point, 2-point ang multi-point crossover

¢. Isit possible for the solutions to alter? Tf Yes. Why and state the process responsible,

d. How does GA deal with the process mentioned above in (c) if it is possible and when it ¢

©. Give any B-configurations of the B-Queen Problem with an appropriate diagram

Question 3:

- a. Describe in your words: (1) Mixed Strategy, (2) Minimax, (3) Maximin, (d) Pure strategy plan
- b. Describe the typical prisoner's dilemma 2mks
- c. State the Expected Payoff theorem Hi
- d. Given the table below, describe the saddle-point in the game "a

le 1 2 |" i0 Ese

4 9 ? 1

' | t Sa AY

| to aT.

Question 4:

the likelihood for the falsity or truth of the argument a

. Sere Gore evaluate penne $\forall AR [RAP] Q > R [Q > P [RIQ]]$ Pon

9. Draw the Probability Matrix table for the roll of two dice

b, P(total of an 11 AND roll of just first die, it yields a 3)

c. P(total of 8 OR total of 6) ty i

d_ Describe the term: (1) Modus Ponens, (b) Modus

Paper 2 of 2 — CSC 414 handout

Level 400 • First Semester • Handout

fo

.in

rs

de

ea

yr

.m

Expert Systems : AI Course Lecture 35 – 36, notes, slides

w

w

,w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

ty

or

www.myreaders.info/html/artificial_intelligence.html

ab

kr

www.myreaders.info

ha

C

C

R

Expert Systems

Artificial Intelligence

Expert system, topics - Introduction, expert system components

and human interfaces, expert system characteristics, expert

system features; Knowledge acquisition - issues and techniques;

Knowledge base - representing and using domain knowledge,

IF-THEN rules, semantic network, frames; Working memory;

Inference engine : Forward chaining - data driven approach,

Backward chaining - goal driven approach; Tree searches - DFS, BFS;

Expert system shells - Shell components and description;

Explanation - example, types of explanation; Application of expert systems.

fo

.in

rs

de

ea

Expert System

yr

.m

w

w

Artificial Intelligence

,w

ty

or

ab

Topics

kr

ha

C

(Lectures 35, 36, 2 hours) Slides

C

R

1. Introduction 03-16

Expert system components and human interfaces, expert system characteristics, expert system features.

2. Knowledge Acquisition 17-20

Issues and techniques.

3. Knowledge Base 21-25

Representing and using domain knowledge - IF-THEN rules, semantic network, frames.

4. Working Memory 26

5. Inference Engine 27-32

Forward chaining - data driven approach, backward chaining - goal driven approach, tree searches - DFS, BFS.

6. Expert System Shells 33-34

Shell components and description.

7. Explanation 35

Example, types of explanation

8. Application of Expert Systems 36-37

9. References 38

02

fo

.in

rs

de

ea

yr

Expert System

.m

w

w

,w

ty

What is Expert System ?

or

ab

kr

ha

- An expert system, is an interactive computer-based decision tool that

C

C

R

uses both facts and heuristics to solve difficult decision making problems, based on knowledge acquired from an expert.

- An expert system is a model and associated procedure that exhibits, within a specific domain, a degree of expertise in problem solving that is comparable to that of a human expert.

- An expert system compared with traditional computer :

Inference engine + Knowledge = Expert system

(Algorithm + Data structures = Program in traditional computer)

- First expert system, called DENDRAL, was developed in the early 70's at Stanford University.

03

fo

.in

rs

de

AI – Expert system - Introduction

ea

1. Introduction

yr

.m

w

w

Expert systems are computer applications which embody some

,w

ty

or

non-algorithmic expertise for solving certain types of problems. For

ab

kr

example, expert systems are used in diagnostic applications. They also play

ha

C

C

chess, make financial planning decisions, configure computers, monitor

R

real time systems, underwrite insurance policies, and perform many

services which previously required human expertise.

1.1 Expert System Components And Human Interfaces

Expert systems have a number of major system components and

interface with individuals who interact with the system in various roles.

These are illustrated below.

User

Domain

Expert

Expertise User Interface

System

Engineer

Knowledge Inference

Engineer

Engine

Encoded

Expertise

Knowledge Working

Base Storage

Components of Expert System

The individual components and their roles are explained in next slides.

04

fo

.in

rs

de

AI – Expert system - Introduction

ea

■ Components and Interfaces

yr

.m

w

w

‡ Knowledge base : A declarative representation of the expertise;

,w

ty

or

often in IF THEN rules ;

ab

kr

‡ Working storage : The data which is specific to a problem being

ha

C

C

solved;

R

‡ Inference engine : The code at the core of the system which derives recommendations from the knowledge base and problem-specific data in working storage;

‡ User interface : The code that controls the dialog between the user and the system.

■ Roles of Individuals who interact with the system

‡ Domain expert : The individuals who currently are experts in solving the problems; here the system is intended to solve;

‡ Knowledge engineer : The individual who encodes the expert's knowledge in a declarative form that can be used by the expert system;

‡ User : The individual who will be consulting with the system to get advice which would have been provided by the expert.

05

fo

.in

rs

de

AI – Expert system - Introduction

ea

■ Expert System Shells

yr

.m

w

w

Many expert systems are built with products called expert

,w

ty

or

system shells. A shell is a piece of software which contains the

ab

kr

user interface, a format for declarative knowledge in the knowledge

ha

C

C

base, and an inference engine. The knowledge and system engineers

R

uses these shells in making expert systems.

‡ Knowledge engineer : uses the shell to build a system for a

particular problem domain.

‡ System engineer : builds the user interface, designs the

declarative format of the knowledge base, and implements the inference engine.

Depending on the size of the system, the knowledge engineer and the system engineer might be the same person.

06

fo

.in

rs

de

AI – Expert system - Introduction

ea

1.2 Expert System Characteristics

yr

.m

w

w

Expert system operates as an interactive system that responds to

,w

ty

or

questions, asks for clarifications, makes recommendations and generally

ab

kr

aids the decision-making process.

ha

C

C

Expert systems have many Characteristics :

R

■ Operates as an interactive system

This means an expert system :

‡ Responds to questions

‡ Asks for clarifications

‡ Makes recommendations

‡ Aids the decision-making process.

■ Tools have ability to sift (filter) knowledge

‡ Storage and retrieval of knowledge

‡ Mechanisms to expand and update knowledge base on a continuing basis.

■ Make logical inferences based on knowledge stored

‡ Simple reasoning mechanisms is used

‡ Knowledge base must have means of exploiting the knowledge stored, else it is useless; e.g., learning all the words in a language, without knowing how to combine those words to form a meaningful sentence.

07

fo

.in

rs

de

AI – Expert system - Introduction

ea

■ Ability to Explain Reasoning

yr

.m

w ‡ Remembers logical chain of reasoning; therefore user may ask

w

,w

ty

◇ for explanation of a recommendation

or

ab

kr

◇ factors considered in recommendation

ha

C

‡ Enhances user confidence in recommendation and acceptance of

C

R

expert system

■ Domain-Specific

‡ A particular system caters a narrow area of specialization;
e.g., a medical expert system cannot be used to find faults in
an electrical circuit.

‡ Quality of advice offered by an expert system is dependent on
the amount of knowledge stored.

■ Capability to assign Confidence Values

‡ Can deliver quantitative information

‡ Can interpret qualitatively derived values

‡ Can address imprecise and incomplete data through assignment
of confidence values.

08

fo

.in

rs

de

AI – Expert system - Introduction

ea

■ Applications

yr

.m

are not best suited for those dealing with expert heuristics for solving
 with
 , with
 types
 problems.

or
 about

are not a suitable choice for those problems that can be solved using
 knowledge
 has
 C
 purely numerical techniques.
 C
 R

■ Cost-Effective alternative to Human Expert

are Expert systems have become increasingly popular because of
 their specialization, albeit in a narrow field.
 are Encoding and storing the domain-specific knowledge is economic
 process due to small size.
 are Specialists in many areas are rare and the cost of consulting
 them is high; an expert system of those areas can be useful
 and cost-effective alternative in the long run.

09

for
 .in
 rs
 de

AI – Expert system - Introduction

ea

1.3 Expert System Features

yr
 .m
 w
 w

The features which commonly exist in expert systems are :

,w
 ty
 or
 ab

■ Goal Driven Reasoning or Backward Chaining

kr

ha

C

An inference technique which uses IF-THEN rules to repetitively

C

R

break a goal into smaller sub-goals which are easier to prove;

■ Coping with Uncertainty

The ability of the system to reason with rules and data which are not precisely known;

■ Data Driven Reasoning or Forward Chaining

An inference technique which uses IF-THEN rules to deduce a problem solution from initial data;

■ Data Representation

The way in which the problem specific data in the system is stored and accessed;

■ User Interface

That portion of the code which creates an easy to use system;

■ Explanations

The ability of the system to explain the reasoning process that it used to reach a recommendation.

Each of these features were discussed in detail in previous lectures on AI. However for completion or easy to recall these are mentioned briefly here.

10

fo

.in

rs

de

AI – Expert system - Introduction

ea

• Goal-Driven Reasoning

yr

.m

w

w

Goal-driven reasoning, or backward chaining, is an efficient way to solve

,w

ty

or

problems. The algorithm proceeds from the desired goal, adding new

ab

kr

assertions found.

ha

C

C

R

Data Rules Conclusion

a=1 if a = 1 & b = 2 then c = 3, if c = 3 then d = 4, d=4

b=2

The knowledge is structured in rules which describe how each of the possibilities might be selected.

The rule breaks the problem into sub-problems.

Example :

KB contains Rule set :

Rule 1: If A and C Then F

Rule 2: If A and E Then G

Rule 3: If B Then E

Rule 4: If G Then D

Problem : prove

If A and B true Then D is true

11

fo

.in

rs

de

AI - Expert system - Introduction

ea

- Uncertainty

yr

.m

w

w

Often the Knowledge is imperfect which causes uncertainty.

,w

ty

or

To work in the real world, Expert systems must be able to deal with
 ab
 kr
 uncertainty.

ha
 C
 C

□ one simple way is to associate a numeric value with each piece of
 R

information in the system.

□ the numeric value represents the certainty with which the information
 is known.

There are different ways in which these numbers can be defined, and
 how they are combined during the inference process.

12

fo
 .in
 rs
 de

AI – Expert system - Introduction

ea

- Data Driven Reasoning

yr
 .m
 w
 w

The data driven approach, or Forward chaining, uses rules similar to

,w
 ty
 or

those used for backward chaining. However, the inference process is

ab
 kr

different. The system keeps track of the current state of problem

ha
 C
 C

solution and looks for rules which will move that state closer to a final

R

solution. The Algorithm proceeds from a given situation to a desired goal, adding new assertions found.

Data Rules Conclusion

a=1 if a = 1 & b = 2 then c = 3, if c = 3 then d = 4, d=4
b=2

The knowledge is structured in rules which describe how each of the possibilities might be selected. The rule breaks the problem into sub-problems.

Example :

KB contains Rule set :

Rule 1: If A and C Then F

Rule 2: If A and E Then G

Rule 3: If B Then E

Rule 4: If G Then D

Problem : prove

If A and B true Then D is true

13

fo

.in

rs

de

AI – Expert system - Introduction

ea

- Data Representation

yr

.m

w

w

Expert system is built around a knowledge base module.

,w

ty

or

□ knowledge acquisition is transferring knowledge from human expert

ab

kr

to computer.

ha

C

C

□ Knowledge representation is faithful representation of what the expert R knows.

No single knowledge representation system is optimal for all applications.

The success of expert system depends on choosing knowledge encoding scheme best for the kind of knowledge the system is based on.

The IF-THEN rules, Semantic networks, and Frames are the most commonly used representation schemes.

14

fo

.in

rs

de

AI - Expert system - Introduction

ea

- User Interface

yr

.m

w

w

The acceptability of an expert system depends largely on the quality of

,w

ty

or

the user interface.

ab

kr

ha

□ Scrolling dialog interface : It is easiest to implement and communicate

C

C

R

with the user.

□ Pop-up menus, windows, mice are more advanced interfaces and powerful tools for communicating with the user; they require graphics support.

15

fo

.in

rs

de

AI – Expert system - Introduction

ea

- Explanations

yr

.m

w

w

An important features of expert systems is their ability to explain

,w

ty

or

themselves.

ab

kr

ha

C

Given that the system knows which rules were used during the

C

R

inference process, the system can provide those rules to the user as means for explaining the results.

By looking at explanations, the knowledge engineer can see how the system is behaving, and how the rules and data are interacting.

This is very valuable diagnostic tool during development.

16

fo

.in

rs

de

AI – Expert system - Knowledge acquisition

ea

2. Knowledge Acquisition

yr

.m

w

w

Knowledge acquisition includes the elicitation, collection, analysis, modeling

,w

ty

or

and validation of knowledge.

ab

kr

ha

C

2.1 Issues in Knowledge Acquisition

C

R

The important issues in knowledge acquisition are:

- knowledge is in the head of experts
- Experts have vast amounts of knowledge
- Experts have a lot of tacit knowledge
- ‡ They do not know all that they know and use
- ‡ Tacit knowledge is hard (impossible) to describe
- Experts are very busy and valuable people
- One expert does not know everything
- Knowledge has a "shelf life"

17

fo

.in

rs

de

AI – Expert system - Knowledge acquisition

ea

2.2 Techniques for Knowledge Acquisition

yr

.m

w

w

The techniques for acquiring, analyzing and modeling knowledge are :

,w

ty

or

Protocol-generation techniques, Protocol analysis techniques, Hierarchy-

ab

kr

generation techniques, Matrix-based techniques, Sorting techniques,

ha

C

C

Limited-information and constrained-processing tasks, Diagram-based

R

techniques. Each of these are briefly stated in next few slides.

■ Protocol-generation techniques

Include many types of interviews (unstructured, semi-structured and structured), reporting and observational techniques.

■ Protocol analysis techniques

Used with transcripts of interviews or text-based information to identify basic knowledge objects within a protocol, such as goals, decisions, relationships and attributes. These act as a bridge between the use of protocol-based techniques and knowledge modeling techniques.

18

fo

.in

rs

de

AI – Expert system - Knowledge acquisition

ea

■ Hierarchy-generation techniques

yr

.m

w

w

Involve creation, reviewing and modification of hierarchical knowledge.

,w

ty

or

Hierarchy-generation techniques, such as laddering, are used to

ab

kr

build taxonomies or other hierarchical structures such as goal trees

ha

C

C

and decision networks. The Ladders are of various forms like concept

R

ladder, attribute ladder, composition ladders.

■ Matrix-based techniques

Involve the construction and filling-in a 2-D matrix (grid, table), indicating such things, as may be, for example, between concepts and properties (attributes and values) or between problems and solutions or between tasks and resources, etc. The elements within the matrix can contain: symbols (ticks, crosses, question marks) , colors , numbers , text.

19

fo

.in

rs

de

AI – Expert system - Knowledge acquisition

ea

■ Sorting techniques

yr

.m

w Used for capturing the way people compare and order concepts; it

w

,w

ty

may reveal knowledge about classes, properties and priorities.

or

ab

kr

ha

■ Limited-information and constrained-processing tasks

C

C

R

Techniques that either limit the time and/or information available to the expert when performing tasks. For example, a twenty-questions technique provides an efficient way of accessing the key information in a domain in a prioritized order.

■ Diagram-based techniques

Include generation and use of concept maps, state transition networks, event diagrams and process maps. These are particularly important in capturing the "what, how, when, who and why" of tasks and events.

20

fo

.in

rs

de

AI – Expert system - Knowledge base

ea

3. Knowledge Base (Representing and Using Domain Knowledge)

yr

.m

w

w

Expert system is built around a knowledge base module. Expert system

,w

ty

or

contains a formal representation of the information provided by the domain

ab

kr

expert. This information may be in the form of problem-solving rules,

ha

C

C

procedures, or data intrinsic to the domain. To incorporate these information into

R

the system, it is necessary to make use of one or more knowledge representation methods. Some of these methods are described here.

Transferring knowledge from the human expert to a computer is often the most difficult part of building an expert system.

The knowledge acquired from the human expert must be encoded in such a way that it remains a faithful representation of what the expert knows, and it can be manipulated by a computer.

Three common methods of knowledge representation evolved over the years are IF-THEN rules, Semantic networks and Frames.

The first two methods were illustrated in the earlier lecture slides on

knowledge representation therefore just mentioned here. The frame based representation is described more.

21

fo

.in

rs

de

AI - Expert system - Knowledge base

ea

3.1 IF-THEN rules

yr

.m

w

w

Human experts usually tend to think along :

,w

ty

or

ab

condition $\Rightarrow$ action or Situation $\Rightarrow$ conclusion

kr

ha

C

Rules "if-then" are predominant form of encoding knowledge in

C

R

expert systems. These are of the form :

If $a_1, a_2, \dots, a_n$

Then $b_1, b_2, \dots, b_n$ where

each a_i is a condition or situation, and

each b_i is an action or a conclusion.

22

fo

.in

rs

de

AI - Expert system - Knowledge base

ea

3.2 Semantic Networks

yr

.m

w

w

In this scheme, knowledge is represented in terms of objects and

,w

ty

or

relationships between objects.

ab

kr

ha

C

The objects are denoted as nodes of a graph. The relationship between

C

R

two objects are denoted as a link between the corresponding two nodes.

The most common form of semantic networks uses the links between nodes to represent IS-A and HAS relationships between objects.

Example of Semantic Network

The Fig. below shows a car IS-A vehicle; a vehicle HAS wheels.

This kind of relationship establishes an inheritance hierarchy in the network, with the objects lower down in the network inheriting properties from the objects higher up.

HAS

Vehicle Wheels

Is-A

HAS

Engine

CAR

HAS

Battery

Is-A Is-A

Honda Nissan

Civic Sentra

HAS

Power

Steering

23

fo

.in

rs

de

AI - Expert system - Knowledge base

ea

3.3 Frames

yr

.m

w

w

In this technique, knowledge is decomposed into highly modular

,w

ty

or

pieces called frames, which are generalized record structures.

ab

kr

Knowledge consist of concepts, situations, attributes of concepts,

ha

C

C

relationships between concepts, and procedures to handle relationships

R

as well as attribute values.

‡ Each concept may be represented as a separate frame.

‡ The attributes, the relationships between concepts, and the procedures are allotted to slots in a frame.

‡ The contents of a slot may be of any data type - numbers, strings, functions or procedures and so on.

‡ The frames may be linked to other frames, providing the same kind of inheritance as that provided by a semantic network.

A frame-based representation is ideally suited for objected-oriented programming techniques. An example of Frame-based representation of knowledge is shown in next slide.

24

fo

.in

rs

de

AI – Expert system - Knowledge base

ea

Example : Frame-based Representation of Knowledge.

yr

.m

w

w

Two frames, their slots and the slots filled with data type are shown.

,w

ty

or

ab

kr

ha

Frame Car Frame Car

C

C

R

Inheritance Slot Is-A Inheritance Slot Is-A

Value Vehicle Value Car

Attribute Slot Engine Attribute Slot Make

Value Vehicle Value Honda

Value 1 Value

Value Value

Attribute Slot Cylinders Attribute Slot Year

Value 4 Value 1989

Value 6 Value

Value 8 Value

Attribute Slot Doors Attribute Slot

Value 2 Value

Value 5 Value

Value 4 Value

25

fo

.in

rs

de

AI – Expert system - Working memory

ea

4. Working Memory

yr

.m

w

w

Working memory refers to task-specific data for a problem. The contents

,w

ty

or

of the working memory, changes with each problem situation. Consequently,

ab

kr

it is the most dynamic component of an expert system, assuming that it is

ha

C

C

kept current.

R

‡ Every problem in a domain has some unique data associated with it.

‡ Data may consist of the set of conditions leading to the problem, its parameters and so on.

‡ Data specific to the problem needs to be input by the user at the time of using, means consulting the expert system. The Working memory is related to user interface

‡ Fig. below shows how Working memory is closely related to user interface of the expert system.

User

User Interface

Working Memory

(Task specific data)

Inference Engine

Knowledge Base

26

fo

.in

rs

de

AI – Expert system – Inference Engine

ea

5. Inference Engine

yr

.m

w

w

The inference engine is a generic control mechanism for navigating through

,w

ty

or

and manipulating knowledge and deduce results in an organized manner.

ab

kr

The inference engine's generic control mechanism applies the axiomatic

ha

C

C

(self-evident) knowledge present in the knowledge base to the task-specific

R

data to arrive at some conclusion.

‡ Inference engine the other key component of all expert systems.

‡ Just a knowledge base alone is not of much use if there are no facilities for navigating through and manipulating the knowledge to deduce something from knowledge base.

‡ A knowledge base is usually very large, it is necessary to have inferencing mechanisms that search through the database and deduce results in an organized manner.

The Forward chaining, Backward chaining and Tree searches are some of the techniques used for drawing inferences from the knowledge base.

These techniques were talked in the earlier lectures on Problem Solving : Search and Control Strategies, and Knowledge Representation. However they are relooked in the context of expert system.

27

fo

.in

rs

de

AI – Expert system – Inference Engine

ea

5.1 Forward Chaining Algorithm

yr

.m

w

w

Forward chaining is a techniques for drawing inferences from Rule

,w

ty

or

base. Forward-chaining inference is often called data driven.

ab

kr

ha

C

‡ The algorithm proceeds from a given situation to a desired goal,

C

R

adding new assertions (facts) found.

‡ A forward-chaining, system compares data in the working memory against the conditions in the IF parts of the rules and determines which rule to fire.

‡ Data Driven

Data Rules Conclusion

a=1 if a = 1 & b = 2 then c = 3,

b=2 if c = 3 then d = 4 d=4

‡ Example : Forward Channing

■ Given : A Rule base contains following Rule set

Rule 1: If A and C Then F

Rule 2: If A and E Then G

Rule 3: If B Then E

Rule 4: If G Then D

■ Problem : Prove

If A and B true Then D is true

[Continued in next slide]

28

fo

.in

rs

de

AI – Expert system – Inference Engine

ea

[Continued from previous slide]

yr

.m

w ■ Solution :

w

,w

ty

(i) ‡ Start with input given A, B is true and then

or

ab

‡ start at Rule 1 and go forward/down till a rule

kr

ha

C

"fires" is found.

C

R

First iteration :

(ii) ‡ Rule 3 fires : conclusion E is true

‡ new knowledge found

(iii) ‡ No other rule fires;

‡ end of first iteration.

(iv) ‡ Goal not found;

‡ new knowledge found at (ii);

‡ go for second iteration

Second iteration :

(v) ‡ Rule 2 fires : conclusion G is true

‡ new knowledge found

(vi) ‡ Rule 4 fires : conclusion D is true

‡ Goal found;

‡ Proved

29

fo

.in

rs

de

AI – Expert system – Inference Engine

ea

5.2 Backward Chaining Algorithm

yr

.m

w

w

Backward chaining is a techniques for drawing inferences from Rule

,w

ty

or

base. Backward-chaining inference is often called goal driven.

ab

kr

ha

C

‡ The algorithm proceeds from desired goal, adding new assertions found.

C

R

‡ A backward-chaining, system looks for the action in the THEN clause of the rules that matches the specified goal.

‡ Goal Driven

Data Rules Conclusion

a=1 if a = 1 & b = 2 then c = 3,

b=2 if c = 3 then d = 4 d=4

‡ Example : Backward Channing

■ Given : Rule base contains following Rule set

Rule 1: If A and C Then F

Rule 2: If A and E Then G

Rule 3: If B Then E

Rule 4: If G Then D

■ Problem : Prove

If A and B true Then D is true

[Continued in next slide]

30

fo

.in

rs

de

AI – Expert system – Inference Engine

ea

[Continued from previous slide]

yr

.m

w ■ Solution :

w

,w

ty

(i) ‡ Start with goal ie D is true

or

ab

‡ go backward/up till a rule "fires" is found.

kr

ha

C

First iteration :

C

R

(ii) ‡ Rule 4 fires :

‡ new sub goal to prove G is true

‡ go backward

(iii) ‡ Rule 2 "fires"; conclusion: A is true

‡ new sub goal to prove E is true

‡ go backward;

(iv) ‡ no other rule fires; end of first iteration.

‡ new sub goal found at (iii);

‡ go for second iteration

Second iteration :

(v) ‡ Rule 3 fires :

‡ conclusion B is true (2nd input found)

‡ both inputs A and B ascertained

‡ Proved

31

fo

.in

rs

de

AI – Expert system – Inference Engine

ea

5.3 Tree Searches

yr

.m

w

w

Often a knowledge base is represented as a branching network or tree.

,w

ty

or

Many tree searching algorithms exists but two basic approaches are

ab

kr

ha

depth-first search and breadth-first search.

C

C

Note : Here these two search are briefly mentioned since they were

R

described with examples in the previous lectures.

■ Depth-First Search

‡ Algorithm begins at initial node

‡ Check to see if the left-most below initial node (call node A) is a goal node.

‡ If not, include node A on a list of sub-goals outstanding.

‡ Then starts with node A and looks at the first node below it, and so on.

‡ If no more lower level nodes, and goal node not reached, then start from last node on outstanding list and follow next route of descent to the right.

■ Breadth-First Search

‡ Algorithm starts by expanding all the nodes one level below the initial node.

‡ Expand all nodes till a solution is reached or the tree is completely expanded.

‡ Find the shortest path from initial assertion to a solution.

32

fo

.in

rs

de

AI – Expert system – Shells

ea

6. Expert System Shells

yr

.m

w

w

An Expert system shell is a software development environment. It

,w

ty

or

contains the basic components of expert systems. A shell is associated

ab

kr

with a prescribed method for building applications by configuring and

ha

C

C

instantiating these components.

R

6.1 Shell components and description

The generic components of a shell : the knowledge acquisition, the knowledge Base, the reasoning, the explanation and the user interface are shown below. The knowledge base and reasoning engine are the core components.

Expert System Shell

E

x Inference U

Knowledge

p Acquisition Knowledge Base Mechanism Explanation User s

e Facts, Heuristics Reasoning with Subsystem Interface e

Subsystem

Uncertainty

r r

t

Knowledge

Engineer

All these components are described in the next slide.

33

fo

.in

rs

de

AI – Expert system – Shells

ea

[continuing from previous slide]

yr

.m

■ Knowledge Base

w

w

,w

A store of factual and heuristic knowledge. Expert system tool

ty

or

ab

provides one or more knowledge representation schemes for

kr

ha

C

expressing knowledge about the application domain. Some tools

C

R

use both Frames (objects) and IF-THEN rules. In PROLOG the knowledge is represented as logical statements.

■ Reasoning Engine

Inference mechanisms for manipulating the symbolic information and knowledge in the knowledge base form a line of reasoning in solving a problem. The inference mechanism can range from simple modus ponens backward chaining of IF-THEN rules to Case-Based reasoning.

■ Knowledge Acquisition subsystem

A subsystem to help experts in build knowledge bases. However, collecting knowledge, needed to solve problems and build the knowledge base, is the biggest bottleneck in building expert systems.

■ Explanation subsystem

A subsystem that explains the system's actions. The explanation can range from how the final or intermediate solutions were arrived at justifying the need for additional data.

■ User Interface

A means of communication with the user. The user interface is generally not a part of the expert system technology. It was not given much attention in the past. However, the user interface can make a critical difference in the perceived utility of an Expert system.

34

fo

.in

rs

de

AI - Expert system - Explanation

ea

7. Explanation

yr

.m

w

w

Most expert systems have explanation facilities that allow the user to ask

,w

ty

or

questions - why and how it reached some conclusion.

ab

kr

The questions are answered by referring to the system goals, the rules

ha

C

C

being used, and existing problem solving. The rules typically reflect

R

empirical, or "compiled" knowledge. They are codes of an expert's rules of thumb, not the expert's deeper understanding.

Example :

Dialog with an expert system designed to give advice on car problems.

System Does the car start ?

User No.

System Does the engine turn over ?

User Yes.

System Do you smell gas ?

User Yes

System Recommendation - Wait 5 minutes and try again.

User Why ?

System I used the rule:

If not start, and engine_turn_over, and smell_gas

Then recommendation is 'Wait 5 minutes and try again'

User

Note : The rule gives the correct advice for a flooded car, and knows the questions to be ask to determine if the car is flooded, but it does not contain the knowledge of what a flooded car is and why waiting will help.

Types of Explanation

There are four types of explanations commonly used in expert systems.

‡ Rule trace reports on the progress of a consultation;

‡ Explanation of how the system reached to the given conclusion;

‡ Explanation of why the system did not give any conclusion.

‡ Explanation of why the system is asking a question;

35

fo

.in

rs

de

AI – Expert system – Application

ea

8. Application of Expert Systems

yr

.m

w

w

The Expert systems have found their way into most areas of knowledge

,w

ty

or

work. The applications of expert systems technology have widely proliferated

ab

kr

to industrial and commercial problems, and even helping NASA to plan

ha

C

C

the maintenance of a space shuttle for its next flight. The main applications

R

are stated in next few slides.

‡ Diagnosis and Troubleshooting of Devices and Systems

Medical diagnosis was one of the first knowledge areas to which Expert system technology was applied in 1976. However, the diagnosis of engineering systems quickly surpassed medical diagnosis.

‡ Planning and Scheduling

The Expert system's commercial potential in planning and scheduling has been recognized as very large. Examples are airlines scheduling their flights, personnel, and gates; the manufacturing process planning and job scheduling;

‡ Configuration of Manufactured Objects from sub-assemblies

Configuration problems are synthesized from a given set of elements related by a set of constraints. The Expert systems have been very useful to find solutions. For example, modular home building and manufacturing involving complex engineering design.

36

fo

.in

rs

de

AI – Expert system – Application

ea

‡ Financial Decision Making

yr

.m

w The financial services are the vigorous user of expert system

w

,w

ty

techniques. Advisory programs have been created to assist bankers

or

ab

in determining whether to make loans to businesses and

kr

ha

C

individuals. Insurance companies to assess the risk presented by

C

R

the customer and to determine a price for the insurance. ES are used in typical applications in the financial markets / foreign exchange trading.

‡ Knowledge Publishing

This is relatively new, but also potentially explosive area. Here the primary function of the Expert system is to deliver knowledge that is relevant to the user's problem. The two most widely known Expert systems are : one, an advisor on appropriate grammatical usage in a text; and the other, is a tax advisor on tax strategy, tactics, and individual tax policy.

‡ Process Monitoring and Control

Here Expert system does analysis of real-time data from physical devices, looking for anomalies, predicting trends, controlling optimality and failure correction. Examples of real-time systems that actively monitor processes are found in the steel making and oil refining industries.

‡ Design and Manufacturing

Here the Expert systems assist in the design of physical devices and processes, ranging from high-level conceptual design of abstract entities all the way to factory floor configuration of manufacturing processes.

37

fo

.in

rs

de

AI – Expert sytem - References

ea

9. References : Textbooks

yr

.m

w

w

1. "Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill

,w

ty

companies Inc., Chapter 20, page 547-557.

or

ab

kr

2. "Introduction To Artificial Intelligence & Expert Systems", by Dan W Patterson,

ha

C

(2007), Prentice-hall, page 1-464.

C

R

3. "Expert Systems: Introduction To First And Second Generation And Hybrid Knowledge Based Systems", by Chris Nikolopoulos, (1997), Merzell Dekker INC, page 1-325

4. "Artificial intelligence and expert systems for engineers", by C. S. Krishnamoorthy, S. Rajeev, (1996), CRC Press INC, page 1-293.

5. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

38

fo

.in

rs

de

ea

yr

Fundamentals of Neural Networks : AI Course lecture 37 – 38, notes, slides

.m

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

www.myreaders.info

kr

ha

C

C

R

Fundamentals of Neural Networks

Artificial Intelligence

Neural network, topics : Introduction, biological neuron model, artificial neuron model, notations, functions; Model of artificial neuron - McCulloch-Pitts neuron equation; Artificial neuron - basic elements, activation functions, threshold function, piecewise linear function, sigmoidal function; Neural network architectures - single layer feed-forward network, multi layer feed-forward network, recurrent networks; Learning Methods in Neural Networks - classification of learning algorithms, supervised learning, unsupervised learning, reinforced learning, Hebbian learning, gradient descent learning, competitive learning, stochastic learning. Single-Layer NN System - single layer perceptron , learning algorithm for training, linearly separable task, XOR Problem, learning algorithm, ADaptive LINEar Element (ADALINE) architecture and training mechanism; Applications of neural networks - clustering, classification, pattern recognition, function approximation, prediction systems.

fo

.in

rs

de

ea

Fundamentals of Neural Networks

yr

.m

w

w

,w

Artificial Intelligence

ty

or

ab

kr

ha

Topics

C

C

(Lectures 37, 38 2 hours)

R

Slides

1. Introduction 03-12

Why neural network ?, Research history, Biological neuron model, Artificial neuron model, Notations, Functions.

2. Model of Artificial Neuron 13-19

McCulloch-Pitts Neuron Equation, Artificial neuron – basic elements, Activation functions – threshold function, piecewise linear function, sigmoidal function.

3. Neural Network Architectures 20-23

Single layer feed-forward network, Multi layer feed-forward network, Recurrent networks.

4 Learning Methods in Neural Networks 24-29

Classification of learning algorithms, Supervised learning, Unsupervised learning, Reinforced learning, Hebbian Learning, Gradient descent learning, Competitive learning, Stochastic learning.

5. Single-Layer NN System 30-36

Single layer perceptron : learning algorithm for training, linearly separable task, XOR Problem, learning algorithm; ADaptive LINEar Element (ADALINE) : architecture, training mechanism

6. Applications of Neural Networks 37

Clustering, Classification / pattern recognition, Function approximation, Prediction systems.

7. References : 38

02

fo

.in

rs

de

ea

yr

.m

w

w

Neural Networks

,w

ty

or

ab

What is Neural Net ?

kr

ha

C

C

- A neural net is an artificial representation of the human brain that

R

tries to simulate its learning process. An artificial neural network (ANN) is often called a "Neural Network" or simply Neural Net (NN).

- Traditionally, the word neural network is referred to a network of biological neurons in the nervous system that process and transmit information.

- Artificial neural network is an interconnected group of artificial neurons that uses a mathematical model or computational model for information processing based on a connectionist approach to computation.

- The artificial neural networks are made of interconnecting artificial neurons which may share some properties of biological neural networks.

- Artificial Neural network is a network of simple processing elements (neurons) which can exhibit complex global behavior, determined by the connections between the processing elements and element parameters.

- Artificial neural network is an adaptive system that changes its structure based on external or internal information that flows through the network.

03

fo

.in

rs

de

AI-Neural Network – Introduction

ea

1. Introduction

yr

.m

w

w

Neural Computers mimic certain processing capabilities of the human brain.

,w

ty

or

ab

- Neural Computing is an information processing paradigm, inspired by

kr

ha

C

biological system, composed of a large number of highly interconnected

C

R

processing elements (neurons) working in unison to solve specific problems.

- Artificial Neural Networks (ANNs), like people, learn by example.

- An ANN is configured for a specific application, such as pattern recognition or data classification, through a learning process.

- Learning in biological systems involves adjustments to the synaptic connections that exist between the neurons. This is true of ANNs as well.

04

fo

.in

rs

de

AI-Neural Network – Introduction

ea

1.1 Why Neural Network

yr

.m

w

w

■ The conventional computers are good for - fast arithmetic and does

,w

ty

or

what programmer programs, ask them to do.

ab

kr

ha

■ The conventional computers are not so good for - interacting with

C

C

R

noisy data or data from the environment, massive parallelism, fault

tolerance, and adapting to circumstances.

■ The neural network systems help where we can not formulate an algorithmic solution or where we can get lots of examples of the behavior we require.

■ Neural Networks follow different paradigm for computing.

The von Neumann machines are based on the processing/memory abstraction of human information processing.

The neural networks are based on the parallel architecture of biological brains.

■ Neural networks are a form of multiprocessor computer system, with

- simple processing elements ,
- a high degree of interconnection,
- simple scalar messages, and
- adaptive interaction between elements.

05

fo

.in

rs

de

AI-Neural Network – Introduction

ea

1.2 Research History

yr

.m

w

w

,w

The history is relevant because for nearly two decades the future of

ty

or

Neural network remained uncertain.

ab

kr

ha

C

McCulloch and Pitts (1943) are generally recognized as the designers of the

C

R

first neural network. They combined many simple processing units together

that could lead to an overall increase in computational power. They suggested many ideas like : a neuron has a threshold level and once that level is reached the neuron fires. It is still the fundamental way in which ANNs operate. The McCulloch and Pitts's network had a fixed set of weights.

Hebb (1949) developed the first learning rule, that is if two neurons are active at the same time then the strength between them should be increased.

In the 1950 and 60's, many researchers (Block, Minsky, Papert, and Rosenblatt worked on perceptron. The neural network model could be proved to converge to the correct weights, that will solve the problem. The weight adjustment (learning algorithm) used in the perceptron was found more powerful than the learning rules used by Hebb. The perceptron caused great excitement. It was thought to produce programs that could think.

Minsky & Papert (1969) showed that perceptron could not learn those functions which are not linearly separable.

The neural networks research declined throughout the 1970 and until mid 80's because the perceptron could not learn certain important functions.

Neural network regained importance in 1985-86. The researchers, Parker and LeCun discovered a learning algorithm for multi-layer networks called back propagation that could solve problems that were not linearly separable.

06

fo

.in

rs

de

AI-Neural Network – Introduction

ea

1.3 Biological Neuron Model

yr

.m

w

w

The human brain consists of a large number, more than a billion of

,w

ty

neural cells that process information. Each cell works like a simple

or

ab

kr

processor. The massive interaction between all cells and their parallel

ha

C

C

processing only makes the brain's abilities possible.

R

Dendrites are branching fibers that

extend from the cell body or soma.

Soma or cell body of a neuron contains the nucleus and other structures, support chemical processing and production of neurotransmitters.

Axon is a singular fiber carries information away from the soma to the synaptic sites of other neurons (dendrites and somas), muscles, or glands.

Axon hillock is the site of summation for incoming information. At any moment, the collective influence of all neurons that conduct impulses to a given neuron will determine whether or not an

Fig. Structure of Neuron

action potential will be initiated at the axon hillock and propagated along the axon.

Myelin Sheath consists of fat-containing cells that insulate the axon from electrical activity. This insulation acts to increase the rate of transmission of signals. A gap exists between each myelin sheath cell along the axon. Since fat inhibits the propagation of electricity, the signals jump from one gap to the next.

Nodes of Ranvier are the gaps (about 1 μm) between myelin sheath cells long axons are Since fat serves as a good insulator, the myelin sheaths speed the rate of transmission of an electrical impulse along the axon.

Synapse is the point of connection between two neurons or a neuron and a muscle or a gland. Electrochemical communication between neurons takes place at these junctions.

Terminal Buttons of a neuron are the small knobs at the end of an axon that release chemicals called neurotransmitters.

07

fo

.in

rs

de

AI-Neural Network – Introduction

ea

- Information flow in a Neural Cell

yr

.m

w

w

The input /output and the propagation of information are shown below.

,w

ty

or

ab

kr

ha

C

C

R

Structure of a Neural Cell in the Human Brain

■ Dendrites receive activation from other neurons.

■ Soma processes the incoming activations and converts them into output activations.

■ Axons act as transmission lines to send activation to other neurons.

■ Synapses the junctions allow signal transmission between the axons and dendrites.

■ The process of transmission is by diffusion of chemicals called neuro-transmitters.

McCulloch-Pitts introduced a simplified model of this real neurons.

08

fo

.in

rs

de

AI-Neural Network – Introduction

ea

1.4 Artificial Neuron Model

yr

.m

w An artificial neuron is a mathematical function conceived as a simple

w

,w

ty

model of a real (biological) neuron.

or

ab

kr

ha

- The McCulloch-Pitts Neuron

C

C

R

This is a simplified model of real neurons, known as a Threshold Logic Unit.

Input1

Input 2

Σ Output

Input n

■ A set of input connections brings in activations from other neurons.

■ A processing unit sums the inputs, and then applies a non-linear activation function (i.e. squashing / transfer / threshold function).

■ An output line transmits the result to other neurons.

In other words ,

- The input to a neuron arrives in the form of signals.
- The signals build up in the cell.
- Finally the cell discharges (cell fires) through the output .
- The cell can start building up signals again.

09

fo

.in

rs

de

AI-Neural Network – Introduction

ea

1.5 Notations

yr

.m

w

w

Recaps : Scalar, Vectors, Matrices and Functions

,w

ty

or

ab

- Scalar : The number x_i can be added up to give a scalar number.

kr

ha

n

C

$$s = x_1 + x_2 + x_3 + \dots + x_n = \sum x_i$$

C

R

i=1

- Vectors : An ordered sets of related numbers. Row Vectors (1 x n)

$$X = (x_1, x_2, x_3, \dots, x_n), Y = (y_1, y_2, y_3, \dots, y_n)$$

Add : Two vectors of same length added to give another vector.

$$Z = X + Y = (x_1 + y_1, x_2 + y_2, \dots, x_n + y_n)$$

Multiply: Two vectors of same length multiplied to give a scalar.

n

$$p = X \cdot Y = x_1 y_1 + x_2 y_2 + \dots + x_n y_n = \sum x_i y_i$$

i=1

10

fo

.in

rs

de

AI-Neural Network – Introduction

ea

- Matrices : m x n matrix , row no = m , column no = n

yr

.m

w

w

w₁₁ w₁₁ w_{1n}

,w

ty

or

w₂₁ w₂₁ w₂₁

ab

kr

W =

ha

C

C

.

R

w_{m1} w₁₁ w_{mn}

Add or Subtract : Matrices of the same size are added or subtracted component by component. $A+B=C$, $c_{ij} = a_{ij} + b_{ij}$

$a_{11} \ a_{12} \ b_{11} \ b_{12} \ c_{11} = a_{11}+b_{11} \ c_{12} = a_{12}+b_{12}$

+ =

$a_{21} \ a_{22} \ b_{21} \ b_{22} \ c_{21} = a_{21}+b_{21} \ c_{22} = a_{22} + b_{22}$

Multiply : matrix A multiplied by matrix B gives matrix C.

(m x n) (n x p) (m x p)

n

elements $c_{ij} = \sum a_{ik} b_{kj}$

k=1

$a_{11} \ a_{12} \ b_{11} \ b_{12} \ c_{11} \ c_{12}$

x =

$a_{21} \ a_{22} \ b_{21} \ b_{22} \ c_{21} \ c_{22}$

$c_{11} = (a_{11} \times b_{11}) + (a_{12} \times b_{21})$

$c_{12} = (a_{11} \times b_{12}) + (a_{12} \times b_{22})$

$c_{21} = (a_{21} \times b_{11}) + (a_{22} \times b_{21})$

$c_{22} = (a_{21} \times b_{12}) + (a_{22} \times b_{22})$

11

fo

.in

rs

de

AI-Neural Network - Introduction

ea

1.6 Functions

yr

.m

w

w

The Function $y= f(x)$ describes a relationship, an input-output mapping,

,w

ty

or

from x to y.

ab

kr

ha

C

■ Threshold or Sign function : $\text{sgn}(x)$ defined as

C

R

Sign(x)

O/P

1

.8

1 if $x \geq 0$.6 $\text{sgn}(x) =$

.4

0 if $x < 0$

.2

0

-4 -3 -2 -1 0 1 2 3 4 I/P

■ Threshold or Sign function : $\text{sigmoid}(x)$ defined as a smoothed
(differentiable) form of the threshold function

Sign(x)

O/P

1

.8

1 .6

 $\text{sigmoid}(x) =$

-x .4

 $1 + e$

.2

0

-4 -3 -2 -1 0 1 2 3 4 I/P

12

fo

.in

rs

de

AI-Neural Network – Model of Neuron

ea

2. Model of Artificial Neuron

yr

.m

w

w

A very simplified model of real neurons is known as a Threshold Logic

,w

ty

or

Unit (TLU). The model is said to have :

ab

kr

- A set of synapses (connections) brings in activations from other neurons.

ha

C

C

- A processing unit sums the inputs, and then applies a non-linear activation

R

function (i.e. squashing / transfer / threshold function).

- An output line transmits the result to other neurons.

2.1 McCulloch-Pitts (M-P) Neuron Equation

McCulloch-Pitts neuron is a simplified model of real biological neuron.

Input 1

Input 2

Σ Output

Input n

Simplified Model of Real Neuron

(Threshold Logic Unit)

The equation for the output of a McCulloch-Pitts neuron as a function of 1 to n inputs is written as

n

Output = $\text{sgn} \left(\sum_{i=1}^n \text{Input } i - \Phi \right)$

i=1

where Φ is the neuron's activation threshold.

n

If $\sum_{i=1}^n \text{Input } i \geq \Phi$ then Output = 1

i=1

n

If $\sum \text{Input } i < \Phi$ then Output = 0

i=1

In this McCulloch-Pitts neuron model, the missing features are :

- Non-binary input and output,
- Non-linear summation,
- Smooth thresholding,
- Stochastic, and
- Temporal information processing.

13

fo

.in

rs

de

AI-Neural Network – Model Neuron

ea

2.2 Artificial Neuron - Basic Elements

yr

.m

w

w

Neuron consists of three basic components - weights, thresholds, and a

,w

ty

or

single activation function.

ab

kr

ha

x1

C

W1 Activation

C

R

Function

x2 W2

$\sum y$

i=1

$x_n W_n \Phi$

Threshold

Synaptic Weights

Fig Basic Elements of an Artificial Linear Neuron

■ Weighting Factors w

The values $w_1, w_2, \dots, w_n$ are weights to determine the strength of input vector $X = [x_1, x_2, \dots, x_n]^T$. Each input is multiplied by the associated weight of the neuron connection $X^T W$. The +ve weight excites and the -ve weight inhibits the node output.

n

T

$$I = X \cdot W = x_1 w_1 + x_2 w_2 + \dots + x_n w_n = \sum_{i=1}^n x_i w_i$$

■ Threshold Φ

The node's internal threshold Φ is the magnitude offset. It affects the activation of the node output y as:

n

$$Y = f(I) = f\left\{\sum_{i=1}^n x_i w_i - \Phi_k\right\}$$

To generate the final output Y , the sum is passed on to a non-linear filter f called Activation Function or Transfer function or Squash function which releases the output Y .

14

fo

.in

rs

de

AI-Neural Network – Model of Neuron

ea

■ Threshold for a Neuron

yr

.m

w

w

In practice, neurons generally do not fire (produce an output) unless

,w

ty

or

their total input goes above a threshold value.

ab

kr

ha

C

The total input for each neuron is the sum of the weighted inputs

C

R

to the neuron minus its threshold value. This is then passed through the sigmoid function. The equation for the transition in a neuron is :

$a = 1/(1 + \exp(-x))$ where

$x = \sum a_i w_i - Q$

i

a is the activation for the neuron

a_i is the activation for neuron i

w_i is the weight

Q is the threshold subtracted

■ Activation Function

An activation function f performs a mathematical operation on the signal output. The most common activation functions are:

- Linear Function, - Threshold Function,
- Piecewise Linear Function, - Sigmoidal (S shaped) function,
- Tangent hyperbolic function

The activation functions are chosen depending upon the type of problem to be solved by the network.

15

fo

.in

rs

de

AI-Neural Network – Model of Neuron

ea

2.2 Activation Functions f - Types

yr

.m

w

w

Over the years, researches tried several functions to convert the input into

,w

ty

or

an outputs. The most commonly used functions are described below.

ab

kr

Horizontal axis shows sum of inputs .

ha

- I/P

C

C

- O/P Vertical axis shows the value the function produces ie output.

R

- All functions f are designed to produce values between 0 and 1.

• Threshold Function

A threshold (hard-limiter) activation function is either a binary type or a bipolar type as shown below.

binary threshold Output of a binary threshold function produces :

O/p 1 if the weighted sum of the inputs is +ve,

1

0 if the weighted sum of the inputs is -ve.

I/P

1 if $I \geq 0$

$Y = f(I) =$

0 if $I < 0$

bipolar threshold Output of a bipolar threshold function produces :

O/p 1 if the weighted sum of the inputs is +ve,

1

-1 if the weighted sum of the inputs is -ve.

I/P 1 if $I \geq 0$

$Y = f(I) =$

-1 -1 if $I < 0$

Neuron with hard limiter activation function is called McCulloch-Pitts model.

16

fo

.in

rs

de

AI-Neural Network – Model of Neuron

ea

• Piecewise Linear Function

yr

.m

w

w

This activation function is also called saturating linear function and can

,w

ty

or

have either a binary or bipolar range for the saturation limits of the output.

ab

kr

The mathematical model for a symmetric saturation function is described

ha

C

C

below.

R

Piecewise Linear This is a sloping function that produces :

-1 for a -ve weighted sum of inputs,

O/p

+1 1 for a +ve weighted sum of inputs.

$\propto I$ proportional to input for values between +1

I/P

and -1 weighted sum,

-1

1 if $I \geq 0$

$Y = f(I) = I$ if $-1 \leq I \leq 1$

-1 if $I < 0$

17

fo

.in

rs

de

AI-Neural Network – Model of Neuron

ea

- Sigmoidal Function (S-shape function)

yr

.m

w

w

The nonlinear curved S-shape function is called the sigmoid function.

,w

ty

or

This is most common type of activation used to construct the neural

ab

kr

networks. It is mathematically well behaved, differentiable and strictly

ha

C

C

increasing function.

R

Sigmoidal function A sigmoidal transfer function can be written in the form:

1 O/P

1

$\alpha = 2.0$

$\alpha = 1.0$ $Y = f(I) = \frac{1}{1 + e^{-\alpha I}}$, $0 \leq f(I) \leq 1$

$1 + e^{-\alpha I}$

$\alpha = 0.5$

$0.5 = 1/(1 + \exp(-\alpha I))$, $0 \leq f(I) \leq 1$

This is explained as

I/P

≈ 0 for large -ve input values,

-4 -2 0 1 2

1 for large +ve values, with a

smooth transition between the two.

α is slope parameter also called shape parameter; symbol the λ is also used to represented this parameter.

The sigmoidal function is achieved using exponential equation.

By varying α different shapes of the function can be obtained which adjusts the abruptness of the function as it changes between the two asymptotic values.

18

fo

.in

rs

de

AI-Neural Network – Model of Neuron

ea

• Example :

yr

.m

w

w

The neuron shown consists of four inputs with the weights.

,w

ty

or

ab

kr

x1=1 +1

ha

Activation

C

Function

C

R

x2=2 I

+1

 Σy

X3=5 -1

Summing

xn=8 Junction

+2 $\Phi=0$

Threshold

Synaptic

Weights

Fig Neuron Structure of Example

The output I of the network, prior to the activation function stage, is

+1

+1

$$I = XT \cdot W = 1 \ 2 \ 5 \ 8 = 14$$

-1

+2

$$= (1 \times 1) + (2 \times 1) + (5 \times -1) + (8 \times 2) = 14$$

With a binary activation function the outputs of the neuron is:

$y(\text{threshold}) = 1;$

19

fo

.in

rs

de

AI-Neural Network – Architecture

ea

3. Neural Network Architectures

yr

.m

w

w

An Artificial Neural Network (ANN) is a data processing system, consisting

,w

ty

or

large number of simple highly interconnected processing elements as

ab

kr

artificial neuron in a network structure that can be represented using a

ha

C

C

directed graph G , an ordered 2-tuple (V, E) , consisting a set V of vertices

R

and a set E of edges.

- The vertices may represent neurons (input/output) and

- The edges may represent synaptic links labeled by the weights attached.

Example :

e5

V1 V3

V5

e2

e4

e5

V2 V4

e3

Fig. Directed Graph

Vertices $V = \{ v_1, v_2, v_3, v_4, v_5 \}$

Edges $E = \{ e_1, e_2, e_3, e_4, e_5 \}$

20

fo

.in

rs

de

AI-Neural Network - Architecture

ea

3.1 Single Layer Feed-forward Network

yr

.m

w

w

The Single Layer Feed-forward Network consists of a single layer of

,w

ty

or

weights, where the inputs are directly connected to the outputs, via a

ab

kr

series of weights. The synaptic links carrying weights connect every input

ha

C

C

to every output, but not other way. This way it is considered a network of

R

feed-forward type. The sum of the products of the weights and the inputs is calculated in each neuron node, and if the value is above some threshold (typically 0) the neuron fires and takes the activated value (typically 1); otherwise it takes the deactivated value (typically -1).

input x_i weights w_{ij} output y_j

$w_{11} y_1$

x_1

w_{21}

w_{12}

$w_{22} y_2$

x_2

w_{2m}

w_{1m}

w_{n1}

w_{n2}

$x_n w_{nm}$

y_m

Single layer

Neurons

Fig. Single Layer Feed-forward Network

21

fo

.in

rs

de

AI-Neural Network – Architecture

ea

3.2 Multi Layer Feed-forward Network

yr

.m

w

w

The name suggests, it consists of multiple layers. The architecture of

,w

ty

or

this class of network, besides having the input and the output layers,

ab

kr

also have one or more intermediary layers called hidden layers. The

ha

C

C

computational units of the hidden layer are known as hidden neurons.

R

Input Output

hidden layer hidden layer

weights v_{ij} weights w_{jk} y_1

w_{11}

$x_1 v_{11}$

w_{12}

v_{21} y_1 y_2
 x_2 w_{11}
 v_{1m}
 v_{2m} y_3
 v_{n1} w_{1m}
 V_ℓ m
 y_m
 Hidden Layer
 x_ℓ neurons y_j
 y_n
 Input Layer
 neurons x_i Output Layer
 neurons z_k

Fig. Multilayer feed-forward network in $(\ell - m - n)$ configuration.

- The hidden layer does intermediate computation before directing the input to output layer.
- The input layer neurons are linked to the hidden layer neurons; the weights on these links are referred to as input-hidden layer weights.
- The hidden layer neurons and the corresponding weights are referred to as output-hidden layer weights.
- A multi-layer feed-forward network with ℓ input neurons, m_1 neurons in the first hidden layers, m_2 neurons in the second hidden layers, and n output neurons in the output layers is written as $(\ell - m_1 - m_2 - n)$.
- Fig. above illustrates a multilayer feed-forward network with a configuration $(\ell - m - n)$.

22

fo

.in

rs

de

AI-Neural Network – Architecture

ea

3.3 Recurrent Networks

yr

.m

w The Recurrent Networks differ from feed-forward architecture.

w

,w

ty

A Recurrent network has at least one feed back loop.

or

ab

Example :

kr

ha

y1

C

C

x1

R

y1 y2

Feedback

links

x2

Yn

ym

X_l

Input Layer Hidden Layer Output Layer

neurons x_i neurons y_j neurons z_k

Fig. Recurrent neural network

There could be neurons with self-feedback links;

that is the output of a neuron is feedback into itself as input.

23

fo

.in

rs

de

AI-Neural Network -Learning methods

ea

4. Learning methods in Neural Networks

yr

.m

w

w

The learning methods in neural networks are classified into three basic types :

,w

ty

or

- Supervised Learning,

ab

kr

- Unsupervised Learning

ha

C

C

- Reinforced Learning

R

These three types are classified based on :

- presence or absence of teacher and
- the information provided for the system to learn.

These are further categorized, based on the rules used, as

- Hebbian,
- Gradient descent,
- Competitive
- Stochastic learning.

24

fo

.in

rs

de

AI-Neural Network -Learning methods

ea

Classification of Learning Algorithms

yr

.m

w

w

,w

Fig. below indicate the hierarchical representation of the algorithms mentioned

ty

or

in the previous slide. These algorithms are explained in subsequent slides.

ab

kr

ha

C

C

R

Neural Network

Learning algorithms

Supervised Learning Reinforced Learning Unsupervised Learning

(Error based) (Output based)

Stochastic Error Correction Hebbian Competitive

Gradient descent

Least Mean Back

Square Propagation

Fig. Classification of learning algorithms

25

fo

.in

rs

de

AI-Neural Network -Learning methods

ea

- Supervised Learning

yr

.m

w - A teacher is present during learning process and presents

w

,w

ty

expected output.

or

ab

- Every input pattern is used to train the network.

kr

ha

C

- Learning process is based on comparison, between network's

C

R

computed output and the correct expected output, generating "error".

- The "error" generated is used to change network parameters that

result improved performance.

- Unsupervised Learning

- No teacher is present.

- The expected or desired output is not presented to the network.
- The system learns of it own by discovering and adapting to the structural features in the input patterns.
- Reinforced Learning
- A teacher is present but does not present the expected or desired output but only indicated if the computed output is correct or incorrect.
- The information provided helps the network in its learning process.
- A reward is given for correct answer computed and a penalty for a wrong answer.

Note : The Supervised and Unsupervised learning methods are most popular forms of learning compared to Reinforced learning.

26

fo

.in

rs

de

AI-Neural Network –Learning methods

ea

- Hebbian Learning

yr

.m

w

w

Hebb proposed a rule based on correlative weight adjustment.

,w

ty

or

ab

In this rule, the input-output pattern pairs (X_i , Y_i) are associated by

kr

ha

C

the weight matrix W , known as correlation matrix computed as

C

R

n

$W = \sum_{i=1}^n X_i Y_i^T$

i=1

T

where Y_i is the transpose of the associated output vector Y_i

There are many variations of this rule proposed by the other researchers (Kosko, Anderson, Lippman) .

27

fo

.in

rs

de

AI-Neural Network -Learning methods

ea

- Gradient Descent Learning

yr

.m

w

w

This is based on the minimization of errors E defined in terms of weights

,w

ty

or

and the activation function of the network.

ab

kr

ha

C

- Here, the activation function of the network is required to be

C

R

differentiable, because the updates of weight is dependent on the gradient of the error E .

- If ΔW_{ij} is the weight update of the link connecting the i th and the j th neuron of the two neighboring layers, then ΔW_{ij} is defined as

$$\Delta W_{ij} = \eta (\partial E / \partial W_{ij})$$

where η is the learning rate parameters and $(\partial E / \partial W_{ij})$ is error gradient with reference to the weight W_{ij} .

Note : The Hoffs Delta rule and Back-propagation learning rule are the examples of Gradient descent learning.

28

fo

.in

rs

de

AI-Neural Network -Learning methods

ea

- Competitive Learning

yr

.m

w

w

- In this method, those neurons which respond strongly to the input

,w

ty

or

stimuli have their weights updated.

ab

kr

- When an input pattern is presented, all neurons in the layer compete,

ha

C

C

and the winning neuron undergoes weight adjustment .

R

- This strategy is called "winner-takes-all".

- Stochastic learning

- In this method the weights are adjusted in a probabilistic fashion.

- Example : Simulated annealing which is a learning mechanism employed by Boltzmann and Cauchy machines.

29

fo

.in

rs

de

AI-Neural Network -Single Layer learning

ea

5. Single-Layer NN Systems

yr

.m

w

w

Here, a simple Perceptron Model and an ADALINE Network Model is presented.

,w

ty

or

ab

5.1 Single Layer Perceptron

kr

ha

C

Definition : An arrangement of one input layer of neurons feed forward

C

R

to one output layer of neurons is known as Single Layer Perceptron.

input x_i weights w_{ij} output y_j

w_{11} y_1

x_1

w_{21}

w_{12}

w_{22} y_2

x_2

w_{2m}

w_{1m}

w_{n1}

w_{n2}

x_n w_{nm}

y_m

Single layer

Perceptron

Fig. Simple Perceptron Model

1 if $\text{net } j \geq 0$

$y_j = f(\text{net } j) = \begin{cases} 1 & \text{if } \text{net } j \geq 0 \\ 0 & \text{if } \text{net } j < 0 \end{cases}$ where $\text{net } j = \sum_{i=1}^n x_i w_{ij}$

$i=1$

0 if $\text{net } j < 0$

30

fo

.in

rs

de

AI-Neural Network -Single Layer learning

ea

- Learning Algorithm for Training Perceptron

yr

.m

w

w

The training of Perceptron is a supervised learning algorithm where

,w

ty

or

weights are adjusted to minimize error when ever the output does

ab

kr

not match the desired output.

ha

C

C

R

- If the output is correct then no adjustment of weights is done.

$K+1$ K

i.e. $W = W$

ij ij

- If the output is 1 but should have been 0 then the weights are decreased on the active input link

$K+1$ K

i.e. $W = W - \alpha \cdot x_i$

ij ij

- If the output is 0 but should have been 1 then the weights are increased on the active input link

$K+1$ K

i.e. $W = W + \alpha \cdot x_i$

ij ij

Where

$K+1$ K

W is the new adjusted weight, W is the old weight

ij ij

x_i is the input and α is the learning rate parameter.

α small leads to slow and α large leads to fast learning.

31

fo

.in

rs

de

AI-Neural Network –Single Layer learning

ea

- Perceptron and Linearly Separable Task

yr

.m

w

w

,w

Perceptron can not handle tasks which are not separable.

ty

or

ab

- Definition : Sets of points in 2-D space are linearly separable if the

kr

ha

C

sets can be separated by a straight line.

C

R

- Generalizing, a set of points in n-dimensional space are linearly separable if there is a hyper plane of (n-1) dimensions separates the sets.

Example

S1 S2 S1

S2

(a) Linearly separable patterns (b) Not Linearly separable patterns

Note : Perceptron cannot find weights for classification problems that are not linearly separable.

32

fo

.in

rs

de

AI-Neural Network –Single Layer learning

ea

- XOR Problem : Exclusive OR operation

yr

.m

w

w

,w

Input x1 Input x2 Output X2

ty

or

(0, 1) (1, 1)

° •

ab

0 0 0

Even parity •

kr

1 1 0

ha

0 1 1

C

Odd parity °

C

1 0 1

• °

R

(0, 0) X1

(0, 1)

XOR truth table

Output of XOR

Even parity means even number of 1 bits in the input in X1 , x2 plane

Odd parity means odd number of 1 bits in the input

- There is no way to draw a single straight line so that the circles are on one side of the line and the dots on the other side.

- Perceptron is unable to find a line separating even parity input patterns from odd parity input patterns.

33

fo

.in

rs

de

AI-Neural Network –Single Layer learning

ea

- Perceptron Learning Algorithm

yr

.m

w The algorithm is illustrated step-by-step.

w

,w

ty

■ Step 1 :

or

ab

Create a peceptron with $(n+1)$ input neurons $x_0, x_1, \dots, x_n$,

kr

ha

C

where $x_0 = 1$ is the bias input.

C

R

Let O be the output neuron.

■ Step 2 :

Initialize weight $W = (w_0, w_1, \dots, w_n)$ to random weights.

■ Step 3 :

Iterate through the input patterns X_j of the training set using the

n

weight set; ie compute the weighted sum of inputs $\text{net } j = \sum_{i=1}^n x_i w_i$

$i=1$

for each input pattern j .

■ Step 4 :

Compute the output y_j using the step function

1 if $\text{net } j \geq 0$

$y_j = f(\text{net } j) =$ where $\text{net } j = \sum_{i=1}^n x_i w_i$

$i=1$

0 if $\text{net } j < 0$

■ Step 5 :

Compare the computed output y_j with the target output y_j for

each input pattern j .

If all the input patterns have been classified correctly, then output
(read) the weights and exit.

■ Step 6 :

Otherwise, update the weights as given below :

If the computed outputs y_j is 1 but should have been 0,

Then $w_i = w_i - \alpha x_i$, $i = 0, 1, 2, \dots, n$

If the computed outputs y_j is 0 but should have been 1,

Then $w_i = w_i + \alpha x_i$, $i = 0, 1, 2, \dots, n$

where α is the learning parameter and is constant.

■ Step 7 :

goto step 3

■ END

34

fo

.in

rs

de

AI-Neural Network -ADALINE

ea

5.2 ADaptive LINEar Element (ADALINE)

yr

.m

w

w

An ADALINE consists of a single neuron of the McCulloch-Pitts type,

,w

ty

or

where its weights are determined by the normalized least mean

ab

kr

square (LMS) training law. The LMS learning rule is also referred to as

ha

C

C

delta rule. It is a well-established supervised training method that

R

has been used over a wide range of diverse applications.

- Architecture of a simple ADALINE

x1 W1

x2 W2

Output

Σ

Neuron

$x_n W_n$

Error

Σ

Desired Output

The basic structure of an ADALINE is similar to a neuron with a linear activation function and a feedback loop. During the training phase of ADALINE, the input vector as well as the desired output are presented to the network.

[The complete training mechanism has been explained in the next slide.]

35

fo

.in

rs

de

AI-Neural Network –ADALINE

ea

- ADALINE Training Mechanism

yr

.m

w (Ref. Fig. in the previous slide - Architecture of a simple ADALINE)

w

,w

ty

or

■ The basic structure of an ADALINE is similar to a linear neuron

ab

kr

ha

with an extra feedback loop.

C

C

R

■ During the training phase of ADALINE, the input vector

T

$X = [x_1, x_2, \dots, x_n]$ as well as desired output are presented to the network.

■ The weights are adaptively adjusted based on delta rule.

■ After the ADALINE is trained, an input vector presented to the network with fixed weights will result in a scalar output.

■ Thus, the network performs an n dimensional mapping to a scalar value.

■ The activation function is not used during the training phase.

Once the weights are properly adjusted, the response of the trained unit can be tested by applying various inputs, which are not in the training set. If the network produces consistent responses to a high degree with the test inputs, it is said that the network could generalize. The process of training and generalization are two important attributes of this network.

Usage of ADLINE :

In practice, an ADALINE is used to

- Make binary decisions; the output is sent through a binary threshold.
- Realizations of logic gates such as AND, NOT and OR .
- Realize only those logic functions that are linearly separable.

36

fo

.in

rs

de

AI-Neural Network -Applications

ea

6. Applications of Neural Network

yr

.m

w Neural Network Applications can be grouped in following categories:

w

,w

ty

or

■ Clustering:

ab

kr

A clustering algorithm explores the similarity between patterns and

ha

C

C

places similar patterns in a cluster. Best known applications include

R

data compression and data mining.

■ Classification/Pattern recognition:

The task of pattern recognition is to assign an input pattern (like handwritten symbol) to one of many classes. This category includes algorithmic implementations such as associative memory.

■ Function approximation :

The tasks of function approximation is to find an estimate of the unknown function subject to noise. Various engineering and scientific disciplines require function approximation.

■ Prediction Systems:

The task is to forecast some future values of a time-sequenced data. Prediction has a significant impact on decision support systems. Prediction differs from function approximation by considering time factor. System may be dynamic and may produce different results for the same input data based on system state (time).

37

fo

.in

rs

de

AI-AI-Neural Network -References

ea

7. References : Textbooks

yr

.m

w

w

1. "Neural Networks: A Comprehensive Foundation", by Simon S. Haykin, (1999),

,w

ty

Prentice Hall, Chapter 1-15, page 1-889.

or

ab

kr

2. "Elements of Artificial Neural Networks", by Kishan Mehrotra, Chilukuri K. Mohan

ha

C

and Sanjay Ranka, (1996), MIT Press, Chapter 1-7, page 1-339.

C

R

3. "Fundamentals of Neural Networks: Architecture, Algorithms and Applications", by Laurene V. Fausett, (1993), Prentice Hall, Chapter1-7, page 1-449.

4. "Neural Network Design", by Martin T. Hagan, Howard B. Demuth and Mark Hudson Beale, (1996) , PWS Publ. Company, Chapter 1-19, page 1-1 to 19-14.

5. "An Introduction to Neural Networks", by James A. Anderson, (1997), MIT Press, Chapter 1- 17, page 1-585.

6. "AI: A New Synthesis", by Nils J. Nilsson, (1998), Morgan Kaufmann Inc., Chapter 3, Page 37-48.

7. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

38

fo

.in

rs

de

ea

yr

Natural Language Processing : AI Course Lecture 41, notes, slides

.m

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

www.myreaders.info

kr

ha

C

C

R

Natural Language Processing

Artificial Intelligence

Natural Language Processing, topics : Introduction, definition, formal language, linguistic and language processing, terms related

to linguistic analysis, grammatical structure of utterances - sentence, constituents, phrases, classifications and structural rules; Syntactic Processing - context free grammar (CFG), terminal, non-terminal and start symbols, parser, Semantics and Pragmatics.

fo

.in

rs

de

ea

Natural Language Processing

yr

.m

w

w

,w

ty

Artificial Intelligence

or

ab

kr

ha

Topics

C

C

R

(Lecture 41 , 1 hours) Slides

1. Introduction 03-19

Natural language : Definition, Processing, Formal language, Linguistic and language processing, Terms related to linguistic analysis, Grammatical structure of utterances - sentence, constituents, phrases, classifications and structural rules.

2. Syntactic Processing : 20-25

Context free grammar (CFG) - Terminal , Non-terminal and start symbols; Parsar.

3. Semantic and Pragmatic 26

4. References 27

02

fo

.in

rs

de

ea

yr

Natural Language Processing

.m

w

w

,w

ty

What is NLP ?

or

ab

kr

- NLP is Natural Language Processing.

ha

C

C

Natural languages are those spoken by people.

R

- NLP encompasses anything a computer needs to understand natural language (typed or spoken) and also generate the natural language.
- Natural Language Processing (NLP) is a subfield of Artificial intelligence and linguistic, devoted to make computers "understand" statements written in human languages.

03

fo

.in

rs

de

AI - NLP- Introduction

ea

1. Introduction

yr

.m

w

w

Natural Language Processing (NLP) is a subfield of artificial intelligence and

,w

ty

or

linguistic, devoted to make computers "understand" statements written in

ab

kr

human languages.

ha

C

C

R

1.1 Natural Language

A natural language (or ordinary language) is a language that is spoken, written by humans for general-purpose communication.

Example : Hindi, English, French, and Chinese, etc.

A language is a system, a set of symbols and a set of rules (or grammar).

- The Symbols are combined to convey new information.

- The Rules govern the manipulation of symbols.

04

fo

.in

rs

de

AI - NLP - Introduction

ea

Natural Language Processing (NLP)

yr

.m

w

w

NLP encompasses anything a computer needs to understand natural

,w

ty

or

language (typed or spoken) and also generate the natural language.

ab

kr

ha

C

‡ Natural Language Understanding (NLU) :

C

R

The NLU task is understanding and reasoning while the input is a natural language.

Here we ignore the issues of natural language generation.

‡ Natural Language Generation (NLG) :

NLG is a subfield of natural language processing NLP.

NLG is also referred to text generation.

Natural Language Processing

NL input Computer NL output

Understanding Generation

05

fo

.in

rs

de

AI – NLP - Introduction

ea

1.2 Formal Language

yr

.m

w

w

Before defining formal language Language, we need to define symbols,

,w

ty

or

alphabets, strings and words.

ab

kr

ha

C

Symbol is a character, an abstract entity that has no meaning by itself.

C

R

e.g., Letters, digits and special characters

Alphabet is finite set of symbols;

an alphabet is often denoted by Σ (sigma)

e.g., $B = \{0, 1\}$ says B is an alphabet of two symbols, 0 and 1.

$C = \{a, b, c\}$ says C is an alphabet of three symbols, a, b and c.

String or a word is a finite sequence of symbols from an alphabet.

e.g., 01110 and 111 are strings from the alphabet B above.

aaabccc and b are strings from the alphabet C above.

Language is a set of strings from an alphabet .

Formal language (or simply language) is a set L of strings over some finite alphabet Σ .

Formal language is described using formal grammars.

06

fo

.in

rs

de

AI - NLP - Introduction

ea

1.3 Linguistic and Language Processing

yr

.m

w

w

Linguistics is the science of language. Its study includes :

,w

ty

or

□ sounds (phonology),

ab

kr

□ word formation (morphology),

ha

C

C

□ sentence structure (syntax),

R

□ meaning (semantics), and understanding (pragmatics) etc.

The levels of linguistic analysis are shown below.

□ higher level corresponds to Speech Recognition (SR)

□ lower levels corresponds to Natural Language Processing (NLP).

Levels Of Linguistic Analysis

Acoustic signal

Phonetics - Production and perception of speech

Phones

Phonology - Sound patterns of language

SR Letter - strings

Lexicon - Dictionary of words in a language

Morphemes

Morphology - Word formation and structure

Words

Syntax - Sentence structure

NLP Phrases & sentences

Semantics - Intended meaning

Meaning out of context

Pragmatics - Understanding from external info

Meaning in context

07

fo

.in

rs

de

AI - NLP - Introduction

ea

- Steps of Natural Language Processing (NLP)

yr

.m

w

w

Natural Language Processing is done at 5 levels, as shown in the previous

,w

ty

or

slide. These levels are briefly stated below.

ab

kr

ha

■ Morphological and Lexical Analysis :

C

C

R

The lexicon of a language is its vocabulary, that include its words

and expressions. Morphology is the identification, analysis and description of structure of words. The words are generally accepted as being the smallest units of syntax. The syntax refers to the rules and principles that govern the sentence structure of any individual language.

Lexical analysis: The aim is to divide the text into paragraphs, sentences and words. the lexical analysis can not be performed in isolation from morphological and syntactic analysis

■ Syntactic Analysis :

Here the analysis is of words in a sentence to know the grammatical structure of the sentence. The words are transformed into structures that show how the words relate to each others. Some word sequences may be rejected if they violate the rules of the language for how words may be combined.

Example : An English syntactic analyzer would reject the sentence say :
" Boy the go the to store ".

08

fo

.in

rs

de

AI – NLP - Introduction

ea

■ Semantic Analysis :

yr

.m

w It derives an absolute (dictionary definition) meaning from context;

w

,w

ty

it determines the possible meanings of a sentence in a context.

or

ab

The structures created by the syntactic analyzer are assigned meaning.

kr

ha

C

Thus, a mapping is made between the syntactic structures and objects

C

R

in the task domain. The structures for which no such mapping is possible are rejected.

Example : the sentence "Colorless green ideas . . ." would be rejected as semantically anomalous because colorless and green make no sense.

■ Discourse Integration :

The meaning of an individual sentence may depend on the sentences that precede it and may influence the meaning of the sentences that follow it.

Example : the word " it " in the sentence, "you wanted it" depends on the prior discourse context.

■ Pragmatic analysis :

It derives knowledge from external commonsense information; it means understanding the purposeful use of language in situations, particularly those aspects of language which require world knowledge; The idea is, what was said is reinterpreted to determine what was actually meant. Example : the sentence "Do you know what time it is ?" should be interpreted as a request.

09

fo

.in

rs

de

AI - NLP - Introduction

ea

1.4 Defining Terms related to Linguistic Analysis

yr

.m

w

w

The following terms are explained in next few slides.

,w

ty

or

Phones, Phonetics, Phonology, Strings, Lexicon, Words, Determiner,

ab

kr

Morphology, Morphemes, Syntax, Semantics, Pragmatics, Phrase, and

ha

C

C

Sentence.

R

• Terms

■ Phones

The Phones are acoustic patterns that are significant and distinguishable in some human language.

Example : In English, the L - sounds at the beginning and end of the word "loyal", are termed "light L" and "dark L" by linguists.

■ Phonetics

Tells how acoustic signals are classified into phones.

■ Phonology

Tells how phones are grouped together to form phonemes in particular human languages.

10

fo

.in

rs

de

AI - NLP - Introduction

ea

■ Strings

yr

.m

w An alphabet is a finite set of symbols.

w

,w

ty

Example : English alphabets

or

ab

{ a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z }

kr

ha

C

C

A String is a sequence of symbols taken from an alphabet.

R

■ Lexicon

Lexicon is collection of information about words of a language.
 The information is about the lexical categories to which words belong.
 Example : "pig" is usually a noun (N), but also occurs as a verb(V)
 and an adjective(ADJ).

Lexicon structure : as collection of lexical entries.

Example : ("pig" N, V, ADJ)

11

fo

.in

rs

de

AI - NLP - Introduction

ea

■ Words

yr

.m

w Word is a unit of language that carries meaning.

w

,w

ty

Example : words like bear, car, house are very different from

or

ab

words like run, sleep, think, and are different from

kr

ha

C

words like in, under, about.

C

R

These and other categories of words have names : nouns, verbs,
 prepositions, and so on.

Words build phrases, which in turn build sentences.

■ Determiner

Determiners occur before nouns and indicate the kind of reference
 which the noun has.

Example below shows determiners marked by "bold letters"

the boy a bus our car

these children both hospitals

12

fo

.in

rs

de

AI - NLP - Introduction

ea

■ Morphology

yr

.m

w Morphology is the analysis of words into morphemes, and conversely

w

,w

ty

the synthesis of words from morphemes.

or

ab

kr

ha

■ Morphemes

C

C

A smallest meaningful unit in the grammar of a language.

R

A smallest linguistic unit that has semantic meaning.

A unit of language immediately below the 'word level'.

A smallest part of a word that can carry a discrete meaning.

Example : the word "unbreakable" has 3 morphemes:

1 "un-" a bound morpheme;

2 "-break-" a free morpheme; and

3 "-able" a bound morpheme;

Also "un-" is also a prefix; "-able" is a suffix; Both are affixes.

Morphemes are of many types, stated in the next slide.

13

fo

.in

rs

de

AI - NLP - Introduction

ea

Types of Morphemes

yr

.m

w

w

‡ Free Morphemes

,w

ty

or

can appear stand alone, or "free" .

ab

kr

Example : "town", "dog" or with other lexemes

ha

C

C

"town hall" , "dog house".

R

‡ Bound Morphemes

appear only together with other morphemes to form a lexeme.

Example : "un-" ; in general it tend to be prefix and suffix.

‡ Inflectional Morphemes

modify a word's tense, number, aspect, etc.

Example : dog morpheme with plural marker morpheme s becomes dogs.

‡ Derivational Morphemes

can be added to a word to derive another word.

Example : addition of "-ness" to "happy" gives "happiness."

‡ Root Morpheme

It is the primary lexical unit of a word; roots can be either free or bound morphemes; sometimes "root" is used to describe word minus its inflectional endings, but with its lexical endings.

Example : word chatters has the inflectional root or lemma chatter, but the lexical root chat.

Inflectional roots are often called stems, and a root in the stricter sense may be thought of as a mono-morphemic stem.

‡ Null Morpheme

It is an "invisible" affix, also called zero morpheme represented as either the figure zero (0), the empty set symbol $\emptyset$, or its variant $\emptyset$.

Adding a null morpheme is called null affixation, null derivation or zero derivation; null morpheme that contrasts singular morpheme with the plural morpheme.

e.g., cat = cat + -0 = ROOT("cat") + SINGULAR

cats = cat + -s = ROOT("cat") + PLURAL

14

fo

.in

rs

de

AI - NLP - Introduction

ea

■ Syntax

yr

.m

w Syntax is the structure of language. It is the grammatical arrangement

w

,w

ty

of words in a sentence to show its relationship to one another in a

or

ab

sentence; Syntax is finite set of rules that specifies a language;

kr

ha

C

Syntax rules govern proper sentence structure;

C

R

Syntax is represented by Parse Tree, a way to show the structure of a language fragment, or by a list.

■ Semantics

Semantic is Meaning of words / phrases/ sentences/ whole texts.

Normally semantic is restricted to "meaning out of context" - that is, meaning as it can be determined without taking context into account.

■ Pragmatics

Pragmatics tell how language is used; that is 'meaning in context'.

Example: if someone says "the door is open" then it is necessary to know which door "the door" refers to;

Need to know what the intention of the speaker :
 could be a pure statement of fact,
 could be an explanation of how the cat got in, or
 could be a request to the person addressed to close the door.

15

fo

.in

rs

de

AI – NLP - Introduction

ea

1.5 Grammatical Structure of Utterances

yr

.m

w

w

Here sentence, constituent, phrase, classification and structural rule are

,w

ty

or

explained.

ab

kr

■ Sentence

ha

C

C

Sentence is a string of words satisfying grammatical rules of a language;

R

Sentences are classified as simple, compound, and complex.

Sentence is often abbreviated to "S".

Sentence (S) : "The dog bites the cat".

■ Constituents

Assume that a phrase is a construction of some kind.

Here construction means a syntactic arrangement that consists of parts, usually two, called "constituents".

Examples : The phrase the man is a construction consists of two constituents the and man. A few more examples are shown below.

Phrase : the man Phrase : traveled slowly

Constituents : the and man Constituents : traveled and slowly.

Construction : Construction :

the man traveled slowly

the man traveled slowly

Phrase : the man traveled slowly

Constituents four : the , man , traveled , slowly

Construction :

the man traveled slowly

the man traveled slowly

the man traveled slowly

16

fo

.in

rs

de

AI - NLP - Introduction

ea

■ Phrase

yr

.m

w A Phrase is a group of words (minimum is two) that functions as a

w

,w

ty

single unit in the syntax of a sentence.

or

ab

kr

e.g., 1: "the house at the end of the street " is a phrase, acts like noun.

ha

C

e.g., 2: "end of the street " is a phrase, acts like adjective;

C

R

How phrases are formed is governed by phrase structure rules.

Most phrases have a head or central word, which defines the type of phrase. Head is often the first word of the phrase. Some phrases, can be headless.

e.g.,3: "the rich" is a noun phrase composed of a determiner and an

adjective, but no noun.

Phrases may be classified by the type of head they take.

[Continued in next slide]

17

fo

.in

rs

de

AI – NLP - Introduction

ea

[Continued from previous slide]

yr

.m

w

w

Classification of Phrases : names (abbreviation)

,w

The most accepted classifications for phrases are stated below.

ty

or

ab

‡ Sentence (S) : often abbreviated to "S".

kr

ha

C

‡ Noun phrase (NP) : noun or pronoun as head, or optionally

C

R

accompanied by a set of modifiers; The possible modifiers include:

determiners: articles (the, a) or adjectives (the red ball) etc ;

example : "the black cat", "a cat on the mat".

‡ Verb phrase (VP) : verb as head,

example : "eat cheese", "jump up and down".

‡ Adjectival phrase (AP) : adjective as head,

example : "full of toys"

‡ Adverbial phrase (AdvP) : adverb as head,

example : "very carefully"

‡ Prepositional phrase (PP) : preposition as head,

example : "in love", "over the rainbow".

‡ Determiner phrase (DP) : determiner as head

example : "a little dog", "the little dogs".

In English, determiners are usually placed before the noun as a noun modifier that includes : articles (the, a), demonstratives (this, that), numerals (two, five, etc.), possessives (my, their, etc.), and quantifiers (some, many, etc.).

18

fo

.in

rs

de

AI – NLP - Introduction

ea

■ Phrase Structure Rules

yr

.m

w Phrase-structure rules are a way to describe language syntax. Rules

w

,w

ty

determine what goes into phrase and how its constituents are ordered.

or

ab

They are used to break a sentence down to its constituent parts namely

kr

ha

C

phrasal categories and lexical categories.

C

R

□ Phrasal category include : noun phrase, verb phrase, prepositional phrase;

□ Lexical category include : noun, verb, adjective, adverb, others.

Phrase structure rules are usually of the form $A \rightarrow B C$,

Meaning "constituent A is separated into two sub-constituents B and C " or simply " A consists of B followed by C " .

Examples :

‡ $S \rightarrow NP VP$ Reads : S consists of an NP followed by a VP ;

means a sentence consists of a noun phrase followed by a verb phrase.

‡ NP → Det N1 Reads : NP consists of an Det followed by a N1 ;
means a noun phrase consists of a determiner followed by a noun.

Phrase Structure Rules and Trees for Noun Phrase (NP)

Noun Phrase (NP)

John N

the boy Det N

A little boy Det Adj N

A boy in a bubble Det N PP

Phrase Structure rules for NPs

NP → (Det) (Adj) N (PP)

Phrase Structure trees for NPs

NP NP NP NP

N Det N Det Adj N Det N PP

Jhon the boy a little boy a boy P NP

in Det N

a Bubble

19

fo

.in

rs

de

AI - NLP - Syntactic Processing

ea

2. Syntactic Processing

yr

.m

w

w

Syntactic Processing converts a flat input sentence into a hierarchical structure

,w

ty

or

that corresponds to the units of meaning in the sentence.

ab

kr

The Syntactic processing has two main components :

ha

C

C

□ one is called grammar, and

R

□ other is called parser.

‡ Grammar :

It is a declarative representation of syntactic facts about the language.

It is the specification of the legal structures of a language.

It has three basic components : terminal symbols, non-terminal symbols, and rules (productions) .

‡ Parser :

It is a procedure that compares the grammar against input sentences to produce a parsed structures called parse tree.

Example 1 : Sentence "Llama pickup ball".

Parse Tree Structure (PS)

Sentence

Noun Phrase Verb Phrase

Noun Verb Noun Phrase

Noun

Llama pick-up ball

20

fo

.in

rs

de

AI - NLP - Syntactic Processing

ea

2.1 Context Free Grammar (CFG)

yr

.m

w In formal language theory, a context free grammar is a grammar where

w

,w

ty

every production rules is of the form: $A \rightarrow \alpha$ where A is a single

or

ab

kr

symbol called non-terminal , and α is a string that is a sequence of

ha

C

symbols of terminals and/or non-terminals (possibly empty).

C

R

Note : The difference with an arbitrary grammars is that the left hand side of a production rule is always a single nonterminal symbol rather than a string of terminal and/or nonterminal symbols.

- Terminal , Non-Terminal and Start Symbols

The terminal and non-terminal symbols are those symbols that are used to construct production rules in a formal grammar.

‡ Terminal Symbol

Any symbol used in the grammar which does not appear on the left-hand-side of some rule (ie. has no definition) is called a terminal symbol. Terminal symbols cannot be broken down into smaller units without losing their literal meaning.

‡ Non-Terminal Symbol

Symbols that are defined by rules are called non-terminal symbol. Each production rule defines the non-terminal symbol. Like the above rule states that "whenever we see an A, we can replace it with α ".

‡ A non-terminal may have more than one definition, in that case we use symbol " $\sqcup$ " as the union operator;

Example 1: $A \rightarrow \alpha \sqcup \beta$ states that "whenever we see A, we can replace it with α or with β ".

Similarly, if a rule is $NP \rightarrow Det N \sqcup Prop$ then the vertical slash on the right side is a convention used to represent that the NP can be replaced either by Det N or by Prop. Thus, this is really two rules.

Example 2: $S \rightarrow NP VP$ states that the symbol S is replaced by the symbols NP and VP.

‡ One special non-terminal is called Start symbol, usually written S. The production rules for this symbol are usually written first in a grammar.

21

fo

.in

rs

de

AI – NLP - Syntactic Processing

ea

- How Grammar works ?

yr

.m

w Grammar starts with the start symbol, then successively applies the

w

,w

ty

production rules (replacing the L.H.S. with the R.H.S.) until reaches to a

or

ab

word which contains no non-terminals. This is known as a derivation.

kr

ha

C

C

R

‡ Anything which can be derived from the start symbol by applying the production rules is called a sentential form.

‡ Any grammar may have an infinite number of sentences;

The set of all such sentences is the language defined by that grammar.

‡ Example of grammar :

$S \rightarrow Xc \quad X \rightarrow YX \quad Y \rightarrow a \mid b$

The above grammar shows that it can derive all words which start arbitrarily and have many 'a's or 'b's and finish with a 'c'. This language is defined by the regular expression $(a \mid b)^* c$. The "*" indicates that the character immediately to its left may be repeated any number of times, including zero. Thus ab^*c would match "ac", "abc", "abbc", "abbbc", "abbbbbbbbc", and any string that starts with an "a", is followed by a sequence of "b"s, and ends with a "c".

‡ Regular Expression

Every regular expression can be converted to a grammar, but not

every grammar can be converted back to a regular expression;

Any grammar which can be converted back to a regular expression is called a regular grammar; the language it defines is a regular language.

Regular Expression $\rightarrow$ Grammar

Regular Expression $\leftarrow$ Regular Grammar

‡ Regular Grammars

A regular grammar is a grammar where all of the production rules are of one of the following forms:

$A \rightarrow aB$ or $A \rightarrow a$

where A and B represent any single non-terminal, and a represents any single terminal, or the empty string.

22

fo

.in

rs

de

AI – NLP - Syntactic Processing

ea

2.2 Parsar

yr

.m

w A parser is a program, that accepts as input a sequence of words

w

,w

ty

in a natural language and breaks them up into parts (nouns, verbs,

or

ab

kr

and their attributes), to be managed by other programming.

ha

C

□ Parsing can be defined as the act of analyzing the grammaticality

C

R

an utterance according to some specific grammar.

□ Parsing is the process to check, that a particular sequence of words

in a sentence correspond to a language defined by its grammar.

□ Parsing means show how we can get from the start symbol of the grammar to the sequence of words using the production rules.

□ The output of a parser is a Parse tree.

Parse Tree is a way of representing the output of a parser.

□ Each phrasal constituent found during parsing becomes a branch node of the parse tree;

□ the words of the sentence become the leaves of the parse tree;

□ there can be more than one parse tree for a single sentence;

23

fo

.in

rs

de

AI – NLP - Syntactic Processing

ea

- Parsing

yr

.m

w To parse a sentence, it is necessary to find a way in which the

w

,w

ty

sentence could have been generated from the start symbol. There two

or

ab

ways to do : One, Top-Down Parsing and the other, Bottom-UP Parsing.

kr

ha

C

C

R

■ Top-Down Parsing

Begin with the start symbol and apply the grammar rules forward until the symbols at the terminals of the tree corresponds to the components of the sentence being parsed.

■ Bottom-UP Parsing

Begin with the sentence to be parsed and apply the grammar rules backward until a single tree whose terminals are the words of the sentence and whose top node is the start symbol has been produced.

Note : The choice between these two approaches is similar to the choice between forward and backward reasoning in other problem solving tasks.

The most important consideration is the branching factors. Some times these two approaches are combined in to a single method called bottom-up parsing with top-down filtering.

24

fo

.in

rs

de

AI – NLP - Syntactic Processing

ea

- Modeling a Sentence using Phase Structure

yr

.m

Every sentence consists of an internal structure which could be

W

,W

ty

modeled with the phrase structure.

or

ab

kr

ha

Algorithm : Steps

C

C

R

‡ Apply rules on an proposition

‡ The base proposition would be :

S (the root, ie the sentence).

‡ The first production rule would be :

(NP = noun phrase, VP = verb phrase)

S -> (NP, VP)

‡ Apply rules for the 'branches'

NP -> noun VP -> verb, NP

‡ The verb and noun have terminal nodes which could be any word in the lexicon for the appropriate category.

‡ The end is a tree with the words as terminal nodes, which is referred as the sentence.

Example : Parse tree

- sentence "He ate the pizza",

- apply the grammar with rules

S -> NP VP, NP -> PRO, NP -> ART N, VP -> V NP,

- the lexicon structure is

("ate" V) ("he" PRO) ("pizza" N) ("the" ART)

- The parse tree is

PRO, V, ART, N - lexical non-terminals

S

S, NP, VP - phrasal non-terminal

NP VP He, ate, the, pizza - words or terminal

PRO V NP

ART N

He ate

the pizza

25

fo

.in

rs

de

AI - NLP - Semantics and Pragmatics

ea

3. Semantics and Pragmatics

yr

.m

w The semantics and pragmatics, are the two stages of analysis concerned

w

,w

ty

with getting at the meaning of a sentence.

or

ab

□ In the first stage (semantics) a partial representation of the meaning

kr

ha

C

is obtained based on the possible syntactic structure(s) of the sentence

C

R

and the meanings of the words in that sentence.

□ In the second stage (pragmatic), the meaning is elaborated based on :
the contextual and the world knowledge.

For the difference between these stages, consider the sentence:

"He asked for the boss".

From knowledge of the meaning of the words and the structure of
the sentence we can work out that :

- Someone (who is male) asked for someone who is a boss.
- We can't say who these people are and why the first guy wanted the second.
- If we know something about the context (including the last few sentences spoken/written) we may be able to work these things out.
- Maybe the last sentence was "Fred had just been sacked."
- From our general knowledge that bosses generally sack people :
if people want to speak to people who sack them it is generally to complain about it.

- We could then really start to get at the meaning of the sentence :

"Fred wants to complain to his boss about getting sacked".

26

fo

.in

rs

de

AI - NLP - References

ea

4. References : Textbooks

yr

.m

w

w

1. "Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill

,w

ty

companies Inc., Chapter 15, page 377-426.

or

ab

kr

2. "Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig,

ha

C

(2002), Prentice Hall, Chapter 23, page 834-861.

C

R

3. "Artificial Intelligence: Structures and Strategies for Complex Problem Solving", by George F. Luger, (2002), Addison-Wesley, Chapter 15, page 619-632.

4. "Artificial Intelligence: Theory and Practice", by Thomas Dean, (1994), Addison-Wesley, Chapter 10, Page 489-538.

5. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

27

fo

.in

rs

de

ea

yr

Common Sense : AI Course Lecture 42, notes, slides

.m

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

www.myreaders.info

kr

ha

C

C

R

Common Sense

Artificial Intelligence

Common sense, topics : Introduction, Common sense knowledge and reasoning, how to teach commonsense to a computer;

Formalization of common sense reasoning - initial attempts of late 60's and early, renewed attempts in late 70's and 80's to recent time; Physical world - modeling the qualitative world, reasoning with qualitative information; Common Sense

Ontologies - time, space, material; Memory organization - short term memory (STM), long term memory (LTM).

fo

.in

rs

de

ea

Common Sense

yr

.m

w

w

,w

Artificial Intelligence

ty

or

ab

kr

Topics

ha

C

C

(Lecture 42 , 1 hours) Slides

R

1. Introduction 03-10

Common sense knowledge and reasoning, How to teach commonsense to a computer.

2. Formalization of Common Sense Reasoning 11-13

Initial attempts of late 60's and early, Renewed attempts in late 70's and 80's to recent time.

3. Physical World 18-17

Modeling the qualitative world, Reasoning with qualitative information.

4. Common Sense Ontologies 22-20

Time, Space, Material.

5. Memory Organization 25

Short term memory (STM), Long term memory (LTM).

6. References 26

02

fo

.in

rs

de

ea

yr

.m

w

w

,w

ty

Common Sense

or

ab

kr

ha

C

C

R

What is Common Sense ?

- Common sense is the mental skills that most people share.
- Common Sense is ability to analyze a situation based on its context, using millions of integrated pieces of common knowledge.
- John McCarthy was the first to talk about commonsense reasoning in his paper in 1959, explains that a program has commonsense if it automatically deduces for itself sufficiently wide class of immediate consequences of any thing it is told and what it already knows.
- Common sense is what people come to know in the process of growing and living in the world (R.Elio, 2002).
- Common sense knowledge includes the basic facts about events and their effects, facts about knowledge and how it is obtained, facts about beliefs and desires. It includes the basic facts about material objects and their properties (John McCarthy, 1990).
- Currently, computers lack common sense .

03

fo

.in

rs

de

AI - Common Sense: Introduction

ea

1. Introduction

yr

.m

w

w

Commonsense is ability to analyze a situation based on its context,

,w

ty

or

using millions of integrated pieces of common knowledge. Ability to use

ab

kr

common sense knowledge depends on being able to do commonsense reasoning.

ha

C

C

Commonsense Reasoning is a central part of intelligent behavior.

R

Formalizing the commonsense knowledge for even simple reasoning problem is a huge task. The reason is that, the most commonsense knowledge is implicit in contrast to expert/specialist knowledge, which is usually explicit. Therefore making commonsense reasoning system is making this knowledge explicit.

Example : Everyone knows that dropping a glass of water, the glass will break and water will spill on podium. However, this information is not obtained by formula or equation for a falling body or equations governing fluid flow.

The goal of the formal commonsense reasoning community is to encode this implicit knowledge using formal logic.

Computers and ordinary real life - issues ?

Computers do many remarkable things. Computer programs can play chess at the level of best players. But no computer program match the capabilities of a three year old child at recognizing objects or can draw simple conclusions about ordinary life. Building machines that can think the way any average person can is a distant reality.

Why computers can not think about the world as any person can ?

Where the problem lies ?

There are two basic types of knowledge. One is the specialist's knowledge which mathematicians, scientists and engineers possess. The other type is the commonsense knowledge which every one has, even a small 6-year-old child. The need is to teach the computer to reason about the world (commonsense knowledge). The researchers have not yet reached to any consensus on many related issues. McCarthy suggested to use logic to represent the knowledge. Understanding common sense capability is an active area of research in artificial intelligence.

04

fo

.in

rs

de

AI - Common Sense: Introduction

ea

1.1 Commonsense Knowledge and Reasoning

yr

.m

w

w

Common sense facts and methods are very little understood today.

,w

ty

or

Extending this understanding is the key problem the AI researchers are

ab

kr

facing. John McCarthy (1984) identified as common sense as :

ha

C

C

Common sense knowledge - what every one knows.

R

Common sense reasoning - ability to use common sense knowledge.

- Common Sense Knowledge

What one can express as a fact using a richer ontology.

Examples

‡ Every person is younger than the person's mother

‡ People do not like being repeatedly interrupted

‡ If you hold a knife by its blade then the blade may cut you

‡ If you drop paper into a flame then the paper will burn

‡ You start getting hungry again a few hours after eating a meal

‡ People go to parties to meet new people

‡ People generally sleep at night

Here the problem is , how to give computers these millions of ordinary pieces of knowledge that every person learns by adulthood.

05

fo

.in

rs

de

AI - Common Sense: Introduction

ea

- Common Sense Reasoning

yr

.m

w What one builds as a reasoning method into his program.

w

,w

ty

Examples

or

ab

‡ If you have a problem, think of a past situation where you solved

kr

ha

C

a similar problem.

C

R

‡ If you take an action, anticipate what might happen next

‡ If you fail at something, imagine how you might have done things differently .

‡ If you observe an event, try to infer what prior event might have caused it.

‡ If you see an object, wonder if anyone owns it

‡ If someone does something, ask yourself what the person's purpose was in doing that.

Here the problem is, how to give computers the capacity for commonsense reasoning, the ways to use the commonsense knowledge to solve the various problems we encounter every day.

06

fo

.in

rs

de

AI - Common Sense: Introduction

ea

1.2 How to Teach Commonsense to a Computer

yr

.m

w

w

There is no clear answer for to this question.

,w

ty

or

Presently, there is no program that can match the common sense

ab

kr

reasoning powers of a 5 year old child. The problem was noticed long

ha

C

C

ago by John McCarthy. We do not yet have enough ideas about how

R

to represent, organize, and use much of commonsense knowledge, let alone build a machine that could learn automatically on its own".

□ Some believe that, prior understanding is not necessary to build a machine, and intelligence can be made to emerge from some generic learning.

□ Others feel that, unless we can acquire some experience in manually engineering systems with common sense, we will not be able to build learning machines that can automatically learn common sense.

07

fo

.in

rs

de

AI - Common Sense: Introduction

ea

- Building Human Commonsense Knowledge Base

yr

.m

w

w

Stated below, the two ongoing AI projects for assembling

,w

ty

or

comprehensive ontology and knowledge base of everyday common

ab

kr

sense knowledge with the goal of enabling AI applications to

ha

C

C

perform human-like reasoning.

R

Project CYC : A commonsense knowledge base, building since two decades, has collected 1.5 million pieces of commonsense knowledge, still far away from several hundred million required; the project faces a challenge because such a large database cannot be engineered by any one group.

Project Open Mind : A common sense knowledge base, building since 1999, has accumulated more than 700,000 facts from over 15,000 contributors; the knowledge collected by has enabled many research projects at MIT and elsewhere.

[Continued in next slide]

08

fo

.in

rs

de

AI - Common Sense: Introduction

ea

[Continued from previous slide]

yr

.m

w

w The CYC and Open Mind projects are ensuring enough commonsense

,w

knowledge so as to work in a given environment enabling AI

ty

or

ab

applications to perform human-like reasoning.

kr

ha

C

Next comes the Common Sense Reasoning :

C

R

It is what one builds as a reasoning method into his program, a very complex task. We want computer to do reasoning as human does.

Human does reasoning in different ways and the one which is

Logic Reasoning (deductive, inductive, abductive), is of main concern in AI reasoning system. The logic reasoning can accomplish the task of common sense reasoning. For instance :

- ‡ Predicate logic can represent knowledge about objects, facts, rules,
- ‡ Frames can describe everyday objects
- ‡ Scripts can describe typical sequences of events
- ‡ Non-monotonic logics can support default reasoning,

09

fo

.in

rs

de

AI - Common Sense: Introduction

ea

- Example of Commonsense System Architecture (Mueller, 2004)

yr

.m

w The system takes as input a template produced by information

w

,w

ty

extraction system about certain aspects of a scenario.

or

ab

kr

ha

Template

C

C

R

Script classifier

Template, script

Reasoning problem

builder for script

Reasoning problem

Common

Commonsense

sense

reasoner

KB

Model

Commonsense System Architecture

‡ The template is a frame with slots and slots fillers

‡ The template is fed to a script classifier, which classifies what script is active in the template.

‡ The template and the script are passed to a reasoning problem builder specific to the script, which converts the template into a commonsense reasoning problem.

‡ The problem and a commonsense knowledge base are passed to a commonsense reasoner. It infers and fills in missing details to produce a model of the input text.

‡ The model provides a deeper representation of the input, than is provided by the template alone.

10

fo

.in

rs

de

AI - Common Sense: Formalization of reasoning

ea

2. Formalization of Common Sense Reasoning

yr

.m

w

w

Commonsense reasoning is a central part of human behavior; no real

,w

ty

or

intelligence is possible without it. The ultimate goal of artificially intelligent

ab

kr

systems is that they exhibit commonsense behavior.

ha

C

C

For the computers, the commonsense reasoning is not an easy task,

R

indeed a very complex task, we all perform about every day world.

Example : There are chess-playing programs that beat champions, and there are expert systems that assist in clinical diagnosis, but there is no program that reason about how far one must bend over to put on one's socks.

The reason is expert knowledge is usually explicit, but most commonsense knowledge is implicit. Therefore, one of the prerequisites for developing commonsense reasoning systems is making this knowledge explicit.

John McCarthy (1990) identified commonsense reasoning as human ability to use common sense knowledge.

Mueller (2006) defines commonsense reasoning as a process, taking information about certain aspects of a scenario in the world and making inference about other aspects of the scenario based on our commonsense knowledge or knowledge about how the world works.

[Continued in next slide]

11

fo

.in

rs

de

AI - Common Sense: Formalization of reasoning

ea

[Continued from previous slide]

yr

.m

w To formalize commonsense reasoning, we need to construct

w

,w

ty

representations for commonsense knowledge and inference algorithms

or

ab

kr

to manipulate that knowledge. McCarthy in 1959 was first to put

ha

C

forward the idea of using a formal logic as the representation language

C

R

for a commonsense reasoning system, with the reasoning done by deductive inference.

Robert C. Moore in his article, "Automatic Deduction for Commonsense Reasoning: An Overview", explained the issues involved in drawing conclusions by means of deductive inference from bodies of commonsense knowledge represented by logical formulas. This article contains first a review of initial attempts of late 60's and early 70's – failures and disappointments, and then the renewed attempts in late 70's and 80's to recent time - how domain-specific control information can offer a solution to the difficulties, the relationship of automatic deduction to the new field of "logic programming" and issues that arise while extending automatic-deduction techniques to nonstandard logic.

Note : Just to complete this section, the issues, arguments and the solutions offered his article (Robert C. Moore, technical note 239, april 1981, Sri International Menlo park CA Artificial intelligence center) are put very briefly in next three slides.

12

fo

.in

rs

de

AI - Common Sense: Formalization of reasoning

ea

2.1 Initial attempts of late 60's and early 70's

yr

.m

w failures and disappointments

w

,w

ty

Many researchers, (Black, Robinson, Green and others) made serious

or

ab

attempt to implement McCarthy's idea, but faced difficulties because :

kr

ha

C

□ search space generated by the resolution method was

C

R

growing exponentially with the number of formulas used to describe a problem; the problems of moderate complexity could not be solved in reasonable time.

□ several domain-independent heuristics proposed to deal with this search space issue, proved too weak to produce satisfactory results.

The failures resulted from two constraints the researchers had imposed:

(a) attempted to use only uniform, domain-independent, proof procedures; and (b) tried to force all reasoning and problem-solving behavior into the framework of logical deduction.

There was widespread condemnation of any use of logic or deduction in commonsense reasoning or problem solving.

However, the interest in deduction-based approaches to commonsense reasoning did not go away, rather revived in late 70's.

13

fo

.in

rs

de

AI - Common Sense: Formalization of reasoning

ea

2.2 Renewed attempts in late 70's and 80's to recent time

yr

.m

w

w

The revival of interest in deduction-based approaches to commonsense

,w

ty

or

reasoning, is noticed since late 70's, from the work of many

ab

kr

researchers (McDermott, Doyle, Moore, Bobrow and others), because,

ha

C

C

the recognition of some important class of problems resist solution by

R

any other method. The understanding came from following issues :

- Representation Formalism based on Logic

■ If one decides to use a representation formalism based on logic, it may not be necessary to use general deductive methods to manipulate expressions in the formalism. If the description (object, properties, relations) of a problem situation is complete then we can answer any question by evaluation; deduction is unnecessary.

■ Representation formalism based on logic gives us the ability to express many kind of generalization, even when we do not have a complete description of the problem situation. Using deduction to manipulate expression in the representation formalism allows us to ask logically complex queries of a knowledge base containing such generalization, even when we cannot "evaluate" a query directly.

■ AI inference systems, not based on automatic deduction technique, but has a knowledge representation formalism that is capable of handling the kinds of incomplete information, that people can understand, must at least be able to say that something has a certain property without saying which thing has that property.

Thus, any representation formalism that has these capabilities will be an extension of classical first-order logic, and any inference system that can deal adequately with these kinds of generalization will have to have at least the capabilities of an automatic-deduction system.

14

fo

.in

rs

de

AI - Common Sense: Formalization of reasoning

ea

- Need for Specific Control Information

yr

.m

w

w

■ The difficulties with domain-independent problem solver on

,w

ty

or

automatic-deduction techniques is that many possible inferences

ab

kr

can be drawn at any one time. Finding relevant inferences to a

ha

C

C

particular problem can be impossible, unless domain-specific guidance

R

is supplied to control the deductive process.

■ In search processes, information about whether to use facts in a forward-chaining or back-chaining manner for efficient system performance. The deductive process can be bidirectional, partly working forward from facts to new one, partly working backward from goals to sub-goals, and meeting somewhere in between. Early theorem-proving systems used every facts both ways, leading to highly redundant searches. More sophisticated methods can eliminated these redundancies.

15

fo

.in

rs

de

AI - Common Sense: Formalization of reasoning

ea

- Logic Programming

yr

.m

w

w

■ One factor that can greatly affect the efficiency of deductive

,w

ty

or

reasoning is the way in which a body of knowledge is formalized.

ab

kr

Logically equivalent formalization can have radically different

ha

C

C

behavior when used with standard deduction techniques. This led
R

to the development of Logic programming and the creation of
a new languages such as Prolog.

■ Prolog combines the use of logic as a representation language with efficient deduction technique, based on backward inference process (goal directed) which allows to consider a set of formulas as program. Prolog is now most widely used logic programming language. Originally logic programming was conceived as a subset of classical logic, it was soon extended with some non-classical features, in particular negation as failure. Prolog tries to prove p ; if p can not be proved, then the goal $\neg p$ succeeds, and vice versa. This simple feature of Prolog has been used to achieve non-monotonic behavior.

- Automatic deduction in nonstandard logics

■ The classical first-order logic is the most general logic for which automatic-deduction techniques are well developed. However, many commonsense concepts, are treated in nonstandard, either higher-order or non-classical logics. This presents a problem and require reformulating representation in nonstandard logics in terms of logically equivalent representations in classical first-order logic. One type of nonstandard logic that has received much attention is non-monotonic logic.

16

fo

.in

rs

de

AI - Common Sense: Formalization of reasoning

ea

[Continued from previous slide]

yr

.m

w

w

Note : The approaches to the representation formalism for

,w

ty

commonsense reasoning are mentioned in the previous few slides.

or

ab

kr

However, in our previous lectures on "Knowledge Representation"

ha

C

C

and "Reasoning System", the slides illustrated with example each of

R

these approaches. The following were covered :

- Logic as a KR Language for Reasoning - a formal system in which the formulas or sentences have true or false values.
- Propositional Logic (PL) - a declarative sentence either TRUE or FALSE.
- Predicate Logic Quantifiers - to make a statement about a collection of objects and to state that an object does exist without naming it.
- Resolution - a procedure, produces proofs by refutation or contradiction.
- KR Using Rules - production rules, semantic net and frames; forward and backward reasoning - ways to generate new knowledge.
- Logic Programming - a formalism for specifying a computation in terms of logical relations, Prolog program,
- Non-monotonic logic - where the truth of a proposition may change when new information (axioms) are added.

17

fo

.in

rs

de

AI - Common Sense: Physical world

ea

3. Physical World

yr

.m

w

w

People know a great deal about how the physical world works.

,w

ty

or

Most people, have no notion of the "laws of physics" that govern

ab

kr

ha

this world, yet they

C

C

R

- can predict that a falling ball will bounce many times before come to halt.

- can predict the projection of cricket ball and even catch it.

- know a pendulum swings back and fore finally coming to rest in the middle.

How can we build a computer program to do such reasoning ?

One answer is to program the equations governing the physical motion of the objects. But most people do not know these equations and also do not have exact numerical measures, yet they can predict what will happen in physical situations. This means people seem to reason more abstractly than the equations would. Here comes qualitative physics, to understand how to build and reason with abstract, number less representation.

Researchers are therefore motivated towards :

□ Modeling the qualitative World and

□ Reasoning with qualitative information

18

fo

.in

rs

de

AI - Common Sense: Physical world

ea

• Modeling the Qualitative World

yr

.m

w Qualitative physics seeks to understand physical processes by building

w

,w

ty

models that may have following entities:

or

ab

kr

ha

Variables A restricted set of values,

C

C

R

e.g. temperature as { frozen, between, boiling }.

Quantity Spaces A small set of discrete values.

Rate of Change Values at different times, modeled qualitatively,

e.g. { decreasing, steady, increasing }.

Expressions Combination of variables.

Equations Assignment of expression to variables.

States Sets of variables, whose values change over time.

19

fo

.in

rs

de

AI - Common Sense: Physical world

ea

- Example : Qualitative Algebra - addition

yr

.m

w

w

Describe the volume of glass as {empty, between, full } .

,w

ty

or

When two qualitative values are added together then :

ab

kr

empty + empty = empty

ha

C

C

R

empty + between = between

empty + full = full

between + between = { between, full, overflow }

between + full = { between, over flow }

full + full = { full, over flow }

20

fo

.in

rs

de

AI - Common Sense: Physical world

ea

- Reasoning with Qualitative Information

yr

.m

w

w

Reasoning with qualitative information is often called qualitative

,w

ty

or

simulation. The basic idea is :

ab

kr

ha

C

‡ Construct a sequence of discrete episodes that occur as qualitative

C

R

variable.

‡ States are linked by qualitative rules that may be general.

‡ Rules may be applied to many objects simultaneously as they may all influence each other.

‡ Ambiguity may arise so split outcomes into different paths.

‡ A network of all possible states and transitions for a qualitative system is called an envisionment (mental images). There are often many paths through an envisionment. Each path is called history.

‡ Programs must know how to represent the behavior of many kinds of processes, materials and the world in which they act.

21

fo

.in

rs

de

AI - Common Sense: Ontologies

ea

4. Common Sense Ontologies

yr

.m

w

w

Some concepts are fundamental to common sense reasoning.

,w

ty

or

A computer program that interacts with the real world must be able

ab

kr

to reason about things like time, space and materials. On each of these,

ha

C

C

here some thought is presented. [Details with examples are available in the text

R

book - "Artificial Intelligence", by Elaine Rich and Kevin Knight]

- Time

The most basic notion of time is events. Events occur during intervals over continuous spaces of time. An interval has a start and end point and a duration between them.

Intervals can be related to one another as :

is-before, is-after, meets, is-met-by, starts, is-started-by, during, contains, ends, is-ended-by and equals.

We can build a axioms with intervals to describe events in time.

22

fo

.in

rs

de

AI - Common Sense: Ontologies

ea

- Space

yr

.m

w The Blocks World is a simple example of what we can model and

w

,w

ty

describe space. However common sense notions such as :

or

ab

place object x near object y

kr

ha

C

are not accommodated.

C

R

Objects have a spatial extent while events have a temporal extent.

We may try to extend of common sense theory of time. But, space

is 3D and has many more relationships than those for time so it is

not a good idea.

Another approach is view objects and space at various levels of abstraction. For example, we can view most printed circuit boards as being a 2D object.

Choosing a representation means selecting relevant properties at particular levels of granularity. For instance we can define relations over spaces such as inside, adjacent etc. We can also define relations for curves, lines, surfaces, planes and volumes. e.g. along, across, perpendicular etc.

23

fo

.in

rs

de

AI - Common Sense: Ontologies

ea

- Material

yr

.m

w Describe the properties of materials as :

w

,w

ty

‡ You cannot walk on water.

or

ab

kr

‡ If you knock a cup of coffee over what happens?

ha

C

C

R

‡ If you pour a full kettle into a cup what happens?

‡ You can squeeze a sponge but not a brick.

The Liquids provide many interesting points, such as, the space occupied by them. Thus we can define their properties such as:

‡ Capacity - a bound to an amount of liquid.

‡ Amount - volume occupied by a liquid.

‡ Full - if amount equals capacity.

Other properties that materials can posses include:

‡ Free - if a space is not wholly contained inside another object.

‡ Surround - if enclosed by a very thin free space.

‡ Rigid

‡ Flexible

‡ Particulate - e.g. sand

24

fo

.in

rs

de

AI - Common Sense: Memory

ea

5. Memory Organization

yr

.m

w

w

Memory is central to common sense behavior and also the basis for

,w

ty

or

learning. Human memory is still not fully understood however

ab

kr

psychologists have proposed several ideas.

ha

C

C

R

■ Short term memory (STM) :

Only a few items at a time can be held here;

perceptual information are stored directly here.

■ Long term memory (LTM) :

Capacity for storage is very large and fairly permanent;

LTM is often divided further as :

‡ Episodic memory :

Contains information about personal experiences.

‡ Semantic memory :

General facts with no personal meaning, e.g. Birds fly;

useful in natural language understanding.

25

fo

.in

rs

de

ea

yr

.m

AI - Common Sense - References

w

w

6. References : Textbooks

,w

ty

or

1. "Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill

ab

kr

companies Inc., Chapter 19, page 529-545.

ha

C

C

2. "AI: A New Synthesis", by Nils J. Nilsson, (1998), Morgan Kaufmann Inc., Chapter

R

18, Page 301-314.

3. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

26

fo

.in

rs

de

ea

yr

Fundamentals of Genetic Algorithms : AI Course Lecture 39 – 40, notes, slides

.m

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

www.myreaders.info

kr

ha

C

C

R

Fundamentals of Genetic Algorithms

Artificial Intelligence

Genetic algorithms, topics : Introduction, search optimization

algorithm; Evolutionary algorithm (EAs); Genetic Algorithms (GAs) :

biological background, search space, working principles, basic genetic

algorithm, flow chart for Genetic programming; Encoding : binary

encoding, value encoding, permutation encoding, and tree encoding; Operators of genetic algorithm : reproduction or selection - roulette wheel selection, Boltzmann selection; fitness function; Crossover – one point crossover, two Point crossover, uniform crossover, arithmetic, heuristic; Mutation - flip bit, boundary, non-uniform, uniform, Gaussian; Basic genetic algorithm - solved

2

examples : maximize function $f(x) = x$ and two bar pendulum.

fo

.in

rs

de

ea

Fundamentals of Genetic Algorithms

yr

.m

w

w

,w

Artificial Intelligence

ty

or

ab

kr

ha

Topics

C

C

(Lectures 39, 40 2 hours)

R

Slides

1. Introduction 03-15

Why genetic algorithms, Optimization, Search optimization algorithm; Evolutionary algorithm (EAs); Genetic Algorithms (GAs) : Biological background, Search space, Working principles, Basic genetic algorithm, Flow chart for Genetic programming.

2. Encoding 16-21

Binary Encoding, Value Encoding, Permutation Encoding, and Tree Encoding.

3. Operators of Genetic Algorithm 22-35

Reproduction or selection : Roulette wheel selection, Boltzmann selection; fitness function; Crossover: one-Point crossover, two-Point crossover, uniform crossover, arithmetic, heuristic; Mutation : flip bit, boundary, non-uniform, uniform, Gaussian.

4. Basic Genetic Algorithm 36-41

Solved examples : maximize function $f(x) = x^2$ and two bar pendulum.

5. References 42

02

fo

.in

rs

de

ea

yr

Fundamentals of Genetic Algorithms

.m

w

w

,w

ty

What are GAs ?

or

ab

kr

ha

- Genetic Algorithms (GAs) are adaptive heuristic search algorithm based

C

C

R

on the evolutionary ideas of natural selection and genetics.

- Genetic algorithms (GAs) are a part of Evolutionary computing, a rapidly growing area of artificial intelligence. GAs are inspired by Darwin's theory about evolution - "survival of the fittest".
- GAs represent an intelligent exploitation of a random search used to solve optimization problems.
- GAs, although randomized, exploit historical information to direct the search into the region of better performance within the search space.
- In nature, competition among individuals for scanty resources results

in the fittest individuals dominating over the weaker ones.

03

fo

.in

rs

de

GA - Introduction

ea

1. Introduction to Genetic Algorithms

yr

.m

w

w

Solving problems mean looking for solutions, which is best among

,w

ty

or

others.

ab

kr

ha

Finding the solution to a problem is often thought :

C

C

R

– In computer science and AI, as a process of search through the space of possible solutions. The set of possible solutions defines the search space (also called state space) for a given problem. Solutions or partial solutions are viewed as points in the search space.

– In engineering and mathematics, as a process of optimization. The problems are first formulated as mathematical models expressed in terms of functions and then to find a solution, discover the parameters that optimize the model or the function components that provide optimal system performance.

04

fo

.in

rs

de

SC - GA - Introduction

ea

- Why Genetic Algorithms ?

yr

.m

w

w

- It is better than conventional AI ; It is more robust.

,w

ty

or

ab

- Unlike older AI systems, the GA's do not break easily even if the

kr

ha

inputs changed slightly, or in the presence of reasonable noise.

C

C

R

- While performing search in large state-space, or multi-modal state-space, or n-dimensional surface, a genetic algorithms offer significant benefits over many other typical search optimization techniques like - linear programming, heuristic, depth-first, breath-first.

"Genetic Algorithms are good at taking large, potentially huge search spaces and navigating them, looking for optimal combinations of things, the solutions one might not otherwise find in a lifetime."

(Salvatore Mangano Computer Design, May 1995)

05

fo

.in

rs

de

SC - GA - Introduction

ea

1.1 Optimization

yr

.m

w

w

Optimization is a process that finds a best, or optimal, solution for a

,w

ty

or

problem. The Optimization problems are centered around three factors :

ab

kr

ha

C

1. An objective function which is to be minimized or maximized;

C

R

Examples

‡ In manufacturing, we want to maximize the profit or minimize the cost .

‡ In designing an automobile panel, we want to maximize the strength.

2. A set of unknowns or variables that affect the objective function,

Examples

‡ In manufacturing, the variables are amount of resources used or the time spent.

‡ In panel design problem, the variables are shape and dimensions of the panel.

3. A set of constraints that allow the unknowns to take on certain values but exclude others;

Examples

‡ In manufacturing, one constrain is, that all "time" variables to be non-negative.

‡ In the panel design, we want to limit the weight and put constrain on its shape.

An optimization problem is defined as : Finding values of the variables that minimize or maximize the objective function while satisfying the constraints.

06

fo

.in

rs

de

SC – GA – Introduction

ea

- Search Optimization Algorithms

yr

.m

w

w

Fig. below shows different types of Search Optimization algorithms.

,w

ty

or

ab

Search

kr

ha

Optimization

C

C

R

Calculus Guided Random Search Enumerative

Based techniques Techniques

Techniques

Indirect Direct

method method Uninformed Informed

Search Search

Newton Finonacci

Tabu Hill Simulated Evolutionary

Search Climbing Annealing Algorithms

Genetic Genetic

Programming Algorithms

Fig. Taxonomy of Search Optimization techniques

We are interested in evolutionary search algorithms.

Our main concern is to understand the evolutionary algorithms :

- how to describe the process of search,
- how to implement and carry out search,
- what are the elements required to carry out search, and
- the different search strategies

The Evolutionary Algorithms include :

- Genetic Algorithms and
- Genetic Programming

07

fo

.in

rs

de

SC – GA - Introduction

ea

1.3 Evolutionary Algorithm (EAs)

yr

.m

w

w

Evolutionary Algorithm (EA) is a subset of Evolutionary Computation (EC)

,w

ty

or

which is a subfield of Artificial Intelligence (AI).

ab

kr

ha

C

Evolutionary Computation (EC) is a general term for several

C

R

computational techniques. Evolutionary Computation represents powerful search and optimization paradigm influenced by biological mechanisms of evolution : that of natural selection and genetic.

Evolutionary Algorithms (EAs) refers to Evolutionary Computational models using randomness and genetic inspired operations. EAs involve selection, recombination, random variation and competition of the individuals in a population of adequately represented potential solutions. The candidate solutions are referred as chromosomes or individuals.

Genetic Algorithms (GAs) represent the main paradigm of Evolutionary Computation.

- GAs simulate natural evolution, mimicking processes the nature uses :

Selection, Crosses over, Mutation and Accepting.

- GAs simulate the survival of the fittest among individuals over consecutive generation for solving a problem.

Development History

$EC = GP + ES + EP + GA$

Evolutionary Genetic Evolutionary Genetic

Computing Programming Strategies Programming Algorithms

Rechenberg Koza Rechenberg Fogel Holland

1960 1992 1965 1962 1970

08

fo

.in

rs

de

SC - GA - Introduction

ea

1.4 Genetic Algorithms (GAs) - Basic Concepts

yr

.m

w

w

Genetic algorithms (GAs) are the main paradigm of evolutionary

,w

ty

or

computing. GAs are inspired by Darwin's theory about evolution - the

ab

kr

"survival of the fittest". In nature, competition among individuals for

ha

C

C

scanty resources results in the fittest individuals dominating over the

R

weaker ones.

– GAs are the ways of solving problems by mimicking processes nature uses; ie., Selection, Crosses over, Mutation and Accepting, to evolve a solution to a problem.

– GAs are adaptive heuristic search based on the evolutionary ideas of natural selection and genetics.

– GAs are intelligent exploitation of random search used in optimization problems.

– GAs, although randomized, exploit historical information to direct the

search into the region of better performance within the search space.

The biological background (basic genetics), the scheme of evolutionary processes, the working principles and the steps involved in GAs are illustrated in next few slides.

09

fo

.in

rs

de

SC – GA - Introduction

ea

- Biological Background – Basic Genetics

yr

.m

w

w

‡ Every organism has a set of rules, describing how that organism

,w

ty

is built. All living organisms consist of cells.

or

ab

kr

ha

‡ In each cell there is same set of chromosomes. Chromosomes are

C

C

R

strings of DNA and serve as a model for the whole organism.

‡ A chromosome consists of genes, blocks of DNA.

‡ Each gene encodes a particular protein that represents a trait (feature), e.g., color of eyes.

‡ Possible settings for a trait (e.g. blue, brown) are called alleles.

‡ Each gene has its own position in the chromosome called its locus.

‡ Complete set of genetic material (all chromosomes) is called a genome.

‡ Particular set of genes in a genome is called genotype.

‡ The physical expression of the genotype (the organism itself after

birth) is called the phenotype, its physical and mental characteristics, such as eye color, intelligence etc.

‡ When two organisms mate they share their genes; the resultant offspring may end up having half the genes from one parent and half from the other. This process is called recombination (cross over) .

‡ The new created offspring can then be mutated. Mutation means, that the elements of DNA are a bit changed. This changes are mainly caused by errors in copying genes from parents.

‡ The fitness of an organism is measured by success of the organism in its life (survival).

10

fo

.in

rs

de

SC – GA - Introduction

ea

[continued from previous slide - Biological background]

yr

.m

w Below shown, the general scheme of evolutionary process in genetic

w

,w

ty

along with pseudo-code.

or

ab

kr

ha

Parents

C

C

Parents

R

Initialization

Recombination

Population

Mutation

Termination

Offspring

Survivor

Fig. General Scheme of Evolutionary process

Pseudo-Code

BEGIN

INITIALISE population with random candidate solution.

EVALUATE each candidate;

REPEAT UNTIL (termination condition) is satisfied DO

1. SELECT parents;

2. RECOMBINE pairs of parents;

3. MUTATE the resulting offspring;

4. SELECT individuals or the next generation;

END.

11

fo

.in

rs

de

SC – GA - Introduction

ea

- Search Space

yr

.m

w

w

In solving problems, some solution will be the best among others.

,w

ty

or

The space of all feasible solutions (among which the desired solution

ab

kr

resides) is called search space (also called state space).

ha

C

C

– Each point in the search space represents one possible solution.

R

– Each possible solution can be "marked" by its value (or fitness) for the problem.

- The GA looks for the best solution among a number of possible solutions represented by one point in the search space.
- Looking for a solution is then equal to looking for some extreme value (minimum or maximum) in the search space.
- At times the search space may be well defined, but usually only a few points in the search space are known.

In using GA, the process of finding solutions generates other points (possible solutions) as evolution proceeds.

12

fo

.in

rs

de

SC – GA - Introduction

ea

• Working Principles

yr

.m

w

w

Before getting into GAs, it is necessary to explain few terms.

,w

ty

or

ab

- Chromosome : a set of genes; a chromosome contains the solution in

kr

ha

form of genes.

C

C

R

- Gene : a part of chromosome; a gene contains a part of solution. It determines the solution. e.g. 16743 is a chromosome and 1, 6, 7, 4 and 3 are its genes.

- Individual : same as chromosome.

- Population: number of individuals present with same length of chromosome.

- Fitness : the value assigned to an individual based on how far or

close a individual is from the solution; greater the fitness value better the solution it contains.

- Fitness function : a function that assigns fitness value to the individual.

It is problem specific.

- Breeding : taking two fit individuals and then intermingling there chromosome to create new two individuals.

- Mutation : changing a random gene in an individual.

- Selection : selecting individuals for creating the next generation.

Working principles :

Genetic algorithm begins with a set of solutions (represented by chromosomes) called the population.

- Solutions from one population are taken and used to form a new population. This is motivated by the possibility that the new population will be better than the old one.

- Solutions are selected according to their fitness to form new solutions (offspring); more suitable they are, more chances they have to reproduce.

- This is repeated until some condition (e.g. number of populations or improvement of the best solution) is satisfied.

13

fo

.in

rs

de

SC - GA - Introduction

ea

- Outline of the Basic Genetic Algorithm

yr

.m

w 1. [Start] Generate random population of n chromosomes (i.e. suitable

w

,w

ty

solutions for the problem).

or

ab

kr

2. [Fitness] Evaluate the fitness $f(x)$ of each chromosome x in the

ha

C

population.

C

R

3. [New population] Create a new population by repeating following steps until the new population is complete.

(a) [Selection] Select two parent chromosomes from a population according to their fitness (better the fitness, bigger the chance to be selected)

(b) [Crossover] With a crossover probability, cross over the parents to form new offspring (children). If no crossover was performed, offspring is the exact copy of parents.

(c) [Mutation] With a mutation probability, mutate new offspring at each locus (position in chromosome).

(d) [Accepting] Place new offspring in the new population

4. [Replace] Use new generated population for a further run of the algorithm

5. [Test] If the end condition is satisfied, stop, and return the best solution in current population

6. [Loop] Go to step 2

Note : The genetic algorithm's performance is largely influenced by two operators called crossover and mutation. These two operators are the most important parts of GA.

14

fo

.in

rs

de

SC – GA – Introduction

ea

- Flow Chart for Genetic Programming

yr

.m

w

w

,w

Start

ty

or

ab

kr
 ha
 Seed Population
 C
 Generate N individuals Genesis
 C
 R
 Scoring : assign fitness
 to each individual
 Natural Select two individuals
 Selection (Parent 1 Parent 2)
 No
 Reproduction Use crossover operator
 Crossover
 Recombination to produce off- springs
 Scoring : assign fitness Crossover
 to off- springs Finished?
 Yes
 Survival of Fittest
 Natural Select one off-spring
 Apply replacement Yes No
 operator to incorporate Selection
 new individual into
 population
 Apply Mutation operator
 Mutation to produce Mutated
 No offspring
 Terminate?
 Mutation Scoring : assign
 Yes Finished? fitness to off- spring
 Finish
 Fig. Genetic algorithm – program flow chart
 15
 fo
 .in
 rs
 de
 SC – GA – Encoding
 ea

2. Encoding

yr

.m

w Before a genetic algorithm can be put to work on any problem, a method is

w

,w

ty

needed to encode potential solutions to that problem in a form so that a

or

ab

computer can process.

kr

ha

C

– One common approach is to encode solutions as binary strings: sequences

C

R

of 1's and 0's, where the digit at each position represents the value of some aspect of the solution.

Example :

A Gene represents some data (eye color, hair color, sight, etc.).

A chromosome is an array of genes. In binary form

a Gene looks like : (11100010)

a Chromosome looks like: Gene1 Gene2 Gene3 Gene4

(11000010, 00001110, 001111010, 10100011)

A chromosome should in some way contain information about solution which it represents; it thus requires encoding. The most popular way of encoding is a binary string like :

Chromosome 1 : 1101100100110110

Chromosome 2 : 1101111000011110

Each bit in the string represent some characteristics of the solution.

– There are many other ways of encoding, e.g., encoding values as integer or real numbers or some permutations and so on.

– The virtue of these encoding method depends on the problem to work on .

16

fo

.in

rs

de

SC – GA - Encoding

ea

- Binary Encoding

yr

.m

w

w

Binary encoding is the most common to represent information contained.

,w

ty

or

In genetic algorithms, it was first used because of its relative simplicity.

ab

kr

ha

C

- In binary encoding, every chromosome is a string of bits : 0 or 1, like

C

R

Chromosome 1: 101100101100101011100101

Chromosome 2: 111111100000110000011111

- Binary encoding gives many possible chromosomes even with a small number of alleles ie possible settings for a trait (features).

- This encoding is often not natural for many problems and sometimes corrections must be made after crossover and/or mutation.

Example 1:

One variable function, say 0 to 15 numbers, numeric values, represented by 4 bit binary string.

Numeric 4-bit Numeric 4-bit Numeric 4-bit
value string value string value string

0 0000 6 0110 12 1100

1 0001 7 0111 13 1101

2 0010 8 1000 14 1110

3 0011 9 1001 15 1111

4 0100 10 1010

5 0101 11 1011

17

fo

.in

rs

de

SC - GA - Encoding

ea

[continued binary encoding]

yr

.m

Example 2 :

w

w

,w

Two variable function represented by 4 bit string for each variable.

ty

or

ab

Let two variables X_1 , X_2 as (1011 0110) .

kr

ha

U

 $\leq \leq$

C

Every variable will have both upper and lower limits as $X_i \leq X_i \leq X_i$

C

R

Because 4-bit string can represent integers from 0 to 15,

so (0000 0000) and (1111 1111) represent the points for X_1 , X_2 as

U U

(X_1 , L

X_2) and (X_1 , X_2) respectively.

Thus, an n-bit string can represent integers from

0 to $2^n - 1$, i.e. 2^n integers.

Binary Coding Equivalent integer Decoded binary substring

2 10 Remainder

Let X_i is coded as a substring

1 0 1 0

2 5 0 0 S_i of length n_i . Then decoded

$0 \times 2 = 0$

1 binary substring S_i is as

2 2 1 $1 \times 2 = 2$

2

$$1 \leq i \leq n-1$$

$$1 \leq i \leq n-1$$

$$3$$

$$= 8 \sum_{k=0}^{n-1} S_k 2^k$$

$$10 \text{ } k=0$$

where S_i can be 0 or 1 and the

string S is represented as

$$S_{n-1} \dots S_3 S_2 S_1 S_0$$

Example : Decoding value

Consider a 4-bit string (0111),

– the decoded value is equal to

$$2^3 \times 0 + 2^2 \times 1 + 2^1 \times 1 + 2^0 \times 1 = 7$$

L

– Knowing X_i and X_{iU} corresponding to (0000) and (1111),

the equivalent value for any 4-bit string can be obtained as

$U \cdot L$

$$(X_i - X_i)$$

L

$$X_i = X_i + \frac{X_{iU} - X_i}{2^n - 1} \times (\text{decoded value of string})$$

$$(2^n - 1)$$

$L \cdot U$

– For e.g. a variable X_i ; let $X_i = 2$, and $X_{iU} = 17$, find what value the

4-bit string $X_i = (1010)$ would represent. First get decoded value for

$$S_i = 1010 = 2^3 \times 1 + 2^2 \times 0 + 2^1 \times 1 + 2^0 \times 0 = 10 \text{ then}$$

$$(17 - 2)$$

$$X_i = 2 + \frac{17 - 2}{2^4 - 1} \times 10 = 12$$

$$(2^4 - 1)$$

The accuracy obtained with a 4-bit code is 1/16 of search space.

By increasing the string length by 1-bit, accuracy increases to 1/32.

18

fo

.in

rs

de

SC – GA – Encoding

ea

• Value Encoding

yr

.m

w

w

The Value encoding can be used in problems where values such as real

,w

ty

or

numbers are used. Use of binary encoding for this type of problems

ab

kr

would be difficult.

ha

C

C

R

1. In value encoding, every chromosome is a sequence of some values.

2. The Values can be anything connected to the problem, such as :

real numbers, characters or objects.

Examples :

Chromosome A 1.2324 5.3243 0.4556 2.3293 2.4545

Chromosome B ABDJEIFJDHDIERJFDLDFLFEGT

Chromosome C (back), (back), (right), (forward), (left)

3. Value encoding is often necessary to develop some new types of crossovers and mutations specific for the problem.

19

fo

.in

rs

de

SC - GA - Encoding

ea

• Permutation Encoding

yr

.m

w

w

Permutation encoding can be used in ordering problems, such as traveling

,w

ty

or

salesman problem or task ordering problem.

ab

kr

1. In permutation encoding, every chromosome is a string of numbers

ha

C

C

that represent a position in a sequence.

R

Chromosome A 1 5 3 2 6 4 7 9 8

Chromosome B 8 5 6 7 2 3 1 4 9

2. Permutation encoding is useful for ordering problems. For some problems, crossover and mutation corrections must be made to leave the chromosome consistent.

Examples :

1. The Traveling Salesman problem:

There are cities and given distances between them. Traveling salesman has to visit all of them, but he does not want to travel more than necessary. Find a sequence of cities with a minimal traveled distance. Here, encoded chromosomes describe the order of cities the salesman visits.

2. The Eight Queens problem :

There are eight queens. Find a way to place them on a chess board so that no two queens attack each other. Here, encoding describes the position of a queen on each row.

20

fo

.in

rs

de

SC - GA - Encoding

ea

• Tree Encoding

yr

.m

w

w

Tree encoding is used mainly for evolving programs or expressions.

,w

ty

or

For genetic programming :

ab

kr

– In tree encoding, every chromosome is a tree of some objects, such as

ha

C

C

functions or commands in programming language.

R

– Tree encoding is useful for evolving programs or any other structures that can be encoded in trees.

– The crossover and mutation can be done relatively easy way .

Example :

Chromosome A Chromosome B

do untill

x /

step wall

5 y

(+ x(/ 5y)) (do until step wall)

Fig. Example of Chromosomes with tree encoding

Note : Tree encoding is good for evolving programs. The programming language LISP is often used. Programs in LISP can be easily parsed as a tree, so the crossover and mutation is relatively easy.

21

fo

.in

rs

de

SC – GA - Operators

ea

3. Operators of Genetic Algorithm

yr

.m

w

w

Genetic operators used in genetic algorithms maintain genetic diversity.

,w

ty

or

Genetic diversity or variation is a necessity for the process of evolution.

ab

kr

Genetic operators are analogous to those which occur in the natural world:

ha

C

C

– Reproduction (or Selection) ;

R

– Crossover (or Recombination); and

– Mutation.

In addition to these operators, there are some parameters of GA.

One important parameter is Population size.

– Population size says how many chromosomes are in population (in one generation).

– If there are only few chromosomes, then GA would have a few possibilities to perform crossover and only a small part of search space is explored.

– If there are many chromosomes, then GA slows down.

– Research shows that after some limit, it is not useful to increase population size, because it does not help in solving the problem faster. The population size depends on the type of encoding and the problem.

22

fo

.in

rs

de

SC – GA – Operators

ea

3.1 Reproduction, or Selection

yr

.m

w

w

Reproduction is usually the first operator applied on population. From

,w

ty

or

the population, the chromosomes are selected to be parents to crossover

ab

kr

and produce offspring.

ha

C

C

R

The problem is how to select these chromosomes ?

According to Darwin's evolution theory "survival of the fittest" - the best ones should survive and create new offspring.

- The Reproduction operators are also called Selection operators.
- Selection means extract a subset of genes from an existing population, according to any definition of quality. Every gene has a meaning, so one can derive from the gene a kind of quality measurement called fitness function. Following this quality (fitness value), selection can be performed.
- Fitness function quantifies the optimality of a solution (chromosome) so that a particular solution may be ranked against all the other solutions. The function depicts the closeness of a given 'solution' to the desired result.

Many reproduction operators exists and they all essentially do same thing. They pick from current population the strings of above average and insert their multiple copies in the mating pool in a probabilistic manner.

The most commonly used methods of selecting chromosomes for parents to crossover are :

- Roulette wheel selection, – Rank selection
- Boltzmann selection, – Steady state selection.
- Tournament selection,

The Roulette wheel and Boltzmann selections methods are illustrated next.

23

fo

.in

rs

de

SC – GA – Operators

ea

- Example of Selection

yr

.m

w

w

,w

Evolutionary Algorithms is to maximize the function $f(x) = x^2$ with x in
ty

or

the integer interval $[0, 31]$, i.e., $x = 0, 1, \dots, 30, 31$.

ab

kr

ha

C

1. The first step is encoding of chromosomes; use binary representation

C

R

for integers; 5-bits are used to represent integers up to 31.

2. Assume that the population size is 4.

3. Generate initial population at random. They are chromosomes or
genotypes; e.g., 01101, 11000, 01000, 10011.

4. Calculate fitness value for each individual.

(a) Decode the individual into an integer (called phenotypes),

01101 $\rightarrow$ 13; 11000 $\rightarrow$ 24; 01000 $\rightarrow$ 8; 10011 $\rightarrow$ 19;

(b) Evaluate the fitness according to $f(x) = x^2$,

13 $\rightarrow$ 169; 24 $\rightarrow$ 576; 8 $\rightarrow$ 64; 19 $\rightarrow$ 361.

5. Select parents (two individuals) for crossover based on their fitness
in p_i . Out of many methods for selecting the best chromosomes, if
th

roulette-wheel selection is used, then the probability of the i string

n

in the population is $p_i = F_i / (\sum_{j=1}^n F_j)$, where

$j=1$

F_i is fitness for the string i in the population, expressed as $f(x)$

p_i is probability of the string i being selected,

n is no of individuals in the population, is population size, $n=4$

$n * p_i$ is expected count

String No Initial X value Fitness F_i p_i Expected count

Population $f(x) = x^2$ $N * \text{Prob } i$

1 01101 13 169 0.14 0.58

2 11000 24 576 0.49 1.97

3 01000 8 64 0.06 0.22

4 10011 19 361 0.31 1.23

Sum 1170 1.00 4.00

Average 293 0.25 1.00

Max 576 0.49 1.97

The string no 2 has maximum chance of selection.

24

fo

.in

rs

de

SC – GA – Operators

ea

- Roulette wheel selection (Fitness-Proportionate Selection)

yr

.m

w

w

,w

Roulette-wheel selection, also known as Fitness Proportionate Selection, is

ty

or

a genetic operator, used for selecting potentially useful solutions for

ab

kr

ha

recombination.

C

C

R

In fitness-proportionate selection :

– the chance of an individual's being selected is proportional to its fitness, greater or less than its competitors' fitness.

– conceptually, this can be thought as a game of Roulette.

The Roulette-wheel simulates 8

1

5% 2 individuals with fitness values F_i ,

8

20% 9%

marked at its circumference; e.g.,

th

3 – the 5 individual has a higher

13%

7 fitness than others, so the wheel

8%

would choose the 5th individual

6 more than other individuals .

8%

17% – the fitness of the individuals is

4 calculated as the wheel is spun

20%

$n = 8$ times, each time selecting

5

an instance, of the string, chosen

Fig. Roulette-wheel Shows 8

individual with fitness by the wheel pointer.

n

Probability of i

th

string is $p_i = F_i / (\sum_{j=1}^n F_j)$, where

$j=1$

n = no of individuals, called population size; p_i = probability of i th string being selected; F_i = fitness for i th string in the population.

Because the circumference of the wheel is marked according to a string's fitness, the Roulette-wheel mechanism is expected to

F

make copies of the i th string.

F

Average fitness = F_j / n ; Expected count = $(n = 8) \times p_i$

$N=5$

Cumulative Probability $_5 = \sum_{i=1}^5 p_i$

$i=1$

25

fo

.in

rs

de

SC – GA – Operators

ea

- Boltzmann Selection

yr

.m

w

w

Simulated annealing is a method used to minimize or maximize a function.

,w

ty

or

ab

– This method simulates the process of slow cooling of molten metal to

kr

ha

C

achieve the minimum function value in a minimization problem.

C

R

– The cooling phenomena is simulated by controlling a temperature like parameter introduced with the concept of Boltzmann probability distribution.

– The system in thermal equilibrium at a temperature T has its energy distribution based on the probability defined by

$P(E) = \exp(-E / kT)$ where k is Boltzmann constant.

– This expression suggests that a system at a higher temperature has almost uniform probability at any energy state, but at lower temperature it has a small probability of being at a higher energy state.

– Thus, by controlling the temperature T and assuming that the search process follows Boltzmann probability distribution, the convergence of the algorithm is controlled.

26

fo

.in

rs

de

SC – GA – Operators

ea

3.2 Crossover

yr

.m

w

w

Crossover is a genetic operator that combines (mates) two chromosomes

,w

ty

or

(parents) to produce a new chromosome (offspring). The idea behind

ab

kr

crossover is that the new chromosome may be better than both of the

ha

C

C

parents if it takes the best characteristics from each of the parents.

R

Crossover occurs during evolution according to a user-definable crossover probability. Crossover selects genes from parent chromosomes and creates a new offspring.

The Crossover operators are of many types.

- one simple way is, One-Point crossover.

- the others are Two Point, Uniform, Arithmetic, and Heuristic crossovers.

The operators are selected based on the way chromosomes are encoded.

27

fo

.in

rs

de

SC – GA – Operators

ea

- One-Point Crossover

yr

.m

w

w

One-Point crossover operator randomly selects one crossover point and

,w

ty

or

then copy everything before this point from the first parent and then

ab

kr

everything after the crossover point copy from the second parent. The

ha

C

C

Crossover would then look as shown below.

R

Consider the two parents selected for crossover.

Parent 1 11011|00100110110

Parent 2 11011|11000011110

Interchanging the parents chromosomes after the crossover points -

The Offspring produced are :

Offspring 1 11011|11000011110

Offspring 2 11011|00100110110

Note : The symbol, a vertical line, | is the chosen crossover point.

28

fo

.in

rs

de

SC - GA - Operators

ea

• Two-Point Crossover

yr

.m

w

w

Two-Point crossover operator randomly selects two crossover points within

,w

ty

or

a chromosome then interchanges the two parent chromosomes between

ab

kr

these points to produce two new offspring.

ha

C

C

R

Consider the two parents selected for crossover :

Parent 1 11011|0010011|0110

Parent 2 11011|1100001|1110

Interchanging the parents chromosomes between the crossover points -

The Offspring produced are :

Offspring 1 11011|0010011|0110

Offspring 2 11011|0010011|0110

29

fo

.in

rs

de

SC - GA - Operators

ea

- Uniform Crossover

yr

.m

w

w

Uniform crossover operator decides (with some probability - know as the

,w

ty

or

mixing ratio) which parent will contribute how the gene values in the

ab

kr

offspring chromosomes. The crossover operator allows the parent

ha

C

C

chromosomes to be mixed at the gene level rather than the segment

R

level (as with one and two point crossover).

Consider the two parents selected for crossover.

Parent 1 1 1 0 1 1 0 0 1 0 0 1 1 0 1 0

Parent 2 1 1 0 1 1 1 1 0 0 0 0 1 1 1 0

If the mixing ratio is 0.5 approximately, then half of the genes in the offspring will come from parent 1 and other half will come from parent 2.

The possible set of offspring after uniform crossover would be:

Offspring 1 1 1 1 2 0 2 1 1 1 1 12 12 0 2 0 1 0 1 0 2 1 1 1 2 1 1 1 1 02

Offspring 2 1 2 1 1 0 1 1 2 1 2 01 01 1 1 0 2 0 2 1 1 1 2 0 1 1 2 1 2 01

Note: The subscripts indicate which parent the gene came from.

30

fo

.in

rs

de

SC - GA - Operators

ea

- Arithmetic

yr

.m

w

w

Arithmetic crossover operator linearly combines two parent chromosome

,w

ty

or

vectors to produce two new offspring according to the equations:

ab

kr

ha

Offspring1 = $a * \text{Parent1} + (1 - a) * \text{Parent2}$

C

C

R

Offspring2 = $(1 - a) * \text{Parent1} + a * \text{Parent2}$

where a is a random weighting factor chosen before each crossover operation.

Consider two parents (each of 4 float genes) selected for crossover:

Parent 1 (0.3) (1.4) (0.2) (7.4)

Parent 2 (0.5) (4.5) (0.1) (5.6)

Applying the above two equations and assuming the weighting factor $a = 0.7$, applying above equations, we get two resulting offspring.

The possible set of offspring after arithmetic crossover would be:

Offspring 1 (0.36) (2.33) (0.17) (6.87)

Offspring 2 (0.402) (2.981) (0.149) (5.842)

31

fo

.in

rs

de

SC - GA - Operators

ea

• Heuristic

yr

.m

w

w

Heuristic crossover operator uses the fitness values of the two parent

,w

ty

or

chromosomes to determine the direction of the search.

ab

kr

ha

C

The offspring are created according to the equations:

C

R

$$\text{Offspring1} = \text{BestParent} + r * (\text{BestParent} - \text{WorstParent})$$

$$\text{Offspring2} = \text{BestParent}$$

where r is a random number between 0 and 1.

It is possible that offspring1 will not be feasible. It can happen if r is chosen such that one or more of its genes fall outside of the allowable upper or lower bounds. For this reason, heuristic crossover has a user defined parameter n for the number of times to try and find an r that results in a feasible chromosome. If a feasible chromosome is not produced after n tries, the worst parent is returned as offspring1.

32

fo

.in

rs

de

SC - GA - Operators

ea

3.3 Mutation

yr

.m

w

w

After a crossover is performed, mutation takes place.

,w

ty

or

Mutation is a genetic operator used to maintain genetic diversity from

ab

kr

one generation of a population of chromosomes to the next.

ha

C

C

R

Mutation occurs during evolution according to a user-definable mutation probability, usually set to fairly low value, say 0.01 a good first choice.

Mutation alters one or more gene values in a chromosome from its initial state. This can result in entirely new gene values being added to the gene pool. With the new gene values, the genetic algorithm may be able to arrive at better solution than was previously possible.

Mutation is an important part of the genetic search, helps to prevent the population from stagnating at any local optima. Mutation is intended to prevent the search falling into a local optimum of the state space.

The Mutation operators are of many type.

– one simple way is, Flip Bit.

– the others are Boundary, Non-Uniform, Uniform, and Gaussian.

The operators are selected based on the way chromosomes are encoded .

33

fo

.in

rs

de

SC - GA - Operators

ea

- Flip Bit

yr

.m

w

w

The mutation operator simply inverts the value of the chosen gene.

,w

ty

or

i.e. 0 goes to 1 and 1 goes to 0.

ab

kr

This mutation operator can only be used for binary genes.

ha

C

C

R

Consider the two original off-springs selected for mutation.

Original offspring 1 1 1 0 1 1 1 1 0 0 0 0 1 1 1 0

Original offspring 2 1 1 0 1 1 0 0 1 0 0 1 1 0 1 1 0

Invert the value of the chosen gene as 0 to 1 and 1 to 0

The Mutated Off-spring produced are :

Mutated offspring 1 1 1 0 0 1 1 1 0 0 0 0 1 1 1 0

Mutated offspring 2 1 1 0 1 1 0 1 1 0 0 1 1 0 1 0 0

34

fo

.in

rs

de

SC - GA - Operators

ea

- Boundary

yr

.m

w The mutation operator replaces the value of the chosen gene with either

w

,w

ty

the upper or lower bound for that gene (chosen randomly).

or

ab

This mutation operator can only be used for integer and float genes.

kr

ha

C

C

R

- Non-Uniform

The mutation operator increases the probability such that the amount of the mutation will be close to 0 as the generation number increases. This mutation operator prevents the population from stagnating in the early stages of the evolution then allows the genetic algorithm to fine tune the solution in the later stages of evolution.

This mutation operator can only be used for integer and float genes.

- Uniform

The mutation operator replaces the value of the chosen gene with a uniform random value selected between the user-specified upper and lower bounds for that gene.

This mutation operator can only be used for integer and float genes.

- Gaussian

The mutation operator adds a unit Gaussian distributed random value to the chosen gene. The new gene value is clipped if it falls outside of the user-specified lower or upper bounds for that gene.

This mutation operator can only be used for integer and float genes.

35

fo

.in

rs

de

SC – GA – Examples

ea

4. Basic Genetic Algorithm :

yr

.m

w

w

Examples to demonstrate and explain : Random population, Fitness, Selection,

,w

ty

or

Crossover, Mutation, and Accepting.

ab

kr

ha

C

• Example 1 :

C

R

Maximize the function $f(x) = x^2$ over the range of integers from 0 . . . 31.

Note : This function could be solved by a variety of traditional methods such as a hill-climbing algorithm which uses the derivative.

One way is to :

- Start from any integer x in the domain of f
- Evaluate at this point x the derivative f'
- Observing that the derivative is +ve, pick a new x which is at a small distance in the +ve direction from current x
- Repeat until $x = 31$

See, how a genetic algorithm would approach this problem ?

36

fo

.in

rs

de

SC - GA - Examples

ea

[continued from previous slide]

yr

.m

w Genetic Algorithm Approach to problem - Maximize the function $f(x) = x^2$

w

,w

ty

or

1. Devise a means to represent a solution to the problem :

ab

kr

Assume, we represent x with five-digit unsigned binary integers.

ha

C

C

2. Devise a heuristic for evaluating the fitness of any particular solution :

R

The function $f(x)$ is simple, so it is easy to use the $f(x)$ value itself to rate the fitness of a solution; else we might have considered a more simpler heuristic that would more or less serve the same purpose.

3. Coding - Binary and the String length :

GAs often process binary representations of solutions. This works well, because crossover and mutation can be clearly defined for binary solutions. A Binary string of length 5 can represents 32 numbers (0 to 31).

4. Randomly generate a set of solutions :

Here, considered a population of four solutions. However, larger populations are used in real applications to explore a larger part of the search. Assume, four randomly generated solutions as : 01101, 11000, 01000, 10011.

These are chromosomes or genotypes.

5. Evaluate the fitness of each member of the population :

The calculated fitness values for each individual are -

(a) Decode the individual into an integer (called phenotypes),

01101 $\rightarrow$ 13; 11000 $\rightarrow$ 24; 01000 $\rightarrow$ 8; 10011 $\rightarrow$ 19;

(b) Evaluate the fitness according to $f(x) = x^2$,

13 $\rightarrow$ 169; 24 $\rightarrow$ 576; 8 $\rightarrow$ 64; 19 $\rightarrow$ 361.

(c) Expected count = $N * \text{Prob } i$, where N is the number of

individuals in the population called population size, here $N = 4$.

Thus the evaluation of the initial population summarized in table below .

String No Initial X value Fitness Prob i Expected count

i Population (Pheno $f(x) = x^2$ (fraction $N * \text{Prob } i$

(chromosome) types) of total)

1 01101 13 169 0.14 0.58

2 11000 24 576 0.49 1.97

3 01000 8 64 0.06 0.22

4 10011 19 361 0.31 1.23

Total (sum) 1170 1.00 4.00

Average 293 0.25 1.00

Max 576 0.49 1.97

Thus, the string no 2 has maximum chance of selection.

37

fo

.in

rs

de

SC – GA – Examples

ea

6. Produce a new generation of solutions by picking from the existing

yr

.m

w pool of solutions with a preference for solutions which are better

w

,w

ty

suited than others:

or

ab

We divide the range into four bins, sized according to the relative fitness of

kr

ha

C

the solutions which they represent.

C

R

Strings Prob i Associated Bin

01101 0.14 0.0 . . . 0.14

11000 0.49 0.14 . . . 0.63

01000 0.06 0.63 . . . 0.69

10011 0.31 0.69 . . . 1.00

By generating 4 uniform (0, 1) random values and seeing which bin they fall into we pick the four strings that will form the basis for the next generation.

Random No Falls into bin Chosen string

0.08 0.0 . . . 0.14 01101

0.24 0.14 . . . 0.63 11000

0.52 0.14 . . . 0.63 11000

0.87 0.69 . . . 1.00 10011

7. Randomly pair the members of the new generation

Random number generator decides for us to mate the first two strings together and the second two strings together.

8. Within each pair swap parts of the members solutions to create

offspring which are a mixture of the parents :

For the first pair of strings: 01101 , 11000

– We randomly select the crossover point to be after the fourth digit.

Crossing these two strings at that point yields:

01101 $\Rightarrow$ 0 1 1 0 | 1 $\Rightarrow$ 01100

11000 $\Rightarrow$ 1 1 0 0 | 0 $\Rightarrow$ 11001

For the second pair of strings: 11000 , 10011

– We randomly select the crossover point to be after the second digit.

Crossing these two strings at that point yields:

11000 $\Rightarrow$ 1 1 | 0 0 0 $\Rightarrow$ 11011

10011 $\Rightarrow$ 1 0 | 0 1 1 $\Rightarrow$ 10000

38

fo

.in

rs

de

SC - GA - Examples

ea

9. Randomly mutate a very small fraction of genes in the population :

yr

.m

w With a typical mutation probability of per bit it happens that none of the bits

w

,w

ty

in our population are mutated.

or

ab

kr

10. Go back and re-evaluate fitness of the population (new generation) :

ha

C

C

This would be the first step in generating a new generation of solutions.

R

However it is also useful in showing the way that a single iteration of the genetic algorithm has improved this sample.

String No Initial X value Fitness Prob i Expected count

Population (Pheno f(x) = x² (fraction

(chromosome) types) of total)

1 01100 12 144 0.082 0.328

2 11001 25 625 0.356 1.424

3 11011 27 729 0.415 1.660
 4 10000 16 256 0.145 0.580
 Total (sum) 1754 1.000 4.000
 Average 439 0.250 1.000
 Max 729 0.415 1.660

Observe that :

1. Initial populations : At start step 5 were

01101, 11000, 01000 , 10011

After one cycle, new populations, at step 10 to act as initial population

01100, 11001, 1 1 0 11 , 10000

2. The total fitness has gone from 1170 to 1754 in a single generation.

3. The algorithm has already come up with the string 11011 (i.e $x = 27$) as a possible solution.

39

fo

.in

rs

de

SC - GA - Examples

ea

- Example 2 : Two bar pendulum

yr

.m

w

w

Two uniform bars are connected by pins at A and B and supported

,w

ty

or

at A. Let a horizontal force P acts at C.

ab

kr

Given : Force $P = 2$, Length of bars $\ell_1 = 2$,

ha

A

C

y $\ell_2 = 2$, Bar weights $W_1 = 2$, $W_2 = 2$. angles = X_i

C

R

ℓ_1 Find : Equilibrium configuration of the system if

θ_1

B friction at all joints are neglected ?

ℓ_2 C

θ_2

P Solution : Since there are two unknowns θ_1 and

θ_2 , we use 4 - bit binary for each unknown.

W1

XU - XL 90 - 0

W2 Accuracy = ----- = ----- = 60

Fig. Two bar pendulum 24 - 1 15

Hence, the binary coding and the corresponding angles X_i are given as

$X_{iU} - X_{iL}$

$X_i = X_{iL} + \text{-----} S_i$ where S_i is decoded Value of the i th chromosome.

24 - 1

e.g. the 6th chromosome binary code (0 1 0 1) would have the corresponding angle given by $S_i = 0 \cdot 1 + 1 \cdot 2 + 0 \cdot 4 + 1 \cdot 8 = 11$

90 - 0

$X_i = 0 + \text{-----} \times 11 = 11$

15

The binary coding and the angles are given in the table below.

S. No. Binary code Angle S. No. Binary code Angle

S_i X_i S_i X_i

1 0000 0 9 1000 48

2 0001 6 10 1001 54

3 0010 12 11 1010 60

4 0011 18 12 1011 66

5 0100 24 13 1100 72

6 0101 30 14 1101 78

7 0110 36 15 1110 84

8 0111 42 16 1111 90

Note : The total potential for two bar pendulum is written as

$$\Pi = -P[(\ell_1 \sin\theta_1 + \ell_2 \sin\theta_2)] - (W_1 \ell_1 / 2) \cos\theta_1 - W_2 [(\ell_2 / 2) \cos\theta_2 + \ell_1 \cos\theta_1] \quad (\text{Eq.1})$$

Substituting the values for P, W_1 , W_2 , ℓ_1 , ℓ_2 all as 2 , we get ,

$$\Pi(\theta_1, \theta_2) = -4 \sin\theta_1 - 6 \cos\theta_1 - 4 \sin\theta_2 - 2 \cos\theta_2 = \text{function } f \quad (\text{Eq. 2})$$

θ_1, θ_2 lies between 0 and 90 both inclusive ie $0 \leq \theta_1, \theta_2 \leq 90$ (Eq. 3)

Equilibrium configuration is the one which makes Π a minimum .

Since the objective function is -ve , instead of minimizing the function f let us maximize $-f = f'$. The maximum value of $f' = 8$ when θ_1 and θ_2 are zero.

Hence the fitness function F is given by $F = -f - 7 = f' - 7$ (Eq. 4)

48

fo

.in

rs

de

SC - GA - Examples

ea

[continued from previous slide]

yr

.m

w First randomly generate 8 population with 8 bit strings as shown in table below.

w

,w

ty

Population Population of 8 bit strings Corresponding Angles $F = -f - 7$

or

ab

No. (Randomly generated) (from table above)

kr

θ_1 , θ_2

ha

C

1 0000 0000 0 0 1

C

2 0010 0000 12 6 2.1

R

3 0001 0000 6 30 3.11

4 0010 1000 12 48 4.01

5 0110 1010 36 60 4.66

6 1110 1000 84 48 1.91

7 1110 1101 84 78 1.93

8 0111 1100 42 72 4.55

These angles and the corresponding to fitness function are shown below.

$F=1$ $F=2.1$ $F=3.11$ $F=3.11$

$\theta_1=0$ $\theta_1=12$ $\theta_1=6$ $\theta_1=12$

$\theta_2=0$ $\theta_2=6$ $\theta_2=30$ $\theta_2=48$

$F=4.6$ $F=1.91$ $F=1.93$ $F=4.55$

$\theta_1=36$ $\theta_1=84$ $\theta_1=84$ $\theta_1=42$

$\theta_2=60$ $\theta_2=48$ $\theta_2=78$ $\theta_2=72$

Fig. Fitness function F for various population

The above Table and the Fig. illustrates that :

- GA begins with a population of random strings.
- Then, each string is evaluated to find the fitness value.
- The population is then operated by three operators –
Reproduction , Crossover and Mutation, to create new population.
- The new population is further evaluated tested for termination.
- If the termination criteria are not met, the population is iteratively operated
by the three operators and evaluated until the termination criteria are met.
- One cycle of these operation and the subsequent evaluation procedure is
known as a Generation in GA terminology.

49

fo

.in

rs

de

Sc – GA – References

ea

5. References : Textbooks

yr

.m

w

w

1. "Genetic Algorithms in Search, Optimization, and Machine Learning", by David E.

,w

Goldberg, (1989), Addison-Wesley, Chapter 1-8, page 1- 432.

ty

or

ab

kr

2. "An Introduction to Genetic Algorithms", by Melanie Mitchell, (1998), MIT Press,

ha

Chapter 1- 6, page 1- 203,

C

C

R

3. "Genetic Algorithms: Concepts And Designs", by K. F. Man, K. S. and Tang, S.

Kwong, (201), Springer, Chapter 1- 10, page 1- 348,

4. "Genetic algorithms and engineering design", by Mitsuo Gen, and Runwei Cheng,

(1997), John Wiley & Sons Inc, chapter 1- 10, page 1-411.

5. "Practical genetic algorithms", by Randy L. Haupt, (2004), John Wiley & Sons Inc, Chapter 1- 7, page 1- 251.

6. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

42

fo

.in

rs

de

ea

yr

.m

Artificial Intelligence : Course Content , Lecture hours – 42 , notes , slides : 565

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

kr

www.myreaders.info

ha

C

C

R

Course Content

Artificial Intelligence

Artificial Intelligence topics : Introduction, Problem solving, Search and control strategies, Knowledge representations issues, predicate logic, rules, Reasoning system - symbolic, statistical, Game playing, Learning systems, Expert systems, Fundamentals of neural networks, Fundamentals of genetic algorithms, Natural language processing, Common sense.

fo

.in

rs

de

Course Content

ea

yr

.m

w

w

Artificial Intelligence

,w

ty

or

ab

Content Hrs

kr

ha

C

01 Introduction to AI 1-6

C

R

Definitions, Goals of AI, AI Approaches, AI Techniques, Branches of AI, Applications of AI.

02 Problem Solving, Search and Control Strategies : 7-14

General problem solving, Search and control strategies, Exhaustive searches, Heuristic search techniques, Constraint satisfaction problems (CSPs) and models .

03 Knowledge Representations Issues, Predicate Logic, Rules : 15-22

Knowledge representation, KR using predicate logic, KR using rules.

04 Reasoning System - Symbolic , Statistical : 23-28

Reasoning - Over view, Symbolic reasoning, Statistical reasoning.

05 Game Playing : 29-30

Overview, Mini-Max search procedure, Game playing with Mini-Max, Alpha-Beta pruning.

06 Learning Systems: 31-34

Rote learning, Learning from example : Induction, Explanation Based Learning (EBL), Discovery, Clustering, Analogy, Neural net and genetic learning, Reinforcement learning.

07 Expert Systems : 35-36

Knowledge acquisition, Knowledge base, Working memory, Inference engine, Expert system shells, Explanation, Application of expert systems.

08 Fundamentals of Neural Networks : 37-38

Research history, Model of artificial neuron, Neural networks architectures, Learning methods in neural networks, Single-layer neural network system, Applications of neural networks.

09 Fundamentals of Genetic Algorithms : 39-40

Search optimization algorithm, Evolutionary algorithm, Encoding, Operators of genetic algorithm, Basic genetic algorithm.

10 Natural Language Processing : 41

Introduction, Syntactic processing , Semantic and Pragmatic analysis.

11 Common Sense : 42

Introduction, Formalization of common sense reasoning, Physical world, Common sense ontologies, Memory organization.

fo

.in

rs

de

ea

yr

.m

w Learning Systems : AI Course Lecture 31 – 34, notes, slides

w

,w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

ty

or

ab

www.myreaders.info/html/artificial_intelligence.html

kr

www.myreaders.info

ha

C

C

R

Learning System

Artificial Intelligence

Learning systems, topics : Definition, learning agents, components of learning system, paradigms of machine learning. Rote Learning : learning by memorization, learning something by repeating.

Learning from example : Induction, Winston's learning, Version

spaces - learning algorithm (generalization and specialization tree),
 Decision trees - ID3 algorithm. Explanation Based Learning (EBL) :
 general approach, EBL architecture, EBL system, generalization
 problem, explanation structure. Discovery : theory driven - AM
 system, data driven - BACON system. Clustering : distance
 functions, K-mean clustering algorithm. Learning by analogy;
 Neural Net - Perceptron; Genetic Algorithm. Reinforcement
 Learning : RL Problem, agent - environment interaction, RL tasks,
 Markov system, Markov decision processes, agent's learning task,
 policy, reward function, maximize reward, value functions.

fo

.in

rs

de

ea

Learning System

yr

.m

w

w

Artificial Intelligence

,w

ty

or

ab

Topics

kr

ha

C

(Lectures 31, 32, 33, 34 4 hours) Slides

C

R

1. What is Learning 03-09

Definition, learning agents, components of learning system; Paradigms
 of machine learning.

2. Rote Learning 10

Learning by memorization, Learning something by repeating.

3. Learning from Example : Induction 11-38

Winston's learning, Version spaces - learning algorithm (generalization

and specialization tree), Decision trees - ID3 algorithm.

4. Explanation Based Learning (EBL) 39-43

General approach, EBL architecture, EBL system, Generalization problem, Explanation structure.

5. Discovery 44-52

Theory driven - AM system, Data driven - BACON system

6. Clustering 53-62

Distance functions, K-mean clustering - algorithm.

7. Analogy 63

8. Neural net and Genetic Learning 64-67

Neural Net - Perceptron; Genetic learning - Genetic Algorithm.

9. Reinforcement Learning 68-80

RL Problem : Agent - environment interaction, key Features; RL tasks, Markov system, Markov decision processes, Agent's learning task, Policy, Reward function, Maximize reward, Value functions.

10 References 81

02

fo

.in

rs

de

ea

yr

Learning

.m

w

w

,w

Machine Learning

ty

or

ab

kr

What is learning ? Some Quotations

ha

C

C

R

- Herbert Simon, 1983

Learning denotes changes in a system that enable a system to do the same task more efficiently the next time.

- Marvin Minsky, 1986

Learning is making useful changes in the workings of our minds.

- Ryszard Michalski, 1986

Learning is constructing or modifying representations of what is being experienced.

- Mitchell, 1997

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

03

fo

.in

rs

de

AI - Learning

ea

1. What is Learning

yr

.m

w

w

Learning denotes changes in a system that enable the system to do the

,w

ty

or

same task more efficiently next time.

ab

kr

Learning is an important feature of “Intelligence”.

ha

C

C

R

1.1 Definition

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at

tasks in T, as measured by P, improves with experience E. (Mitchell 1997)

This means :

Given : A task T

A performance measure P

Some experience E with the task

Goal : Generalize the experience in a way that allows to improve your performance on the task.

Why do you require Machine Learning ?

- Understand and improve efficiency of human learning.
- Discover new things or structure that is unknown to humans.
- Fill in skeletal or incomplete specifications about a domain.

04

fo

.in

rs

de

AI - Learning

ea

1.2 Learning Agents

yr

.m

w

w

An agent is an entity that is capable of perceiving and do action.

,w

ty

or

An agent can be viewed as perceiving its environment through sensors and

ab

kr

acting upon that environment through actuators.

ha

C

C

R

Environment Sensors Actuators

Agent

Human agent Eyes, ears, etc Leg, hands, mouth

Robotic agent Cameras, IR range finders motors

Software agent Key stroke, File contents Displays to screen,
write files

In computer science an agent is a software agent that assists users
and acts in performing computer-related tasks.

05

fo

.in

rs

de

AI - Learning

ea

- Intelligent Agent (Learning Agent)

yr

.m

w Agent is an entity that is capable of perceiving and do action.

w

,w

ty

In computer science an agent is a software agent.

or

ab

In artificial intelligence, the term used for agent is an intelligent agent.

kr

ha

C

Learning is an important feature of “Intelligence”.

C

R

Percept : agent's perceptual inputs

Percept sequence : history of everything the agent has perceived

Agent function : describes agent's behavior

Agent program : Implements agent's function

Learning Agent consist of four main components :

◇ Learning element,

◇ Performance element,

◇ Critic, and

◇ Problem generator.

Components of a Learning System

Performance
 standard
 Percepts
 Critic Sensors
 E
 N
 feed back V
 changes R
 Learning Performance O
 Element Element N
 knowledge M
 learning E
 goals N
 Effectors T
 Actions
 Problem
 Generator
 Learning Agent

The components are described in the next slide.

06

fo

.in

rs

de

AI - Learning

ea

- Components of a Learning System

yr

.m

w

w

■ Performance Element: The Performance Element is the agent itself that

,w

ty

or

acts in the world. It takes in percepts and decides on external actions.

ab

kr

ha

■ Learning Element: It responsible for making improvements, takes

C
C
R

knowledge about performance element and some feedback,
determines how to modify performance element.

■ Critic: Tells the Learning Element how agent is doing (success or failure)
by comparing with a fixed standard of performance.

■ Problem Generator: Suggests problems or actions that will generate
new examples or experiences that will aid in training the system further.
Example : Automated Taxi on city roads

■ Performance Element: Consists of knowledge and procedures for
driving actions.

e.g., turning, accelerating, braking are performance element on roads.

■ Learning Element: Formulates goals.

e.g., learn rules for braking, accelerating, learn geography of the city.

■ Critic: Observes world and passes information to learning element.
e.g. , quick right turn across three lanes of traffic, observe reaction of
other drivers.

■ Problem Generator: Try south city road .

07

fo

.in

rs

de

AI - Learning

ea

1.2 Paradigms of Machine Learning

yr

.m

w

w

■ Rote Learning: Learning by memorization; One-to-one mapping from

,w

ty

or

inputs to stored representation; Association-based storage and

ab

kr

retrieval.

ha

C

C

R

■ Induction: Learning from examples; A form of supervised learning, uses specific examples to reach general conclusions; Concepts are learned from sets of labeled instances.

■ Clustering: Discovering similar group; Unsupervised, Inductive learning in which natural classes are found for data instances, as well as ways of classifying them.

■ Analogy: Determine correspondence between two different representations that come from Inductive learning in which a system transfers knowledge from one database into another database of a different domain.

08

fo

.in

rs

de

AI - Learning

ea

■ Discovery: Learning without the help from a teacher; Learning is

yr

.m

w both inductive and deductive. It is deductive if it proves

w

,w

ty

theorems and discovers concepts about those theorems. It is

or

ab

inductive when it raises conjectures (guess). It is unsupervised,

kr

ha

C

specific goal not given.

C

R

■ Genetic Algorithms: Inspired by natural evolution; In the natural world, the organisms that are poorly suited for an environment die off,

while those well-suited for it prosper. Genetic algorithms search the space of individuals for good candidates. The "goodness" of an individual is measured by some fitness function. Search takes place in parallel, with many individuals in each generation.

■ **Reinforcement:** Learning from feedback (+ve or -ve reward) given at end of a sequence of steps. Unlike supervised learning, the reinforcement learning takes place in an environment where the agent cannot directly compare the results of its action to a desired result. Instead, it is given some reward or punishment that relates to its actions. It may win or lose a game, or be told it has made a good move or a poor one. The job of reinforcement learning is to find a successful function using these rewards.

09

fo

.in

rs

de

AI – Rote Learning

ea

2. Rote Learning

yr

.m

w

w

Rote learning technique avoids understanding the inner complexities but

,w

ty

or

focuses on memorizing the material so that it can be recalled by the

ab

kr

learner exactly the way it was read or heard.

ha

C

C

R

- Learning by Memorization which avoids understanding the inner complexities the subject that is being learned; Rote learning instead focuses on memorizing the material so that it can be recalled by the learner exactly the way it was read or heard.

- Learning something by Repeating over and over and over again; saying the same thing and trying to remember how to say it; it does not help us to understand; it helps us to remember, like we learn a poem, or a song, or something like that by rote learning.

10

fo

.in

rs

de

AI -Learning from example : Induction

ea

3. Learning from Example : Induction

yr

.m

w

w

A process of learning by example. The system tries to induce a general rule

,w

ty

or

from a set of observed instances. The learning methods extract rules and

ab

kr

patterns out of massive data sets.

ha

C

C

The learning processes belong to supervised learning, does classification and

R

constructs class definitions, called induction or concept learning.

The techniques used for constructing class definitions (or concept leaning) are :

- Winston's Learning program
- Version Spaces
- Decision Trees

Note : The Decision tree technique has been explained in all details while other two techniques are briefly described in the next few slides.

11

fo

.in

rs

de

AI -Learning from example : Induction

ea

3.1 Winston's Learning

yr

.m

w

w

Winston (1975) described a Blocks World Learning program. This

,w

ty

or

program operated in a simple blocks domain. The goal is to construct

ab

kr

representation of the definition of concepts in the blocks domain.

ha

C

C

R

Example : Concepts such a "house".

■ Start with input, a line drawing of a blocks world structure.

It learned Concepts House, Tent, Arch as :

brick (rectangular block) with a wedge (triangular block) suitably placed on top of it, tent – as 2 wedges touching side by side, or an arch – as 2 non-touching bricks supporting a third wedge or brick.

■ The program for Each concept is learned through near miss. A near miss is an object that is not an instance of the concept but a very similar to such instances.

■ The program uses procedures to analyze the drawing and construct a semantic net representation.

■ An example of such an structural for the house is shown below.

Object - house Semantic net

A

has-part has-part

Supported - by

B C

isa isa

Wedge Brick

■ Node A represents entire structure, which is composed of two parts :
node B, a Wedge, and node C, a Brick.

Links in network include supported-by, has-part, and isa.

12

fo

.in

rs

de

AI -Learning from example : Induction

ea

- Winston's Program

yr

.m

w

w

■ Winston's program followed 3 basic steps in concept formulation:

,w

ty

or

ab

1. Select one known instance of the concept.

kr

ha

C

Call this the concept definition.

C

R

2. Examine definitions of other known instance of the concept.

Generalize the definition to include them.

3. Examine descriptions of near misses.

Restrict the definition to exclude these.

■ Both steps 2 and 3 of this procedure rely heavily on comparison
process by which similarities and differences between structures
can be detected.

■ Winston's program can be similarly applied to learn other concepts
such as "ARCH".

13

fo

.in

rs

de

AI -Learning from example: Induction

ea

3.2 Version Spaces

yr

.m

w

w

A Version Space is a hierarchical representation of knowledge that

,w

ty

or

keeps track of all the useful information supplied by a sequence of

ab

kr

learning examples without remembering any of the examples. The

ha

C

C

version space method is a concept learning process. It is another

R

approach to learning by Mitchell (1977).

• Version Space Characteristics

■ Represents all the alternative plausible descriptions of a heuristic.

A plausible description is one that is applicable to all known +ve examples and no known -ve example.

■ A version space description consists of two complementary trees:

‡ one, contains nodes connected to overly general models, and

‡ other, contains nodes connected to overly specific models.

■ Node values/attributes are discrete.

• Fundamental Assumptions

■ The data is correct ie no erroneous instances.

■ A correct description is a conjunction of some attributes with values.

14

fo

.in

rs

de

AI -Learning from example : Induction

ea

- Version Space Methods

yr

.m

w

w

A version space is a hierarchical representation of knowledge.

,w

ty

or

Version space method is a concept learning process accomplished by

ab

kr

ha

managing multiple models within a version space.

C

C

R

It is specialization of general models and generalization of specific models.

Version Space diagram described below consists : a Specialization tree (colored Blue) and a Generalization tree (colored green) .

Top 1st row : the top of the tree, we have the most general hypotheses.

Top 2nd row : This row is an expanded version of the first. This row of hypotheses is slightly more specific than the root node.

Top 3rd row : As training data (positive examples) is processed, the inconsistent nodes are removed from the general specification.

Finally, we converge to a solution.

Bottom 3rd row : Any hypothesis that is inconsistent with the training data (negative instances) is removed from the tree.

Bottom 2nd row : The specific hypothesis is expanded to form more nodes that are slightly more general.

Bottom 1st row : This is the most specific hypothesis.

Version Space diagram

Tree nodes :

‡ each node is connected to a model.

‡ nodes in the generalization tree are connected to a model that matches everything in its sub-tree.

‡ nodes in the specialization tree are connected to a model that matches only one thing in its sub-tree.

Links between nodes :

‡ denotes generalization relations in a generalization tree, and

‡ denotes specialization relations in a specialization tree.

15

fo

.in

rs

de

AI -Learning from example : Induction

ea

- Version Space Convergence

yr

.m

w

w

Generalization and Specialization leads to Version Space convergence.

,w

ty

or

ab

The key idea in version space learning is that specialization of the general

kr

ha

models and generalization of the specific models may ultimately lead

C

C

R

to just one correct model that matches all observed positive examples and does not match any negative examples. This means :

■ Each time a +ve example is used to specialize the general models, then those specific models that match the -ve example are eliminated.

■ Each time a -ve example is used to generalize the specific models, those general models that fail to match the +ve example are eliminated.

■ Finally, the positive and negative examples may be such that only one general model and one identical specific model survive.

16

fo

.in

rs

de

AI -Learning from example : Induction

ea

• Version Space Learning Algorithm: Candidate - Elimination

yr

.m

w

w

The Candidate - Elimination algorithm finds all describable hypotheses

,w

ty

or

that are consistent with the observed training examples.

ab

kr

The version space method handles +ve and -ve examples symmetrically.

ha

C

C

R

■ Given : A representation language and a set of positive and negative examples expressed in that language.

■ Compute : A concept description that is consistent with all the positive examples and none of the negative examples.

Note : The methodology / algorithm is explained in the next slide.

17

fo

.in

rs

de

AI -Learning from example : Induction

ea

[Continued from previous slide - version space]

yr

.m

w • Version Space Search Algorithm

w

,w

ty

or

‡ Let G be the set of maximally general hypotheses.

ab

kr

ha

Initialize G , to contain most general pattern, a null description.

C

C

‡ Let S be the set of maximally specific hypotheses.

R

Initialize S , to contain one element : the first positive example.

‡ Accept a new training example.

◊ For each new positive training example p do :

1. Delete all members of G that fail to match p ;
2. For every s in S do

If s does not match p , then replace s with its maximally specific generalizations that match p ;

3. Delete from S any hypothesis that is subsumed, a more comprehensive, by other hypothesis in S ;
4. Delete from S any hypothesis more general than some hypothesis in G .

◊ For each new negative training example n do :

1. Delete all members of S that match n ;
2. For each g in G do

If g match n , then replace g with its maximally general specializations that do not match n ;

3. Delete from G any hypothesis more specific than some other hypothesis in G ;
4. Delete from G any hypothesis that is not more general than some hypothesis in S ;

◊ If S and G are both singleton sets, then

1. If they are identical, output their value and halt.

2. If they are different, the training cases were inconsistent; output this result and halt.

3. Else continue accepting new training examples.

‡ The algorithm stops when it runs out of data;

1. If 0 , no consistent description for the data in the language.

2. If 1 , answer (version space converges).

3. If 2 , all descriptions in the language are implicitly included.

18

fo

.in

rs

de

AI -Learning from example : Induction

ea

• Version Space - Problem 1

yr

.m

w Learning the concept of "Japanese Economy Car"

w

,w

ty

[Ref: E. Rich, K. Knight, "Artificial Intelligence", McGraw Hill, Second Edition]

or

ab

kr

Examples 1 to 5 of features

ha

C

Country of Origin, Manufacturer, Color, Decade, Type

C

R

Example Origin Manufacturer Color Decade Type Example Type

1. Japan Honda Blue 1980 Economy Positive

2. Japan Toyota Green 1970 Sports Negative

3. Japan Toyota Blue 1990 Economy Positive

4. USA Chrysler Red 1980 Economy Negative

5. Japan Honda White 1980 Economy Positive

Step by step solution : Symbol for Prune

1. Initialize : to maximally general and maximally specific hypotheses

‡ Initialize G to a null description, means all features are variables.

$G = \{ (?, ?, ?, ?, ?) \}$

‡ Initialize S to a set of maximally specific hypotheses;

Apply a positive example;

Apply Example 1 (Japan, Honda, Blue, 1980, Economy)

$S = \{ (\text{Japan, Honda, Blue, 1980, Economy}) \}$

‡ Resulting Specialization (blue) and Generalization (green)

tree nodes

$G = (?, ?, ?, ?, ?)$

$S = \{ (\text{Japan, Honda, Blue, 1980, Economy}) \}$

These two models represent the most general and the most specific heuristics one might learn.

The actual heuristic to be learned, "Japanese economy car", probably lies between them somewhere within the version space.

19

fo

.in

rs

de

AI -Learning from example : Induction

ea

2. Apply Example 2 (Japan, Toyota, Green, 1970, Sports)

yr

.m

w It is a negative example.

w

,w

ty

‡ Specialize G, to exclude the negative example;

or

ab

i.e., G must be specialized in such a way that the negative example is

kr

ha

C

no longer in the version space.

C

R

The available specializations are :

$G = \{ (?, \text{Honda}, ?, ?, ?), (?, ?, \text{Blue}, ?, ?), (?, ?, ?, 1980, ?),$
 $(?, ?, ?, ?, \text{Economy}) \}$

‡ Resulting Specialization (blue) and Generalization (green)

tree nodes and links

$G = (?, ?, ?, ?, ?)$

$(?, \text{Honda}, ?, ?, ?) (?, ?, \text{Blue}, ?, ?) (?, ?, ?, 1980, ?) (?, ?, ?, ?, \text{Economy})$

$S = \{(\text{Japan}, \text{Honda}, \text{Blue}, 1980, \text{Economy})\}$

20

fo

.in

rs

de

AI -Learning from example : Induction

ea

3. Apply Example 3 (Japan, Toyota, Blue, 1990, Economy)

yr

.m

w It is a positive example.

w

,w

ty

‡ Prune G , to exclude descriptions inconsistent with the positive

or

ab

example; i.e., remove from the G set any descriptions that are

kr

ha

C

inconsistent with the positive example. The new G set is

C

R

$G = \{ (?, ?, \text{Blue}, ?, ?), (?, ?, ?, ?, \text{Economy}) \}$

‡ Generalize S , to include the new example. The new S set is

$S = \{ (\text{Japan}, ?, \text{Blue}, ?, \text{Economy}) \}$

‡ Resulting Specialization (blue) and Generalization (green)

tree nodes and links :

$G = (?, ?, ?, ?, ?)$

$(?, \text{Honda}, ?, ?, ?) (?, ?, \text{Blue}, ?, ?) (?, ?, ?, 1980, ?) (?, ?, ?, ?, \text{Economy})$

$S = \{(\text{Japan}, ?, \text{Blue}, ?, \text{Economy})\}$

$S = \{(\text{Japan}, \text{Honda}, \text{Blue}, 1980, \text{Economy})\}$

‡ At this point, the new G and S sets specify a version space that can be translated roughly in to English as : The concept may be as specific as "Japanese, blue economy car".

21

fo

.in

rs

de

AI -Learning from example : Induction

ea

4. Apply Example 4 (USA, Chrysler, Red, 1980, Economy)

yr

.m

w It is another negative example.

w

,w

ty

‡ Specialize G to exclude the negative example; (but stay consistent

or

ab

with S) i.e.; G set must avoid covering this new example.

kr

ha

C

The new G set is

C

R

$G = \{ (?, ?, \text{Blue}, ?, ?), (\text{Japan}, ?, ?, ?, \text{Economy}) \}$

‡ Prune away all the specific models that match the negative example.

No match found therefore no change in S

$S = \{ (\text{Japan}, ?, \text{Blue}, ?, \text{Economy}) \}$

‡ Resulting Specialization (blue) and Generalization (green)

tree nodes and links :

$G = (?, ?, ?, ?, ?)$

$(?, ?, ?, ?, \text{Economy})$

$(\text{Japan}, ?, \text{Blue}, ?, ?)$

$S = \{(\text{Japan}, ?, ?, ?, \text{Economy})\}$

$S = \{(\text{Japan}, ?, \text{Blue}, ?, \text{Economy})\}$

$S = \{(\text{Japan}, \text{Honda}, \text{Blue}, 1980, \text{Economy})\}$

‡ It is now clear that the car must be Japanese, because all description in the version space contain Japan as origin.

22

fo

.in

rs

de

AI -Learning from example : Induction

ea

5. Apply Example 5 (Japan, Honda, White, 1980, Economy)

yr

.m

w It is positive example.

w

,w

ty

‡ Prune G to exclude descriptions inconsistent with positive example.

or

ab

$G = \{(\text{Japan}, ?, ?, ?, \text{Economy})\}$

kr

ha

C

C

‡ Generalize S to include this positive example.

R

$S = \{(\text{Japan}, ?, ?, ?, \text{Economy})\}$

‡ Resulting Specialization (blue) and Generalization (green)

tree nodes and links :

$G = (?, ?, ?, ?, ?)$

$(?, ?, ?, ?, \text{Economy})$

$(?, ?, \text{Blue}, ?, ?)$

$S = \{(\text{Japan}, ?, ?, ?, \text{Economy})\}$

$S = \{(\text{Japan}, ?, ?, ?, \text{Economy})\}$

$S = \{(\text{Japan}, ?, \text{Blue}, ?, \text{Economy})\}$

$S = \{(\text{Japan}, \text{Honda}, \text{Blue}, 1980, \text{Economy})\}$

‡ Thus, finally G and S are singleton sets and $S = G$.

The algorithm has converged. No more examples needed.

Algorithm stops. $G = (?, ?, ?, ?, ?)$

$G = \{(?, ?, ?, ?, \text{Economy})\}$

$G = \{(\text{Japan}, ?, ?, ?, \text{Economy})\}$

$S = \{(\text{Japan}, ?, ?, ?, \text{Economy})\}$

$S = \{(\text{Japan}, ?, \text{Blue}, ?, \text{Economy})\}$

$S = \{(\text{Japan}, \text{Honda}, \text{Blue}, 1980, \text{Economy})\}$

23

fo

.in

rs

de

AI –Learning from example : Induction

ea

3.3 Decision Trees

yr

.m

w Decision trees are powerful tools for classification and prediction.

w

,w

ty

Decision trees represent rules. Rules are easily expressed so that humans

or

ab

can understand them or even directly use in a database access language

kr

ha

C

like SQL so that records falling into a particular category may be retrieved.

C

R

- Description

■ Decision tree is a classifier in the form of a tree structure where each node is either a leaf or decision node.

‡ leaf node – indicates the target attribute (class) values of examples.

‡ decision node – specify test to be carried on an attribute-value.

■ Decision tree is a typical inductive approach to learn knowledge on classification. The conditions are :

- ‡ Attribute-value description: Object or case must be expressible as a fixed collection of properties or attributes having discrete values.
- ‡ Predefined classes : Categories to which examples are to be assigned must already be defined (ie supervised data).
- ‡ Discrete classes: Classes must be sharply delineated; continuous classes broken up into vague categories as "hard", "flexible", "soft".
- ‡ Sufficient data: Enough training cases to distinguish valid patterns.

- Example : A simple decision tree

A Decision node

A

C=red C=blue K=y Leaf node

A A

$B < 4.5$ $B \geq 4.5$ $B \geq 8.1$ $B < 8.1$

K=y K=y K=y A

C=true C=false

K=y K=y

24

fo

.in

rs

de

AI -Learning from example : Induction

ea

- Algorithm to generate a Decision Tree

yr

.m

w

w

A decision tree program constructs a decision tree T from a set of

,w

ty

or

training cases. The advantage of a decision tree learning is that a

ab

kr

program, rather than a knowledge engineer, elicits knowledge from an

ha

C

C

expert.

R

ID3 Algorithm (Iterative Dichotomiser 3)

ID3 is based on the Concept Learning System (CLS) algorithm, developed J. Ross Quinlan, at the University of Sydney in 1975.

■ Description

‡ ID3 builds a decision tree from a set of "examples".

The "examples" have several attributes.

They belong to a class (yes or no).

‡ Leaf nodes contain the class name.

‡ Decision node is an attribute test .

25

fo

.in

rs

de

AI -Learning from example : Induction

ea

■ Attribute Selection

yr

.m

w How does ID3 decide which attribute is the best?

w

,w

ty

There are different criteria to select which attribute will become a test

or

ab

attribute in a given branch of a tree. A well known criteria is

kr

ha

C

information gain. It uses a log function with a base of 2, to determine

C

R

the number of bits necessary to represent a piece of information.

‡ Entropy measures information content in an attribute.

Shannon defines information entropy in terms of a discrete random variable X , with possible states (or outcomes) $x_1, \dots, x_n$ as:

n

$$H(X) = \sum_{i=1}^n p(x_i) \cdot \log_2 \left(\frac{1}{p(x_i)} \right) = - \sum_{i=1}^n p(x_i) \cdot \log_2 p(x_i)$$

$i=1$

where $p(x_i) = P(X = x_i)$ is the probability of the i th outcome of X .

‡ Information gain of an attribute X with respect to class attribute Y .

Let Y and X are discrete variables that take values in

$\{y_1 \dots y_i \dots y_k\}$ and $\{x_1 \dots x_j \dots x_l\}$.

The information gain of a given attribute X with respect to the class attribute Y is the reduction in uncertainty about the value of Y when we know the value of X , is called $I(Y; X)$.

Uncertainty

Uncertainty about the value of Y is measured by its entropy, $H(Y)$.

uncertainty about the value of Y when we know the value of X is given by the conditional entropy of Y given X , i.e., $H(Y|X)$.

Information Gain of a given attribute X with respect to the class attribute Y is given by : $I(Y; X) = H(Y) - H(Y|X)$. where

k

$H(Y) = - \sum_{i=1}^k p(y_i) \cdot \log_2 p(y_i)$ is the initial entropy in Y ,

l

$H(Y|X) = \sum_{j=1}^l p(x_j) \cdot H(Y|x_j)$ is conditional entropy of Y

given X where $H(Y|x_j) = - \sum_{i=1}^k p(y_i | x_j) \cdot \log_2 p(y_i | x_j)$ is

the entropy in Y given a particular answer x_j .

26

fo

.in

rs

de

AI -Learning from example : Induction

ea

■ Example

yr

.m

w Decide if the weather is amenable to play Golf or Baseball.

w

,w

ty

or

‡ Training Data Collected are Over 2 weeks

ab

kr

ha

Day Outlook Temp Humidity wind Play

C

C

R

1 sunny 85 85 week no

2 sunny 80 90 strong no

3 cloudy 83 78 week yes

4 rainy 70 96 week yes

5 rainy 68 80 week yes

6 rainy 65 70 strong no

7 cloudy 64 65 strong yes

8 sunny 72 95 week no

9 sunny 69 70 week yes

10 rainy 75 80 week yes

11 sunny 75 70 strong yes

12 cloudy 72 90 strong yes

13 cloudy 81 75 week yes

14 rainy 71 85 strong no

‡ Learning set

In the above example, two attributes, the Temperature and Humidity have continuous ranges. ID3 requires them to be discrete like hot, medium, cold, high, normal. Table below indicates the acceptable values.

Attribute Possible Values

Outlook Sunny Cloudy Rainy

Temperature Hot Medium Cold

Humidity High Normal

Wind Strong Weak

Class play no play

Decision n (negative) p (positive)

27

fo

.in

rs

de

AI –Learning from example : Induction

ea

‡ Assign discrete values to the Attributes

yr

.m

w

w

Partition the continuous attribute values to make them discrete,

,w

ty

or

following the key mentioned below.

ab

kr

ha

C

Temperature : Hot (H) 80 to 85 Medium (M) 70 to 75 Cold (C) 64 to 69

C

R

Humidity : High (H) 81 to 96 Normal (N) 65 to 80

Class : Yes (Y) play No (N) no play

Day Outlook Temp Humidity Wind Class (play)

1 Sunny 85 Hot 85 High week no

2 Sunny 80 Hot 90 High strong no

3 Cloudy 83 Hot 78 High week yes

4 Rainy 70 Medium 96 High week yes

5 Rainy 68 Cold 80 Normal week yes

6 Rainy 65 Cold 70 Normal strong no

7 Cloudy 64 Cold 65 Normal strong yes

8 Sunny 72 Medium 95 High week no

9 Sunny 69 Cold 70 Normal week yes

10 Rainy 75 Medium 80 Normal week yes

11 Sunny 75 Medium 70 Normal strong yes

12 Cloudy 72 Medium 90 High strong yes

13 Cloudy 81 Hot 75 Normal week yes

14 Rainy 71 Medium 85 High strong no

28

fo

.in

rs

de

AI -Learning from example : Induction

ea

‡ Attribute Selection

yr

.m

w By applying definitions of Entropy and Information gain

w

,w

ty

or

◇ Entropy : From the definition, When Y is a discrete variables

ab

kr

ha

that take values in $\{y_1 \dots y_i \dots y_k\}$ then the entropy of

C

C

R

Y is given by $H(Y)$

k k

$H(Y) = \sum_{i=1}^k p(y_i) \cdot \log_2 (1/p(y_i)) = - \sum_{i=1}^k p(y_i) \cdot \log_2 p(y_i)$

$i=1$

where $p(y_i)$ is the probability of the i th outcome of Y.

Apply his definition to the example stated before :

Given, S is a collection of 14 examples with 9 YES and 5 NO

then $\text{Entropy}(S) = - (9/14) \log_2 (9/14) - (5/14) \log_2 (5/14)$

$= 0.940$

Note that S is not an attribute but the entire sample set.

Entropy range from 0 ("perfectly classified") to 1 ("totally random").

[Continued in next slide]

29

fo

.in

rs

de

AI -Learning from example : Induction

ea

[Continued from previous slide - Attribute election]

yr

.m

w ◇ Information Gain : From the definition, a given attribute X

w

,w

ty

with respect to the class attribute Y is given by :

or

ab

$I(Y; X) = H(Y) - H(Y|X)$ where

kr

ha

C

k

C

$H(Y) = - \sum p(y_i) \cdot \log_2 p(y_i)$ is the initial entropy in Y,

R

$y_i=1$

I

$H(Y|X) = \sum p(x_j) \cdot H(Y|x_j)$ is conditional entropy of Y

$x_j=1$

I

given X where $H(Y|x_j) = - \sum p(y_i|x_j) \cdot \log_2 p(y_i|x_j)$ is

$x_j=1$

entropy in Y given a particular answer x_j .

Apply this definition to the example stated above :

Information Gain (S, A) of the example set S on attribute A is

$\text{Gain}(S, A) = \text{Entropy}(S) - \sum ((|S_v| / |S|) * \text{Entropy}(S_v))$ where

$\sum$ sum over each value v of all possible values of attribute A

S_v is subset of S for which attribute A has value v

$|S_v|$ is number of elements in S_v

$|S|$ is number of elements in S

Given, S is a set of 14 examples, in which one of the attributes is wind speed. The values of Wind can be Weak or Strong. The classification of these 14 examples are 9 YES and 5 NO. Given, for attribute wind :

8 occurrences of wind = weak and

6 occurrences of wind = strong.

For wind = weak, 6 of the examples are YES and 2 are NO.

For wind = strong, 3 of the examples are YES and 3 are NO.

$\text{Gain}(S, \text{wind}) = \text{Entropy}(S) - (8/14) \cdot \text{Entropy}(S_{\text{weak}})$

$- (6/14) \cdot \text{Entropy}(S_{\text{strong}})$

$= 0.940 - (8/14) \cdot 0.811 - (6/14) \cdot 1.00 = 0.048$

$\text{Entropy}(S_{\text{weak}}) = - (6/8) \cdot \log_2(6/8) - (2/8) \cdot \log_2(2/8) = 0.811$

$$\text{Entropy}(S_{\text{strong}}) = - (3/6) \cdot \log_2(3/6) - (3/6) \cdot \log_2(3/6) = 1.00$$

For each attribute, the gain is calculated and the highest gain is used in the decision node.

30

fo

.in

rs

de

AI -Learning from example : Induction

ea

‡ Step-by-Step Calculations : Ref. Training data stated before

yr

.m

w

w

◇ Step 01 : "example" set S

,w

ty

The set S of 14 examples is with 9 'yes' and 5 'no' then

or

ab

kr

$$\text{Entropy}(S) = - (9/14) \log_2 (9/14) - (5/14) \log_2 (5/14) = 0.940$$

ha

C

C

R

◇ Step 02 : Attribute Outlook

Outlook value can be sunny, cloudy or rainy.

Outlook = sunny is of occurrences 5

Outlook = cloudy is of occurrences 4

Outlook = rainy is of occurrences 5

Outlook = sunny, 2 of the examples are 'yes' and 3 are 'no'

Outlook = cloudy, 4 of the examples are 'yes' and 0 are 'no'

Outlook = rainy, 3 of the examples are 'yes' and 2 are 'no'

$$\text{Entropy}(S_{\text{sunny}}) = - (2/5) \times \log_2(2/5) - (3/5) \times \log_2(3/5) = 0.970950$$

$$\text{Entropy}(S_{\text{cloudy}}) = - (4/4) \times \log_2(4/4) - (0/4) \times \log_2(0/4) = 0$$

$$\text{Entropy}(S_{\text{rainy}}) = - (3/5) \times \log_2(3/5) - (2/5) \times \log_2(2/5) = 0.970950$$

$$\text{Gain}(S, \text{Outlook}) = \text{Entropy}(S) - (5/14) \times \text{Entropy}(\text{Sunny})$$

$$- (4/14) \times \text{Entropy}(\text{Scloud})$$

$$\begin{aligned}
& - (5/14) \times \text{Entropy}(\text{Srainy}) \\
& = 0.940 - (5/14) \times 0.97095059 - (4/14) \times 0 \\
& - (5/14) \times 0.97095059 \\
& = 0.940 - 0.34676 - 0 - 0.34676 \\
& = 0.246
\end{aligned}$$

31

fo

.in

rs

de

AI -Learning from example : Induction

ea

◇ Step 03 : Attribute Temperature

yr

.m

w

w Temp value can be hot, medium or cold.

,w

Temp = hot is of occurrences 4

ty

or

ab

Temp = medium is of occurrences 6

kr

ha

Temp = cold is of occurrences 4

C

C

Temp = hot, 2 of the examples are 'yes' and 2 are 'no'

R

Temp = medium, 4 of the examples are 'yes' and 2 are 'no'

Temp = cold, 3 of the examples are 'yes' and 1 are 'no'

$$\text{Entropy}(\text{Shot}) = - (2/4) \times \log_2(2/4) - (2/4) \times \log_2(2/4) = -0.99999999$$

$$\text{Entropy}(\text{Smedium}) = - (4/6) \times \log_2(4/6) - (2/6) \times \log_2(2/6) = -0.91829583$$

$$\text{Entropy}(\text{Scold}) = - (3/4) \times \log_2(3/4) - (1/4) \times \log_2(1/4) = -0.81127812$$

$$\text{Gain}(\text{S}, \text{Temp}) = \text{Entropy}(\text{S}) - (4/14) \times \text{Entropy}(\text{Shot})$$

$$- (6/14) \times \text{Entropy}(\text{Smedium})$$

$$- (4/14) \times \text{Entropy}(\text{Scold})$$

$$= 0.940 - (4/14) \times 0.99999 - (6/14) \times 0.91829583$$

$$\begin{aligned}
 & - (4/14) \times 0.81127812 \\
 & = 0.940 - 0.2857142 - 0.393555 - 0.2317937 \\
 & = 0.0289366072
 \end{aligned}$$

32

fo

.in

rs

de

AI - Learning from example : Induction

ea

◇ Step 04 : Attribute Humidity

yr

.m

w

w Humidity value can be high, normal.

,w

Humidity = high is of occurrences 7

ty

or

ab

Humidity = normal is of occurrences 7

kr

ha

Humidity = high, 3 of the examples are 'yes' and 4 are 'no'

C

C

Humidity = normal, 6 of the examples are 'yes' and 1 are 'no'

R

$$\text{Entropy(Shigh)} = - (3/7) \times \log_2(3/7) - (4/7) \times \log_2(4/7) = -0.9852281$$

$$\text{Entropy(Snormal)} = - (6/7) \times \log_2(6/7) - (1/7) \times \log_2(1/7) = -0.5916727$$

$$\text{Gain(S, Humidity)} = \text{Entropy(S)} - (7/14) \times \text{Entropy(Shigh)}$$

$$- (7/14) \times \text{Entropy(Snormal)}$$

$$= 0.940 - (7/14) \times 0.9852281 - (7/14) \times 0.5916727$$

$$= 0.940 - 0.49261405 - 0.29583635$$

$$= 0.1515496$$

33

fo

.in

rs

de

AI -Learning from example : Induction

ea

◇ Step 05 : Attribute wind

yr

.m

w

w Wind value can be weak or strong.

,w

Wind = weak is of occurrences 8

ty

or

ab

Wind = strong is of occurrences 6

kr

ha

Wind = weak, 6 of the examples are 'yes' and 2 are 'no'

C

C

Wind = strong, 3 of the examples are 'yes' and 3 are 'no'

R

$$\text{Entropy}(S_{\text{weak}}) = - (6/8) \times \log_2(6/8) - (2/8) \times \log_2(2/8) = 0.811$$

$$\text{Entropy}(S_{\text{strong}}) = - (3/6) \times \log_2(3/6) - (3/6) \times \log_2(3/6) = 1.00$$

$$\text{Gain}(S, \text{Wind}) = \text{Entropy}(S) - (8/14) \times \text{Entropy}(S_{\text{weak}}) -$$

$$(6/14) \times \text{Entropy}(S_{\text{strong}})$$

$$= 0.940 - (8/14) \times 0.811 - (6/14) \times 1.00$$

$$= 0.048$$

34

fo

.in

rs

de

AI -Learning from example : Induction

ea

◇ Step 06 : Summary of results are

yr

.m

w

$$w \text{ Entropy}(S) = 0.940$$

,w

Gain(S, Outlook) = 0.246

ty

or

ab

Gain(S, Temp) = 0.0289366072

kr

ha

Gain(S, Humidity) = 0.1515496

C

C

Gain(S, Wind) = 0.048

R

◇ Step 07 : Find which attribute is the root node.

Gain(S, Outlook) = 0.246 is highest.

Therefore "Outlook" attribute is the decision attribute in the root node.

"Outlook" as root node has three possible values - sunny, cloudy, rain.

S.No Day Outlook Temp Humidity Wind Play

1 11 sunny 75 Medium 70 Normal strong yes

2 2 sunny 80 Hot 90 High strong no

3 1 sunny 85 Hot 85 High week no

4 8 sunny 72 Medium 95 High week no

5 9 sunny 69 Cold 70 Normal week yes

6 12 cloudy 72 Medium 90 High strong yes

7 3 cloudy 83 Hot 78 High week yes

8 7 cloudy 64 Cold 65 Normal strong yes

9 13 cloudy 81 Hot 75 Normal week yes

10 14 rainy 71 Medium 85 High strong no

11 6 rainy 65 Cold 70 Normal strong no

12 10 rainy 75 Medium 80 Normal week yes

13 5 rainy 68 Cold 80 Normal week yes

14 4 rainy 70 Medium 96 High week yes

35

fo

.in

rs

de

AI -Learning from example : Induction

ea

◇ Step 08 : Find which attribute is next decision node.

yr

.m

w

w

Outlook has three possible values,

,w

ty

so root node has three branches (sunny, cloudy, rain).

or

ab

Ssunny = {D1, D2, D8, D9, D11} = 5 "examples". "D" represent "Days".

kr

ha

C

C

With outlook = sunny

R

Gain(Ssunny, Humidity) = 0.970

Gain(Ssunny, Temp) = 0.570

Gain(Ssunny, Wind) = 0.019

Humidity has the highest gain 0.970 is next decision node.

◇ Step 09 : This process goes on until all days data are classified

perfectly or run out of attributes.

36

fo

.in

rs

de

AI -Learning from example : Induction

ea

■ Thus, the classification tree built using ID3 algorithm is shown

yr

.m

w below. It tells if the weather was amenable to play ?

w

,w

ty

or

ab

kr

Outlook

ha

C

C

R

sunny cloudy rainy

Hu midity Yes play Wind

D3 D7 D12 D13

High Normal Strong Weak

No play Yes play No play Yes play

D1 D2 D8 D9 D11 D6 D14 D4 D5

Decision tree

37

fo

.in

rs

de

AI -Learning from example : Induction

ea

■ Applications

yr

.m

w

w

‡ ID3 is easy to use.

,w

ty

or

‡ Its primary use is replacing the expert who would normally

ab

kr

build a classification tree by hand.

ha

C

C

‡ ID3 has been incorporated in a number of commercial rule-

R

induction packages.

‡ Some specific applications include

- ◇ medical diagnosis,
- ◇ credit risk assessment of loan applications,
- ◇ equipment malfunctions by their cause,
- ◇ classification of plant diseases, and
- ◇ web search classification.

38

fo

.in

rs

de

AI – Explanation Based Learning (EBL)

ea

4. Explanation Based Learning (EBL)

yr

.m

w

w

Humans appear to learn quite a lot from one example.

,w

ty

or

Human learning is accomplished by examining particular situations and relating

ab

kr

them to the background knowledge in the form of known general principles.

ha

C

C

This kind of learning is called "Explanation Based Learning (EBL)".

R

4.1 General Approach

EBL is abstracting a general concept from a particular training example.

EBL is a technique to formulate general concepts on the basis of a specific training example. EBL analyses the specific training example in terms of domain knowledge and the goal concept. The result of EBL is an explanation structure, that explains why the training example is an instance of the goal concept. The explanation-structure is then used as the basis for formulating

the general concept.

Thus, EBL provides a way of generalizing a machine-generated explanation of a situation into rules that apply not only to the current situation but to similar ones as well.

4.2 EBL Architecture

The overall architecture of the EBL learning method.

Inputs

Specific Partial

goal / external

problem solution

Knowledge Problem solver

base (under-stander)

New general

concept Explanation

Generalizer

39

fo

.in

rs

de

AI – Explanation Based Learning (EBL)

ea

4.3 EBL System Schematic

yr

.m

w The schematic below shows explanation based learning.

w

,w

ty

or

Explanation Based Learning

ab

kr

ha

Domain Training Operationality

C

C

theory Example criterion

R

Goal Results

Concept

general

trace

Problem trace Operationality

Generalizer

solver pruner

• Input to EBL :

The blocks color yellow are external to EBL.

They are the 4 different kinds of input that EBL algorithm accepts.

1. Training example - Situation description, facts .

An example is a set of facts describing an instance of the goal concept.

2. Goal concept - Some predicate calculus statements .

A high level description of concept that the program is supposed to learn; e.g., the definition of an object "Cup" in function-structure involves functional properties (lift-able, stable, open-vessel . . .) rather than structural features (light, part of, pointing . . .).

3. Domain theory - Inference rules .

It represents facts and rules that constitute what the learner knows.

The facts describe an instance of the goal concept and the rules describe relationships between objects and actions in a domain; e.g., the cup domain includes facts: concavities, bases, and lugs, as well as rules: about lift ability, stability and what makes an open vessel.

4. Operational criterion - description of concepts .

A description in an appropriate form of the final concept.

A predicate from domain theory over concept definition, specifying the form in which the learned concept definition must be expressed.

40

fo

.in

rs

de

AI – Explanation Based Learning (EBL)

ea

■ Description of EBL algorithm

yr

.m

w Given the four inputs just described, the task is to construct an

w

,w

ty

explanation in terms of the domain theory and then determine a set of

or

ab

sufficient conditions under which the explanation holds.

kr

ha

C

Thus, EBL algorithm consisting of two stages:

C

R

1. Explain: Construct an explanation in terms of the domain theory that shows how the training example satisfies the goal concept definition. This explanation must be constructed so that each branch of the explanation structure terminates in an expression that satisfies the operationality criterion.

2. Generalize: Determine a set of sufficient conditions under which the explanation holds, stated in terms that satisfy the operationality criterion. This is accomplished by regressing (back propagating) the goal concept through the explanation structure. The conjunction of the resulting expressions constitutes the desired concept definition.

Algorithm : The step involved are

1. Given an example, an "explanation" is first constructed by applying a problem solver to relate the example to some general domain theory.

The result of this operation is a trace or "proof" of the example with respect to the theory.

2. Next, generalizes the explanation using the method of goal regression. This involves traversing the tree from top to bottom, replacing constants by variables, but just those constants that are not embedded in the rules or facts used to create the proof. The result of this operation is the generalized proof tree.

3. The tree is then pruned by removing irrelevant leaf nodes until no operational predicates appear at internal nodes. The result of this operation is that explanation structure satisfies the operationality criterion that applies to other examples too.

4. Finally, the operational rules are extracted from the general explanation. The definition of the target concept is conjunction of the remaining leaf nodes.

41

fo

.in

rs

de

AI – Explanation Based Learning (EBL)

ea

4.4 Example : "CUP" Generalization problem

yr

.m

w

w

The EBL methodology described before is applied to concept "CUP".

,w

ty

or

The training example, domain theory, goal concept, operability

ab

kr

criterion, and a proof tree generalization using a goal regression

ha

C

C

technique are presented. The logical operators used are

R

 $\wedge$ and; implies; if .. then; if and only if; iff

Goal Concept

 $\text{lift-able}(x) \wedge \text{stable}(x) \wedge \text{open-vessel}(x) \text{ cup}(x)$

Training example

 $\text{colour}(\text{Obj23}, \text{Blue}) \wedge \text{has-part}(\text{Obj23}, \text{Handle16}) \wedge \text{has-part}(\text{Obj23}, \text{Bottom19}) \wedge$ $\text{owner}(\text{Obj23}, \text{Ralph}) \wedge \text{has-part}(\text{Obj23}, \text{Concavity12}) \wedge \text{is}(\text{Obj23}, \text{Light}) \wedge$ $\text{is}(\text{Ralph}, \text{Male}) \wedge \text{isa}(\text{Handle16}, \text{Handle}) \wedge \text{isa}(\text{Bottom19}, \text{Bottom}) \wedge$ $\text{is}(\text{Bottom19}, \text{Flat}) \wedge \text{isa}(\text{Concavity12}, \text{Concavity}) \wedge$ $\text{is}(\text{Concavity12}, \text{Upward-Pointing})$

Domain theory

 $\text{has-part}(x, y) \wedge \text{isa}(y, \text{Concavity}) \wedge \text{is}(y, \text{Upward-Pointing}) \text{ open-vessel}(x)$ $\text{is}(x, \text{Light}) \wedge \text{has-part}(x, y) \wedge \text{isa}(y, \text{Handle}) \text{ liftable}(x)$ $\text{has-part}(x, y) \wedge \text{isa}(y, \text{Bottom}) \wedge \text{is}(y, \text{Flat}) \text{ stable}(x)$

Operability Criterion

The concept definition is expressed in terms of structural features.

The first stage of the EBL process is to show why the training example is an example of a cup. This represents a proof.

This proof is expressed only in terms of the operability criterion and irrelevant details discarded relating to the Owner and Color.

Explanation structure of the cup.

42

fo

.in

rs

de

AI – Explanation Based Learning (EBL)

ea

Explanation Structure of the cup

yr

.m

w The structure below represents the first stage of the EBL process.

w

,w

ty

It proves why the training example is an example of a cup.

or

ab

kr

ha

Cup(Obj1)

C

C

R

Lift-able(Obj1) Stable(Obj1) Open-vessel(Obj1)

I s (Ob j 1,light) i sa (Concavit y1, Co ncavit y)

isa (Handle1, Ha ndle) ha s-part(o bj1, Co ncavity)

has-part (Ob j 1, Ha ndle1) is(Concavit y1, Upw ard- Po int ing)

i sa(Conca vit y1, Fl at)

i sa(Conca vity1, B ott om)

ha s-part (Ob j1, Concavity 1)

Proof tree generalization using a goal regression technique.

The above proof is now generalized by using a goal regression technique. In this example replacing the constants with variables gives the required generalization of the cup.

has-part(x, y) U isa(y, Concavity) U is(y, Upward-Pointing) U

has-part(x, z) U isa(z, Bottom) U is(z, Flat) U has-part(x,w) U
isa(w, Handle) U is(x, Light)

43

fo

.in

rs

de

AI -Learning : Discovery

ea

5. Discovery

yr

.m

w Simon (1966) first proposed the idea that we might explain scientific discovery

w

,w

ty

in computational terms and automate the processes involved on a computer.

or

ab

kr

Project DENDRAL (Feigenbaum 1971) demonstrated this by inferring structures

ha

C

of organic molecules from mass spectra, a problem previously solved only by

C

R

experienced chemists.

Later, a knowledge based program called AM the Automated Mathematician
(Lenat 1977) discovered many mathematical concepts.

After this, an equation discovery systems called BACON (Langley, 1981)
discovered a wide variety of empirical laws such as the ideal gas law. The
research continued during the 1980s and 1990s but reduced because the
computational biology, bioinformatics and scientific data mining have convinced
many researchers to focus on domain-specific methods. But need for research
on general principles for scientific reasoning and discovery very much exists.

Discovery system AM relied strongly on theory-driven methods of discovery.

BACON employed data-driven heuristics to direct its search for empirical laws.

These two discovery programs are illustrated in the next few slides.

44

fo

.in

rs

de

AI -Learning : Discovery

ea

5.1 Theory Driven Discovery

yr

.m

w

w

The Simon's theory driven science, means AI-modeling for theory building.

,w

ty

or

It starts with an existing theory represented in some or all aspects in form

ab

kr

of a symbolic model and one tries to transform the theory to a runnable

ha

C

C

program. One important reason for modeling a theory is scientific discovery

R

in the theory driven approach, this means the discovery of new theoretical conclusions, gaps, or inconsistencies.

Many computational systems have been developed for modeling different types of discoveries. The Logic Theorist (1956) was designed to prove theorems in logic when AI did not exist. Among the more recent systems, the Automated Mathematician AM (Lenat, 1979) is a good example in modeling mathematical discovery.

- AM (Automated Mathematician)

AM is a heuristic driven program that discovers concepts in elementary mathematics and set theory. AM has 2 inputs:

(a) description of some concepts of set theory: e.g. union, intersection;

(b) information on how to perform mathematics. e.g. functions.

AM have successively rediscovered concepts such as :

(a) Integers , Natural numbers, Prime Numbers;

(b) Addition, Multiplication, Factorization theorem ;

(c) Maximally divisible numbers, e.g. 12 has six divisors 1, 2, 3, 4, 6, 12.

[AM is described in the next slide.]

45

fo

.in

rs

de

AI –Learning : Discovery

ea

- How does AM work ?

yr

.m

w

w

AM employs many general-purpose AI techniques.

,w

ty

or

The system has around 115 basic elements such as sets, lists, elementary

ab

kr

relations. The mathematical concepts are represented as frames. Around

ha

C

C

250 heuristic rules are attached to slots in the concepts. The rules present

R

hints as how to employ functions, create new concepts, generalization etc. about activities that might lead to interesting discoveries.

The system operates from an agenda of tasks. It selects the most interesting task as determined by a set of over 50 heuristics. It then performs all heuristics it can find which should help in executing it. The heuristics represented as operators are used to generalize, to specialize or to combine the elementary concepts or relations to make more complex ones. Heuristics can fill in concept slots, check the content of the slots, create new concepts, modify task agenda, interestingness levels, etc. Because it selects the most interesting task to perform at all times, AM is performing the best-first search in a space of mathematical concepts. However, its numerous heuristics (over 200) guide its search very effectively, limiting the number of concepts it creates and improving their mathematical quality.

46

fo

.in

rs

de

AI –Learning : Discovery

ea

5.2 Data Driven Discovery

yr

.m

w

w

Data driven science, in contrast to theory driven, starts with empirical data

,w

ty

or

or the input-output behavior of the real system without an explicitly given

ab

kr

theory. The modeler tries to write a computer program which generates the

ha

C

C

empirical data or input-output behavior of the system. Typically, models are

R

produced in a generate-and-test-procedure. Generate-and-test means

writing program code which tries to model the i-o-behavior of the real

system first approximately and then improve as long as the i-o-behavior

does not correspond to the real system. A family of such discovery models

are known as BACON programs.

- BACON System

Equation discovery is the area of machine learning that develops methods

for automated discovery of quantitative laws, expressed in the form of

equations, in collections of measured data.

BACON is pioneer among equation discovery systems. BACON is a family of

algorithms for discovering scientific laws from data.

BACON.1 discovers simple numeric laws.

BACON.3 is a knowledge based system, has discovered simple empirical

laws like physicists and shown its generality by rediscovering the

Ideal gas law, Kepler's third law, Ohm's law and more.

The next few slides shows how BACON1 rediscovers Kepler's third Law and BACON3 rediscovers Ideal Gas Law.

47

fo

.in

rs

de

AI -Learning : Discovery

ea

- BACON.1 : Discovers simple numeric laws.

yr

.m

w Given a set of observed values about two variables X and Y, BACON.1 finds

w

,w

ty

a function $Y = f(X)$ using four heuristics:

or

ab

■ Heuristic 1 : If Y has value V in several observed cases, make the

kr

ha

C

hypothesis that $Y = V$ in all cases.

C

R

■ Heuristic 2 : If X and Y are linearly related with slope S and intercept I

in several observed cases, then make the hypothesis that $Y = S \cdot X + I$

is in all cases.

■ Heuristic 3 : If X increases as Y decreases, and X and Y are not linearly related define a new term T as the product of X and Y

ie., $T = X \cdot Y$

■ Heuristic 4 : If X increases as Y increases, and X and Y are not linearly related, define a new term T as the division of X by Y

ie., $T = X / Y$

Note : BACON1 iteratively applies these 4 heuristics until a scientific law is discovered. Heuristics 1 and 2 detect linear relationships. Heuristics 3 and 4 detect simple non-linear relationships. Heuristics 1 and 2 produce scientific laws. Heuristics 3 and 4 are intermediate steps.

Example : Rediscovering Kepler's third Law

Kepler's third Law is stated below. Assume the law is not discovered or known.

"The square of the orbital period T is proportional to the cube of the mean distance a from the Sun." i.e., $T^2 = k a^3$, k is constant number, is same for all planets.

If we measure T in years and all distances in "astronomical units AUs" with 1 AU the mean distance between the Earth and the Sun, then if $a = 1$ AU, T is one year, and k with these units just equals 1, i.e. $T^2 = a^3$.

■ Input : Planets, Distance from Sun (D), orbit time Period (P)

Planet D P

Mercury 0.382 0.241

Venus 0.724 0.616

Earth 1.0 1.0

Mars 1.524 1.881

Jupiter 5.199 11.855

Saturn 9.539 29.459

[continued in next slide]

48

fo

.in

rs

de

AI -Learning : Discovery

ea

■ Apply heuristics 1 to 4 : (Iteration 1)

yr

.m

w

w

Try heuristic 1: not applicable, neither D nor P is constant

,w

ty

Try heuristic 2: not applicable, no linear relationship

or

ab

Try heuristic 3: not applicable, D increasing P not decreasing

kr

ha

C

Try heuristic 4: applicable, D increases as P increase,

C

R

so add new variable D/P to the data set.

Adding new variable D/P to the data set:

Planet D P D/P

Mercury 0.382 0.241 1.607

Venus 0.724 0.616 1.175

Earth 1.0 1.0 1.0

Mars 1.524 1.881 0.810

Jupiter 5.199 11.855 0.439

Saturn 9.539 29.459 0.324

■ Apply heuristics 1 to 4 : (Iteration 2)

Try heuristic 1: not applicable, D/P is not constant

Try heuristic 2: not applicable, no linear relationship between D/P and D or P

Try heuristic 3: applicable, D/P decreases as D increases and as P

increases, so the system could add two new

variables: $D \times (D/P) = D^2/P$ or $P \times (D/P) = D$

but D already exists, so add new variable D²/P

Adding new variable D²/P to the data set:

Planet D P D/P D²/P

Mercury 0.382 0.241 1.607 0.622

Venus 0.724 0.616 1.175 0.851

Earth 1.0 1.0 1.0 1.0

Mars 1.524 1.881 0.810 1.234

Jupiter 5.199 11.855 0.439 2.280

Saturn 9.539 29.459 0.324 3.088

49

fo

.in

rs

de

AI -Learning : Discovery

ea

■ Apply heuristics 1 to 4 : (Iteration 3)

yr

.m

w

w Try heuristic 1: not applicable, D²/P is not constant

,w

Try heuristic 2: not applicable, D²/P is not linearly related with any other

ty

or

variable

ab

Try heuristic 3: applicable, D_2/P decreases as D/P increases, so add the

kr

ha

new variable: $(D_2/P) \times (D/P) = D_3/P^2$

C

C

R

Adding new variable D_3/P^2 to the data set:

Planet D P D/P D_2/P D_3/P^2

Mercury 0.382 0.241 1.607 0.622 1.0

Venus 0.724 0.616 1.175 0.851 1.0

Earth 1.0 1.0 1.0 1.0 1.0

Mars 1.524 1.881 0.810 1.234 1.0

Jupiter 5.199 11.855 0.439 2.280 1.0

Saturn 9.539 29.459 0.324 3.088 1.0

■ Apply heuristics 1 to 4 : (Iteration 4)

Try heuristic 1: applicable,, D_3/P^2 is constant

■ Conclusion : D_3/P^2

This is Kepler's third law. (took about 20 years to discover it !)

■ A limitation of BACON.1

It works only for target equation relating at most two variable.

50

fo

.in

rs

de

AI -Learning : Discovery

ea

BACON.3 :

yr

.m

w BACON.3 is a knowledge based system production system that discovers

w

,w

ty

empirical laws. The main heuristics detect constancies and trends in data,
or
ab

and lead to the formulation of hypotheses and the definition of theoretical
kr
ha
C

terms. The program represents information at varying levels of description.
C
R

The lowest levels correspond to direct observations, while the highest
correspond to hypotheses that explain everything so far observed. BACON.3
is built on top of BACON.1.

■ It starts with a set of variables for a problem. For example, to derive
the ideal gas law, it started with four variables, p , V , n , T .

‡ p - gas pressure,

‡ V - gas volume,

‡ T - gas temperature,

‡ n - is the number of moles.

■ Values from experimental data are inputted.

■ BACON holds some constant and try to notice trends in the data.

■ Finally draws inferences. Recall $pV/nT = k$ where k is a constant.

■ BACON has also been applied to Kepler's 3rd law, Ohm's law,
conservation of momentum and Joule's law.

51

fo

.in

rs

de

AI -Learning : Discovery

ea

• Example :

yr

.m

w Rediscovering the ideal gas law $pV/nT = 8.32$, where p is the pressure on a

w

,w

ty

gas, n is the number of moles, T is the temperature and V the volume of

or

ab

the gas.

kr

[The step-by-step complete algorithm is not given like previous

ha

C

example, but the procedure is explained below]

C

R

■ At the first level of description we hold $n = 1$ and $T = 300$ and vary p and V . Choose V to be the dependent variable.

■ At this level, BACON discovers the law $pV = 2496.0$.

■ Now the program examines this phenomenon :

when $n = 1$ and $T = 310$ then $pV = 2579.2$. Similarly,

when $n = 1$ and $T = 320$ then $pV = 2662.4$.

■ At this point, BACON has enough information to relate the values of pV and the temperature T . These terms are linearly related with an intercept of 0, making the ratio pV/T equal to 8.32.

■ Now the discovery system can vary its third independent term.

while $n = 2$, the pV/T is found to be 16.64,

while $n = 3$, the pV/T is found to be 24.96.

■ When it compares the values of n and pV/T , BACON finds another linear relation with a zero intercept. The resulting equation, $pV/nT = 8.32$, is equivalent to the ideal gas law.

52

fo

.in

rs

de

AI -Learning : Clustering

ea

6. Clustering

yr

.m

w Clustering is a way to form natural groupings or clusters of patterns.

w

,w

ty

Clustering is often called an unsupervised learning.

or

ab

kr

ha

Example : Three natural groups of data points, i.e., 3 natural clusters.

C

Y

C

R

• • •

• • • • •

• • • • •

■ ■ ■ ■

■ ■ ■

X

■ Clustering is one of the most utilized data mining techniques.

■ The data have no target attribute.

The data set is explored to find some intrinsic structures in them.

■ Unlike Classification where the task is to learn to assign instances to predefined classes, in Clustering no predefined classification is required.

The task is to learn a classification from the data.

53

fo

.in

rs

de

AI -Learning : Clustering

ea

6.1 Distance Functions

yr

.m

w

w

■ Euclidean geometry is study of relationships between angles and

,w

ty

or

distances in space. Mathematical spaces may be extended to any

ab

kr

dimension, called an n-dimensional Euclidean space or an n-space.

ha

C

C

R

Let R denote the field of real numbers. The space of all n -tuples of real numbers forms an n -dimensional vector space over R , denoted by R^n .

An element of R^n is written as $X = (x_1, x_2, \dots, x_n)$, where x_i is a real number. Similarly the other element $Y = (y_1, y_2, \dots, y_n)$.

The vector space operations on R^n are defined by

$X + Y = (x_1 + y_1, x_2 + y_2, \dots, x_n + y_n)$ and

$aX = (ax_1, ax_2, \dots, ax_n)$

The standard inner product (ie dot product) on R^n , given by

n

$X \cdot Y = \sum_{i=1}^n (x_i y_i + x_2 y_2 + \dots + x_n y_n)$ is a real number.

This product defines a distance function (or metric) on R^n by

n^2

$d(X, Y) = \|X - Y\| = \sum_{i=1}^n (x_i - y_i)^2$

The (interior) angle θ between x and y is then given by

$X \cdot Y$

$\theta = \cos^{-1} \left(\frac{\|X\| \|Y\|}{X \cdot Y} \right)$

The inner product of X with itself is always nonnegative.

n

$\|X\|^2 = \sum_{i=1}^n (x_i - y_i)^2$

54

fo

.in

rs

de

AI -Learning : Clustering

ea

■ Euclidean distance or Euclidean metric is the "ordinary" distance

yr

.m

w between two points that one would measure with a ruler. The Euclidean

w

,w

ty

distance between two points $P = (p_1, p_2, \dots, p_i, \dots, x_n)$ and $Q = (q_1, q_2,$

or

ab

$\dots, q_i, \dots, q_n)$ in Euclidean n-space, is defined as :

kr

ha

C

C

n

R

$$(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2 = \sum_{i=1}^n (p_i - q_i)^2$$

Example : 3-dimensional distance

For two 3D points, $P = (p_x, p_y, \dots, p_z)$ and $Q = (q_x, q_y, \dots, q_z)$

The Euclidean 3-space, is computed as :

$$(p_x - q_x)^2 + (p_y - q_y)^2 + (p_z - q_z)^2$$

■ Manhattan distance also known as city block distance between two points in an Euclidean space with fixed cartesian coordinate system is the sum of the lengths of the projections of the line segment between the points onto the coordinate axes.

Example : In a plane, the Manhattan distance between the point P1 with coordinates (x_1, y_1) and the point P2 with coordinates (x_2, y_2) is $|x_1 - x_2| + |y_1 - y_2|$

Manhattan versus Euclidean distance:

The red, blue, and yellow lines represent Manhattan distance. They all are of same length as 12.

The green line represent Euclidian distance of length $6\sqrt{2} \approx 8.48$.

55

fo

.in

rs

de

AI -Learning : Clustering

ea

■ Minkowski Metric

yr

.m

w

w

Let X_i and X_j are data points (vectors) then Minkowski distance

,w

ty

or

between these two data points is :

ab

kr

h h h $1/h$

$\text{dist}(X_i, X_j) = ((x_{i1} - x_{j1})^h + (x_{i2} - x_{j2})^h + \dots + (x_{ir} - x_{jr})^h)^{1/h}$

ha

C

C

where h is a positive integer.

R

‡ Euclidean distance IF $h = 2$

$\text{dist}(X_i, X_j) = ((x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{ir} - x_{jr})^2)^{1/2}$

$= ((x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{ir} - x_{jr})^2)^{1/2}$

Weighted Euclidean distance

$\text{dist}(X_i, X_j) = (w_1(x_{i1} - x_{j1})^2 + w_2(x_{i2} - x_{j2})^2 + \dots + w_r(x_{ir} - x_{jr})^2)^{1/2}$

‡ Manhattan distance IF $h = 1$

$\text{dist}(X_i, X_j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ir} - x_{jr}|$

‡ Squared Euclidean distance: to place progressively greater weight on data points that are further apart.

$\text{dist}(X_i, X_j) = (x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{ir} - x_{jr})^2$

‡ Chebychev distance: to define two data points as "different" if they are different on any one of the attributes.

$\text{dist}(X_i, X_j) = \max(|x_{i1} - x_{j1}|, |x_{i2} - x_{j2}|, \dots, |x_{ir} - x_{jr}|)$

56

fo

.in

rs

de

AI -Learning : Clustering

ea

6.2 K-Mean Clustering

yr

.m

w

w

k-means clustering is an algorithm to classify or to group objects based on

,w

ty

or

attributes/features into K number of group.

ab

kr

K is positive integer number.

ha

C

C

The grouping is done by minimizing the sum of squares of distances

R

between data and the corresponding cluster centroid.

K-Mean Clustering algorithm

Start The algorithm consists of 3

steps. First take any random

Number of object as initial centroid, then

cluster P

the three steps mentioned

below are iterated until

Centroid converge.

No Object

END Step-1 Determine centroid

Move to group

Objects distance

from Centroid

coordinate

Step-2 Determine distance of each

object to the centroids

Grouping based on

min distance Step-3 Group the object based on

minimum distance

The next slide explains step by step the complete algorithm

57

fo

.in

rs

de

AI -Learning : Clustering

ea

Example : K-Mean Clustering

yr

.m

w 4.5

w

Objects : 4 medicines as A, B, C, D.

,w

4.0 D

ty

3.5

or

Attributes : 2 as X is weight & Y is PH 3.0 C

ab

kr

2.5

Y

ha

Objects Attributes 2.0

C

X Y 1.5

C

R

1.0 A B

A 1 1 $\diamond \diamond$

0.5

B 2 1 0

C 4 3 0 1 2 3 4 5

D 5 4 X

Initial value of centroids:

Suppose medicine A and medicine B 4.5

4.0 D

be first centroids. If P1 and P2 denote 3.5

3.0 C

coordinates of the centroids, 2.5 $\diamond$

Y 2.0

then $P1 = (1, 1)$ and $P2 = (2, 1)$.

1.5

1.0 A B

Let centroid P1 be for cluster group-1 0.5 $\diamond \diamond$

0

Let centroid P2 be for cluster group-2. 0 1 2 3 4 5

X

58

fo

.in

rs

de

AI -Learning : Clustering

ea

1. Iteraion 0

yr

.m

w

w (a) Objects Clusters centers stated before : Objects : A , B, C, D

,w

Group-1 has center $P1 = (1, 1)$; Attributes : X and Y

ty

or

ab

Group-2 has center $P2 = (2, 1)$; A B C D

kr

ha

X 1 2 4 5

C

Y 1 1 3 4

C

R

(b) Calculate distances between cluster center to each object

□ 1st, calculate the Euclidean distances from cetroid P1 to each point A, B, C, D. It is the 1st row of the distance matrix.

□ 2nd, calculate the Euclidean distances from cetroid P2 to each point A, B, C, D. It is the 2nd row of the distance matrix.

The ways to calculate just two distance matrix elements D13 and D23 are :

$$D_{13} = (C_x - P_{1x})^2 + (C_y - P_{1y})^2 = (4 - 1)^2 + (3 - 1)^2 = 3.61$$

$$D_{23} = (C_x - P_{2x})^2 + (C_y - P_{2y})^2 = (4 - 2)^2 + (3 - 1)^2 = 2.83$$

Similarly calculate other elements D_{11} , D_{12} , D_{14} , D_{21} , D_{22} , D_{24}

(c) Distance matrix becomes

0 1 3.61 5 1st row indicates group-1 cluster

$D_0 =$

2nd row indicates group-2 cluster

1 0 2.83 4

A B C D

(d) Objects clustering into groups:

Assign group to each object based on the minimum distance. Thus,

medicine A is assigned to group 1;

medicine B is assigned to group 2,

medicine C is assigned to group 2, and

medicine D is assigned to group 2.

Group Matrix : matrix element is 1 if the object is assigned to that group

1 0 0 0 1st row as group 1

$G_0 =$

0 1 1 1 2nd row as group 2

A B C D

59

fo

.in

rs

de

AI -Learning : Clustering

ea

2. Iteration 1 :

yr

.m

w The cluster groups have new members. Compute the new centroids of

w

,w

ty

each group. Repeat the process of iteration indicated below.

or

ab

Group-1 has one member A, the

kr

ha

C

centroid remains as $P1 = (1, 1)$. 4.5

C

4.0 D

R

Group-2 now has 3 members B, 3.5

3.0 C

C, D, so centroid is the average $2.5 \diamond$

Y 2.0

of their coordinates: 1.5

$2+4+5$ $1+3+5$ 1.0 A B

$P2 = (,) \diamond \diamond$

3 0.5

3

0

$= (11/3, 9/3)$ 0 1 2 3 4 5

X

$= (3.66, 3)$

(a) Objects Clusters centers stated above : Objects : A , B, C, D

Group-1 has center $P1 = (1, 1)$; Attributes : X and Y

Group-2 has center $P2 = (3.66, 3)$; A B C D

X 1 2 4 5

Y 1 1 3 4

(b) Calculate distances between cluster center to each object

[The method is same as in Iteration 0 (b)]

□ 1st, calculate the Euclidean distances from centroid $P1$ to each point A, B,

C, D You get $D11, D12, D13, D14$ as the 1st row of the distance matrix.

□ 2nd, calculate the Euclidean distances from centroid $P2$ to each point A, B,

C, D. You get $D21, D22, D23, D24$ as the 2nd row of the distance matrix.

(c) Distance matrix becomes

0 1 3.61 5 1st row indicates group-1 cluster

$D1 =$

2nd row indicates group-2 cluster

3.14 2.36 0.47 1.89

(d) Objects clustering into groups:

Assign group to each object based on the minimum distance.

medicine A no change, remains in group 1;

medicine B moves to group 1,

medicine C no change, remains in group 2 ,

medicine D no change, remains in group 2.

Group Matrix : matrix element is 1 if the object is assigned to that group

1 1 0 0 1st row indicates group-1 cluster

G1 =

2nd row indicates group-2 cluster

0 0 1 1

A B C D

60

fo

.in

rs

de

AI -Learning : Clustering

ea

3. Iteration 2 :

yr

.m

w The cluster groups have new members. Compute the new centroids of

w

,w

ty

each group. Repeat the process of iteration indicated below.

or

ab

Group-1 has 2 members, so the

kr

ha

C

new centroid is the average 4.5

C

4.0 D

R

coordinate among A and B : 3.5

1+2 1+1 3.0 C

$P1 = (,) = (1.5, 1) 2.5 \diamond$

2 2 Y 2.0

Group-2 has 2 members, so the 1.5

1.0 A B

new centroid is the average $0.5 \diamond \diamond$

coordinate among C and D : 0

0 1 2 3 4 5

4+5 3+4

$P2 = (,) = (4.5, 3.5)$ X

2 2

(a) Objects Clusters centers stated above : Objects : A , B, C, D

Group-1 has center $P1 = (1.5, 1)$; Attributes : X and Y

Group-2 has center $P2 = (4.5, 3.5)$; A B C D

X 1 2 4 5

Y 1 1 3 4

(b) Calculate distances between cluster center to each object

[The method is same as in Iteration 0 (b) or Iteration 1 (b)]

□ 1st, calculate the Euclidean distances from centroid $P1$ to each point A, B, C, D You get $D11, D12, D13, D14$ as the 1st row of the distance matrix.

□ 2nd, calculate the Euclidean distances from centroid $P2$ to each point A, B, C, D. You get $D21, D22, D23, D24$ as the 2nd row of the distance matrix.

(c) Distance matrix becomes

0.5 0.5 3.2 4.61 1st row indicates group-1 cluster

$D2 =$

2nd row indicates group-2 cluster

4.3 3.54 0.71 0.71

(d) Objects clustering into groups:

Assign group to each object based on the minimum distance.

medicine A no change, remains in group 1;

medicine B no change, remains in group 1;

medicine C no change, remains in group 2 ,

medicine D no change, remains in group 2.

Group Matrix : matrix element is 1 if the object is assigned to that group

1 1 0 0 1st row indicates group-1 cluster

2

$G =$

2nd row indicates group-2 cluster

0 0 1 1

A B C D

61

fo

.in

rs

de

AI –Learning : Clustering

ea

4. Finally, the result state that Group Matrix $G2 = G1$.

yr

.m

w This means the grouping of objects in this last iteration and the one

w

,w

ty

before does not change anymore. Thus, the computation of the k-mean

or

ab

clustering has reached its stability and no more iteration is needed.

kr

ha

C

Results of final grouping are :

C

R

Objects Feature 1 (X) Feature 2 (Y) Cluster Group

Weight index pH

Medicine A 1 1 1

Medicine B 2 1 1

Medicine C 4 3 2

Medicine A 3 4 2

62

fo

.in

rs

de

AI –Learning by analogy

ea

7. Analogy

yr

.m

w

w

Learning by analogy means acquiring new knowledge about an input entity by

,w

ty

or

transferring it from a known similar entity.

ab

kr

This technique transforms the solutions of problems in one domain to the

ha

C

C

solutions of the problems in another domain by discovering analogous states and

R

operators in the two domains.

Example: Infer by analogy the hydraulics laws that are similar to Kirchoff's laws.

$$Q_a = 3 \quad Q_b = 9$$

$$I_1 \quad I_2$$

$$I_3 = I_1 + I_2$$

$$Q_c = ?$$

Hydraulic Problem Kirchoff's First Law

The other similar examples are :

■ Pressure Drop is like Voltage Drop

■ Hydrogen Atom is like our Solar System :

The Sun has a greater mass than the Earth and attracts it, causing the Earth to revolve around the Sun. The nucleus also has a greater mass than the electron and attracts it. Therefore it is plausible that the electron also revolves around the nucleus.

63

fo

.in

rs

de

AI -Learning - Neural net and Genetic learning

ea

8. Neural net and Genetic Learning

yr

.m

w The Neural net, the Genetic learning and the Reinforcement learning are the

w

,w

ty

or

Biology-inspired AI techniques. In this section the Neural net and Genetic

ab

kr

ha

learning are briefly described.

C

C

R

8.1 Neural Net (NN)

A neural net is an artificial representation of the human brain that tries to simulate its learning process. An artificial neural network (ANN) is often just called a "neural network" (NN).

■ Neural Networks model a brain learning by example.

■ Neural networks are structures "trained" to recognize input patterns.

■ Neural networks typically take a vector of input values and produce a vector of output values; inside, they train weights of "neurons".

■ A Perceptron is a model of a single 'trainable' neuron.

64

fo

.in

rs

de

AI -Learning - Neural net

ea

■ Perceptron is a model of a single 'trainable' neuron.

yr

.m

w

w

,w

X1

ty

w1

or

ab

X2 w2

kr

ha

C

w3

C

X3 T Y

R

wn

Xn

‡ Inputs are $x_1, x_2, \dots, x_n$ as real numbers or boolean values depending on the problem.

‡ $w_1, w_2, \dots, w_n$ are weights of the edges and are real valued.

‡ T is the threshold and is real valued.

‡ Y is the output and is boolean.

■ If the net input which is $(w_1 x_1 + w_2 x_2 + \dots + w_n x_n)$ is greater than the threshold T then output Y is 1 else 0.

■ Neural network uses supervised learning, in which inputs and outputs are known and the goal is to build a representation of a function that will approximate the input to output mapping.

65

fo

.in

rs

de

AI -Learning - Genetic learning

ea

8.2 Genetic Learning

yr

.m

w

w

Genetic algorithms (GAs) are part of evolutionary computing.

,w

ty

or

GA is a rapidly growing area of AI.

ab

kr

ha

C

■ Genetic algorithms are implemented as a computer simulation,

C

R

where techniques are inspired by evolutionary biology.

■ Mechanics of biological evolution:

‡ Every organism has a set of rules, describing how that organism is built, and encoded in the genes of an organism.

‡ The genes are connected together into long strings called chromosomes.

‡ Each gene represents a specific trait (feature) of the organism and has several different settings, e.g. setting for a hair color gene may be black or brown.

‡ The genes and their settings are referred as an organism's genotype.

‡ When two organisms mate they share their genes. The resultant offspring may end up having half the genes from one parent and half from the other. This process is called cross over.

‡ A gene may be mutated and expressed in the organism as a completely new trait.

■ Thus, Genetic Algorithms are a way of solving problems by mimicking processes the nature uses ie Selection, Crosses over, Mutation and Accepting to evolve a solution to a problem.

66

fo

.in

rs

de

AI –Learning - Genetic learning

ea

■ Genetic Algorithm steps

yr

.m

w

w

(1) [Start] Generate random population of n chromosomes (Encode

,w

ty

or

suitable solutions for the problem)

ab

kr

(2) [Fitness] Evaluate the fitness $f(x)$ of each chromosome x in the population.

R

(3) [New population] Create a new population by repeating following steps until the new population is complete.

(a) [Selection] Select two parent chromosomes from a population according to their fitness.

(b) [Crossover] With a crossover probability, cross over the parents to form new offspring (children). If no crossover is performed, the offspring would be the exact copy of parents.

(c) [Mutation] With a mutation probability, mutate the new offspring at each locus (position in chromosome).

(d) [Accepting] Place new offspring in the new population.

(4) [Replace] Use new generated population for a further run of the algorithm.

(5) [Test] If the end condition is satisfied, stop, and return the best solution in the current population.

(6) [Loop] Go to step 2.

■ Genetic Algorithms does unsupervised learning - the right answer is not known beforehand.

67

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

9. Reinforcement Learning

yr

.m

w

w

Reinforcement learning refers to a class of problems in machine learning which

,w

ty

or

postulate an agent exploring an environment.

ab

kr

□ The agent perceives its current state and takes actions.

ha

C

C

□ The environment, in return, provides a reward positive or negative.

R

□ The algorithms attempt to find a policy for maximizing cumulative reward for the agent over the course of the problem.

In other words, the definition of Reinforcement learning is :

" A computational approach to learning whereby an agent tries to maximize the total amount of reward it receives when interacting with a complex, uncertain environment."

Reinforcement learning is "a way of programming agents by reward and punishment without needing to specify how the task is to be achieved".

[Kaelbling, Littman, & Moore, 96]

The RL Problem and the tasks are illustrated in the next few slides.

68

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

9.1 Reinforcement Learning (RL) Problem

yr

.m

w Reinforcement Learning is a computational approach to understanding

w

,w

ty

and automating goal-directed learning and decision-making.

or

ab

- Agent - Environment interaction

kr

ha

C

RL emphasizes learning from interaction with its environment.

C

R

□ RL uses a formal framework defining the interaction between a learning agent and its environment in terms of states, actions, and rewards.

□ The state-action pairs form value functions that does efficient search in the space of policies guided by evaluations of all policies.

Agent - Environment interaction in Reinforcement learning

Agent

state s_t reward r_t

action a_t

$r_t + 1$

Environment

s_{t+1}

‡ The agent and environment interact in a sequence of discrete time steps, $t = 0, 1, 2, 3, 4, \dots$

‡ At each discrete time t , the agent (learning system) observes state $s_t \in S$ and chooses action $a_t \in A$

‡ Then agent receives an immediate numerical reward $r_{t+1} \in \mathcal{R}$ and

the state changes to s_{t+1}

‡ At each time step, the agent implements a mapping from states to probabilities of selecting each possible action.

‡ This mapping is called the agent's policy π_t , where $\pi_t(s, a)$ is the probability at $a_t = a$ if $s_t \in S$.

‡ RL methods specify how the agent changes its policy as a result of its experience.

‡ The agent's goal, is to maximize the total amount of reward it receives over the long run.

69

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

• Key Features of RL

yr

.m

w

w

■ The learner is not told what actions to take, instead it finds out

,w

ty

or

what to do by trial-and-error search.

ab

kr

ha

C

■ The environment is stochastic; ie., the behavior is non-deterministic

C

R

means a "state" does not fully determine its next "state".

■ The reward may be delayed, so the learner may need to sacrifice short-term gains for greater long-term gains.

■ The learner has to balance between the need to explore its environment and the need to exploit its current knowledge.

70

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

9.2 Reinforcement Learning tasks

yr

.m

w

w

,w

Most RL research is conducted within the mathematical framework of

ty

or

Markov decision processes (MDPs).

ab

kr

ha

□ The MDPs involve a decision-making agent interacting with its

C

C

R

environment so as to maximize the cumulative reward it receives over time.

□ The agent perceives aspects of the environment's state and selects actions.

□ The agent may estimate a value function and use it to construct better and better decision-making policies over time.

Thus the elements of Reinforcement Learning are

‡ Markov Decision Process

‡ Policy

‡ Agent's Learning Task

‡ Reward

‡ Value Functions

Each of these elements are explained in the next few slides. However, first the Stochastic process, Markov chain, and few notations are explained.

71

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

• Recall : Stochastic processes, Markov chain , and the notations.

yr

.m

w

w

,w

■ Stochastic processes

ty

or

A physical stochastic processes is a sequences of events or path

ab

kr

ha

governed by probabilistic laws.

C

C

R

A stochastic process $X = \{ X(t), t \in T \}$ is a collection of random variables. For each t in the index set T , the $X(t)$ is a random variable. The t is often interpreted as time and $X(t)$ the state of the process at time t .

If the index set T is a countable set,

then we have a discrete-time stochastic process;

Else if T is a non-countable continuous set,

then we have a continuous-time stochastic process.

Any realization of X is named a sample path, which can be discrete or continuous.

■ Markov chain

A Markov chain, is a stochastic process with the Markov property.

Markov property means, given the present state, the future states are independent of the past states; means the future states depends only on its present state, and not on any past states.

In other words, the description of the present state fully captures all the information that could influence the future evolution of the process.

Future states will be reached through a probabilistic process instead of a deterministic one.

At each step the system may change its state from the current state to another state, or remain in the same state, according to a certain probability distribution. The changes in state are called transitions, and the probabilities associated with various state-changes are called transition probabilities.

72

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

■ Notations followed

yr

.m

w

w

$,w$

t discrete time step

t_y

or

T final time step of an episode

ab

kr

ha

s_t state perceived by agent at time t

C

C

a_t action selected by agent at time t

R

r_{t+1} reward received by agent at time t

s_{t+1} resulting next state perceived by agent at time t

$r_{t+1} \ r_{t+2} \ r_{t+3}$

$s_t \ s_{t+1} \ s_{t+2} \ s_{t+3}$

$a_t \ a_{t+1} \ a_{t+2} \ a_{t+3}$

R_t return (cumulative discounted reward) following t

$R_t(n)$ n - step return

$R_t \lambda$ λ - return

π policy, decision-making rule

$\pi(s)$ action taken in state s under deterministic policy π

$\pi(s,a)$ probability of taking action a in state s under stochastic policy π

S set of all non-terminal states

S^+ set of all states, including the terminal state

$A(s)$ set of actions possible in state s

$p_{ss'}$ probability of transition from state s to state s' under action a

$r_{ss'}$ expected immediate reward on transition from state s to state s' under action a

73

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

- Markov System

yr

.m

w

w

A system that can be in one of several specified states with a specified

,w

ty

or

time step, and at each step the system will randomly change states

ab

kr

or remain where it is according to some fix probability. The probability

ha

C

C

of going from State i to State j is a fix number p_{ij} , called the

R

transition probability.

Example : Laundry Detergent Switching

A market analyst for Detergents is interested to know whether consumers prefer powdered detergents or liquid detergents.

Two market surveys taken 1 year apart revealed that :

□ 20% of all powdered detergent users had switched to liquid 1 year later, while the rest were using powder.

□ 10% of liquid detergent users had switched to powder 1 year later, with the rest using liquid.

Analysis of the survey

1. Every year a consumer may be in one of two possible states:

State 1, he is a powdered detergent user;

State 2, he is a liquid detergent user.

2. There is a basic time step of 1 year;

3. If a consumer during a given year is in State 1, a powdered detergent user, then the probability that he will be in State 2, a liquid detergent user, in the next year is $20\% = 0.2$ denoted by $p_{12} = 0.2$.

Therefore probability for him to remain in State 1 is thus $(1 - 0.2 = 0.8)$, denoted by $p_{11} = 0.8$.

4. If a consumer during a given year is in State 2, a liquid detergent user, then the probability that he will be in State 1, a powered detergent

user, in the next year is $10\% = 0.1$ denoted by $p_{21} = 0.1$.

Therefore probability for him to remain in State 2 is thus $(1 - 0.1 = 0.9)$, denoted by $p_{22} = 0.9$.

The state transition diagram is shown in the next slide.

74

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

[Continued from previous slide - Example : Laundry Detergent Switching]

yr

.m

w

w

We can draw a state transition diagram or use a matrix to represent these

,w

ty

or

probabilities called transition probabilities.

ab

kr

ha

0.2 0.8 0.2

C

C

0.8 1 2 0.9 $P=$

R

0.1 0.9 0.1

State transition diagram State transition matrix

The Figure 1 and 2 : the numbers are transition probabilities, from State i to State j , a number p_{ij} , and the arrows show the switching directions.

75

fo

.in

rs

de

AI –Learning - Reinforcement learning

ea

■ Markov State Transition

yr

.m

w State transition probabilities mentioned before are explained here.

w

,w

ty

Transition probability :

or

ab

If a Markov system is in state i , there is a fixed probability p_{ij} of it

kr

ha

C

going into state j in the next time step. The probability p_{ij} is called

C

R

a transition probability.

Transition diagram :

A Markov system is illustrated by a state transition diagram, that shows all the states and transition probabilities.

Example : Transition Diagram (Missing arrows indicate zero probability.)

0.8

0.2 1 2

0.4

0.6

0.5 0.3

3

0.1

■ Transition matrix :

The matrix P whose ij th entry is p_{ij} is called transition matrix associated with the system. The entries in each row add up to 1.

Example :

A 2×2 transition matrix P

To

1 2

1 P_{11} P_{12} Arrows originating in state 1

From

2 P21 P22 Arrows originating in state 2

The 3x3 Matrix corresponding to Transition Diagram

To

1 2 3

1 0.2 0.8 0

From 2 0.4 0 0.6

3 0.5 0.35 0.15

76

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

- Markov Decision Processes (MDPs)

yr

.m

w Markov decision processes (MDPs), provide a mathematical framework

w

,w

ty

for modeling decision-making in situations where outcomes are partly

or

ab

kr

random and partly under the control of the decision maker.

ha

C

C

R

Description :

□ MDP is a discrete time stochastic control process characterized by a set of states;

□ In each state there are several actions from which the decision maker must choose. For a state s and an action a , a state transition function $P_a(s)$ determines the transition probabilities to next state.

□ The decision maker earns a reward r for each state transition. The state transitions possess the Markov property, ie., given the state of the MDP at time t , the transition probabilities to the state at time $t+1$

are independent of all previous states or actions.

□ MDPs are an extension of Markov chains. The differences are the addition of actions (allowing choice) and the rewards (giving motivation). If there are only one action, or if the action to take were fixed for each state, a Markov decision process would reduce to a Markov chain.

r_{t+1} r_{t+2} r_{t+3}

s_t s_{t+1} s_{t+2} s_{t+3}

a_t a_{t+1} a_{t+2} a_{t+3}

State transitions, Actions and Rewards

‡ Set of states S

‡ Set of actions $A(s)$ available in each state S

‡ Markov assumption: s_{t+1} and r_{t+1} depend only on s_t , a_t and not on anything that happened before t

‡ Rewards: on transition from state S to state S' under action a

$r_{ss'} = E r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'$

‡ Transition probabilities : from state S to state S' under action a

$p_{ss'} = P(s_{t+1} = s' \mid s_t = s, a_t = a)$

77

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

• Agent's Learning Task

yr

.m

w Agent execute actions in the environment.

w

,w

ty

or

Observe results, and learn policy which is a strategy (a way of behaving).

ab

kr

ha

$\pi : S \times A \rightarrow [0, 1]$ with $\pi(s, a) = P(a_t = a \mid s_t = s)$

C

C

R

If the policy is deterministic, then write more simply as

$\pi : S \rightarrow A$ with $\pi(s) = a$

giving the action chosen in the state S .

- Policy

It defines the learning agent's way of behaving at a given time.

A mapping from states to actions (deterministic policy), or the distributions over actions (stochastic policy).

■ It is a mapping from perceived states of the environment to actions to be taken in those states.

■ It in some cases may be a simple function or lookup table; it may involve extensive computation like a search process.

■ It is the core of a reinforcement learning agent; it alone is sufficient to determine behavior.

■ In general it may be stochastic (random).

78

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

- Reward Function

yr

.m

w The function defines the goal in a reinforcement learning problem.

w

,w

ty

or

■ Maps each perceived state (or state-action pair) of the environment

ab

kr

ha

to a single number, indicating intrinsic desirability of that state.

C

C

R

■ Indicates what is good in an immediate sense.

■ Tells what are the good and bad events for the agent;

Example : a biological system identify rewards high with pleasure and low with pain

■ RL agent's sole objective is to maximize the total reward it receives in the long run.

■ Rewards serves a basis for altering the policy;

Example : an action selected by the policy if followed by low reward, then the policy may be changed to select some other action in that situation.

■ In general, the reward function may be stochastic.

79

fo

.in

rs

de

AI -Learning - Reinforcement learning

ea

- Maximize Reward

yr

.m

w Agent's goal is to maximize reward it receives in long run.

w

,w

ty

"Agent-Environment" interaction is generally a sequence of episodes.

or

ab

If the interaction result into a sequence of separate episodes, it is known

kr

ha

C

as episodic tasks. If the interaction does not break into identifiable

C

R

episodes, but continue without limit is called continuing tasks.

■ If Episodic task then the simplest return R_t is sum of rewards

$$R_t = r_{t+1} + r_{t+2} + \dots + r_T,$$

where T is the time when a terminal state is reached.

■ If Continuing tasks then the discounted return R_t

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=1}^{\infty} \gamma^{k-1} r_{t+k},$$

$$\gamma = 1$$

where parameter γ is discount factor, $0 \leq \gamma \leq 1$.

■ If Average-reward tasks then return R_t

$$R_t = \lim_{T \rightarrow \infty} (1/T) (r_{t+1} + r_{t+2} + \dots + r_T)$$

$$T \rightarrow \infty$$

• Value Functions

The value of state s under policy π is the expected return when starting from

s and choosing actions according to π :

$$V_{\pi}(s) = E \{ R | s = s \} = E \{ \sum_{k=1}^{\infty} \gamma^{k-1} r_{t+k} | s_t = s \}$$

$$V_{\pi}(s) = E \{ R | s = s \} = E \{ \sum_{k=1}^{\infty} \gamma^{k-1} r_{t+k} | s_t = s \}$$

$$V_{\pi}(s) = E \{ R | s = s \} = E \{ \sum_{k=1}^{\infty} \gamma^{k-1} r_{t+k} | s_t = s \}$$

80

fo

.in

rs

de

AI -Learning - References

ea

10. References : Textbooks

yr

.m

w

w

1. "Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill

,w

companies Inc., Chapter 17, page 447-484.

ty

or

ab

kr

2. "Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig,

ha

(2002), Prentice Hall, Chapter 18-21, page 649-788.

C

C

R

3. "Computational Intelligence: A Logical Approach", by David Poole, Alan Mackworth,

and Randy Goebel, (1998), Oxford University Press, Chapter 11, page 397-438.

4. "Artificial Intelligence: Structures and Strategies for Complex Problem Solving", by George F. Luger, (2002), Addison-Wesley, Chapter 10- 13, page 385-570.

5. "AI: A New Synthesis", by Nils J. Nilsson, (1998), Morgan Kaufmann Inc., Chapter 10, Page 163-178.

6. "Artificial Intelligence: Theory and Practice", by Thomas Dean, (1994), Addison-Wesley, Chapter 5, Page 179-254

7. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

81

fo

.in

rs

de

ea

yr

Artificial Intelligence - Introduction : AI Course Lecture 1- 6, notes, slides

.m

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

www.myreaders.info

kr

ha

C

C

R

Introduction

Artificial Intelligence

Introduction to Artificial Intelligence, topics : Definitions, goals, approaches, techniques, and branches; Intelligent behavior, understanding AI, hard or strong AI, soft or weak AI, cognitive science. General, engineering and science based AI Goals. AI approaches - cognitive science, laws of thought, turing test, rational

agent. AI Techniques that make system to behave as intelligent - describe and match, goal reduction, constraint satisfaction, tree searching, generate and test, rule based systems. Biology-inspired AI techniques - neural networks, genetic algorithms, reinforcement learning. Branches of AI - logical AI, search in AI, pattern recognition, knowledge representation, inferencing, common sense knowledge and reasoning, learning, planning, epistemology, ontology, heuristics, genetic programming. Applications of AI - game playing, speech recognition, understanding natural language, computer vision, expert systems.

fo

.in

rs

de

ea

Introduction

yr

.m

w

w

,w

Artificial Intelligence

ty

or

ab

kr

ha

Topics

C

C

R

(Lectures 01, 02, 03, 04, 05, 06 6 hours) Slides

1. Definitions 03-10

Artificial Intelligence, Intelligence, Intelligent behavior, Understanding AI, Hard or Strong AI, Soft or Weak AI, Cognitive Science.

2. Goals of AI 11-12

General AI Goal, Engineering based AI Goal, Science based AI Goal.

3. AI Approaches 13-16

Cognitive science, Laws of thought, Turing Test, Rational agent.

4. AI Techniques 17-32

Techniques that make system to behave as Intelligent

Describe and match, Goal reduction, Constraint satisfaction, Tree

Searching, Generate and test, Rule based systems,

Biology-inspired AI techniques

Neural Networks, Genetic Algorithms, Reinforcement learning.

5. Branches of AI 33-45

Logical AI, Search in AI, Pattern Recognition, Knowledge Representation,

Inference, Common sense knowledge and reasoning, Learning, Planning,

Epistemology, Ontology, Heuristics, Genetic programming.

6. Applications of AI 46-50

Game playing, Speech Recognition, Understanding Natural Language,

Computer Vision, Expert Systems.

7. References 51

02

fo

.in

rs

de

ea

yr

Introduction

.m

w

w

,w

ty

What is Artificial Intelligence ?

or

ab

kr

ha

• John McCarthy, who coined the term Artificial Intelligence in 1956,

C

C

R

defines it as "the science and engineering of making intelligent machines", especially intelligent computer programs.

• Artificial Intelligence (AI) is the intelligence of machines and the branch

of computer science that aims to create it.

- Intelligence is the computational part of the ability to achieve goals in the world. Varying kinds and degrees of intelligence occur in people, many animals and some machines.
- AI is the study of the mental faculties through the use of computational models.
- AI is the study of : How to make computers do things which, at the moment, people do better.
- AI is the study and design of intelligent agents, where an intelligent agent is a system that perceives its environment and takes actions that maximize its chances of success.

03

fo

.in

rs

de

AI - Definitions

ea

1. Definitions

yr

.m

w

w

1.1 Artificial Intelligence (AI)

,w

ty

or

ab

The definitions of AI outlined in textbooks

kr

ha

C

C

(a) 'The exciting new effort to make (b) 'The study of mental faculties through
R

computers think ... machines with the use of computational models'
minds, in the full and literal sense' (Charniak and McDermott, 1985)
(Haugeland, 1985)

'The automation of activities that we 'The study of the computations that

associate with human thinking, make it possible to perceive, reason, activities such as decision-making, and act' (Winston, 1992)
 problem solving, learning ...'
 (Bellman, 1978)

(C) 'The art of creating machines that (d) 'A field of study that seeks to explain perform functions that require and emulate intelligent behavior in intelligence when performed by terms of computational processes' people' (Kurzweil, 1990) (Schalkoff, 1990)

'The study of how to make computers 'The branch of computer science that do things at which, at the moment, is concerned with the automation of people are better' (Rich and Knight, intelligent behavior' (Luger and 1991) Stubblefield, 1993)

■ The definitions on the top,
 (a) and (b) are concerned with reasoning, whereas those on the bottom,
 (c) and (d) address behavior.

■ The definitions on the left,
 (a) and (c) measure success in terms of human performance, whereas those on the right,
 (b) and (d) measure ideal concept of intelligence called rationality.

Note : A system is rational if it does the right thing.

04

fo

.in

rs

de

AI - Definitions

ea

1.2 Intelligence

yr

.m

w

w

,w

□ Relate to tasks involving higher mental processes.

ty

or

Examples:

ab

kr

ha

creativity, solving problems, pattern recognition, classification, learning,

C

C

R

induction, deduction, building analogies, optimization, language

processing, knowledge and many more.

□ Intelligence is the computational part of the ability to achieve goals.

05

fo

.in

rs

de

AI - Definitions

ea

1.3 Intelligent Behavior

yr

.m

w

w

,w

□ Perceiving one's environment,

ty

or

ab

kr

□ Acting in complex environments,

ha

C

C

R

□ Learning and understanding from experience,

□ Reasoning to solve problems and discover hidden knowledge,

□ Knowledge applying successfully in new situations,

□ Thinking abstractly, using analogies,

□ Communicating with others, and more like

□ Creativity, Ingenuity, Expressive-ness, Curiosity.

06

fo

.in

rs

de

AI - Definitions

ea

1.4 Understanding AI

yr

.m

w

w

,w

□ How knowledge is acquired, represented, and stored;

ty

or

ab

kr

□ How intelligent behavior is generated and learned;

ha

C

C

R

□ How motives, emotions, and priorities are developed and used;

□ How sensory signals are transformed into symbols;

□ How symbols are manipulated to perform logic, to reason about past,
and plan for future;

□ How mechanisms of intelligence produce the phenomena of illusion,
belief, hope, fear, dreams, kindness and love.”

07

fo

.in

rs

de

AI - Definitions

ea

1.5 Hard or Strong AI

yr

.m

w

w

,w

■ Generally, artificial intelligence research aims to create AI that can

ty

or

ab

replicate human intelligence completely.

kr

ha

C

C

R

■ Strong AI refers to a machine that approaches or supersedes human intelligence,

◇ If it can do typically human tasks,

◇ If it can apply a wide range of background knowledge and

◇ If it has some degree of self-consciousness.

■ Strong AI aims to build machines whose overall intellectual ability is indistinguishable from that of a human being.

08

fo

.in

rs

de

AI - Definitions

ea

1.6 Soft or Weak AI

yr

.m

w

w

,w

■ Weak AI refers to the use of software to study or accomplish specific

ty

or

ab

problem solving or reasoning tasks that do not encompass the full

kr

ha

C

range of human cognitive abilities.

C

R

- Example : a chess program such as Deep Blue.

- Weak AI does not achieve self-awareness; it demonstrates wide range of human-level cognitive abilities; it is merely an intelligent, a specific problem-solver.

09

fo

.in

rs

de

AI - Definitions

ea

1.7 Cognitive Science

yr

.m

w

w

,w

- Aims to develop, explore and evaluate theories of how the mind

ty

or

ab

works through the use of computational models.

kr

ha

C

C

R

- The important is not what is done but how it is done; means intelligent behavior is not enough, the program must operate in an intelligent manner.

- Example : The Chess programs are successful, but say little about the ways humans play chess.

10

fo

.in

rs

de

Goals of AI

ea

2. Goals of AI

yr

.m

w

w

,w

- The definitions of AI gives four possible goals to pursue :

ty

or

ab

kr

1. Systems that think like humans. 2. Systems that think rationally.

ha

C

C

3. Systems that act like humans 4. Systems that act rationally

R

- Traditionally, all four goals have been followed and the approaches were:

Human-like Rationally

Think (1) Cognitive science Approach (2) Laws of thought Approach

Act (3) Turing Test Approach (4) Rational agent Approach

- Most of AI work falls into category (2) and (4).

11

fo

.in

rs

de

Goals of AI

ea

- General AI Goal

yr

.m

w

w

□ Replicate human intelligence : still a distant goal.

,w

ty

or

□ Solve knowledge intensive tasks.

ab

kr

ha

□ Make an intelligent connection between perception and action.

C

C

R

□ Enhance human-human, human-computer and computer to computer interaction / communication.

Engineering based AI Goal

□ Develop concepts, theory and practice of building intelligent machines

□ Emphasis is on system building.

Science based AI Goal

□ Develop concepts, mechanisms and vocabulary to understand biological intelligent behavior.

□ Emphasis is on understanding intelligent behavior.

12

fo

.in

rs

de

AI Approaches

ea

3. AI Approaches

yr

.m

w

w

The approaches followed are defined by choosing goals of the computational

,w

ty

or

model, and basis for evaluating performance of the system.

ab

kr

ha

C

3.1 Cognitive science : Think human-like

C

R

- An exciting new effort to make computers think; that it is, the machines with minds, in the full and literal sense.
- Focus is not just on behavior and I/O, but looks at reasoning process.
- Computational model as to how results were obtained.
- Goal is not just to produce human-like behavior but to produce a sequence of steps of the reasoning process, similar to the steps followed by a human in solving the same task.

13

fo

.in

rs

de

AI Approaches

ea

3.2 Laws of Thought : Think Rationally

yr

.m

w

w

- The study of mental faculties through the use of computational models;

,w

ty

or

that it is, the study of the computations that make it possible to

ab

kr

perceive, reason, and act.

ha

C

C

R

- Focus is on inference mechanisms that are provably correct and guarantee an optimal solution.
- Develop systems of representation to allow inferences to be like "Socrates is a man. All men are mortal. Therefore Socrates is mortal."

- Goal is to formalize the reasoning process as a system of logical rules and procedures for inference.
- The issue is, not all problems can be solved just by reasoning and inferences.

14

fo

.in

rs

de

AI Approaches

ea

3.3 Turing Test : Act Human-like

yr

.m

w

w

- The art of creating machines that perform functions requiring

,w

ty

or

intelligence when performed by people; that it is the study of, how to

ab

kr

make computers do things which at the moment people do better.

ha

C

C

- Focus is on action, and not intelligent behavior centered around

R

representation of the world.

- A Behaviorist approach, is not concerned with how to get results but to the similarity to what human results are.

- Example : Turing Test

◇ 3 rooms contain: a person, a computer, and an interrogator.

◇ The interrogator can communicate with the other 2 by teletype (to avoid the machine imitate the appearance or voice of the person).

◇ The interrogator tries to determine which is the person and which is the machine.

◇ The machine tries to fool the interrogator to believe that it is the human, and the person also tries to convince the interrogator that it

is the human.

◇ If the machine succeeds in fooling the interrogator, then conclude that the machine is intelligent.

■ Goal is to develop systems that are human-like.

15

fo

.in

rs

de

AI Approaches

ea

3.4 Rational Agent : Act Rationally

yr

.m

w

w

■ Tries to explain and emulate intelligent behavior in terms of

,w

ty

or

computational processes; that it is concerned with the automation of

ab

kr

intelligence.

ha

C

C

R

■ Focus is on systems that act sufficiently if not optimally in all situations;

■ It is passable to have imperfect reasoning if the job gets done.

■ Goal is to develop systems that are rational and sufficient.

16

fo

.in

rs

de

AI Techniques

ea

4. AI Techniques

yr

.m

w

w

Various techniques that have evolved, can be applied to a variety of AI tasks.

,w

ty

or

The techniques are concerned with how we represent, manipulate and reason

ab

kr

with knowledge in order to solve problems.

ha

C

C

R

Example

- Techniques, not all "intelligent" but used to behave as intelligent

- Describe and match □ Goal reduction

- Constraint satisfaction □ Tree Searching

- Generate and test □ Rule based systems

- Biology-inspired AI techniques are currently popular

- Neural Networks □ Genetic Algorithms

- Reinforcement learning

17

fo

.in

rs

de

AI Techniques

ea

4.1 Techniques that make system to behave as "Intelligent"

yr

.m

w

w

,w

- Describe and Match

ty

or

ab

■ Model is a description of a system's behavior.

kr

ha

C

C

■ Finite state model consists of a set of states, a set of input events and

R

the relations between them. Given a current state and an input event you can determine the next current state of the model.

■ Computation model is a finite state machine. It includes of a set of states, a set of start states, an input alphabet, and a transition function which maps input symbols and current states to a next state.

■ Representation of computational system include start and end state descriptions and a set of possible transition rules that might be applied. Problem is to find the appropriate transition rules.

■ Transition relation: If a pair of states (S, S') is such that one move takes the system from S to S' , then the transition relation is represented by $S \Rightarrow S'$

■ State-transition system is called deterministic if every state has at most one successor; it is called non-deterministic if at least one state has more than one successor.

■ Examples of some possible transitions between states are shown for the Towers of Hanoi puzzle.

18

fo

.in

rs

de

AI Techniques

ea

■ Puzzle : Towers of Hanoi with only 2 disks

yr

.m

w

w

,w

Solve the puzzle :

ty

or

ab

Initial state Goal state

kr

ha

C

C

R

Move the disks from the leftmost post to the rightmost post while never putting a larger disk on top of a smaller one; move one disk at a time, from one peg to another; middle post can be used for intermediate storage. Play the game in the smallest number of moves possible .

■ Possible state transitions in the Towers of Hanoi puzzle with 2 disks.

[1, 2] [] []

[2] [1] [] [2] [] [1]

[] [1] [2] [] [2] [1]

[] [] [1, 2] [1] [] [2] [1] [2] [] [] [1, 2] []

Towers of Hanoi puzzle

■ Shortest solution is the sequence of transitions from the top state downward to the lower left.

19

fo

.in

rs

de

AI Techniques

ea

• Goal Reduction

yr

.m

w

w

,w

■ Goal-reduction procedures are a special case of the procedural

ty

or

ab

representations of knowledge in AI; an alternative to declarative, logic-

kr

ha

based representations.

C

C

R

■ The process involves the hierarchical sub-division of goals into sub-goals, until the sub-goals which have an immediate solution are reached and said “goal has been satisfied”.

■ Goal-reduction process is illustrated in the form of AND/OR tree drawn upside-down.

◇ Goal levels : Higher-level goals are higher in the tree, and lower-level goals are lower in the tree.

◇ Arcs are directed from a higher-to-lower level node represents the reduction of higher-level goal to lower-level sub-goal.

◇ Nodes at the bottom of the tree represent irreducible action goals.

■ An AND-OR tree/graph structure can represent relations between goals and sub-goals, alternative sub-goals and conjoint sub-goals.

[example in the next slide]

20

fo

.in

rs

de

AI Techniques

ea

■ Example Goal Reduction

yr

.m

w

w

AND-OR tree/graph structure to represent facts such as “enjoyment”,

,w

ty

or

“earning/save money”, “old age” etc.

ab

kr

ha

C

Improve enjoyment of life

C

R

OR

Provide for old age Improve standard
of living

AND Work less hard

Save money Earn more money

OR

Improve productivity

Go on strike

AND-OR tree/graph structure

The above AND-OR tree/graph structure describes

◇ Hierarchical relationships between goals and subgoals

The “going on strike” is a sub-goal of “earning more money”, is a
sub-goal of “improving standard of living”, is a sub-goal of
“improving enjoyment of life”.

◇ Alternative ways of trying to solve a goal

The “going on strike” and “increasing productivity” are alternative
ways of trying to “earn more money” (increase pay).

e.g.: “improving standard of living” and “working less hard” are
alternative ways of trying to “improve enjoyment of life”.

◇ Conjoint sub-goals

To “provide for old age”, not only need to “earn more money”, but
as well need to “save money”.

21

fo

.in

rs

de

AI Techniques

ea

• Constraint Satisfaction Techniques

yr

.m

w

w

,w

■ Constraint is a logical relation among variables. e.g. “circle is inside

ty

or

ab

the square” – The constraints relate objects without precisely specifying

kr

ha

their positions; moving any one, the relation is still maintained.

C

C

R

■ Constraint satisfaction is a process of finding a solution to a set of constraints – the constraints express allowed values for variables and finding solution is evaluation of these variables that satisfies all constraints.

■ Constraint Satisfaction Problem (CSP) and its solution

◇ A Constraint Satisfaction Problem (CSP) consists of :

‡ Variables, a finite set $X = \{x_1, \dots, x_n\}$,

‡ Domain, a finite set D_i of possible values which each variable x_i can take,

‡ Constraints, a set of values that the variables can simultaneously satisfy the constraint (e.g. $D_1 \neq D_2$)

◇ A solution to a CSP is an assignment of a value from its domain to every variable satisfying every constraint; that could be :

‡ one solution, with no preference as to which one,

‡ all solutions,

‡ an optimal, or a good solution - Constraint Optimization Problem (COP).

22

fo

.in

rs

de

AI Techniques

ea

[continued from Constraint Satisfaction Techniques]

yr

.m

w

w

■ Constraint satisfaction has application in Artificial Intelligence,

,w

Programming Languages, Symbolic Computing, Computational Logic.

ty

or

ab

kr

■ Example 1 : N-Queens puzzle

ha

C

C

R

Problem : Given any integer N , place N queens on $N \times N$ chessboard

satisfying constraint that no two queens threaten each other.

(a queen threatens other queens on same row, column and diagonal).

Solution : To model this problem

◇ Assume that each queen is in different column;

◇ Assign a variable R_i ($i = 1$ to N) to the queen in the i -th column indicating the position of the queen in the row.

◇ Apply "no-threatening" constraints between each couple R_i and R_j of queens and evolve the algorithm.

◇ Example : 8 - Queens puzzle

a b c d e f g h a b c d e f g h

8 8 8 8

7 7 7 7

6 6 6 6

5 5 5 5

4 4 4 4

3 3 3 3

2 2 2 2

1 1 1 1

a b c d e f g h a b c d e f g h

Unique solution 1 Unique solution 2

◇ The eight queens puzzle has 92 distinct solutions. If solutions that differ only by symmetry operations (rotations and reflections) of the board are counted as one. The puzzle has 12 unique solutions. Only two solutions are presented above.

23

fo

.in

rs

de

AI Techniques

ea

■ Example 2 : Map Coloring

yr

.m

w

w

Problem : Given a map (graph) and a number of colors, the problem is

,w

ty

or

to assign colors to those areas in the map (nodes) satisfying the

ab

kr

constraint that no adjacent nodes (areas) have the same color assigned

ha

C

C

to them.

R

Solution : To model this Map Coloring problem

◇ Label each node of the graph with a variable (domain corresponding to the set of colors);

◇ Introduce the non-equality constraint between two variables labeling adjacent nodes.

◇ A 4 - color map

◇ The "Four Color Theorem", states that 4 - colors are sufficient to color any map so that regions sharing a common border receive different colors.

24

fo

.in

rs

de

AI Techniques

ea

- Tree Searching

yr

.m

w

w

■ Many problems (e.g. goal reduction, constraint networks) can be

,w

ty

or

described in the form of a search tree. A solution to the problem is

ab

kr

obtained by finding a path through this tree.

ha

C

C

■ A search through the entire tree, until a satisfactory path is found, is

R

called exhaustive search.

■ Tree search strategies:

◇ Depth-first search

* Assumes any one path is as good as any other path.

* At each node, pick an arbitrary path and work forward until a solution is found or a dead end is reached.

* In the case of a dead end - backtrack to the last node in the tree where a previously unexplored path branches off, and test this path.

* Backtracking can be of two types :

- Chronological backtracking : undo everything as we move back "up" the tree to a suitable node.

- Dependency directed backtracking : only withdraw choices that "matter" (ie those on which dead end depends).

The four other types of search strategies are :

◇ Hill climbing ◇ Breadth-first search

* Like depth first but involving * Look for a solution amongst some quantitative decision on all nodes at a given level the "most likely" path to before proceeding to the next. follow at each node.

◇ Beam search ◇ Best-first search

* Like breadth first (level by * Like beam search but only level) but selecting only those proceeding from one "most

N nodes at each level that are likely" node at each level.

"most likely" to lead to a solution.

25

fo

.in

rs

de

AI Techniques

ea

- Generate and Test (GT)

yr

.m

w

w

■ Most algorithms for solving Constraint Satisfaction Problems (CSPs)

,w

ty

or

search systematically through the possible assignments of values.

ab

kr

◇ CSP algorithms guarantee to find a solution, if one exists, or to

ha

C

C

prove that the problem is unsolvable.

R

◇ the disadvantage is that they take a very long time to do so.

■ Generate-and-test method

The method first guesses the solution and then tests whether this solution is correct, means solution satisfies the constraints.

◇ This paradigm involves two processes:

- * Generator to enumerate possible solutions (hypotheses).

- * Test to evaluate each proposed solution

◇ The algorithm is

Generate labels

Test satisfaction

Disadvantages

- ◇ Not very efficient; generates many wrong assignments of values to variables which are rejected in the testing phase.
- ◇ Generator leaves conflicting instantiations and it generates other assignments independently of the conflict.
- ◇ For better efficiency GT approach need to be supported by backtracking approach.

Example: Opening a combination lock without knowing the combination.

26

fo

.in

rs

de

AI Techniques

ea

- Rule-Based Systems (RBSs)

yr

.m

w

w

- Rule-based systems are simple and successful AI technique.

,w

ty

or

ab

- ◇ Rules are of the form: IF <condition> THEN <action>.

kr

ha

C

- ◇ Rules are often arranged in hierarchies ("and/or" trees).

C

R

- ◇ When all conditions of a rule are satisfied the rule is triggered.

- RBS Components : Working Memory, Rule Base, Interpreter.

Change Interpreter

Conditions

Observed Rules

Working

Data Memory

Rule

Base

Rule-Based System

Each of these 3 components are described in the next slide

27

fo

.in

rs

de

AI Techniques

ea

■ RBS components - Description

yr

.m

w

w

◇ Working Memory (WM)

,w

ty

or

‡ Contains facts about the world observed or derived from a rule;

ab

kr

stored as a triplet < object, attribute, values >

ha

C

C

e.g. < car, color, red > : "The color of my car is red".

R

‡ Contains temporary knowledge about problem-solving session.

‡ Can be modified by the rules.

◇ Rule Base (RB)

‡ RB contains rules; each rule is a step in a problem solving.

‡ Rules are domain knowledge and modified only from outside.

‡ Rule syntax is IF <condition> THEN <action>

e.g. IF <(temperature, over, 20>

THEN <add (ocean, swimable, yes)>

‡ If the conditions are matched to the working memory and if

fulfilled then rule may be fired.

‡ RB actions are :

"Add" fact(s) to WM;

"Remove" fact(s) from WM;

"Modify" fact(s) in WM;

◇ Interpreter

‡ It is the domain independent reasoning mechanism for RBS.

‡ It selects rule from Rule Base and applies by performing action.

‡ It operates on a cycle:

Retrieval - Finds the rules that matches the current WM;

Refinement - Prunes, reorders and resolves conflicts;

Execution - Executes the actions of the rules in the Conflict Set,
then applies the rule by performing action.

28

fo

.in

rs

de

AI Techniques

ea

4.2 Biology-Inspired AI Techniques

yr

.m

w

w

• Neural Networks (NN)

,w

ty

or

ab

■ Neural Networks model a brain learning by example.

kr

ha

C

■ Neural networks are structures trained to recognize input patterns.

C

R

■ Neural networks typically take a vector of input values and produce a
vector of output values; inside, they train weights of "neurons".

■ A Perceptron is a model of a single 'trainable' neuron, shown below :

◇ $x_1, x_2, \dots, x_n$ are inputs as real

X1

w1 numbers or boolean values depends

X2 w2

on problem.

w3

X3 T Y $\diamond$ w1, w2, ..., wn are weights of the

wn edges and are real valued.

Xn $\diamond$ T is the threshold and is real valued.

Perceptron Model $\diamond$ y is the output and is boolean.

If the net input which is $w_1 x_1 + w_2 x_2 + \dots + w_n x_n$ is greater than the threshold T then output y is 1 else 0.

■ Neural networks use supervised learning, in which inputs and outputs are known and the goal is to build a representation of a function that will approximate the input to output mapping.

29

fo

.in

rs

de

AI Techniques

ea

- Genetic Algorithms (GA)

yr

.m

w

w

■ GAs are part of evolutionary computing, a rapidly growing area of AI.

,w

ty

or

ab

■ Genetic algorithms are implemented as a computer simulation, where

kr

ha

techniques are inspired by evolutionary biology.

C

C

R

■ Mechanics of biological evolution

◇ Every organism has a set of rules, describing how that organism is built, and encoded in the genes of an organism.

◇ The genes are connected together into long strings called chromosomes.

◇ Each gene represents a specific trait (feature) of the organism and has several different settings, e.g. setting for a hair color gene may be black or brown.

◇ The genes and their settings are referred as an organism's genotype.

◇ When two organisms mate they share their genes. The resultant offspring may end up having half the genes from one parent and half from the other. This process is called cross over.

◇ A gene may be mutated and expressed in the organism as a completely new trait.

■ Thus, Genetic Algorithms are a way of solving problems by mimicking processes, the nature uses, Selection, Crosses over, Mutation and Accepting to evolve a solution to a problem.

30

fo

.in

rs

de

AI Techniques

ea

■ Genetic Algorithm Steps

yr

.m

w

w

(1) [Start] Generate random population of n chromosomes (Encode

,w

ty

or

suitable solutions for the problem)

ab

kr

(2) [Fitness] Evaluate the fitness $f(x)$ of each chromosome x in the

ha

C

C

population.

R

(3) [New population] Create a new population by repeating following steps until the new population is complete.

(a) [Selection] Select two parent chromosomes from a population according to their fitness.

(b) [Crossover] With a crossover probability, cross over the parents to form new offspring (children). If no crossover was performed, offspring is the exact copy of parents.

(c) [Mutation] With a mutation probability, mutate new offspring at each locus (position in chromosome).

(d) [Accepting] Place new offspring in the new population.

(4) [Replace] Use new generated population for a further run of the algorithm.

(5) [Test] If the end condition is satisfied, stop, and return the best solution in current population.

(6) [Loop] Go to step 2.

■ Genetic Algorithms does unsupervised learning – the right answer is not known beforehand.

31

fo

.in

rs

de

AI Techniques

ea

• Reinforcement Learning (RL)

yr

.m

w

w

■ RL is learning from interaction with an environment; from the

,w

ty

or

consequences of action, rather than from explicit teaching.

ab

kr

■ RL is conducted within the mathematical framework of Markov decision

ha

C

C

processes (MDPs).

R

■ The basic RL model consists : a set of “environment states - S ” ;
a set of “actions - A ”; and a set of scalar “rewards - R ”.

■ The decision-making agent interacts with its environment so as to
maximize the cumulative reward it receives over time. The steps are:

◇ At each time t , the agent perceives its environment state s_t and
the set of possible actions $A(s_t)$.

◇ It chooses an action $a \in A(s_t)$ and receives from the environment
the new state s_{t+1} and a reward r_{t+1} .

◇ Based on these interactions, the agent develops a policy $\pi : S \rightarrow A$
which maximizes the quantity $R = r_0 + r_1 + \dots + r_n$ for MDPs.

■ RL methods focus on the kind of learning and decision making
problems that people face in their normal, everyday lives.

32

fo

.in

rs

de

Branches of AI

ea

5. Branches of AI

yr

.m

w

w

• Logical AI

,w

ty

or

ab

■ Logic is a language for reasoning; a collection of rules used while

kr

ha

doing logical reasoning.

C

C

R

■ Types of logic

◇ Propositional logic - logic of sentences

◇ predicate logic - logic of objects

◇ logic involving uncertainties

◇ Fuzzy logic - dealing with fuzziness

◇ Temporal logic, etc

■ Propositional logic and Predicate logic are fundamental to all logic

◇ Propositional logic

‡ Propositions are “Sentences”; either true or false but not both.

‡ A sentence is smallest unit in propositional logic

‡ If proposition is true, then truth value is "true"; else “false”

‡ Example : Sentence "Grass is green"; Truth value “ true”;

Proposition is “yes”

◇ Predicate logic

‡ Predicate is a function may be true or false for arguments

‡ Predicate logic are rules that govern quantifiers

‡ Predicate logic is propositional logic added with quantifiers

‡ Examples:

“The car Tom is driving is blue”,

"The sky is blue",

"The cover of this book is blue"

Predicate is blue, give a name B ;

Sentence represented as B(x); read B(x) as "x is blue" ;

Object represented as x .

33

fo

.in

rs

de

Branches of AI

ea

• Search in AI

yr

.m

w

w

■ Search is a problem-solving technique that systematically consider

,w

ty

or

all possible action to find a path from initial state to target state.

ab

kr

ha

C

■ Search techniques are many; the most fundamental are

C

R

◇ Depth first ◇ Hill climbing

◇ Breadth first ◇ Least cost

■ Search components

◇ Initial state - First location

◇ Available actions - Successor function : reachable states

◇ Goal test - Conditions for goal satisfaction

◇ Path cost - Cost of sequence from initial state to reachable state

■ Search objective

◇ Transform initial state into goal state - find a sequence of actions.

■ Search solution

◇ Path from initial state to goal - optimal if lowest cost.

34

fo

.in

rs

de

Branches of AI

ea

• Pattern Recognition (PR)

yr

.m

w

w

■ Definitions : from the literature

,w

ty

or

ab

◇ 'The assignment of a physical object or event to one of pre-specified

kr

ha

categories' – Duda and Hart

C

C

R

◇ 'The science that concerns the description or classification (recognition) of measurements' – Schalkoff

◇ 'The process of giving names Ω to observations X ' – Schürmann

◇ Pattern Recognition is concerned with answering the question 'What is this?' – Morse

◇ 'A problem of estimating density functions in a high-dimensional space and dividing the space into the regions of categories or classes' – Fukunaga

■ Pattern recognition problems

◇ Machine vision - Visual inspection, ATR

◇ Character recognition – Mail sorting, processing bank cheques

◇ Computer aided diagnosis - Medical image/EEG/ECG signal analysis

◇ Speech recognition - Human Computer Interaction, access

■ Approaches for Pattern recognition

◇ Template Matching

◇ Statistical classification

◇ Syntactic or Structural matching

Each of these approaches are explained in the next slide.

35

fo

.in

rs

de

Branches of AI

ea

[continued from previous slide Approaches for Pattern Recognition]

yr

.m

w Template Matching

w

,w

ty

Match with stored template considering translation, rotation and scale

or

ab

kr

changes; measure similarity (correlation) based on training set.

ha

C

C

R

Statistical classification

Each pattern is represented in terms of d features (measurements) and viewed as a point in a d -dimensional space. Using training sets establish decision boundaries in the feature space - following decision theoretic or discriminant analysis approaches.

Syntactic or Structural matching

Complex pattern is composed of sub-patterns and the relations; they themselves are built from simpler / elementary sub-patterns are called primitives. The patterns are viewed as sentences belonging to a language, primitives are viewed as the alphabet of the language. The sentences are generated according to a grammar. A large collection of complex patterns can be described by a small number of primitives and grammatical rules. The grammar for each pattern class are inferred from the training samples.

Neural networks

Neural networks are viewed as weighted directed graphs in which the nodes are artificial neurons and directed edges (with weights) are connections between neurons input-output. Neural networks have the ability to learn complex nonlinear input-output relationships from the sequential training procedures, and adapt themselves to input data.

■ Applications requiring Pattern recognition

◇ Image Proc / Segmentation ◇ Seismic Analysis

◇ Computer Vision ◇ Industrial Inspection

◇ Medical Diagnosis ◇ Financial Forecast

◇ Man and Machine Diagnostics

36

fo

.in

rs

de

Branches of AI

ea

• Knowledge Representation

yr

.m

w

w

■ How do we represent what we know?

,w

ty

or

Knowledge is a collection of facts. To manipulate these facts by a

ab

kr

program, a suitable representation is required. A Good representation

ha

C

C

facilitates problem solving.

R

■ Knowledge representation formalisms (techniques)

Different types of knowledge require different types of representation.

◇ Predicate logic :

Predicate is a function may be TRUE for some arguments, and FALSE for others.

◇ Semantic networks :

A semantic net is just a graph, where the nodes represent concepts, and the arcs represent binary relationships between concepts.

◇ Frames and scripts :

A frame is a data structure that typically consists of : Frame name, Slot-filler (relations target), Pointers (links) to other Frames, Instantiation Procedure (inheritance, default, consistency). The Scripts are linked sentences using frame-like structures; e.g., a record of sequence of events for a given type of occurrence.

◇ Production rules :

Consists of a set of rules about behavior; a production consists two parts: a precondition (or IF) and an action (or THEN); if a production's precondition matches the current state of the world, then the production is said to be triggered.

37

fo

.in

rs

de

Branches of AI

ea

- Inference

yr

.m

w

w

Inference is the act or process of deriving a conclusion based solely on what

,w

ty

or

one already knows; it is deduction of new facts from old ones; Logic

ab

kr

captures inference.

ha

C

C

R

■ Deductive Inference

◇ It is never false; inference is true if premise is true.

◇ A traditional logic is based on deduction; it is a method of exact inference; there is no possibility of mistake if rules are followed exactly.

◇ The information required is complete, precise, and consistent .

◇ A logic is monotonic, if the truth of a proposition does not change when new information are added to the system.

■ Inductive Inference

◇ It may be correct or incorrect inference, because in real world the information is incomplete, inexact, inconsistent;

◇ A logic is inductive, also called Induction or Inductive reasoning, if the process of reasoning in which the premises of an argument are believed to support the conclusion but do not ensure it.

◇ A logic is non-monotonic, if the truth of a proposition may change when a new information (axioms) is added to or an old information is deleted from the system.

◇ The reasoner draws conclusions tentatively reserving the right

to retract them in the light of further information.

◇ Example: When we hear of a bird, we human infer that bird can fly, but this conclusion can be reversed when we hear that it is a penguin; the bird penguin cannot fly.

38

fo

.in

rs

de

Branches of AI

ea

- Common Sense Knowledge and Reasoning

yr

.m

w

w

Common sense is the mental skills that most people have.

,w

ty

or

ab

■ It is the ability to analyze a situation based on its context.

kr

ha

C

◇ People can think because the brain contain vast libraries of

C

R

common sense knowledge and has means for organizing, acquiring, and using such knowledge.

◇ Computer can not think; the computers programs do many things, they can play chess at the level of best players but cannot match capabilities of a 3 year old child at recognizing objects. Currently, computers lack common sense.

■ Researchers have divided common sense capability into :

◇ Common sense knowledge and

◇ Common sense reasoning.

■ Teaching computers common sense

◇ Project “OpenMind” at MIT - Here the goal is to teach a

computer things that human take them for granted; here the knowledge is represented in the form of Semantic net, Probabilistic graphical models, and Story scripts.

◇ Project “Cyc” – It is an attempt to manually build a database of human common sense knowledge; it has 1.5 million collection of common sense facts, but still far away from several hundred million needed.

39

fo

.in

rs

de

Branches of AI

ea

- Learning

yr

.m

w

w

Programs learn from what the facts or the behaviors can represent.

,w

ty

or

ab

■ Definitions

kr

ha

◇ Herbert Simon 1983 – “Learning denotes changes in the system

C

C

R

that are adaptive in the sense that they enable the system to do the same task or tasks more efficiently and more effectively the next time.”

◇ Marvin Minsky 1986 – “Learning is making useful changes in the working of our mind.”

◇ Ryszard Michalski 1986 – “Learning is constructing or modifying representations of what is being experienced.”

◇ Mitchell 1997 – “A computer program is said to learn from experience E with respect to some class of tasks T and performance

measure P, if its performance at tasks in T, as measured by P, improves with experience E.”

■ Major Paradigms of Machine Learning

◇ Rote : Learning by memorization; Saving knowledge so that it can be used again.

◇ Induction : Learning by example; Process of learning by example where a system tries to induce a general rule from a set of observed instances.

◇ Analogy : Learning from similarities; Recognize similarities in information already stored; can determine correspondence between two different representations.

◇ Genetic Algorithms : Learning by mimicking processes nature uses; Part of evolutionary computing, a way of solving problems by mimicking processes, nature uses, selection, crosses over, mutation and accepting to evolve a solution to a problem.

◇ Reinforcement : Learning from actions; Assign rewards, +ve or -ve; at the end of a sequence of steps, it learns which actions are good or bad.

40

fo

.in

rs

de

Branches of AI

ea

• Planning

yr

.m

w

w

A plan is a representation of a course of action.

,w

ty

or

Planning is a problem solving technique.

ab

kr

Planning is a reasonable series of actions to accomplish a goal.

ha

C

C

R

■ Planning programs

Start with facts about the world, particularly

- ◇ facts about the effects of actions,
- ◇ facts about the particular situation, and
- ◇ statement of a goal.

■ Benefits of planning

- ◇ reducing search,
- ◇ resolving goal conflicts, and
- ◇ providing a basis for error recovery.

■ Strategy for planning

A strategy is just a sequence of actions. From facts the program generate a strategy for achieving the goal.

41

fo

.in

rs

de

Branches of AI

ea

- Epistemology

yr

.m

w

w

Epistemology is the theory of knowledge.

,w

ty

or

ab

■ There are various kinds of knowledge:

kr

ha

C

- ◇ knowing how to do something (e.g., how to ride a bicycle),

C

R

- ◇ knowing someone in person, and
- ◇ knowing a place or a city.

■ Epistemology is the study of knowledge and justified belief.

Propositions

Know-

Truths ledge Beliefs

◇ Consider a knowledge of proposition - a schema 'S knows that P' ;

◇ Question asked - what are the necessary and sufficient conditions?

S knows that P if and only if

P is true;

S believes that P is true; and

S is justified in believing that P is true.

Gettier claimed that the above account of knowledge is insufficient.

■ Epistemology is the study of the kinds of knowledge that are required

for solving problems in the world.

42

fo

.in

rs

de

Branches of AI

ea

- Ontology

yr

.m

w

w

Ontology is concerned with existence; a study of the categories of things

,w

ty

or

that exist or may exist in some domain.

ab

kr

ha

C

■ Ontology is a data model, represents a domain and is used to reason

C

R

about the objects in that domain and the relations between them.

■ Ontology is used in artificial intelligence, as a form of knowledge

representation about the world or some part of it.

■ Ontology generally describe:

◇ Individuals (instances): the basic or ground level objects

◇ Classes: sets, collections, or types of objects.

◇ Attributes: properties, features, characteristics, or parameters that objects can have and share.

◇ Relations: ways the objects can be related to one another.

■ Ontology is a specification of a conceptualization.

43

fo

.in

rs

de

Branches of AI

ea

• Heuristics

yr

.m

w

w

Heuristics are simple, efficient rules;

,w

ty

or

ab

■ Heuristics are in common use as Rule of thumb;

kr

ha

C

In computer science, a heuristic is an algorithm with provably good run

C

R

times and with provably good or optimal solution.

■ Heuristics are intended to gain computational performance or conceptual simplicity, potentially at the cost of accuracy or precision.

■ People use heuristics to make decisions, come to judgments, and solve problems, when facing complex problems or incomplete information. These rules work well under most circumstances.

■ In AI programs, the heuristic functions are :

◇ used to measure how far a node is from goal state.

◇ used to compare two nodes, find if one is better than the other.

44

fo

.in

rs

de

Branches of AI

ea

- Genetic programming (GP)

yr

.m

w

w

Genetic programming is an automated method for creating program

,w

ty

or

from a high-level problem statement.

ab

kr

ha

C

■ GP starts from a high-level statement of the requirements of a problem

C

R

and attempts to produce a computer program that solves the problem.

■ The user (human) communicates the high-level statement of the problem to the GP system by performing certain well-defined preparatory steps.

■ The major five preparatory steps, the human user require to specify for the GP to be evolved are :

◇ Set of terminals (variables , functions, and constants)

◇ Set of primitive functions

◇ Fitness measure (for fitness of individuals in the population),

◇ Controlling parameters for the run, and

◇ Termination criterion for designating the result of the run.

■ The “Run” of genetic programming executes a series of well-defined problem-independent steps (the flowchart).

45

fo

.in

rs

de

Applications of AI

ea

6. Applications of AI

yr

.m

w

w

- Game playing

,w

ty

or

ab

■ Games are Interactive computer program, an emerging area in which

kr

ha

the goals of human-level AI are pursued.

C

C

R

■ Games are made by creating human level artificially intelligent entities, e.g. enemies, partners, and support characters that act just like humans.

■ Game play is a search problem defined by:

◇ Initial state - board

◇ Expand function - build all successor states

◇ Cost function - payoff of the state

◇ Goal test - ultimate state with maximal payoff

■ Game playing is characterized by:

◇ "Unpredictable" opponent

◇ Need to specify move for every possible opponent reply.

◇ Time limits - games become boring if there is no action for too long a time; opponents are unlikely to find goal, must approximate.

■ Computer Games

◇ Computers perform at champion level games, examples :

Checkers, Chess, Othello, Backgammon.

◇ Computers perform well games, example : Bridge

◇ Computers still do badly, example : Go, Hex

■ The Deep Blue Chess program won over world champion Gary Kasparov.

46

fo

.in

rs

de

Applications of AI

ea

• Speech Recognition

yr

.m

w

w

■ A process of converting a speech signal to a sequence of words;

,w

ty

or

ab

■ In 1990s, computer speech recognition reached a practical level for

kr

ha

limited purposes.

C

C

R

■ Using computers recognizing speech is quite convenient, but most users find the keyboard and the mouse still more convenient.

■ The typical usages are :

◇ Voice dialing (Call home),

◇ Call routing (collect call),

◇ Data entry (credit card number).

◇ Speaker recognition.

■ The spoken language interface PEGASUS in the American Airlines'

EAASY SABRE reservation system, allows users to obtain flight information and make reservations over the telephone.

47

fo

.in

rs

de

Applications of AI

ea

- Understanding Natural Language

yr

.m

w

w

Natural language processing (NLP) does automated generation and

,w

ty

or

understanding of natural human languages.

ab

kr

ha

C

■ Natural language generation system

C

R

Converts information from computer databases into normal-sounding human language

■ Natural language understanding system

Converts samples of human language into more formal representations that are easier for computer programs to manipulate.

■ Some major tasks in NLP

◇ Text-to-Speech (TTS) system :

converts normal language text into speech.

◇ Speech recognition (SR) system :

process of converting a speech signal to a sequence of words;

◇ Machine translation (MT) system :

translate text or speech from one natural language to another.

◇ Information retrieval (IR) system :

search for information from databases such as Internet or World

Wide Web or Intranets.

48

fo

.in

rs

de

Applications of AI

ea

- Computer Vision

yr

.m

w

w

■ It is a combination of concepts, techniques and ideas from : Digital

,w

ty

or

Image Processing, Pattern Recognition, Artificial Intelligence and

ab

kr

Computer Graphics.

ha

C

C

R

■ The world is composed of 3-D objects, but the inputs to the human eye and computers' TV cameras are 2-D.

■ Some useful programs can work solely in 2-D, but full computer vision requires partial 3-D information that is not just a set of 2-D views.

■ At present there are only limited ways of representing 3-D information directly, and they are not as good as what humans evidently use.

■ Examples

◇ Face recognition :

the programs in use by banks

◇ Autonomous driving :

The ALVINN system, autonomously drove a van from Washington, D.C. to San Diego, averaging 63 mph day and night, and in all weather conditions.

◇ Other usages

Handwriting recognition, Baggage inspection, Manufacturing inspection, Photo interpretation, etc .

49

fo

.in

rs

de

Applications of AI

ea

• Expert Systems

yr

.m

w

w

Systems in which human expertise is held in the form of rules

,w

ty

or

ab

■ It enable the system to diagnose situations without the human expert

kr

ha

C

being present.

C

R

■ A Man-machine system with specialized problem-solving expertise.

The "expertise" consists of knowledge about a particular domain, understanding of problems within that domain, and "skill" at solving some of these problems.

■ Knowledge base

A knowledge engineer interviews experts in a certain domain and tries to embody their knowledge in a computer program for carrying out some task.

■ One of the first expert systems was MYCIN in 1974, which diagnosed bacterial infections of the blood and suggested treatments.

■ Expert systems rely on knowledge of human experts, e.g.

◇ Diagnosis and Troubleshooting :

deduces faults and suggest corrective actions for a malfunctioning

device or process

◇ Planning and Scheduling :

analyzing a set of goals to determine and ordering a set of actions
taking into account the constraints; e.g. airline scheduling of
flights

◇ Financial Decision Making :

advisory programs assists bankers to make loans, Insurance
companies to assess the risk presented by the customer, etc.

◇ Process Monitoring and Control :

analyzes real-time data, noticing anomalies, predicting trends, and
controlling optimality and do failure correction.

50

fo

.in

rs

de

AI- Introduction - References

ea

7. References : Textbooks

yr

.m

w

w

1. "Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill

,w

ty

companies Inc., Chapter 1-22, page 1-613.

or

ab

kr

2. "Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig,

ha

C

(2002), Prentice Hall, Chapter 1-27, page 1-1057.

C

R

3. "Computational Intelligence: A Logical Approach", by David Poole, Alan Mackworth,
and Randy Goebel, (1998), Oxford University Press, Chapter 1-12, page 1-608.

4. "Artificial Intelligence: Structures and Strategies for Complex Problem Solving", by
George F. Luger, (2002), Addison-Wesley, Chapter 1- 16, page 1-743.

5. "AI: A New Synthesis", by Nils J. Nilsson, (1998), Morgan Kaufmann Inc., Chapter 1-25, Page 1-493.
6. "Artificial Intelligence: Theory and Practice", by Thomas Dean, (1994), Addison-Wesley, Chapter 1-10, Page 1-650.
7. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

51

fo

.in

rs

de

ea

yr

Problem Solving, Search & Control Strategies: AI Course Lecture 7 – 14 , notes, slides

.m

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

kr

www.myreaders.info

ha

C

C

R

Problem Solving,

Search and Control Strategies

Artificial Intelligence

General Problem Solving in AI , topics : Definitions, problem space, problem solving, state space, state change, problem solution, problem description. Search and Control Strategies - search related terms, algorithm's performance and complexity, "Big - o" notations, tree structure, stacks and queues; Search - search algorithms, hierarchical representation, search space, formal statement, search notations, estimate cost function and heuristic function; Control strategies - strategies for search, forward and backward chaining.

Exhaustive search - depth-first search algorithm, breadth-first search algorithm, compare depth-first and breadth-first search. Heuristic search techniques - characteristics of heuristic search, heuristic search compared with other search, types of heuristic search algorithms. Constraint satisfaction problems (CSPs) and models - generate and test, backtracking algorithm.

fo

.in

rs

de

ea

Problem Solving,

yr

.m

w

w

Search and Control Strategies

,w

ty

or

Artificial Intelligence

ab

kr

ha

C

C

Topics

R

(Lectures 07, 08, 09, 10, 11, 12, 13, 14, 8 hours) Slides

1. General Problem Solving 03-17

Problem solving definitions : problem space, problem solving, state space, state change, structure of state space, problem solution, problem description; Examples of problem definition.

2. Search and Control Strategies 18-43

Search related terms : algorithm's performance and complexity, computational complexity, "Big - o" notations, tree structure, stacks and queues; Search: search algorithms, hierarchical representation, search space, formal statement, search notations, estimate cost and heuristic function; Control strategies : strategies for search, forward and backward chaining.

3. Exhaustive Searches 44-52

Depth-first search Algorithm; Breadth-first search Algorithm; Compare depth-first and breadth-first search;

4. Heuristic Search Techniques 53-59

Characteristics of heuristic search; Heuristic search compared with other search; Example of heuristic search; Types of heuristic search algorithms.

5. Constraint Satisfaction Problems (CSPs) and Models 60-74

Examples of CSPs; Constraint Satisfaction Models : Generate and Test, Backtracking algorithm, Constraint Satisfaction Problems (CSPs) : definition, properties and algorithms.

6. References 75

02

fo

.in

rs

de

ea

yr

Problem Solving,

.m

w

w

,w

Search and Control Strategies

ty

or

ab

kr

ha

What are problem solving, search and control strategies ?

C

C

R

- Problem solving is fundamental to many AI-based applications.

There are two types of problems.

- The Problems like, computation of the sine of an angle or the square root of a value. These can be solved through the use of deterministic procedure and the success is guaranteed.

– In the real world, very few problems lend themselves to straightforward solutions.

Most real world problems can be solved only by searching for a solution.

AI is concerned with these type of problems solving.

- Problem solving is a process of generating solutions from observed data.

- a problem is characterized by a set of goals,

- a set of objects, and

- a set of operations.

These could be ill-defined and may evolve during problem solving.

- Problem space is an abstract space.

- A problem space encompasses all valid states that can be generated by the application of any combination of operators on any combination of objects.

- The problem space may contain one or more solutions.

Solution is a combination of operations and objects that achieve the goals.

- Search refers to the search for a solution in a problem space.

- Search proceeds with different types of search control strategies.

- The depth-first search and breadth-first search are the two common search strategies.

03

fo

.in

rs

de

AI - General Problem solving

ea

1. General Problem solving

yr

.m

w

w

Problem solving has been the key areas of concern for Artificial Intelligence.

,w

ty

or

ab

■ Problem solving is a process of generating solutions from observed or

kr

ha

given data. It is however not always possible to use direct methods (i.e. go
C
C
R

directly from data to solution). Instead, problem solving often need to use
indirect or model-based methods.

■ General Problem Solver (GPS) was a computer program created in 1957
by Simon and Newell to build a universal problem solver machine. GPS was
based on Simon and Newell's theoretical work on logic machines. GPS in
principle can solve any formalized symbolic problem, like : theorems proof
and geometric problems and chess playing.

■ GPS solved many simple problems such as the Towers of Hanoi, that could
be sufficiently formalized, but GPS could not solve any real-world
problems.

To build a system to solve a particular problem, we need to

■ Define the problem precisely – find input situations as well as final
situations for acceptable solution to the problem.

■ Analyze the problem – find few important features that may have impact
on the appropriateness of various possible techniques for solving the
problem.

■ Isolate and represent task knowledge necessary to solve the problem

■ Choose the best problem solving technique(s) and apply to the particular
problem.

04

fo

.in

rs

de

AI - General Problem solving

ea

1.1 Problem Definitions :

yr

.m

w

w

A problem is defined by its elements and their relations.

,w

ty

or

ab

To provide a formal description of a problem, we need to do following:

kr

ha

C

C

a. Define a state space that contains all the possible configurations of

R

the relevant objects, including some impossible ones.

b. Specify one or more states, that describe possible situations, from which the problem-solving process may start. These states are called initial states.

c. Specify one or more states that would be acceptable solution to the problem. These states are called goal states.

d. Specify a set of rules that describe the actions (operators) available.

The problem can then be solved by using the rules, in combination with an appropriate control strategy, to move through the problem space until a path from an initial state to a goal state is found.

This process is known as search.

- Search is fundamental to the problem-solving process.

- Search is a general mechanism that can be used when more direct method is not known.

- Search provides the framework into which more direct methods for solving subparts of a problem can be embedded.

A very large number of AI problems are formulated as search problems.

05

fo

.in

rs

de

AI - General Problem solving

ea

- Problem Space

yr

.m

w

w

A problem space is represented by directed graph, where nodes

,w

ty

or

represent search state and paths represent the operators applied to

ab

kr

ha

change the state.

C

C

R

To simplify a search algorithms, it is often convenient to logically and programmatically represent a problem space as a tree. A tree usually decreases the complexity of a search at a cost. Here, the cost is due to duplicating some nodes on the tree that were linked numerous times in the graph; e.g., node B and node D shown in example below.

A tree is a graph in which any two vertices are connected by exactly one path. Alternatively, any connected graph with no cycles is a tree.

Examples

Graph Trees

A B A

B C

C D

D B D

D

06

fo

.in

rs

de

AI - General Problem solving

ea

- Problem Solving

yr

.m

w

w

The term Problem Solving relates analysis in AI. Problem solving may

,w

ty

or

be characterized as a systematic search through a range of possible

ab

kr

ha

actions to reach some predefined goal or solution. Problem-solving

C

C

R

methods are categorized as special purpose and general purpose.

– Special-purpose method is tailor-made for a particular problem, often exploits very specific features of the situation in which the problem is embedded.

– General-purpose method is applicable to a wide variety of problems.

One general-purpose technique used in AI is "means-end analysis". It is a step-by-step, or incremental, reduction of the difference between current state and final goal.

Examples : Tower of Hanoi puzzle

– For a Robot this might consist of PICKUP, PUTDOWN, MOVEFORWARD, MOVEBACK, MOVELEFT, and MOVERIGHT—until the goal is reached.

– Puzzles and Games have explicit rules : e.g., the Tower of Hanoi puzzle.

(a) (b)

Start Final

Tower of Hanoi puzzle.

◇ This puzzle may involves a set of rings of different sizes that can be placed on three different pegs.

◇ The puzzle starts with the rings arranged as shown in Fig. (a)

◇ The goal of this puzzle is to move them all as to Fig. (b)

◇ Condition : Only the top ring on a peg can be moved, and it may only be placed on a smaller ring, or on an empty peg.

In this Tower of Hanoi puzzle : Situations encountered while solving the problem are described as states. The set of all possible configurations of rings on the pegs is called problem space.

07

fo

.in

rs

de

AI - General Problem solving

ea

- States

yr

.m

w

w

A state is a representation of elements at a given moment. A problem is

,w

ty

or

defined by its elements and their relations.

ab

kr

ha

At each instant of a problem, the elements have specific descriptors and

C

C

R

relations; the descriptors tell - how to select elements ?

Among all possible states, there are two special states called :

– Initial state is the start point

– Final state is the goal state

08

fo

.in

rs

de

AI - General Problem solving

ea

- State Change: Successor Function

yr

.m

w

w

A Successor Function is needed for state change.

,w

ty

or

The successor function moves one state to another state.

ab

kr

ha

C

Successor Function :

C

R

◇ Is a description of possible actions; a set of operators.

◇ Is a transformation function on a state representation, which converts that state into another state.

◇ Defines a relation of accessibility among states.

◇ Represents the conditions of applicability of a state and corresponding transformation function

09

fo

.in

rs

de

AI - General Problem solving

ea

• State Space

yr

.m

w

w

,w

A State space is the set of all states reachable from the initial state.

ty

or

ab

Definitions of terms :

kr

ha

C

C

◇ A state space forms a graph (or map) in which the nodes are states

R

and the arcs between nodes are actions.

◇ In state space, a path is a sequence of states connected by a sequence

of actions.

◇ The solution of a problem is part of the map formed by the state space.

10

fo

.in

rs

de

AI - General Problem solving

ea

- Structure of a State Space

yr

.m

w

w

The Structures of state space are trees and graphs.

,w

ty

or

ab

– Tree is a hierarchical structure in a graphical form; and

kr

ha

C

– Graph is a non-hierarchical structure.

C

R

◇ Tree has only one path to a given node;

i.e., a tree has one and only one path from any point to any other point.

◇ Graph consists of a set of nodes (vertices) and a set of edges (arcs).

Arcs establish relationships (connections) between the nodes;

i.e., a graph has several paths to a given node.

◇ operators are directed arcs between nodes.

Search process explores the state space. In the worst case, the search explores all possible paths between the initial state and the goal state.

11

fo

.in

rs

de

AI - General Problem solving

ea

- Problem Solution

yr

.m

w

w

In the state space, a solution is a path from the initial state to a goal

,w

ty

or

state or sometime just a goal state.

ab

kr

ha

C

◇ A Solution cost function assigns a numeric cost to each path;

C

R

It also gives the cost of applying the operators to the states.

◇ A Solution quality is measured by the path cost function; and

An optimal solution has the lowest path cost among all solutions.

◇ The solution may be any or optimal or all.

◇ The importance of cost depends on the problem and the type of solution asked.

12

fo

.in

rs

de

AI - General Problem solving

ea

- Problem Description

yr

.m

w

w

A problem consists of the description of :

,w

ty

or

– current state of the world,

ab

kr

ha

– actions that can transform one state of the world into another,

C

C

R

– desired state of the world.

◇ State space is defined explicitly or implicitly

A state space should describe everything that is needed to solve a problem and nothing that is not needed to solve the problem.

◇ Initial state is start state

◇ Goal state is the conditions it has to fulfill

– A description of a desired state of the world;

– The description may be complete or partial.

◇ Operators are to change state

– Operators do actions that can transform one state into another.

– Operators consist of : Preconditions and Instructions;

□ Preconditions provide partial description of the state of the world

that must be true in order to perform the action,

□ Instructions tell on how to create next state.

– Operators should be as general as possible, to reduce their number.

◇ Elements of the domain has relevance to the problem

– Knowledge of the starting point.

◇ Problem solving is finding solution

– Finding an ordered sequence of operators that transform the current (start) state into a goal state;

◇ Restrictions are solution quality any, optimal, or all

– Finding the shortest sequence, or

– Finding the least expensive sequence defining cost , or

– Finding any sequence as quickly as possible.

13

fo

.in

rs

de

Recall algebraic function

ea

Recall : Algebraic Function

yr

.m

w

w

Algebraic Function

,w

ty

or

A function may take the form of a set of ordered pair, a graph, or a

ab

kr

equation. Regardless of the form it takes, a function must obey the

ha

C

C

condition that, no two of its ordered pair have the same first member with

R

different second members.

Relation : A set of ordered pair of the form (x, y) is called a relation.

Function : A relation in which no two ordered pair have the same x-value but may have same or different y-value is called a function. Functions are usually named by lower-case letters such as f , g , and h .

For example, $f = \{(-3, 9), (0, 0), (3, 9)\}$, $g = \{(4, -2), (4, 2)\}$

Here f is a function, g is not a function.

Domain and Range : The domain of a function is the set of all the first members of its ordered pairs, and the range of a function is the set of all second members of its ordered pairs.

if function $f = \{(a, A), (b, B), (c, C)\}$,

then its domain is $\{a, b, c\}$ and its range is $\{A, B, C\}$.

Function and mapping : A function may be viewed as a mapping or a pairing of one set with elements of a second set such that each element of the first set (called domain) is paired with exactly one element of the second set (called codomain). Example :

If a function f maps $\{a, b, c\}$ into $\{A, B, C, D\}$

such that $a \rightarrow A$ (read "a is mapped into A"), $b \rightarrow B$, $c \rightarrow C$

Then the domain is $\{a, b, c\}$ and the codomain is $\{A, B, C, D\}$.

Each element of the co-domain that corresponds to an element of the domain is called the image of that element. Since a is paired with A in codomain, A is called the image of a .

The set of image points, $\{A, B, C\}$, is called the range. Thus, the range is a subset of the codomain.

Onto mappings : Set A is mapped onto set B if each element of set B is image of an element of a set A . Thus, every function maps its domain onto its range.

[Continued in next slide]

14

fo

.in

rs

de

Recall algebraic function

ea

[Continued from previous slide]

yr

.m

w Describing a function by an equation : The rule by which each x -value gets

w

,w

ty

paired with the corresponding y -value may be specified by an equation. For

or

ab

example, the function described by the equation $y = x + 1$ requires that for

kr

ha

C

any choice of x in the domain, the corresponding range value is $x + 1$.

C

R

Thus, $2 \rightarrow 3$, $3 \rightarrow 4$, and $4 \rightarrow 5$.

Restricting domains of functions : Unless otherwise indicated, the domain of a function is assumed to be the largest possible set of real numbers.

Thus, the domain of $y = x / (x^2 - 4)$ is the set of all real numbers except ± 2 since for these values of x the denominator is 0.

The domain of $y = (x - 1)^{1/2}$ is the set of real numbers greater than or equal to 1 since for any value of x less than 1, the root radical has a

negative radicand so the radical does not represent a real number.

Example : Find which of the relation describe function ?

(a) $y = x^{1/2}$, (b) $y = x^3$, (c) $y < x$, (d) $x = y^2$

The equations (a) and (b) produce exactly one value of y for each value of x. Hence, equations (a) and (b) describe functions.

The equation (c), $y < x$ does not represent a function since it contains ordered pair such as (1, 2) and (1, 3) where same value of x is paired with different values of y.

The equation (d), $x = y^2$ is not a function since ordered pair such as (4, 2) and (4, -2) satisfy the equation but have the same value of x paired with different values of y.

Function notation : For any function f, the value of y that corresponds to a given value of x is denoted by $f(x)$.

If $y = 5x - 1$, then $f(2)$, read as "f of 2" , represents the value of y.

when $x = 2$, then $f(2) = 5 \cdot 2 - 1 = 9$;

when $x = 3$, then $f(3) = 5 \cdot 3 - 1 = 14$;

In an equation that describes function f, then $f(x)$ may be used in place of y, for example , $f(x) = 5x - 1$.

If $y = f(x)$, then y is said to be a function of x.

Since the value of y depends on the value of x, then y is called the dependent variable and x is called the independent variable.

15

fo

.in

rs

de

AI - General Problem solving

ea

1.2 Examples of Problem Definitions

yr

.m

w

w

• Example 1 :

,w

ty

or

ab

A game of 8-Puzzle

kr

ha

C

◇ State space : configuration of 8 - tiles on the board

C

R

◇ Initial state : any configuration 1 2 3

◇ Goal state : tiles in a specific order 4 5 6

7 8

◇ Action : "blank moves"

Condition: the move is within the board Solution

Transformation: blank moves Left, Right, Up, Dn

◇ Solution : optimal sequence of operators

16

fo

.in

rs

de

AI - General Problem solving

ea

• Example 2 :

yr

.m

w

w

A game of n - queens puzzle; n = 8

,w

ty

or

ab

◇ State space : configurations n = 8

kr

a b c d e f g h

ha

8 8

C

queens on the board with only one

C

R

7 7

queen per row and column

6 6

◇ Initial state : configuration without

5 5

queens on the board

4 4

◇ Goal state : configuration with $n = 8$

3 3

queens such that no queen attacks any

2 2

other 1 1

◇ Operators or actions : place a queen on a b c d e f g h

the board.

One Solution

Condition: the new queen is not

attacked by any other already

placed

Transformation: place a new queen

in a particular square of the board

◇ Solution : one solution (cost is not considered)

17

fo

.in

rs

de

AI - Search and Control strategies

ea

2. Search and Control Strategies

yr

.m

w

w

,w

Word "Search" refers to the search for a solution in a problem space.

ty

or

ab

– Search proceeds with different types of "Search Control strategies".

kr

ha

C

– A strategy is defined by picking the order in which the nodes expand.

C

R

The Search strategies are evaluated in the following dimensions:

Completeness, Time complexity, Space complexity, Optimality.

(the search related terms are first explained, then the search algorithms and control strategies are illustrated).

2.1 Search related terms

• Algorithm's Performance and Complexity

Ideally we want a common measure so that we can compare approaches in order to select the most appropriate algorithm for a given situation.

◇ Performance of an algorithm depends on internal and external factors.

Internal factors External factors

‡ Time required, to run ‡ Size of input to the algorithm

‡ Space (memory) required to run ‡ Speed of the computer

‡ Quality of the compiler

◇ Complexity is a measure of the performance of an algorithm.

It measures the internal factors, usually in time than space.

18

fo

.in

rs

de

AI - Search and Control strategies

ea

• Computational Complexity

yr

.m

w

w

,w

A measure of resources in terms of Time and Space.

ty

or

ab

◇ If A is an algorithm that solves a decision problem f

kr

ha

C

then run time of A is the number of steps taken on the input of length n.

C

R

◇ Time Complexity $T(n)$ of a decision problem f is the run time of the 'best' algorithm A for f .

◇ Space Complexity $S(n)$ of a decision problem f is the amount of memory used by the 'best' algorithm A for f .

19

fo

.in

rs

de

AI - Search and Control strategies

ea

- "Big - O" notation

yr

.m

w

w

The "Big-O" is theoretical measure of the execution of an algorithm,

,w

ty

or

usually indicates the time or the memory needed, given the problem

ab

kr

size n, which is usually the number of items.

ha

C

C

R

◇ Big-O notation

The Big-O notation is used to give an approximation to the run-time-efficiency of an algorithm ; the letter "O" is for order of magnitude of operations or space at run-time.

◇ The Big-O of an Algorithm A

– If an algorithm A requires time proportional to $f(n)$, then the algorithm A is said to be of order $f(n)$, and it is denoted as $O(f(n))$.

2

– If algorithm A requires time proportional to n^2 , then order of the algorithm is said to be $O(n^2)$.

– If algorithm A requires time proportional to n , then order of the algorithm is said to be $O(n)$.

The function $f(n)$ is called the algorithm's growth-rate function.

If an algorithm has performance complexity $O(n)$, this means that the run-time t should be directly proportional to n , ie $t \propto n$ or $t = kn$ where k is constant of proportionality.

Similarly, for algorithms having performance complexity $O(\log^2(n))$, $O(\log N)$, $O(N \log N)$, $O(2N)$ and so on.

20

fo

.in

rs

de

AI - Search and Control strategies

ea

◇ Example 1 :

yr

.m

w

w

1-D array, determine the Big-O of an algorithm ;

,w

ty

or

Calculate the sum of the n elements in an integer array $a[0 \dots n-1]$.

ab

kr

ha

C

Line no Instructions No of execution steps

C

R

line 1 $sum = 0$ 1

line 2 for ($i = 0; i < n; i++$) $n + 1$

line 3 $sum += a[i]$ n

line 4 print sum 1

Total $2n + 3$

For the polynomial $(2n + 3)$ the Big-O is dominated by the 1st term as n while the number of elements in the array becomes very large.

Also in determining the Big-O

‡ Ignoring constants such as 2 and 3, the algorithm is of the order n .

‡ So the Big-O of the algorithm is $O(n)$.

‡ In other words the run-time of this algorithm increases roughly as the size of the input data n , say an array of size n .

21

fo

.in

rs

de

AI - Search and Control strategies

ea

◇ Example 2 :

yr

.m

w

w

2-D array, determine the Big-O of an algorithm;

,w

ty

or

In a square 2-D array $a[0 \dots n-1][0 \dots n-1]$, find the

ab

kr

largest element .

ha

C

C

R

Line no Instructions No of executions

line 1 $\text{max} = a[0][0]$ 1

line 2 for ($\text{row} = 0; \text{row} < n; \text{row}++$) $n + 1$

line 3 for ($\text{col} = 0; \text{col} < n; \text{col}++$) $n * (n + 1)$

line 4 if ($a[\text{row}][\text{col}] > \text{max}$) $\text{max} = a[\text{row}][\text{col}]$. $n * (n)$

line 5 print max 1

Total $2n^2 + 2n + 3$

‡ Ignoring the constants such as 2, 2 and 3, the algorithm is of

order n^2 .

‡ So Big-O of the algorithm is $O(n^2)$.

‡ In other words, run-time of this algorithm increases roughly as the square of the size of the input data which is n^2 , say an array of size $n \times n$.

22

fo

.in

rs

de

AI - Search and Control strategies

ea

◇ Example 3 :

yr

.m

w

w

Polynomial in n with degree k

,w

ty

or

ab

‡ Let the number of steps needed to carry out an algorithm is

kr

ha

C

expressed as

C

R

$k \ k-1 \ 1$

$f(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$

Then $f(n)$ is a polynomial in n with degree k and $f(n) \in O(n^k)$.

‡ To obtain the order of a polynomial function, use the term which is of highest degree and disregard the constants and the terms which are of lower degrees.

The Big-O of the algorithm is $O(n^k)$.

In other words, run-time of this algorithm increases exponentially.

23

fo

.in

rs

de

AI - Search and Control strategies

ea

◇ Growth Rates Variation

yr

.m

w

w

The Growth rates vary with number of inputs and time.

,w

ty

or

ab

kr

2

$O(n)$

ha

C

C

Time

R

$O(n \log n)$

For a short time (n^2) is

better than ($n \log n$)

Number of Inputs n

Fig. Growth rates variation with number of inputs and time

If n is sufficiently large

then $\log n < n < n \log n < n^2 < n^3 < 2n$.

So is the Big-O are

$2^3 n$

$O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$.

Here $\log n$ is interpreted as base-2 logarithm. The base of logarithm does not matter because in determining the Big-O of an algorithm constants are ignored as mentioned.

Note : $\log_2 n = \log_{10} n / \log_{10} 2$, where $\log_{10} 2 = 0.3010299957$.

24

fo

.in

rs

de

AI - Search and Control strategies

ea

◇ Example of Growth Rates Variation

yr

.m

w

w

Problem : If an algorithm requires 1 second run time for a problem of

,w

ty

or

size 8 , then find run time for that algorithm for the problem of size 16 ?

ab

kr

Solutions: If the order of the algorithm is $O(f(n))$ then the calculated

ha

C

C

execution time $T(n)$ of the algorithm as problem size increases are

R

as below.

Order of the Run time $T(n)$ required

algorithm $O(f(n))$ as the problem size increases

$O(1) \Rightarrow T(n) = 1 \text{ second ;}$

constant time

run time is independent of the size of the

problem.

$O(\log_2 n) \Rightarrow T(n) = (1 \cdot \log_2 16) / \log_2 8 = 4/3 \text{ seconds ;}$

logarithmic time

run time increases slowly with the size of the

problem.

$O(n) \Rightarrow T(n) = (1 \cdot 16) / 8 = 2 \text{ seconds}$

linear time

run time increases directly with the size of the

problem.

$O(n \cdot \log_2 n) \Rightarrow T(n) = (1 \cdot 16 \cdot \log_2 16) / 8 \cdot \log_2 8 = 8/3 \text{ seconds}$

log-linear time

run time increases more rapidly than the linear algorithm with the size of the problem.

$$O(n^2) \Rightarrow T(n) = (1 \cdot 16^2) / 82 = 4 \text{ seconds}$$

quadratic time

the run time increases rapidly with the size of the problem.

$$O(n^3) \Rightarrow T(n) = (1 \cdot 16^3) / 83 = 8 \text{ seconds}$$

cubic time

the run time increases more rapidly than quadratic algorithm with the size of the problem.

$$O(2^n) \Rightarrow T(n) = (1 \cdot 2^{16}) / 28 = 28 \text{ seconds} = 256 \text{ seconds}$$

exponential time

the run time increases too rapidly to be practical.

25

fo

.in

rs

de

AI - Search and Control strategies

ea

- Tree Structure

yr

.m

w

w

Tree is a way of organizing objects, related in a hierarchical fashion.

,w

ty

or

Tree is a type of data structure where

ab

kr

ha

– each element is attached to one or more elements directly beneath it.

C

C

R

– the connections between elements are called branches.

- tree is often called inverted trees because its root is at the top.
- the elements that have no elements below them are called leaves.
- a binary tree is a special type, each element has two branches below it.

◇ Example

Tree depth Root

0 Parent A

1 Child B C

2 D E F G H

Leaves Goal Node

Fig. A simple example unordered tree

◇ Properties

‡ Tree is a special case of a graph.

‡ The topmost node in a tree is called the root node; at root node all operations on the tree begin.

‡ A node has at most one parent. The topmost node called root node has no parents.

‡ Each node has either zero or more child nodes below it .

‡ The Nodes at the bottom most level of the tree are called leaf nodes. The leaf nodes do not have children.

‡ A node that has a child is called the child's parent node.

‡ The depth of a node n is the length of the path from the root to the node; The root node is at depth zero.

26

fo

.in

rs

de

AI - Search and Control strategies

ea

• Stacks and Queues

yr

.m

w The Stacks and Queues are data structures .

w

,w

ty

It maintains the order last-in first-out and first-in first-out respectively.

or

ab

Both stacks and queues are often implemented as linked lists, but that is

kr

ha

C

not the only possible implementation.

C

R

◇ Stack

An ordered list works as Last-In First-Out (LIFO);

Here the items are in a sequence and piled one on top of the other.

‡ The insertions and deletions are made at one end only, called Top.

‡ If Stack $S = a[1], a[2], \dots, a[n]$ then $a[1]$ is bottom most element

‡ Any intermediate element $a[i]$ is on top of element $a[i-1]$ where $1 < i \leq n$.

‡ In a Stack all operations take place on Top.

The Pop operation removes item from top of the stack.

The Push operation adds an item on top of the stack.

Items enter Stack at Top and leave from Top

$a[i] \in \text{Top}$

$a[i-1] \in \text{D}$

15 9

C 11 11 11

B 8 8 8

$a[1] \in \text{A}$ 23 23 23

5 5 5

Stack

Original after after

Stack Pop (15) Push (9)

27

fo

.in

rs

de

AI - Search and Control strategies

ea

◇ Queue

yr

.m

w An ordered list works as First-In First-Out (FIFO)

w

,w

ty

Here the items are in a sequence. There are restrictions about how

or

ab

items can be added to and removed from the list.

kr

ha

C

C

R

‡ A queue has two ends.

‡ All insertions (enqueue) take place at one end, called Rear or Back.

‡ All deletions (dequeue) take place at other end, called Front.

‡ If Queue has $a[n]$ as rear element then $a[i+1]$ is behind $a[i]$

where $1 < i \leq n$.

‡ All operation take place at one end of queue or the other end.

a dequeue operation removes the item at Front of the queue.

a enqueue operation adds an item to the Rear of the queue.

Items enter Queue at Rear and leave from Front

Front Rear

A B C D E 11 25 9 11 19

$a[n]$ $a[n+1]$ Front Rear

Queue Original Queue

25 9 11 19

After Dequeue (11)

25 9 11 19 15

After Enqueue (15)

28

fo

.in

rs

de

AI - Search and Control strategies

ea

2.2 Search

yr

.m

w

w

Search is the systematic examination of states to find path from the

,w

ty

or

start/root state to the goal state.

ab

kr

– search usually results from a lack of knowledge.

ha

C

C

– search explores knowledge alternatives to arrive at the best answer.

R

– search algorithm output is a solution, ie, a path from the initial state to a state that satisfies the goal test.

For general-purpose problem solving : "Search" is an approach.

– search deals with finding nodes having certain properties in a graph that represents search space.

– search methods explore the search space "intelligently", evaluating possibilities without investigating every single possibility.

Example : Search tree

The search trees are multilevel indexes used to guide the search for data items, given some search criteria.

Nodes The search starts at root and

A Root

explores nodes looking for

Goal Nodes

a goal node, that satisfies

B C certain conditions depending on the problem.

D E F G For some problems, any goal node, N or J , is acceptable;

H I J K For other problems, it may be a minimum depth goal node, say J which is nearest

L M N O P Q

to root.

Fig. Tree Search.

29

fo

.in

rs

de

AI - Search and Control strategies

ea

• Search Algorithms :

yr

.m

w

w

Many traditional search algorithms are used in AI applications. For complex

,w

ty

or

problems, the traditional algorithms are unable to find the solution within

ab

kr

some practical time and space limits. Consequently, many special techniques

ha

C

C

are developed, using heuristic functions .

R

The algorithms that use heuristic functions are called heuristic algorithms.

– Heuristic algorithms are not really intelligent; they appear to be intelligent because they achieve better performance.

– Heuristic algorithms are more efficient because they take advantage of feedback from the data to direct the search path.

– Uninformed search algorithms or Brute-force algorithms search, through the search space, all possible candidates for the solution checking whether each candidate satisfies the problem's statement.

– Informed search algorithms use heuristic functions, that are specific to the problem, apply them to guide the search through the search space to try to reduce the amount of time spent in searching.

A good heuristic can make an informed search dramatically out-perform any uninformed search. For example, the Traveling Salesman Problem (TSP) where the goal is to find a good solution instead of finding the best solution.

In TPS like problems, the search proceeds using current information about the problem to predict which path is closer to the goal and follow it, although it does not always guarantee to find the best possible solution. Such techniques help in finding a solution within reasonable time and space. Some prominent intelligent search algorithms are stated below.

1. Generate and Test Search 4. A* Search
2. Best-first Search 5. Constraint Search
3. Greedy Search 6. Means-ends analysis

There are more algorithms, either an improvement or combinations of these.

30

fo

.in

rs

de

AI - Search and Control strategies

ea

- Hierarchical Representation of Search Algorithms

yr

.m

w

w

A representation of most search algorithms is illustrated below. It begins

,w

ty

or

with two types of search - Uninformed and Informed.

ab

kr

Also called blind, exhaustive or brute-force search,

ha

Uninformed Search :

C

C

uses no information about the problem to guide the search and therefore

R

may not be very efficient.

Informed Search : Also called heuristic or intelligent search, uses information about the problem to guide the search, usually guesses the distance to a goal state and therefore efficient, but the search may not be always possible.

Search Algorithms

G (State, Operator, Cost)

No heuristics User heuristics $h(n)$

Uninformed Search Informed Search

Priority

LIFO Stack FIFO Queue

Queue: $g(n)$

Depth- Breadth- Cost- Generate Hill

First Search First Search First Search -and- Climbing

(DFS) (BFS) test

Impose fixed

depth limit

Priority

Depth Queue: $h(n)$

Limited

Search Best first Problem Constraint Mean-end-

search Reduction satisfaction analysis

Gradually increase

fixed depth limit

Iterative Priority Queue:

$f(n)=h(n)+g(n)$

Deepening

DFS AO*

A* Search Search

Fig. Different Search Algorithms

31

fo

.in

rs

de

AI - Search and Control strategies

ea

- Search Space

yr

.m

w

w

A set of all states , which can be reached, constitute a search space.

,w

ty

or

This is obtained by applying some combination of operators defining their

ab

kr

connectivity.

ha

C

C

R

Example :

Find route from Start to Goal state.

Consider the vertices as city and the edges as distances.

4 4

3 A B C

S

5 5 G

4 2 4 3

D E F

Fig. Search Space

– Initial State S

– Goal State G

– Nodes represent cities

– Arcs represent distances

32

fo

.in

rs

de

AI - Search and Control strategies

ea

• Formal Statement :

yr

.m

w

w

Problem solving is a set of statements describing the desired states

,w

ty

or

expressed in a suitable language; e.g., first-order logic.

ab

kr

ha

C

The solution of many problems can be described by finding a sequence of

C

R

actions that lead to a desired goal (e.g., problems in chess and cross).

– the aim is to find the sequence of actions that lead from the initial (start) state to a final (goal) state.

– each action changes the state.

A well-defined problem can be described by an example stated below.

Example

◇ Initial State: (S)

◇ Operator or successor function : for any state x , returns $s(x)$, the set of states reachable from x with one action.

◇ State space : all states reachable from initial by any sequence of actions.

◇ Path : sequence through state space.

◇ Path cost : function that assigns a cost to a path;

cost of a path is the sum of costs of individual actions along the path.

◇ Goal state : (G)

◇ Goal test : test to determine if at goal state.

33

fo

.in

rs

de

AI - Search and Control strategies

ea

• Search notations

yr

.m

w

w

Search is the systematic examination of states to find path from the

,w

ty

or

start or root state to the goal state. The notations used for defining

ab

kr

search are

ha

C

C

– $f(n)$ is evaluation function that estimates least cost solution

R

through node n.

– $h(n)$ is heuristic function that estimates least cost path from node n to goal node.

– $g(n)$ is cost function that estimates least cost path from start node to node n.

The relation among three parameters are expressed

$$f(n) = g(n) + h(n)$$

actual estimate

Start n Goal

$g(n)$ $h(n)$

$f(n)$

◇ If $h(n) \leq$ actual cost of shortest path from node n to goal then $h(n)$ is an under-estimate.

◇ The estimated values of f , g , h are expressed as $\hat{f}$, $\hat{g}$, $\hat{h}$

$\hat{f}$, $\hat{g}$, $\hat{h}$

$\hat{g}$

$\hat{h}$

$\hat{f}(n)$

$\hat{g}(n) + \hat{h}(n)$

$\hat{h}(n)$

Note: For easy to write, symbol $\hat{f}$ is replaced by symbol f^* and written

g is replaced with g^* and

h is replaced with h^*

◇ The estimates of g^* and h^* are given in the next slide

34

fo

.in

rs

de

AI - Search and Control strategies

ea

- Estimate Cost Function g^*

yr

.m

w An estimated least cost path from start node to node n , is

w

,w

ty

written as $g^*(n)$.

or

ab

kr

ha

◇ g^* is calculated as the actual cost so far of the explored path.

C

C

R

◇ g^* is known by summing all path costs from start to current state.

◇ If search space is a tree, then $g^* = g$, because there is only one path from start node to current node.

◇ In general, the search space is a graph.

◇ If search space is a graph, then $g^* \geq g$,

– g^* can never be less than the cost of the optimal path; rather it can only over estimate the cost.

– g^* can be equal to g in a graph if chosen properly.

35

fo

.in

rs

de

AI - Search and Control strategies

ea

- Estimate Heuristic Function h^*

yr

.m

w

w

,w

An estimated least cost path from node n to goal node ,

ty

or

is written as $h^*(n)$

ab

kr

ha

C

◇ h^* is a heuristic information, it represents a guess; example :

C

R

"how hard it is to reach from current node to goal state ? ".

◇ h^* may be estimated using an evaluation function $f(n)$ that measures "goodness" of a node.

◇ h^* may have different values;

the values lie between $0 \leq h^*(n) \leq h(n)$;

they mean a different search algorithm.

◇ If $h^* = h$, it is a "perfect heuristic";

means no unnecessary nodes are ever expanded.

36

fo

.in

rs

de

AI - Search and Control strategies

ea

2.3 Control Strategies

yr

.m

w

w

,w

Search for a solution in a problem space, requires "Control Strategies" to

ty

or

control the search processes.

ab

kr

ha

C

The search control strategies are of different types, and are realized

C

R

by some specific type of "Control Structures".

- Strategies for search

Some widely used control strategies for search are stated below.

◇ Forward search : Here, the control strategies for exploring search proceeds forward from initial state to wards a solution; the methods are called data-directed.

◇ Backward search : Here, the control strategies for exploring search proceeds backward from a goal or final state towards either a solvable sub problem or the initial state; the methods are called goal directed.

◇ Both forward and backward search : Here, the control strategies for exploring search is a mixture of both forward and backward strategies .

◇ Systematic search : Where search space is small, a systematic (but blind) method can be used to explore the whole search space.

One such search method is depth-first search and the other is breath-first search.

37

fo

.in

rs

de

AI - Search and Control strategies

ea

◇ Heuristic search : Many search depend on the knowledge of the

yr

.m

w problem domain. They have some measure of relative merits to guide

w

,w

ty

the search. The search so guided are called heuristic search and the

or

ab

methods used are called heuristics .

kr

ha

C

C

R

Note : A heuristic search might not always find the best solution

but it is guaranteed to find a good solution in reasonable time.

Heuristic Search Algorithms :

- First, generate a possible solution which can either be a point in the problem space or a path from the initial state.
- Then, test to see if this possible solution is a real solution by comparing the state reached with the set of goal states.
- Lastly, if it is a real solution, return, else repeat from the first again.

38

fo

.in

rs

de

AI - Search and Control strategies

ea

- More on Search Strategies : Related terms

yr

.m

w

w

The Condition-action rules and Chaining : Forward and Backward

,w

ty

or

ab

◇ Condition-action rules

kr

ha

C

– one way of encoding Knowledge is condition-action rules

C

R

– the rules are written as if < condition> then < conclusion >

Rule: Red_Light IF the light is red THEN Stop

Rule: Green_Light IF the light is green THEN Go

antecedent consequent

◇ Chaining

– Chaining refers to sharing conditions between rules, so that the same condition is evaluated once for all rules.

– When one or more conditions are shared between rules, they are considered "chained."

– Chaining are of two types : Forward and Backward chaining.

– Forward chaining is called data-driven and

– Backward chaining is called query-driven.

◇ Activation of Rules

– The Forward chaining and Backward chaining are the two different strategies for activation of rules in the system.

– Forward and Backward chaining are some techniques for drawing inferences from Rule base.

39

fo

.in

rs

de

AI - Search and Control strategies

ea

• Forward Chaining Algorithm

yr

.m

w

w

Forward chaining is a techniques for drawing inferences from Rule

,w

ty

or

base. Forward-chaining inference is often called data driven.

ab

kr

ha

C

◇ The algorithm proceeds from a given situation to a desired goal,

C

R

adding new assertions (facts) found.

◇ A forward-chaining, system compares data in the working memory against the conditions in the IF parts of the rules and determines which rule to fire.

◇ Data Driven

Data Rules Conclusion

a=1 if a = 1 & b = 2 then c = 3,

b=2 if c = 3 then d = 4 d=4

◇ Example : Forward Channing

■ Given : A Rule base contains following Rule set;

Rule 1: If A and C Then F

Rule 2: If A and E Then G

Rule 3: If B Then E

Rule 4: If G Then D

■ Problem : Prove that

If A and B true Then D is true

40

fo

.in

rs

de

AI - Search and Control strategies

ea

■ Solution : (continued - the Forward Chaining problem)

yr

.m

w (i) ‡ Start with input given A, B is true and then

w

,w

ty

‡ start at Rule 1 and go forward / down till a rule

or

ab

"fires" is found.

kr

ha

C

First iteration :

C

R

(ii) ‡ Rule 3 fires : conclusion E is true

‡ new knowledge found

(iii) ‡ No other rule fires;

‡ end of first iteration.

(iv) ‡ Goal not found;

‡ new knowledge found at (ii);

‡ go for second iteration

Second iteration :

(v) ‡ Rule 2 fires : conclusion G is true

‡ new knowledge found

(vi) ‡ Rule 4 fires : conclusion D is true

‡ Goal found;

‡ Proved

41

fo

.in

rs

de

AI - Search and Control strategies

ea

- Backward Chaining Algorithm

yr

.m

w

w

Backward chaining is a techniques for drawing inferences from Rule base.

,w

ty

or

Backward-chaining inference is often called goal driven.

ab

kr

ha

C

◇ The algorithm proceeds from desired goal, adding new assertions

C

R

found.

◇ A backward-chaining, system looks for the action in the THEN clause of the rules that matches the specified goal.

◇ Goal Driven

Data Rules Conclusion

a=1 if a = 1 & b = 2 then c = 3,

b=2 if c = 3 then d = 4 d=4

◇ Example : Backward Channing

◇ Given : Rule base contains following Rule set

Rule 1: If A and C Then F

Rule 2: If A and E Then G

Rule 3: If B Then E

Rule 4: If G Then D

◇ Problem : Prove

If A and B true Then D is true

42

fo

.in

rs

de

AI - Search and Control strategies

ea

◇ Solution : (continued - the Backward Chaining problem)

yr

.m

w (i) ‡ Start with goal ie D is true

w

,w

ty

‡ go backward/up till a rule "fires" is found.

or

ab

First iteration :

kr

ha

C

(ii) ‡ Rule 4 fires :

C

R

‡ new sub goal to prove G is true

‡ go backward

(iii) ‡ Rule 2 "fires"; conclusion: A is true

‡ new sub goal to prove E is true

‡ go backward;

(iv) ‡ no other rule fires; end of first iteration.

‡ new sub goal found at (iii);

‡ go for second iteration

Second iteration :

(v) ‡ Rule 3 fires :

‡ conclusion B is true (2nd input found)

‡ both inputs A and B ascertained

‡ Proved

43

fo

.in

rs

de

AI - Exhaustive Search

ea

3. Exhaustive Search

yr

.m

w

w

Besides Forward and Backward chaining explained, there are many other

,w

ty

or

search strategies used in computational intelligence. Among the most

ab

kr

ha

commonly used approaches are :

C

C

R

Breadth-first search (BFS) and depth-first search (DFS).

A search is said to be exhaustive if the search is guaranteed to generate all reachable states (outcomes) before it terminates with failure.

A graphical representation of all possible reachable states and the paths by which they may be reached is called decision tree.

Breadth-first search (BFS) : A Search strategy, in which the highest layer of a decision tree is searched completely before proceeding to the next layer is called Breadth-first search (BFS).

- In this strategy, no viable solution is omitted and therefore guarantee that optimal solution is found.
- This strategy is often not feasible when the search space is large.

Depth-first search (DFS) : A search strategy that extends the current path as far as possible before backtracking to the last choice point and trying the next alternative path is called Depth-first search (DFS).

- This strategy does not guarantee that the optimal solution has been found.
- In this strategy, search reaches a satisfactory solution more rapidly than breadth first, an advantage when the search space is large.

The Breadth-first search (BFS) and depth-first search (DFS) are the foundation for all other search techniques.

44

fo

.in

rs

de

AI - Exhaustive Search

ea

- Breadth-First Search Strategy (BFS)

yr

.m

w

w

This is an exhaustive search technique.

,w

ty

or

ab

◇ The search generates all nodes at a particular level before proceeding

kr

ha

to the next level of the tree.

C

C

R

◇ The search systematically proceeds testing each node that is reachable from a parent node before it expands to any child of those nodes.

◇ The control regime guarantees that the space of possible moves is systematically examined; this search requires considerable memory resources.

◇ The space that is searched is quite large and the solution may lie a thousand steps away from the start node. It does, however, guarantee that if we find a solution it will be the shortest possible.

◇ Search terminates when a solution is found and the test returns true.

45

fo

.in

rs

de

AI - Exhaustive Search

ea

- Depth-First Search Strategy (DFS)

yr

.m

w

w

This is an exhaustive search technique to an assigned depth.

,w

ty

or

ab

◇ Here, the search systematically proceeds to some depth d, before

kr

ha

another path is considered.

C

C

R

◇ If the maximum depth of search tree is three, then if this limit is

reached and if the solution has not been found, then the search backtracks to the previous level and explores any remaining alternatives at this level, and so on.

◇ It is this systematic backtracking procedure that guarantees that it will systematically and exhaustively examine all of the possibilities.

◇ If the tree is very deep and the maximum depth searched is less than the maximum depth of the tree, then this procedure is "exhaustive modulo of depth" that has been set.

46

fo

.in

rs

de

AI - Exhaustive Search

ea

- Depth-First Iterative-Deepening (DFID)

yr

.m

w

w

DFID is another kind of exhaustive search procedure which is a blend of

,w

ty

or

depth first and breadth first search.

ab

kr

ha

C

Algorithm : Steps

C

R

- First perform a Depth-first search (DFS) to depth one.

- Then, discarding the nodes generated in the first search, starts all over and do a DFS to level two.

- Then three until the goal state is reached.

47

fo

.in

rs

de

AI - Exhaustive Search

ea

3.1 Depth-First Search (DFS)

yr

.m

w

w

Here explained the Depth-first search tree, the backtracking to the

,w

ty

or

previous level, and the Depth-first search algorithm

ab

kr

ha

C

◇ DFS explores a path all the way to a leaf before backtracking and

C

R

exploring another path.

◇ Example: Depth-first search tree

A Goal Nodes

B C

D E F G

H I J K

L M N O P Q

Fig. Depth-first search (DFS)

■ Node are explored in the order :

A B D E H L M N I O P C F G J K Q

■ After searching node A, then B, then D, the search backtracks and tries another path from node B .

■ The goal node N will be found before the goal node J.

48

fo

.in

rs

de

AI - Exhaustive Search

ea

◇ Algorithm - Depth-first search

yr

.m

w ■ Put the root node on a stack;

w

,w

ty

while (stack is not empty)

or

ab

kr

{ remove a node from the stack;

ha

C

if (node is a goal node) return success;

C

R

put all children of node onto the stack; }

return failure;

Note :

‡ At every step, the stack contains some nodes from each level.

‡ The stack size required depends on the branching factor b .

‡ Searching level n , the stack contains approximately $b * n$ nodes.

‡ When this method succeeds, it does not give the path.

‡ To hold the search path the algorithm required is “Recursive depth-first search” and stack size large.

49

fo

.in

rs

de

AI - Exhaustive Search

ea

3.2 Breadth-First Search (BFS)

yr

.m

w

w

Here explained, the Breadth-first search tree, and the Breadth-first

,w

ty

or

search algorithm.

ab

kr

ha

C

◇ BFS explores nodes nearest to the root before exploring nodes

C

R

that are father or further away.

◇ Example: Breadth-first search tree

A Goal nodes

B C

D E F G

H I J K

L M N O P Q

Fig. Breadth-first search (BFS)

■ Node are explored in the order :

A B C D E F G H I J K L M N O P Q

■ After searching A, then B, then C, the search proceeds with D, E, F, G,

■ The goal node J will be found before the goal node N.

50

fo

.in

rs

de

AI - Exhaustive Search

ea

◇ Algorithm - Breadth-first search

yr

.m

w

```

w
■ Put the root node on a queue;
,w
ty
or
while (queue is not empty)
ab
kr
{ remove a node from the queue;
ha
C
C
if (node is a goal node) return success;
R
put all children of node onto the queue; }
return failure;

```

Note :

- ‡ Just before the start to explore level n , the queue holds all the nodes at level $n-1$.
- ‡ In a typical tree, the number of nodes at each level increases exponentially with the depth.
- ‡ Memory requirements may be infeasible.
- ‡ When this method succeeds, it does not give the path.
- ‡ There is no “recursive” breadth-first search equivalent to recursive depth-first search.

51

fo

.in

rs

de

AI - Exhaustive Search

ea

3.3 Compare Depth-First and Breadth-First Search

yr

.m

w

w

Here the Depth-first and Breadth-first search are compared at algorithm

,w

ty
or
ab

level, at feature level and how to over come the limitations.

kr
ha
C

Depth-first search Breadth-first search

C
R

◇ Compare Algorithms

Put the root node on a stack; Put the root node on a queue;
while (stack is not empty) while (queue is not empty)
{ {
remove a node from the stack; remove a node from the queue;
if (node is a goal node) if (node is a goal node)
return success; return success;
put all children of node put all children of node
onto the stack; onto the queue;
} }
return failure; return failure;

◇ Compare Features

‡ When succeeds, the goal node ‡ When succeeds, it finds a
found is not necessarily minimum-depth (nearest to
minimum depth root) goal node
‡ Large tree, may take ‡ Large tree, may require
excessive long time to find excessive memory
even a nearby goal node

◇ How to over come limitations of DFS and BFS ?

‡ Requires, how to combine the advantages and avoid disadvantages?
‡ The answer is “Depth-limited search” .

This means, perform Depth-first searches with a depth limit.

52

fo
.in
rs
de

AI - Heuristic Search Techniques

ea

4. Heuristic Search Techniques

yr

.m

w

w

For complex problems, the traditional algorithms, presented above, are

,w

ty

or

unable to find the solution within some practical time and space limits.

ab

kr

Consequently, many special techniques are developed, using heuristic functions

ha

C

C

R

– Blind search is not always possible, because they require too much time or Space (memory).

– Heuristics are rules of thumb; they do not guarantee for a solution to a problem.

– Heuristic Search is a weak techniques but can be effective if applied correctly; they require domain specific information

53

fo

.in

rs

de

AI - Heuristic Search Techniques

ea

4.1 Characteristics of Heuristic Search

yr

.m

w

w

◇ Heuristics, are knowledge about domain, which help search and

,w

ty

or

reasoning in its domain.

ab

kr

ha

C

◇ Heuristic search incorporates domain knowledge to improve efficiency

C

R

over blind search.

◇ Heuristic is a function that, when applied to a state, returns value as estimated merit of state, with respect to goal.

■ Heuristics might (for reasons) under estimate or over estimate the merit of a state with respect to goal.

■ Heuristics that under estimates are desirable and called admissible.

◇ Heuristic evaluation function estimates likelihood of given state leading to goal state.

◇ Heuristic search function estimates cost from current state to goal, presuming function is efficient.

54

fo

.in

rs

de

AI - Heuristic Search Techniques

ea

4.2 Heuristic Search compared with other search

yr

.m

w

w

The Heuristic search is compared with Brute force or Blind search

,w

ty

or

techniques

ab

kr

ha

C

Compare Algorithms

C

R

Brute force / Blind search Heuristic search

◊ Only have knowledge about ◊ Estimates “distance” to goal state
already explored nodes

◊ No knowledge about how far a ◊ Guides search process toward
node is from goal state goal state

◊ Prefer states (nodes) that lead
close to and not away from goal
state

55

fo

.in

rs

de

AI - Heuristic Search Techniques

ea

- Example : 8 - Puzzle

yr

.m

w

w

◊ State space: Configuration of 8- tiles on the board

,w

ty

or

ab

◊ state Initial: any configuration Goal: tiles in a specific order

kr

ha

C

C

R

1 2 3 1 2 3

7 8 4 8 4

6 5 7 6 5

◊ Solution: optimal sequence of operators

◊ Action: “blank moves”

- Condition: the move is within the board

- Directions: Left, Right, Up, Dn

◇ Problem

- which 8-puzzle move is best?

- what heuristic(s) can decide?

- which move is “best” (worth considering first) ?

56

fo

.in

rs

de

AI - Heuristic Search Techniques

ea

◇ Actions

yr

.m

w

w

Figure below shows: three possible moves - left , up, right

,w

ty

or

ab

kr

1 2 3 Goal 1 2 3

ha

Initial State

C

C

8 4 7 8 4

R

7 6 5 6 5

Left Up

Right

1 2 3 1 2 3 1 2 3

7 8 4 7 8 4 7 4

6 5 6 5 6 8 5

Count correct h=6 h=4 h=5

positions

Find Which move is best ?

◇ Apply the Heuristic : Three different approaches

- Count correct position of each tile, compare to goal state
- Count incorrect position of each tile, compare to goal state
- Count how far away each tile is from its correct position.

Approaches Left Right Up

1. Count correct position 6 4 5
2. Count incorrect position 2 4 3
3. Count how far away 2 4 4

Each of these three approaches are explained below.

57

fo

.in

rs

de

AI - Heuristic Search Techniques

ea

◇ Heuristic :

yr

.m

w

w

Three different approaches

,w

ty

or

ab

■ 1st approach :

kr

ha

Count correct position of each tile, compare to goal state.

C

C

R

‡ Higher the number the better it is.

‡ Easy to compute (fast and takes little memory).

‡ Probably the simplest possible heuristic.

■ 2nd approach

Count incorrect position of each tile, compare to goal state

‡ Lower the number the better it is.

‡ The “best” move is where lowest number returned by heuristic.

■ 3rd approach

Count how far away each tile is from it's correct position

‡ Count how far away (how many tile movements) each tile is from it's correct position.

‡ Sum up these count over all the tiles.

‡ The “best” move is where lowest number returned by heuristic.

58

fo

.in

rs

de

AI - Heuristic Search Techniques

ea

4.3 Heuristic Search Algorithms : types

yr

.m

w

w

◇ Generate-And-Test

,w

ty

or

ab

◇ Hill climbing ■ Simple

kr

ha

C

■ Steepest-Ascent Hill climbing

C

R

■ Simulated Annealing

◇ Best First Search ■ OR Graph

■ A* (A-Star) Algorithm

■ Agendas

◇ Problem Reduction ■ AND-OR_Graph

■ AO* (AO-Star) Algorithm

◇ Constraint Satisfaction

◇ Mean-end Analysis

59

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

5. Constraint Satisfaction Problems (CSPs) and Models

yr

.m

w

w

Constraints arise in most areas of human endeavor.

,w

ty

or

Constraints are a natural medium for people to express problems in many fields.

ab

kr

Many real problems in AI can be modeled as Constraint Satisfaction Problems

ha

C

C

(CSPs) and are solved through search.

R

Examples of constraints :

- The sum of three angles of a triangle is 180 degrees,
- The sum of the currents flowing into a node must equal zero.

◇ Constraint is a logical relation among variables.

- the constraints relate objects without precisely specifying their positions; moving any one, the relation is still maintained.
- example : “circle is inside the square”.

◇ Constraint satisfaction

The Constraint satisfaction is a process of finding a solution to a set of constraints.

- the constraints articulate allowed values for variables, and
- finding solution is evaluation of these variables that satisfies all constraints.

◇ Constraint Satisfaction problems (CSPs)

The CSPs are all around us while managing work, home life, budgeting expenses and so on;

- where we do not get success in finding solution, there we run into problems.
- we need to find solution to such problems satisfying all constraints.
- the Constraint Satisfaction problems (CSPs) are solved through search.

60

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

5.1 Examples of CSPs

yr

.m

w

w

Some popular puzzles like, the Latin Square, the Eight Queens, and Sudoku

,w

ty

or

are stated below.

ab

kr

ha

C

◇ Latin Square Problem : How can one fill an $n \times n$ table with n different

C

R

symbols such that each symbol occurs exactly once in each row and each column ?

Solutions : The Latin squares for $n = 1, 2, 3$ and 4 are :

1 2 3 4

1 2 3

1 2 2 3 4 1

1 2 3 1

2 1 3 4 1 2

3 1 2

4 1 2 3

◇ Eight Queens Puzzle Problem : How can one put 8 queens on a (8 x 8) chess board such that no queen can attack any other queen ?

Solutions: The puzzle has 92 distinct solutions. If rotations and reflections of the board are counted as one, the puzzle has 12 unique solutions.

◇ Sudoku Problem : How can one fill a partially completed (9 × 9) grid such that each row, each column, and each of the nine (3 × 3) boxes contains the numbers from 1 to 9.

Problem Solution

2 6 8 1 7 2 6 4 9 3 8 1 5

3 7 8 6 3 1 5 7 2 8 9 4 6

4 5 7 4 8 9 6 5 1 2 3 7

5 1 7 9 8 5 2 1 4 7 6 9 3

3 9 5 1 6 7 3 9 8 5 1 2 4

4 3 2 5 9 4 1 3 6 2 7 5 8

1 3 2 1 9 4 8 3 6 5 7 2

5 2 4 9 5 6 7 2 1 4 3 8 9

3 8 4 6 2 3 8 5 7 9 4 6 1

61

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

5.2 Constraint Satisfaction Models

yr

.m

w

w

Humans solve the puzzle problems stated above while

,w

ty

or

– trying with different configurations, and

ab

kr

– using various insights about the problem to explore only a small number

ha

C

C

of configurations.

R

It is not clear what these insights are ?

For $n = 8$ queens, on a standard chess board (8×8), such that no queen can attack any other queen, the puzzle has 92 distinct solutions.

Humans would find it hard to solve N-Queens puzzle while N becomes more.

Example : The possible number of configurations are :

– For 4-Queens there are 256 different configurations.

– For 8-Queens there are 16,777,216 configurations.

– For 16-Queens there are 18,446,744,073,709,551,616 configurations.

N

– In general, for N - Queens there are we have N configurations.

– For $N = 16$, this would take about 12,000 years on a fast machine.

How do we solve such problems ?

Three computer based approaches or models are stated below. They are

‡ Generate and Test (GT) ,

‡ Backtracking (BT) and

‡ Constrain Satisfaction Problems (CSPs)

62

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

◇ Generate and Test (GT) : $n = 4$ Queens puzzle

yr

.m

w

w

One possible solution is to systematically try every placement of queens

,w

ty

or

until we find a solution.

ab

kr

The process is known as "Generate and Test".

ha

C

C

R

Examples of Generate and Test conditions for solutions :

Q Q Q Q

Q Q Q Q

Q Q Q Q

Q Q Q Q

Q Q Q

Q Q Q

Q Q Q

- - - - -

Q Q Q

63

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

◇ Backtracking (BT) : $n = 4$ Queens puzzle

yr

.m

w

w

The Backtracking method is based on systematic examination of the

,w

ty

or

possible solutions.

ab

kr

– The algorithms try each possibility until they find the right one.

ha

C

C

- It differs from simple brute force, which generates all solutions,

R

even those arising from infeasible partial solutions.

Backtracking is similar to a depth-first search but uses less space, keeping just one current solution state and updating it.

‡ during search, if an alternative does not work, the search backtracks to the choice point, the place which presented different alternatives, and tries the next alternative.

‡ when the alternatives are exhausted, the search returns to the previous choice point and tries the next alternative there.

‡ if there are no more choice points, the search fails.

This is usually achieved in a recursive function where each instance takes one more variable and alternatively assigns all the available values to it, keeping the one that is consistent with subsequent recursive calls.

Note : feasible, infeasible, pruning the solution tree.

- The partial solutions are evaluated for feasibility.

- A partial solution is said to be feasible if it can be developed by further choices without violating any of the problem's constraints.

- A partial solution is said to be infeasible if there are no legitimate options for any remaining choices.

- The abandonment of infeasible partial solutions is called pruning the solution tree.

Backtracking is most efficient technique for problems, like n-Queens problem. An example below illustrates to solve $N = 4$ Queens problem.

64

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

Example : Backtracking to solve $N = 4$ Queens problem.

yr

.m

w

w
 ,w
 ty
 Start H
 or
 A
 ab
 0,0
 kr
 Q Q
 ha
 C
 C
 R
 1,1 1,2
 B D I
 Q Q Q
 Q Q Q Q Q Q Q Q
 2,1 2,2 2,3 2,4 2,1 2,2 2,3 2,4
 C G J
 Q Q Q
 Q Q Q
 3,1 3,2 3,3 3,4 3,1 3,2 3,3 3,4 3,1
 Q Q Q Q Q Q Q
 E
 Dead end, Dead end,
 backtrack Q
 backtrack
 Up to (1,1) Q Up to (0,0)
 F Q Q K L
 Q Q
 Q Q Q
 Q Q Q
 4,1 4,2 4,3 4,4 4,1 4,2 4,3
 Q Q Q
 Q Q Q Q Q Q Q Q
 Dead end,
 backtrack Solution
 Up to (2,4)

Fig. Backtracking to solve $N = 4$ Queens problem.

The Fig above illustrates the state space tree for the instance of the $N=4$ Queens problem. The ordered pair (i, j) in each node indicates a possible (row, column) placement of a queen.

65

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

Algorithm : Backtracking to solve N Queens problem.

yr

.m

w

w

The problem proceeds either by rows or by columns.

,w

ty

or

‡ for no particularly good reason, select columns to proceed.

ab

kr

ha

‡ for each column, select a row to place the queen.

C

C

R

‡ a partial solution is feasible if no two queens can attack each other;

Note : No feasible solution can contain an infeasible partial solution.

1. Move “left to right” processing one column at a time.

2. For column J , select a row position for the queen. Check for feasibility.

a. If there are one or more attacks possible from queens in columns 1 through $(J - 1)$, discard the solution.

b. For each feasible placement in column J , make the placement and try placement in column $(J + 1)$.

c. If there are no more feasible placements in column J , return to column $(J - 1)$ and try another placement.

3. Continue until all N columns are assigned or until no feasible solution

is found.

66

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

◇ Constrain Satisfaction Problems (CSPs)

yr

.m

w

w

The Backtracking just illustrated, is one main method for solving

,w

ty

or

problems like N-Queens but what is required is a generalized solution.

ab

kr

ha

C

Design algorithms for solving the general class of problems.

C

R

For an algorithm not just for solving N-Queens problem, we need to express N-Queens as an instance of a general class of problems;

The N-queens problem can be represented as a constraint satisfaction problem (CSPs).

In CSPs, we find states or objects that satisfy a number of constraints or criteria. The CSPs are solved through search.

Before we deal with CSPs, we need to define CSPs and the domain related properties of the constraints. Here we illustrate :

- the definition of CSPs,
- the properties of CSPs and
- the algorithms for CSPs.

67

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

■ Definition of a CSPs

yr

.m

w

w

The formal definition of a CSP involves variables, their domains, and

,w

ty

or

constraints. A constraint network is defined by

ab

kr

– a set of variables, $V_1, V_2, \dots, V_n$,

ha

C

C

– a domain of values, $D_1, D_2, \dots, D_n$ for each variable;

R

– all variables V_i have a value in their respective domain D_i .– a set of constraints, $C_1, C_2, \dots, C_m$;– a constraint C_i restricts the possible values in the domain of some subset of the variables.

– Problem : Is there a solution of the network, i.e., an assignment of values to the variables such that all constraints are satisfied ?

– Solution to a CSP : Is an assignment to every variable by some value in its domain such that every constraint is satisfied. Also each assignment of a value to a variable must be consistent, i.e. it must not violate any of the constraints.

Example :

Variables ■ $V_1, V_2, V_3, \dots$ with Domains $D_1, D_2, D_3, \dots$ Constraints ■ Set of allowed value pairs $\{(red, blue), (green, blue), (green, red)\}$ ■ V_1 "not equal to" V_2 ,

Solution ■ Assign values to variables that satisfy all constraints

■ $V1 = \text{red}, V2 = \text{blue}, V3 = \text{green},$

$V1 D1 = \{(\text{red}, \text{green})\}$

$\neq \neq$

$D2 = \{(\text{blue}, \text{red})\} V2 V3 D3 = \{(\text{green})\}$

68

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

■ Properties of CSPs

yr

.m

w

w

Constraints are used to guide reasoning of everyday common

,w

ty

or

sense. The constraints enjoy several interesting properties.

ab

kr

– Constraints may specify partial information; constraint need not

ha

C

C

uniquely specify the values of its variables;

R

– Constraints are non-directional, typically a constraint on (say)

two variables $V1, V2$ can be used to infer a constraint on $V1$ given

a constraint on $V2$ and vice versa;

– Constraints are declarative; they specify what relationship must

hold without specifying a computational procedure to enforce that relationship;

– Constraints are additive; the order of imposition of constraints

does not matter, all that matters at the end is that the

conjunction of constraints is in effect;

– Constraints are rarely independent; typically constraints in the constraint store share variables.

69

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

■ Algorithms for CSPs : Consider $n = 8$ Queens puzzle

yr

.m

w

w

As stated above, a CSP consists of 3 components :

,w

ty

or

– A set of variables,

ab

kr

– A set of values for each of the variables and

ha

C

C

– A set of constraints imposed between the variables.

R

Find a value for each variables that satisfies all the constraints.

‡ Constraints

– A constraint is a relation between a local collection of variables.

– The constraint restricts the values that these variables can simultaneously have.

– Examples : one constraint All-diff(X_1, X_2, X_3).

This constraint says that X_1, X_2, X_3 must take on different values.

If $\{1, 2, 3\}$ is the set of values for each of these variables

then: $X_1 = 1, X_2 = 2, X_3 = 3$ is ok and $X_1 = 1, X_1 = 1, X_1 = 3$

not ok.

‡ Finding a Solution

– Finding a global assignment to all of the variables that

satisfies all of the constraints is hard:NP-Complete

(nondeterministic polynomial-time hard)

– The solution techniques work by cleverly searching through

the space of possible assignments of values to variables.

– If each variable has d values and there are n variables, then we have d^n possible assignments.

70

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

‡ Representations : $N = 8$ Queens as a CSP

yr

.m

w This problem can be represented as a CSP in different ways.

w

,w

ty

or

Representation 1

ab

kr

ha

– we need to know where to place each of the N queens.

C

C

R

– we could have N variables each of which has as a value

2

1. . . N .

th

– The values represent where we will place the i variable.

$Q_1 = 1$ This representation has

$Q_2 = 15$

$Q_3 = 21 \ 648 = 281,474,976,710,656$

Q4 = 32 different possible assignments

Q5 = 34

Q6 = 44 in the search space.

Q7 = 54

Q8 = 59

Representation 2

– Here we know, we can not place two queens in the same column.

– Assign one queen to each column, and then find out the rows where each of these queens is to placed.

– We can have N variables: $Q_1, \dots, Q_N$.

– The set of values for each of these variables are

$\{1, 2, \dots, N\}$.

This representation has

Q1 = 18

8 = 16,777,216

Q7 = 2

Q5 = 3 different possible assignments

Q8 = 4

in the search space.

Q2 = 5

Q4 = 6

Q6 = 7 It is still too large to examine

Q3 = 8

all of them, but definitely a big improvement.

71

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

‡ Constraints :

yr

.m

w

w

Translate each of individual conditions into a separate constraint.

,w

ty

or

ab

Condition 1 : Queens Cannot attack each other

kr

ha

C

If Q_i cannot attack Q_j for $(i \neq j)$

C

R

Then Q_i is a queen to be placed in column i , and

Q_j is a queen to be placed in column j .

The value of Q_i and Q_j are the rows the queens are to be placed.

Condition 2 : Queens can attack each other

– Vertically, if they are in any same column; it is impossible as Q_i and Q_j are placed in different columns.

– Horizontally, if they are in any same row; we need the constraint $Q_i \neq Q_j$.

– If along any diagonal; they cannot be in the same number of columns apart as they are rows apart: we need the constraint $|i - j| \neq |Q_i - Q_j|$; (symbol $| \cdot |$ is absolute value)

72

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

‡ Representing the Constraints

yr

.m

w

w

– Between every pair of variables (Q_i, Q_j) for $(i \neq j)$, the

,w

ty

or

constraint is C_{ij} .

ab

kr

– For each C_{ij} , an assignment of values to the variables

ha

C

C

$Q_i = A$ and $Q_j = B$, satisfies this constraint if and only

R

if $A \neq B$ and $|A - B| \neq |i - j|$.

‡ Solutions

A solution to N - Queens problem is any assignment of values to the variables $Q_1, \dots, Q_N$ that satisfies all of the constraints.

– Constraints can be over any collection of variables.

– The N - Queens problems, need only binary constraints, i.e.

constraints over pairs of variables.

– By simply enumerating and testing all possible assignments,

we can recognize a subset of the variables that make a

solution impossible. Thus we can largely improve upon

different possible assignments in the search space.

– Finally, by expressing the problem as a CSP we have a

systematic way of achieving extra efficiency.

73

fo

.in

rs

de

AI - Constraint Satisfaction problems and Models

ea

5.3 Remarks

yr

.m

w Few remarks on Generic Backtracking, Forward Checking and Variable

w

,w

ty

Ordering are stated below.

or

ab

kr

ha

◇ Generic Backtracking

C

C

R

- Generic Backtracking is the simplest and oldest algorithm for solving CSP problems.
- The idea is to search in a tree of variable assignments. As we move down the tree we assign a value to a new variable.
- Once we have assigned all of the variables that participate in a constraint, we check that constraint.
- At any point if a constraint is violated we backtrack up the tree.

Note :

- The idea of searching in a tree of variable assignments is very powerful. However generic backtracking (BT) is not a very good algorithm.
- Although BT is much faster than any simple enumeration, but then all algorithms for solving CSPs take time that can grow exponentially with the size of the problem.

◇ Forward Checking

- Forward Checking is based on the idea of looking ahead in the tree to see if we have already assigned a value to one of the unassigned variable impossible.
- It is based on the idea of pruning the domains of the unassigned variables.

◇ Variable Ordering

- Choosing a variable is critical to performance.
- The efficiency of search algorithms depends considerably on the order in which variables are considered for instantiations.
- This ordering effects the efficiency of the algorithm.
- There exist various heuristics for dynamic or static ordering of values and variables.

74

fo

.in

rs

de

AI-Problem solving, search and control - References

ea

6. References : Textbooks

yr

.m

w

w

1. "Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill

,w

ty

companies Inc., Chapter 2-3, page 29-98.

or

ab

kr

2. "Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig,

ha

C

(2002), Prentice Hall, Chapter 3-6, page 59-189.

C

R

3. "Computational Intelligence: A Logical Approach", by David Poole, Alan Mackworth, and Randy Goebel, (1998), Oxford University Press, Chapter 4, page 113-163.

4. "Artificial Intelligence: Structures and Strategies for Complex Problem Solving", by George F. Luger, (2002), Addison-Wesley, Chapter 2-6, page 35-193.

5. "AI: A New Synthesis", by Nils J. Nilsson, (1998), Morgan Kaufmann Inc., Chapter 7-9, Page 117-160.

6. "Artificial Intelligence: Theory and Practice", by Thomas Dean, (1994), Addison-Wesley, Chapter 3-4, Page 71-131.

7. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

75

fo

.in

rs

de

ea

yr

Knowledge Representations : AI Course Lecture 15 – 22, notes, slides

.m

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

www.myreaders.info

kr

ha

C

C

R

Knowledge Representation

Issues, Predicate Logic, Rules

Artificial Intelligence

Knowledge representation in AI, topics : knowledge progression, model, category, typology map, and relationship; Mapping between facts and representation, forward and backward representation, KR system requirements; KR schemes – relational, inheritable, inferential, declarative and procedural; Issues in KR - attributes, relationship, granularity. Knowledge representation using logic : Propositional logic - statements, variables, symbols, connective, truth value, contingencies, tautologies, contradictions, antecedent, consequent, argument; Predicate logic – predicate, logic expressions, quantifiers, formula; Representing “IsA” and “Instance” relationships; Computable functions and predicates. Knowledge representation using rules : declarative, procedural, and meta rules; Logic programming characteristics - statement, language, syntax, data objects, clause, predicate, sentence, subject, queries; Programming paradigms – models of computation, imperative, functional, and logic model; Reasoning, and conflict.

fo

.in

rs

de

ea

Knowledge Representation

yr

.m

w

w

Issues, Predicate Logic, Rules

,w

ty

or

Artificial Intelligence

ab

kr

ha

C

Topics

C

R

(Lectures 15, 16, 17, 18, 19, 20, 21, 22 8 hours)

Slides

1. Knowledge Representation 03-26

Introduction – Knowledge Progression, KR model, category : typology map, type, relationship, framework, mapping, forward & backward representation, KR system requirements; KR schemes - relational, inheritable, inferential, declarative, procedural; KR issues - attributes, relationship, granularity.

2. KR Using Predicate Logic 27-47

Logic as language; Logic representation : Propositional logic, statements, variables, symbols, connective, truth value, contingencies, tautologies, contradictions, antecedent, consequent, argument; Predicate logic – predicate, logic expressions, quantifiers, formula; Representing “IsA” and “Instance” relationships; Computable functions and predicates; Resolution.

3. KR Using Rules 48-78

Types of Rules - declarative, procedural, meta rules; Procedural verses declarative knowledge & language; Logic programming – characteristics, statement, language, syntax & terminology, Data components - simple & structured data objects, Program Components - clause, predicate, sentence, subject, queries; Programming paradigms – models of computation, imperative model, functional model, logic model; Reasoning - Forward and backward chaining, conflict resolution; Control knowledge.

4. References 79

02

fo

.in

rs

de

ea

yr

Knowledge Representation

.m

w

w

,w

Issues, Predicate Logic, Rules

ty

or

ab

kr

How do we Represent what we know ?

ha

C

C

R

- Knowledge is a general term.

An answer to the question, "how to represent knowledge", requires an analysis to distinguish between knowledge "how" and knowledge "that".

■ knowing "how to do something".

e.g. "how to drive a car" is a Procedural knowledge.

■ knowing "that something is true or false".

e.g. "that is the speed limit for a car on a motorway" is a Declarative knowledge.

- knowledge and Representation are two distinct entities. They play a central but distinguishable roles in intelligent system.

■ Knowledge is a description of the world.

It determines a system's competence by what it knows.

■ Representation is the way knowledge is encoded.

It defines a system's performance in doing something.

- Different types of knowledge require different kinds of representation. The Knowledge Representation models/mechanisms are often based on:

◇ Logic ◇ Rules

◇ Frames ◇ Semantic Net

- Different types of knowledge require different kinds of reasoning.

03

fo

.in

rs

de

KR -Introduction

ea

1. Introduction

yr

.m

w

w

Knowledge is a general term.

,w

ty

or

Knowledge is a progression that starts with data which is of limited utility.

ab

kr

□ By organizing or analyzing the data, we understand what the data means,

ha

C

C

and this becomes information.

R

□ The interpretation or evaluation of information yield knowledge.

□ An understanding of the principles embodied within the knowledge is wisdom.

• Knowledge Progression

Organizing Interpretation Understanding

Data Information Knowledge Wisdom

Analyzing Evaluation Principles

Fig 1 Knowledge Progression

■ Data is viewed as collection of : Example : It is raining.

disconnected facts.

■ Information emerges when : Example : The temperature dropped 15

relationships among facts are degrees and then it started raining.

established and understood;

Provides answers to "who",

"what", "where", and "when".

■ Knowledge emerges when : Example : If the humidity is very high

relationships among patterns and the temperature drops substantially, are identified and understood; then atmospheres is unlikely to hold the moisture, so it rains.

Provides answers as "how" .

■ Wisdom is the pinnacle of : Example : Encompasses understanding understanding, uncovers the of all the interactions that happen principles of relationships that between raining, evaporation, air currents, temperature gradients and describe patterns. changes.

Provides answers as "why" .

04

fo

.in

rs

de

KR -Introduction

ea

- Knowledge Model (Bellinger 1980)

yr

.m

w

w

A knowledge model tells, that as the degree of “connectedness” and

,w

ty

or

“understanding” increases, we progress from data through information and

ab

kr

knowledge to wisdom.

ha

C

C

R

Degree of

Connectedness

Wisdom

Understanding

principles

Knowledge

Understanding

patterns

Information

Understanding

relations

Degree of

Data

Understanding

Fig. Knowledge Model

The model represents transitions and understanding.

□ the transitions are from data, to information, to knowledge, and finally to wisdom;

□ the understanding support the transitions from one stage to the next stage.

The distinctions between data, information, knowledge, and wisdom are not very discrete. They are more like shades of gray, rather than black and white (Shedroff, 2001).

□ "data" and "information" deal with the past; they are based on the gathering of facts and adding context.

□ "knowledge" deals with the present that enable us to perform.

□ "wisdom" deals with the future, acquire vision for what will be, rather than for what is or was.

05

fo

.in

rs

de

KR -Introduction

ea

- Knowledge Category

yr

.m

w

w

Knowledge is categorized into two major types: Tacit and Explicit.

,w

ty

or

ab

□ term “Tacit” corresponds to "informal" or "implicit" type of knowledge,

kr

ha

C

□ term “Explicit” corresponds to "formal" type of knowledge.

C

R

Tacit knowledge Explicit knowledge

◇ Exists within a human being; ◇ Exists outside a human being;
it is embodied. it is embedded.

◇ Difficult to articulate formally. ◇ Can be articulated formally.

◇ Difficult to communicate or ◇ Can be shared, copied, processed
share. and stored.

◇ Hard to steal or copy. ◇ Easy to steal or copy

◇ Drawn from experience, ◇ Drawn from artifact of some type as
action, subjective insight. principle, procedure, process,
concepts.

(The next slide explains more about tacit and explicit knowledge).

06

fo

.in

rs

de

KR -Introduction

ea

■ Knowledge Typology Map

yr

.m

w The map shows two types of knowledge - Tacit and Explicit knowledge.

w

,w

ty

□ Tacit knowledge comes from "experience", "action", "subjective" , "insight"

or

ab

□ Explicit knowledge comes from "principle", "procedure", "process",

kr

ha

C

"concepts", via transcribed content or artifact of some type.

C

R

Doing

Experience Principles Procedure

(action)

Tacit Explicit

Knowledge Process

Knowledge

Subjective Knowledge Concept

Insight

Context

Information

Data

Fig. Knowledge Typology Map

Facts

◇ Facts : are data or instance that are specific and unique.

◇ Concepts : are class of items, words, or ideas that are known by a common name and share common features.

◇ Processes : are flow of events or activities that describe how things work rather than how to do things.

◇ Procedures : are series of step-by-step actions and decisions that result in the achievement of a task.

◇ Principles : are guidelines, rules, and parameters that govern; principles allow to make predictions and draw implications;

These artifacts are used in the knowledge creation process to create two types of knowledge: declarative and procedural explained below.

07

fo

.in

rs

de

KR -Introduction

ea

• Knowledge Type

yr

.m

w

w

Cognitive psychologists sort knowledge into Declarative and Procedural

,w

ty

or

category and some researchers added Strategic as a third category.

ab

kr

ha

C

‡ About procedural knowledge, there is some disparity in views.

C

R

– One, it is close to Tacit knowledge, it manifests itself in the doing of something yet cannot be expressed in words; e.g., we read faces and moods.

– Another, it is close to declarative knowledge; the difference is that a task or method is described instead of facts or things.

‡ All declarative knowledge are explicit knowledge; it is knowledge that can be and has been articulated.

‡ The strategic knowledge is thought as a subset of declarative knowledge.

Procedural knowledge Declarative knowledge

◊ Knowledge about "how to do" ◊ Knowledge about "that something"; e.g., to determine if something is true or false". e.g., Peter or Robert is older, first find A car has four tyres; Peter is their ages. older than Robert;

◊ Focuses on tasks that must be performed to reach a particular objects and events; knowledge objective or goal. ◊ Refers to representations of about facts and relationships;

◊ Examples : procedures, rules, ◊ Example : concepts, objects, strategies, agendas, models. facts, propositions, assertions, semantic nets, logic and descriptive models.

08

fo

.in

rs

de

KR -Introduction

ea

- Relationship among Knowledge Type

yr

.m

w

w

The relationship among explicit, implicit, tacit, declarative and procedural

,w

ty

or

knowledge are illustrated below.

ab

kr

ha

C

C

R

Knowledge

Start

No Can not be

Yes

Has been Implicit

articulated articulated

Yes No

Explicit Tacit

Facts and Motor Skill

things (Manual)

Describing Declarative Procedural Doing

Tasks and Mental Skill

methods

Fig. Relationship among types of knowledge

The Figure shows :

Declarative knowledge is tied to "describing" and

Procedural knowledge is tied to "doing."

Vertical arrows connecting explicit with declarative and tacit with procedural, indicate the strong relationships exist among them.

Horizontal arrow connecting declarative and procedural indicates that we often develop procedural knowledge as a result of starting with declarative knowledge. i.e., we often "know about" before we "know how".

Therefore, we may view :

- all procedural knowledge as tacit knowledge, and
- all declarative knowledge as explicit knowledge.

09

fo

.in

rs

de

KR -framework

ea

1.1 Framework of Knowledge Representation (Poole 1998)

yr

.m

w

w

Computer requires a well-defined problem description to process and

,w

ty

or

provide well-defined acceptable solution.

ab

kr

ha

C

To collect fragments of knowledge we need first to formulate a description

C

R

in our spoken language and then represent it in formal language so that computer can understand. The computer can then use an algorithm to compute an answer. This process is illustrated below.

Solve

Problem Solution

Represent Interpret Informal

Formal

Compute

Representation Output

Fig. Knowledge Representation Framework

The steps are

- The informal formalism of the problem takes place first.

- It is then represented formally and the computer produces an output.
- This output can then be represented in a informally described solution that user understands or checks for consistency.

Note : The Problem solving requires

- formal knowledge representation, and
- conversion of informal knowledge to formal knowledge , that is conversion of implicit knowledge to explicit knowledge.

10

fo

.in

rs

de

KR - framework

ea

- Knowledge and Representation

yr

.m

w

w

Problem solving requires large amount of knowledge and some

,w

ty

or

mechanism for manipulating that knowledge.

ab

kr

ha

C

The Knowledge and the Representation are distinct entities, play a

C

R

central but distinguishable roles in intelligent system.

- Knowledge is a description of the world;

it determines a system's competence by what it knows.

- Representation is the way knowledge is encoded;

it defines the system's performance in doing something.

In simple words, we :

- need to know about things we want to represent , and
- need some means by which things we can manipulate.

◇ know things ‡ Objects - facts about objects in the domain.

to represent

‡ Events - actions that occur in the domain.

‡ Performance - knowledge about how to do things

‡ Meta- - knowledge about what we know

knowledge

◇ need means ‡ Requires - to what we represent ;

to manipulate some formalism

Thus, knowledge representation can be considered at two levels :

(a) knowledge level at which facts are described, and

(b) symbol level at which the representations of the objects, defined in terms of symbols, can be manipulated in the programs.

Note : A good representation enables fast and accurate access to knowledge and understanding of the content.

11

fo

.in

rs

de

KR - framework

ea

- Mapping between Facts and Representation

yr

.m

w

w

Knowledge is a collection of "facts" from some domain.

,w

ty

or

ab

We need a representation of "facts" that can be manipulated by a program.

kr

ha

C

Normal English is insufficient, too hard currently for a computer program to

C

R

draw inferences in natural languages.

Thus some symbolic representation is necessary.

Therefore, we must be able to map "facts to symbols" and "symbols to facts" using forward and backward representation mapping.

Example : Consider an English sentence

Reasoning

programs

Internal

Facts

Representation

English English

understanding generation

English

Representation

Facts Representations

◇ Spot is a dog A fact represented in English sentence

◇ dog (Spot) Using forward mapping function the above fact is represented in logic

◇ $\forall x : \text{dog}(x) \rightarrow \text{hastail}(x)$ A logical representation of the fact that "all dogs have tails"

Now using deductive mechanism we can generate a new representation of object :

◇ hastail (Spot) A new object representation

◇ Spot has a tail Using backward mapping function to [it is new knowledge] generate English sentence

12

fo

.in

rs

de

KR - framework

ea

■ Forward and Backward Representation

yr

.m

w

w

The forward and backward representations are elaborated below :

,w

ty

or

ab

Desired real

kr

ha

Initial reasoning Final

C

Facts Facts

C

R

Forward Backward

representation representation

mapping mapping

Internal English

Representation Operated by Representation

program

‡ The dotted line on top indicates the abstract reasoning process that a program is intended to model.

‡ The solid lines on bottom indicates the concrete reasoning process that the program performs.

13

fo

.in

rs

de

KR - framework

ea

- KR System Requirements

yr

.m

w

w

A good knowledge representation enables fast and accurate access to

,w

ty

or

knowledge and understanding of the content.

ab

kr

A knowledge representation system should have following properties.

ha

C

C

R

◇ Representational The ability to represent all kinds of knowledge

Adequacy

that are needed in that domain.

◇ Inferential Adequacy The ability to manipulate the representational structures to derive new structure corresponding to new knowledge inferred from old .

◇ Inferential Efficiency The ability to incorporate additional information into the knowledge structure that can be used to focus the attention of the inference mechanisms in the most promising direction.

◇ Acquisitional The ability to acquire new knowledge using Efficiency automatic methods wherever possible rather than reliance on human intervention.

Note : To date no single system can optimizes all of the above properties.

14

fo

.in

rs

de

KR - schemes

ea

1.2 Knowledge Representation Schemes

yr

.m

w

w

There are four types of Knowledge representation :

,w

ty

or

Relational, Inheritable, Inferential, and Declarative/Procedural.

ab

kr

ha

C

◇ Relational Knowledge :

C

R

– provides a framework to compare two objects based on equivalent attributes.

– any instance in which two different objects are compared is a relational type of knowledge.

◇ Inheritable Knowledge

– is obtained from associated objects.

– it prescribes a structure in which new objects are created which may inherit all or a subset of attributes from existing objects.

◇ Inferential Knowledge

– is inferred from objects through relations among objects.

– e.g., a word alone is a simple syntax, but with the help of other words in phrase the reader may infer more from a word; this inference within linguistic is called semantics.

◇ Declarative Knowledge

– a statement in which knowledge is specified, but the use to which that knowledge is to be put is not given.

– e.g. laws, people's name; these are facts which can stand alone, not dependent on other knowledge;

Procedural Knowledge

– a representation in which the control information, to use the knowledge, is embedded in the knowledge itself.

– e.g. computer programs, directions, and recipes; these indicate specific use or implementation;

These KR schemes are detailed in next few slides

15

fo

.in

rs

de

KR - schemes

ea

• Relational Knowledge :

yr

.m

w

w

This knowledge associates elements of one domain with another domain.

,w

ty

or

ab

– Relational knowledge is made up of objects consisting of attributes and

kr

ha

their corresponding associated values.

C

C

R

– The results of this knowledge type is a mapping of elements among different domains.

The table below shows a simple way to store facts.

– The facts about a set of objects are put systematically in columns.

– This representation provides little opportunity for inference.

Table - Simple Relational Knowledge

Player Height Weight Bats - Throws

Aaron 6-0 180 Right - Right

Mays 5-10 170 Right - Right

Ruth 6-2 215 Left - Left

Williams 6-3 205 Left - Right

‡ Given the facts it is not possible to answer simple question such as :

" Who is the heaviest player ? ".

but if a procedure for finding heaviest player is provided, then these facts will enable that procedure to compute an answer.

‡ We can ask things like who "bats – left" and "throws – right".

16

fo

.in

rs

de

KR - schemes

ea

• Inheritable Knowledge :

yr

.m

w

w

Here the knowledge elements inherit attributes from their parents.

,w

ty

or

ab

The knowledge is embodied in the design hierarchies found in the

kr

ha

functional, physical and process domains. Within the hierarchy, elements

C

C

R

inherit attributes from their parents, but in many cases not all attributes of the parent elements be prescribed to the child elements.

The inheritance is a powerful form of inference, but not adequate. The basic KR needs to be augmented with inference mechanism.

The KR in hierarchical structure, shown below, is called “semantic network” or a collection of “frames” or “slot-and-filler structure”. The structure shows property inheritance and way for insertion of additional knowledge.

Property inheritance : The objects or elements of specific classes inherit attributes and values from more general classes. The classes are organized in a generalized hierarchy.

Baseball knowledge

Person Right

– isa : show class inclusion handed

– instance : show class membership isa

Adult height

5.10

Male

isa height 6.1

EQUAL bats

Baseball

handed Player batting-average

0.252

isa isa

batting-average batting-average

0.106 Pitcher Fielder 0.262

instance instance

Chicago team Three Finger Pee-Wee- team Brooklyn-
Cubs Brown Reese Dodger

Fig. Inheritable knowledge representation (KR)

‡ The directed arrows represent attributes (isa, instance, team) originates
at object being described and terminates at object or its value.

‡ The box nodes represents objects and values of the attributes.

[Continued in the next slide]

17

fo

.in

rs

de

KR - schemes

ea

[from previous slide - example]

yr

.m

w ◇ Viewing a node as a frame

w

,w

Example : Baseball-player

ty

or

ab

isa : Adult-Male

kr

ha

Bates : EQUAL handed

C

C

R

Height : 6.1

Batting-average : 0.252

◇ Algorithm : Property Inheritance

Retrieve a value V for an attribute A of an instance object O.

Steps to follow:

1. Find object O in the knowledge base.

2. If there is a value for the attribute A then report that value.
3. Else, if there is a value for the attribute instance; If not, then fail.
4. Else, move to the node corresponding to that value and look for a value for the attribute A; If one is found, report it.
5. Else, do until there is no value for the "isa" attribute or until an answer is found :
 - (a) Get the value of the "isa" attribute and move to that node.
 - (b) See if there is a value for the attribute A; If yes, report it.

This algorithm is simple. It describes the basic mechanism of inheritance. It does not say what to do if there is more than one value of the instance or "isa" attribute.

This can be applied to the example of knowledge base illustrated, in the previous slide, to derive answers to the following queries :

- team (Pee-Wee-Reese) = Brooklyn-Dodger
- batting-average(Three-Finger-Brown) = 0.106
- height (Pee-Wee-Reese) = 6.1
- bats (Three Finger Brown) = right

[For explanation - refer book on AI by Elaine Rich & Kevin Knight, page 112]

18

fo

.in

rs

de

KR - schemes

ea

- Inferential Knowledge :

yr

.m

w

w

This knowledge generates new information from the given information.

,w

ty

or

ab

This new information does not require further data gathering form source,

kr

ha

but does require analysis of the given information to generate new

C

C

R

knowledge.

Example :

- given a set of relations and values, one may infer other values or relations.
- a predicate logic (a mathematical deduction) is used to infer from a set of attributes.
- inference through predicate logic uses a set of logical operations to relate individual data.
- the symbols used for the logic operations are :
 $\rightarrow$ (implication), $\neg$ (not), $\vee$ (or), $\wedge$ (and),
 $\forall$ (for all), $\exists$ (there exists).

Examples of predicate logic statements :

1. "Wonder" is a name of a dog : $\text{dog}(\text{wonder})$
2. All dogs belong to the class of animals : $\forall x : \text{dog}(x) \rightarrow \text{animal}(x)$
3. All animals either live on land or in water : $\forall x : \text{animal}(x) \rightarrow \text{live}(x, \text{land}) \vee \text{live}(x, \text{water})$

From these three statements we can infer that :

" Wonder lives either on land or on water."

Note : If more information is made available about these objects and their relations, then more knowledge can be inferred.

19

fo

.in

rs

de

KR - schemes

ea

- Declarative/Procedural Knowledge

yr

.m

w

w

Differences between Declarative/Procedural knowledge is not very clear.

,w

ty

or

ab

Declarative knowledge :

kr

ha

Here, the knowledge is based on declarative facts about axioms and

C

C

R

domains .

– axioms are assumed to be true unless a counter example is found to invalidate them.

– domains represent the physical world and the perceived functionality.

– axiom and domains thus simply exists and serve as declarative statements that can stand alone.

Procedural knowledge:

Here, the knowledge is a mapping process between domains that specify “what to do when” and the representation is of “how to make it” rather than “what it is”. The procedural knowledge :

– may have inferential efficiency, but no inferential adequacy and acquisitional efficiency.

– are represented as small programs that know how to do specific things, how to proceed.

Example : A parser in a natural language has the knowledge that a noun phrase may contain articles, adjectives and nouns. It thus accordingly call routines that know how to process articles, adjectives and nouns.

20

fo

.in

rs

de

KR - issues

ea

1.3 Issues in Knowledge Representation

yr

.m

w

w

The fundamental goal of Knowledge Representation is to facilitate

,w

ty

or

inferencing (conclusions) from knowledge.

ab

kr

ha

C

The issues that arise while using KR techniques are many. Some of these

C

R

are explained below.

◇ Important Attributes :

Any attribute of objects so basic that they occur in almost every problem domain ?

◇ Relationship among attributes:

Any important relationship that exists among object attributes ?

◇ Choosing Granularity :

At what level of detail should the knowledge be represented ?

◇ Set of objects :

How sets of objects be represented ?

◇ Finding Right structure :

Given a large amount of knowledge stored, how can relevant parts be accessed ?

Note : These issues are briefly explained, referring previous example, Fig.

Inheritable KR. For detail readers may refer book on AI by Elaine Rich & Kevin Knight- page 115 - 126.

21

fo

.in

rs

de

KR - issues

ea

• Important Attributes : (Ref. Example - Fig. Inheritable KR)

yr

.m

w

w

There are attributes that are of general significance.

,w

ty

or

ab

There are two attributes "instance" and "isa", that are of general

kr

ha

importance. These attributes are important because they support

C

C

R

property inheritance.

- Relationship among Attributes : (Ref. Example- Fig. Inheritable KR)

The attributes to describe objects are themselves entities they represent.

The relationship between the attributes of an object, independent of specific knowledge they encode, may hold properties like:

Inverses, existence in an isa hierarchy, techniques for reasoning about values and single valued attributes.

◇ Inverses :

This is about consistency check, while a value is added to one attribute.

The entities are related to each other in many different ways. The figure shows attributes (isa, instance, and team), each with a directed arrow, originating at the object being described and terminating either at the object or its value.

There are two ways of realizing this:

‡ first, represent two relationships in a single representation; e.g., a logical representation, team(Pee-Wee-Reese, Brooklyn-Dodgers), that can be interpreted as a statement about Pee-Wee-Reese or Brooklyn-Dodger.

‡ second, use attributes that focus on a single entity but use them in pairs, one the inverse of the other; for e.g., one, team = Brooklyn-Dodgers , and the other, team = Pee-Wee-Reese,

This second approach is followed in semantic net and frame-based systems, accompanied by a knowledge acquisition tool that guarantees the consistency of inverse slot by checking, each time a value is added to one attribute then the corresponding value is added to the inverse.

22

fo

.in

rs

de

KR - issues

ea

◇ Existence in an "isa" hierarchy :

yr

.m

w This is about generalization-specialization, like, classes of objects and

w

,w

ty

specialized subsets of those classes. There are attributes and

or

ab

specialization of attributes.

kr

ha

C

Example: the attribute "height" is a specialization of general attribute

C

R

"physical-size" which is, in turn, a specialization of "physical-attribute".

These generalization-specialization relationships for attributes are

important because they support inheritance.

◇ Techniques for reasoning about values :

This is about reasoning values of attributes not given explicitly.

Several kinds of information are used in reasoning, like,

height : must be in a unit of length,

age : of person can not be greater than the age of

person's parents.

The values are often specified when a knowledge base is created.

◇ Single valued attributes :

This is about a specific attribute that is guaranteed to take a unique value.

Example : A baseball player can at time have only a single height and be a member of only one team. KR systems take different approaches to provide support for single valued attributes.

23

fo

.in

rs

de

KR - issues

ea

- Choosing Granularity

yr

.m

w What level should the knowledge be represented and

w

,w

ty

what are the primitives ?

or

ab

kr

- Should there be a small number or should there be a large number of

ha

C

low-level primitives or High-level facts.

C

R

- High-level facts may not be adequate for inference while Low-level primitives may require a lot of storage.

Example of Granularity :

- Suppose we are interested in following facts

John spotted Sue.

- This could be represented as

Spotted (agent(John), object (Sue))

- Such a representation would make it easy to answer questions such as

Who spotted Sue ?

- Suppose we want to know

Did John see Sue ?

- Given only one fact, we cannot discover that answer.

- We can add other facts, such as

Spotted (x , y) → saw (x , y)

- We can now infer the answer to the question.

24

fo

.in

rs

de

KR - issues

ea

• Set of Objects

yr

.m

w

w

Certain properties of objects that are true as member of a set but

,w

ty

or

not as individual;

ab

kr

ha

C

Example : Consider the assertion made in the sentences

C

R

"there are more sheep than people in Australia", and

"English speakers can be found all over the world."

To describe these facts, the only way is to attach assertion to the sets representing people, sheep, and English.

The reason to represent sets of objects is :

If a property is true for all or most elements of a set,

then it is more efficient to associate it once with the set

rather than to associate it explicitly with every elements of the set .

This is done in different ways :

- in logical representation through the use of universal quantifier, and
- in hierarchical structure where node represent sets, the inheritance propagate set level assertion down to individual.

Example: assert large (elephant);

Remember to make clear distinction between,

- whether we are asserting some property of the set itself, means, the set of elephants is large, or
- asserting some property that holds for individual elements of the set , means, any thing that is an elephant is large.

There are three ways in which sets may be represented :

(a) Name, as in the example – Ref Fig. Inheritable KR, the node - Baseball-

Player and the predicates as Ball and Batter in logical representation.

(b) Extensional definition is to list the numbers, and

(c) Intensional definition is to provide a rule, that returns true or false depending on whether the object is in the set or not.

[Readers may refer book on AI by Elaine Rich & Kevin Knight- page 122 - 123]

25

fo

.in

rs

de

KR - issues

ea

• Finding Right Structure

yr

.m

w

w

Access to right structure for describing a particular situation.

,w

ty

or

ab

It requires, selecting an initial structure and then revising the choice.

kr

ha

C

While doing so, it is necessary to solve following problems :

C

R

- how to perform an initial selection of the most appropriate structure.
- how to fill in appropriate details from the current situations.
- how to find a better structure if the one chosen initially turns out not to be appropriate.
- what to do if none of the available structures is appropriate.
- when to create and remember a new structure.

There is no good, general purpose method for solving all these problems.

Some knowledge representation techniques solve some of them.

[Readers may refer book on AI by Elaine Rich & Kevin Knight- page 124 - 126]

26

fo

.in

rs

de

KR – using logic

ea

2. KR Using Predicate Logic

yr

.m

w

w

In the previous section much has been illustrated about knowledge and KR

,w

ty

or

related issues. This section, illustrates :

ab

kr

How knowledge can be represented as “symbol structures” that characterize

ha

C

C

bits of knowledge about objects, concepts, facts, rules, strategies;

R

Examples : “red” represents colour red;

“car1” represents my car ;

"red(car1)" represents fact that my car is red.

Assumptions about KR :

– Intelligent Behavior can be achieved by manipulation of symbol structures.

– KR languages are designed to facilitate operations over symbol structures, have precise syntax and semantics;

Syntax tells which expression is legal ?,

e.g., red1(car1), red1 car1, car1(red1), red1(car1 & car2) ?; and

Semantic tells what an expression means ?

e.g., property “dark red” applies to my car.

– Make Inferences, draw new conclusions from existing facts.

To satisfy these assumptions about KR, we need formal notation that allow automated inference and problem solving. One popular choice is use of logic.

27

fo

.in

rs

de

KR – Logic

ea

• Logic

yr

.m

w

w

Logic is concerned with the truth of statements about the world.

,w

ty

or

ab

Generally each statement is either TRUE or FALSE.

kr

ha

Logic includes : Syntax , Semantics and Inference Procedure.

C

C

R

◇ Syntax :

Specifies the symbols in the language about how they can be combined to form sentences. The facts about the world are represented as sentences in logic.

◇ Semantic :

Specifies how to assign a truth value to a sentence based on its meaning in the world. It Specifies what facts a sentence refers to. A fact is a claim about the world, and it may be TRUE or FALSE.

◇ Inference Procedure :

Specifies methods for computing new sentences from the existing sentences.

Note

Facts : are claims about the world that are True or False.

Representation : is an expression (sentence), stands for the objects and relations.

Sentences : can be encoded in a computer program.

28

fo

.in

rs

de

KR - Logic

ea

- Logic as a KR Language

yr

.m

w

w

Logic is a language for reasoning, a collection of rules used while doing

,w

ty

or

logical reasoning. Logic is studied as KR languages in artificial intelligence.

ab

kr

ha

C

◇ Logic is a formal system in which the formulas or sentences have true

C

R

or false values.

◇ Problem of designing KR language is a tradeoff between that which is

(a) Expressive enough to represent important objects and relations in a problem domain.

(b) Efficient enough in reasoning and answering questions about implicit information in a reasonable amount of time.

◇ Logics are of different types : Propositional logic, Predicate logic, Temporal logic, Modal logic, Description logic etc;

They represent things and allow more or less efficient inference.

◇ Propositional logic and Predicate logic are fundamental to all logic.

Propositional Logic is the study of statements and their connectivity.

Predicate Logic is the study of individuals and their properties.

29

fo

.in

rs

de

KR – Logic

ea

2.1 Logic Representation

yr

.m

w

w

Logic can be used to represent simple facts.

,w

ty

or

The facts are claims about the world that are True or False.

ab

kr

ha

C

To build a Logic-based representation :

C

R

◇ User defines a set of primitive symbols and the associated semantics.

◇ Logic defines ways of putting symbols together so that user can define legal sentences in the language that represent TRUE facts.

◇ Logic defines ways of inferring new sentences from existing ones.

◇ Sentences - either TRUE or false but not both are called propositions.

◇ A declarative sentence expresses a statement with a proposition as content; example:

the declarative "snow is white" expresses that snow is white;

further, "snow is white" expresses that snow is white is TRUE.

In this section, first Propositional Logic (PL) is briefly explained and then the Predicate logic is illustrated in detail.

30

fo

.in

rs

de

KR - Propositional Logic

ea

- Propositional Logic (PL)

yr

.m

w

w

A proposition is a statement, which in English would be a declarative

,w

ty

or

sentence. Every proposition is either TRUE or FALSE.

ab

kr

Examples:

ha

(a) The sky is blue., (b) Snow is cold. , (c) $12 * 12 = 144$

C

C

R

‡ Propositions are “sentences” , either true or false but not both.

‡ A sentence is smallest unit in propositional logic.

‡ If proposition is true, then truth value is "true" .

If proposition is false, then truth value is "false" .

Example :

Sentence Truth value Proposition (Y/N)

"Grass is green" "true" Yes

" $2 + 5 = 5$ " "false" Yes

"Close the door" - No

"Is it hot out side ?" - No

" $x > 2$ " where x is variable - No

(since x is not defined)

" $x = x$ " - No

(don't know what is "x" and "=";

" $3 = 3$ " or "air is equal to air" or

"Water is equal to water"

has no meaning)

– Propositional logic is fundamental to all logic.

– Propositional logic is also called Propositional calculus, Sentential calculus, or Boolean algebra.

– Propositional logic tells the ways of joining and/or modifying entire propositions, statements or sentences to form more complicated propositions, statements or sentences, as well as the logical

relationships and properties that are derived from the methods of combining or altering statements.

31

fo

.in

rs

de

KR - Propositional Logic

ea

■ Statement, Variables and Symbols

yr

.m

w

w

These and few more related terms, such as, connective, truth value,

,w

ty

or

contingencies, tautologies, contradictions, antecedent, consequent,

ab

kr

argument are explained below.

ha

C

C

R

◇ Statement

Simple statements (sentences), TRUE or FALSE, that does not contain any other statement as a part, are basic propositions; lower-case letters, p, q, r, are symbols for simple statements. Large, compound or complex statement are constructed from basic propositions by combining them with connectives.

◇ Connective or Operator

The connectives join simple statements into compounds, and joins compounds into larger compounds.

Table below indicates, the basic connectives and their symbols :

- listed in decreasing order of operation priority;
- operations with higher priority is solved first.

Example of a formula : $((((a \wedge \neg b) \vee c \rightarrow d) \leftrightarrow \neg (a \vee c)))$

Connectives and Symbols in decreasing order of operation priority

Connective Symbols Read as

assertion P "p is true"

negation $\neg p \sim !$ NOT "p is false"

conjunction $p \wedge q \cdot \&\&$ AND "both p and q are true"

disjunction $P \vee q ||$ OR "either p is true, or q is true, or both "

implication $p \rightarrow q \supset \Rightarrow$ if ..then "if p is true, then q is true"

" p implies q "

equivalence $\leftrightarrow \equiv \Leftrightarrow$ if and only if "p and q are either both true or both false"

Note : The propositions and connectives are the basic elements of propositional logic.

32

fo

.in

rs

de

KR - Propositional Logic

ea

◇ Truth Value

yr

.m

w

w

The truth value of a statement is its TRUTH or FALSITY ,

,w

ty

or

Example :

ab

kr

p is either TRUE or FALSE,

ha

C

C

$\sim p$ is either TRUE or FALSE,

R

$p \vee q$ is either TRUE or FALSE, and so on.

use " T " or " 1 " to mean TRUE.

use " F " or " 0 " to mean FALSE

Truth table defining the basic connectives :

p	q	$\neg p$	$\neg q$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$	$q \rightarrow p$
T	T	F	F	T	T	T	T	T
T	F	F	T	F	T	F	F	T
F	T	T	F	F	T	T	F	F
F	F	T	T	F	F	T	T	F

[The next slide shows the truth values of a group of propositions, called tautology, contradiction, contingency, antecedent, consequent. They form argument where one proposition claims to follow logically other proposition]

33

fo

.in

rs

de

KR - Propositional Logic

ea

◇ Tautologies

yr

.m

w A proposition that is always true is called a "tautology".

w

,w

ty

e.g., $(P \vee \neg P)$ is always true regardless of the truth value of the

or

ab

proposition P.

kr

ha

C

◇ Contradictions

C

R

A proposition that is always false is called a "contradiction".

e.g., $(P \wedge \neg P)$ is always false regardless of the truth value of the proposition P.

◇ Contingencies

A proposition is called a "contingency" , if that proposition is

neither a tautology nor a contradiction .

e.g., $(P \vee Q)$ is a contingency.

◇ Antecedent, Consequent

These two are parts of conditional statements.

In the conditional statements, $p \rightarrow q$, the

1st statement or "if - clause" (here p) is called antecedent ,

2nd statement or "then - clause" (here q) is called consequent.

34

fo

.in

rs

de

KR - Propositional Logic

ea

◇ Argument

yr

.m

w An argument is a demonstration or a proof of some statement.

w

,w

ty

Example : "That bird is a crow; therefore, it's black."

or

ab

Any argument can be expressed as a compound statement.

kr

ha

C

In logic, an argument is a set of one or more meaningful declarative

C

R

sentences (or "propositions") known as the premises along with another meaningful declarative sentence (or "proposition") known as the conclusion.

□ Premise is a proposition which gives reasons, grounds, or evidence for accepting some other proposition, called the conclusion.

□ Conclusion is a proposition, which is purported to be established on the basis of other propositions.

Take all the premises, conjoin them, and make that conjunction the

antecedent of a conditional and make the conclusion the consequent. This implication statement is called the corresponding conditional of the argument.

Note : Every argument has a corresponding conditional, and every implication statement has a corresponding argument. Because the corresponding conditional of an argument is a statement, it is therefore either a tautology, or a contradiction, or a contingency.

‡ An argument is valid

"if and only if" its corresponding conditional is a tautology.

‡ Two statements are consistent

"if and only if" their conjunction is not a contradiction.

‡ Two statements are logically equivalent

"if and only if" their truth table columns are identical;

"if and only if" the statement of their equivalence using " $\equiv$ " is

a tautology.

Note : The truth tables are adequate to test validity, tautology, contradiction, contingency, consistency, and equivalence.

35

fo

.in

rs

de

KR - Predicate Logic

ea

• Predicate Logic

yr

.m

w

w

The propositional logic, is not powerful enough for all types of assertions;

,w

ty

or

ab

Example : The assertion " $x > 1$ ", where x is a variable, is not a proposition

kr

ha

C

because it is neither true nor false unless value of x is defined.

C

R

For $x > 1$ to be a proposition ,

- either we substitute a specific number for x ;
- or change it to something like

"There is a number x for which $x > 1$ holds";

- or "For every number x , $x > 1$ holds".

Consider example :

" All men are mortal.

Socrates is a man.

Then Socrates is mortal" ,

These cannot be expressed in propositional logic as a finite and logically valid argument (formula).

We need languages : that allow us to describe properties (predicates) of objects, or a relationship among objects represented by the variables .

Predicate logic satisfies the requirements of a language.

- Predicate logic is powerful enough for expression and reasoning.
- Predicate logic is built upon the ideas of propositional logic.

36

fo

.in

rs

de

KR - Predicate Logic

ea

■ Predicate :

yr

.m

w

w

Every complete "sentence" contains two parts : a "subject" and a

,w

ty

or

"predicate".

ab

kr

The subject is what (or whom) the sentence is about.

ha

C

C

The predicate tells something about the subject;

R

Example :

A sentence "Judy {runs}".

The subject is Judy and the predicate is runs .

Predicate, always includes verb, tells something about the subject.

Predicate is a verb phrase template that describes a property of objects, or a relation among objects represented by the variables.

Example:

"The car Tom is driving is blue" ;

"The sky is blue" ;

"The cover of this book is blue"

Predicate is "is blue" , describes property.

Predicates are given names; Let 'B' is name for predicate "is_blue".

Sentence is represented as "B(x)" , read as "x is blue";

Symbol "x" represents an arbitrary Object .

37

fo

.in

rs

de

KR - Predicate Logic

ea

■ Predicate Logic Expressions :

yr

.m

w The propositional operators combine predicates, like

w

,w

ty

If ($p(\dots) \ \&\& \ (\ !q(\dots) \ || \ r(\dots)) \)$)

or

ab

kr

ha

Logic operators :

C

C

R

Examples of disjunction (OR) and conjunction (AND).

Consider the expression with the respective logic symbols $\parallel$ and $\&\&$

$$x < y \parallel (y < z \&\& z < x)$$

which is true $\parallel (true \&\& true)$;

Applying truth table, found True

Assignment for $<$ are 3, 2, 1 for x, y, z and then

the value can be FALSE or TRUE

$$3 < 2 \parallel (2 < 1 \&\& 1 < 3)$$

It is False

38

fo

.in

rs

de

KR - Predicate Logic

ea

■ Predicate Logic Quantifiers

yr

.m

w As said before, $x > 1$ is not proposition and why ?

w

,w

ty

Also said, that for $x > 1$ to be a proposition what is required ?

or

ab

kr

ha

Generally, a predicate with variables (is called atomic formula) that

C

C

R

can be made a proposition by applying one of the following two operations to each of its variables :

1. Assign a value to the variable; e.g., $x > 1$, if 3 is assigned to x becomes $3 > 1$, and it then becomes a true statement, hence a proposition.

2. Quantify the variable using a quantifier on formulas of predicate logic (called wff well-formed formula), such as $x > 1$ or $P(x)$, by using Quantifiers on variables.

Apply Quantifiers on Variables

‡ Variable x

* $x > 5$ is not a proposition, its truth depends upon the value of variable x

* to reason such statements, x need to be declared

‡ Declaration $x : a$

* $x:a$ declares variable x

* $x:a$ read as “ x is an element of set a ”

‡ Statement p is a statement about x

* $\forall x:a \cdot p$ is quantification of statement
statement

declaration of variable x as element of set a
quantifier

* Quantifiers are two types :

universal quantifiers , denoted by symbol $\forall$ and
existential quantifiers , denoted by symbol $\exists$

Note : The next few slide tells more on these two Quantifiers.

39

fo

.in

rs

de

KR - Predicate Logic

ea

■ Universe of Discourse

yr

.m

w The universe of discourse, also called domain of discourse or universe.

w

,w

ty

This indicates :

or

ab

a set of entities that the quantifiers deal.

kr

ha

C

– entities can be set of real numbers, set of integers, set of all cars

C

R

on a parking lot, the set of all students in a classroom etc.

– universe is thus the domain of the (individual) variables.

– propositions in the predicate logic are statements on objects of a universe.

The universe is often left implicit in practice, but it should be obvious from the context.

Examples:

– About "natural numbers" forAll $x, y (x < y \text{ or } x = y \text{ or } x > y)$,

there is no need to be more precise and say forAll $x, y \text{ in } N$,

because N is implicit, being the universe of discourse.

– About a property that holds for natural numbers but not for real numbers, it is necessary to qualify what the allowable values of x and y are.

40

fo

.in

rs

de

KR - Predicate Logic

ea

■ Apply Universal Quantifier " For All "

yr

.m

w Universal Quantification allows us to make a statement about a

w

,w

ty

collection of objects.

or

ab

kr

ha

‡ Universal quantification: $x:a \bullet p$

C

C

R

* read " for all x in a , p holds "

* a is universe of discourse

* x is a member of the domain of discourse.

* p is a statement about x

‡ In propositional form it is written as : $\forall x P(x)$

* read " for all x , $P(x)$ holds "

" for each x , $P(x)$ holds " or

" for every x , $P(x)$ holds "

* where $P(x)$ is predicate,

x means all the objects x in the universe

$P(x)$ is true for every object x in the universe

‡ Example : English language to Propositional form

* "All cars have wheels"

x : car • x has wheel

* $\forall x P(x)$

where $P(x)$ is predicate tells : ' x has wheels'

x is variable for object 'cars' that populate

universe of discourse

41

fo

.in

rs

de

KR - Predicate Logic

ea

■ Apply Existential Quantifier " There Exists "

yr

.m

w

w

Existential Quantification allows us to state that an object does exist

,w

ty

or

without naming it.

ab

kr

ha

C

‡ Existential quantification: $x:a \bullet p$

C

R

* read " there exists an x such that p holds "

* a is universe of discourse

* x is a member of the domain of discourse.

* p is a statement about x

‡ In propositional form it is written as : $x P(x)$

* read " there exists an x such that $P(x)$ " or

" there exists at least one x such that $P(x)$ "

* Where $P(x)$ is predicate

x means at least one object x in the universe

$P(x)$ is true for least one object x in the universe

‡ Example : English language to Propositional form

* " Someone loves you "

x : Someone • x loves you

* $x P(x)$

where $P(x)$ is predicate tells : ' x loves you '

x is variable for object ' someone ' that populate

universe of discourse

42

fo

.in

rs

de

KR - Predicate Logic

ea

■ Formula

yr

.m

w In mathematical logic, a formula is a type of abstract object.

w

,w

ty

A token of a formula is a symbol or string of symbols which may be

or

ab

interpreted as any meaningful unit in a formal language.

kr

ha

C

C

R

‡ Terms

Defined recursively as variables, or constants, or functions like $f(t_1, \dots, t_n)$, where f is an n -ary function symbol, and $t_1, \dots, t_n$ are terms. Applying predicates to terms produces atomic formulas.

‡ Atomic formulas

An atomic formula (or simply atom) is a formula with no deeper propositional structure, i.e., a formula that contains no logical connectives or a formula that has no strict sub-formulas.

- Atoms are thus the simplest well-formed formulas of the logic.
- Compound formulas are formed by combining the atomic formulas using the logical connectives.
- Well-formed formula ("wiff") is a symbol or string of symbols (a formula) generated by the formal grammar of a formal language.

An atomic formula is one of the form :

- $t_1 = t_2$, where t_1 and t_2 are terms, or
- $R(t_1, \dots, t_n)$, where R is an n -ary relation symbol, and $t_1, \dots, t_n$ are terms.
- $\neg a$ is a formula when a is a formula.
- $(a \wedge b)$ and $(a \vee b)$ are formula when a and b are formula

‡ Compound formula : example

$((((a \wedge b) \wedge c) \vee ((\neg a \wedge b) \wedge c)) \vee ((a \wedge \neg b) \wedge c))$

43

fo

.in

rs

de

KR – logic relation

ea

2.2 Representing “IsA” and “Instance” Relationships

yr

.m

w

w

Logic statements, containing subject, predicate, and object, were

,w

ty

or

explained. Also stated, two important attributes "instance" and "isa", in

ab

kr

a hierarchical structure (Ref. Fig. Inheritable KR).

ha

C

C

Attributes "IsA" and "Instance" support property inheritance and play

R

important role in knowledge representation.

The ways these two attributes "instance" and "isa", are logically expressed are shown in the example below :

■ Example : A simple sentence like "Joe is a musician"

◇ Here "is a" (called IsA) is a way of expressing what logically is called a class-instance relationship between the subjects represented by the terms "Joe" and "musician".

◇ "Joe" is an instance of the class of things called "musician".

"Joe" plays the role of instance,

"musician" plays the role of class in that sentence.

◇ Note : In such a sentence, while for a human there is no confusion, but for computers each relationship have to be defined explicitly.

This is specified as: [Joe] IsA [Musician]

i.e., [Instance] IsA [Class]

44

fo

.in

rs

de

KR – functions & predicates

ea

2.3 Computable Functions and Predicates

yr

.m

w

w

The objective is to define class of functions C computable in terms of F.

,w

ty

or

ab

This is expressed as $C \{ F \}$ is explained below using two examples :

kr

ha

C

(1) "evaluate factorial n" and (2) "expression for triangular functions".

C

R

■ Example 1 : A conditional expression to define factorial n ie n!

◇ Expression

" if p1 then e1 else if p2 then e2 ... else if pn then en" .

ie. $(p1 \rightarrow e1, p2 \rightarrow e2, \dots, pn \rightarrow en)$

Here p1, p2, pn are propositional expressions taking the values T or F for true and false respectively.

◇ The value of $(p1 \rightarrow e1, p2 \rightarrow e2, \dots, pn \rightarrow en)$ is the value of the e corresponding to the first p that has value T.

◇ The expressions defining n! , n= 5, recursively are :

$n! = n \times (n-1)!$ for $n \geq 1$

$5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$

$0! = 1$

The above definition incorporates an instance that :

if the product of no numbers ie $0! = 1$,

then only, recursive relation $(n + 1)! = (n+1) \times n!$ works for $n = 0$

◇ Use of the above conditional expressions to define functions n!

recursively is $n! = (n = 0 \rightarrow 1, n \neq 0 \rightarrow n \cdot (n - 1) !)$

◇ Example: Evaluate 2! according to above definition.

$2! = (2 = 0 \rightarrow 1, 2 \neq 0 \rightarrow 2 \cdot (2 - 1) !)$

$= 2 \times 1!$

$= 2 \times (1 = 0 \rightarrow 1, 1 \neq 0 \rightarrow 1 \cdot (1 - 1) !)$

$= 2 \times 1 \times 0!$

$= 2 \times 1 \times (0 = 0 \rightarrow 1, 0 \neq 0 \rightarrow 0 \cdot (0 - 1) !)$

$= 2 \times 1 \times 1$

$= 2$

45

fo

.in

rs

de

KR – functions & predicates

ea

■ Example 2 : A conditional expression for triangular functions

yr

.m

w

w

◇ The graph of a well known triangular function is shown below

,w

ty

or

ab

Y

kr

ha

0,1

C

C

R

X

-1,0 1,0

Fig. A Triangular Function

the conditional expressions for triangular functions are

$\square x \square = (x < 0 \rightarrow -x, x \geq 0 \rightarrow x)$

◇ the triangular function of the above graph is represented by the conditional expression

$\text{tri}(x) = (x \leq -1 \rightarrow 0, x \leq 0 \rightarrow -x, x \leq 1 \rightarrow x, x > 1 \rightarrow 0)$

46

fo

.in

rs

de

KR - Predicate Logic - resolution

ea

2.4 Resolution

yr

.m

w

w

Resolution is a procedure used in proving that arguments which are

,w

ty

or

expressible in predicate logic are correct.

ab

kr

ha

C

Resolution is a procedure that produces proofs by refutation or

C

R

contradiction.

Resolution lead to refute a theorem-proving technique for sentences in propositional logic and first-order logic.

– Resolution is a rule of inference.

– Resolution is a computerized theorem prover.

– Resolution is so far only defined for Propositional Logic. The strategy is that the Resolution techniques of Propositional logic be adopted in Predicate Logic.

47

fo

.in

rs

de

KR Using Rules

ea

3. KR Using Rules

yr

.m

w

w

In the earlier slides, the Knowledge representations using predicate logic

,w

ty

or

have been illustrated. The other popular approaches to Knowledge

ab

kr

representation are called production rules , semantic net and frames.

ha

C

C

R

Production rules, sometimes called IF-THEN rules are most popular KR.

□ production rules are simple but powerful forms of KR.

□ production rules provide the flexibility of combining declarative and procedural representation for using them in a unified form.

Examples of production rules :

- IF condition THEN action
- IF premise THEN conclusion
- IF proposition p1 and proposition p2 are true THEN proposition p3 is true

Advantages of production rules :

- they are modular,
- each rule define a small and independent piece of knowledge.
- new rules may be added and old ones deleted
- rules are usually independently of other rules.

The production rules as knowledge representation mechanism are used in the design of many "Rule-based systems" also called "Production systems" .

48

fo

.in

rs

de

KR Using Rules

ea

• Types of Rules

yr

.m

w Three types of rules are mostly used in the Rule-based production systems.

w

,w

ty

or

■ Knowledge Declarative Rules :

ab

kr

These rules state all the facts and relationships about a problem.

ha

C

C

Example :

R

IF inflation rate declines

THEN the price of gold goes down.

These rules are a part of the knowledge base.

■ Inference Procedural Rules

These rules advise on how to solve a problem, while certain facts are known.

Example :

IF the data needed is not in the system

THEN request it from the user.

These rules are part of the inference engine.

■ Meta rules

These are rules for making rules. Meta-rules reason about which rules should be considered for firing.

Example :

IF the rules which do not mention the current goal in their premise,

AND there are rules which do mention the current goal in their premise,

THEN the former rule should be used in preference to the latter.

– Meta-rules direct reasoning rather than actually performing reasoning.

– Meta-rules specify which rules should be considered and in which order they should be invoked.

49

fo

.in

rs

de

KR – procedural & declarative

ea

3.1 Procedural versus Declarative Knowledge

yr

.m

w

w

These two types of knowledge were defined in earlier slides.

,w

ty

or

ab

■ Procedural Knowledge : knowing 'how to do'

kr

ha

Includes : rules, strategies, agendas, procedures, models.

C

C

R

These explains what to do in order to reach a certain conclusion.

Example

Rule: To determine if Peter or Robert is older, first find their ages.

It is knowledge about 'how to do' something. It manifests itself in the doing of something, e.g., manual or mental skills cannot reduce to words. It is held by individuals in a way which does not allow it to be communicated directly to other individuals.

Accepts a description of the steps of a task or procedure. It Looks similar to declarative knowledge, except that tasks or methods are being described instead of facts or things.

■ Declarative Knowledge : knowing 'what', knowing 'that'

Includes : concepts, objects, facts, propositions, assertions, models.

It is knowledge about facts and relationships, that

- can be expressed in simple and clear statements,
- can be added and modified without difficulty.

Examples : A car has four tyres; Peter is older than Robert.

Declarative knowledge and explicit knowledge are articulated knowledge and may be treated as synonyms for most practical purposes.

Declarative knowledge is represented in a format that can be manipulated, decomposed and analyzed independent of its content.

50

fo

.in

rs

de

KR – procedural & declarative

ea

■ Comparison :

yr

.m

w

w

Comparison between Procedural and Declarative Knowledge :

,w

ty

or

ab

Procedural Knowledge Declarative Knowledge

kr

ha

- Hard to debug • Easy to validate

C

C

R

- Black box • White box

- Obscure • Explicit

- Process oriented • Data - oriented

- Extension may effect stability • Extension is easy

- Fast , direct execution • Slow (requires interpretation)

- Simple data type can be used • May require high level data type

- Representations in the form of • Representations in the form of sets of rules, organized into production system, the entire set routines and subroutines. of rules for executing the task.

51

fo

.in

rs

de

KR – procedural & declarative

ea

■ Comparison :

yr

.m

w Comparison between Procedural and Declarative Language :

w

,w

ty

or

Procedural Language Declarative Language

ab

kr

ha

• Basic, C++, Cobol, etc. • SQL

C

C

• Most work is done by interpreter of • Most work done by Data Engine

R

the languages within the DBMS

• For one task many lines of code • For one task one SQL statement

• Programmer must be skilled in • Programmer must be skilled in
translating the objective into lines clearly stating the objective as a
of procedural code SQL statement

• Requires minimum of management • Relies on SQL-enabled DBMS to
around the actual data hold the data and execute the SQL
statement .

• Programmer understands and has • Programmer has no interaction
access to each step of the code with the execution of the SQL
statement

• Data exposed to programmer • Programmer receives data at end
during execution of the code as an entire set

• More susceptible to failure due to • More resistant to changes in the
changes in the data structure data structure

• Traditionally faster, but that is • Originally slower, but now setting
changing speed records

• Code of procedure tightly linked to • Same SQL statements will work
front end with most front ends

Code loosely linked to front end.

• Code tightly integrated with • Code loosely linked to structure of
structure of the data store data; DBMS handles structural
issues

• Programmer works with a pointer • Programmer not concerned with
or cursor positioning

• Knowledge of coding tricks applies • Knowledge of SQL tricks applies to
only to one language any language using SQL

52

fo

.in

rs

de

KR – Logic Programming

ea

3.2 Logic Programming

yr

.m

w

w

Logic programming offers a formalism for specifying a computation in terms

,w

ty

or

of logical relations between entities.

ab

kr

– logic program is a collection of logic statements.

ha

C

C

– programmer describes all relevant logical relationships between the

R

various entities.

– computation determines whether or not, a particular conclusion follows from those logical statements.

• Characteristics of Logic program

Logic program is characterized by set of relations and inferences.

– program consists of a set of axioms and a goal statement.

– rules of inference determine whether the axioms are sufficient to ensure the truth of the goal statement.

– execution of a logic program corresponds to the construction of a proof of the goal statement from the axioms.

– programmer specify basic logical relationships, does not specify the manner in which inference rules are applied.

Thus Logic + Control = Algorithms

• Examples of Logic Statements

– Statement

A grand-parent is a parent of a parent.

– Statement expressed in more closely related logic terms as

A person is a grand-parent if she/he has a child and that child is a parent.

– Statement expressed in first order logic as

(for all) x : grandparent (x, y) :- parent (x, z), parent (z, y)

read as x is the grandparent of y

if x is a parent of z and z is a parent of y

53

fo

.in

rs

de

KR – Logic Programming

ea

• Logic Programming Language

yr

.m

w

w

A programming language includes :

,w

ty

or

– the syntax

ab

kr

– the semantics of programs and

ha

C

C

– the computational model.

R

There are many ways of organizing computations. The most familiar paradigm is procedural. The program specifies a computation by saying "how" it is to be performed. FORTRAN, C, and Object-oriented languages fall under this general approach.

Another paradigm is declarative. The program specifies a computation by giving the properties of a correct answer. Prolog and logic data language (LDL) are examples of declarative languages, emphasize the logical properties of a computation.

Prolog and LDL are called logic programming languages.

PROLOG (PROgramming LOGic) is the most popular Logic programming language rose within the realm of Artificial Intelligence (AI). It became popular with AI researchers, who know more about "what" and "how" intelligent behavior is achieved.

54

fo

.in

rs

de

KR – Logic Programming

ea

- Syntax and Terminology (relevant to Prolog programs)

yr

.m

w

w

In any language, the formation of components (expressions, statements,

,w

ty

or

etc.), is guided by syntactic rules.

ab

kr

The components are divided into two parts:

ha

C

C

(A) data components and (B) program components.

R

(A) Data components :

Data components are collection of data objects that follow hierarchy.

Data object of any kind is also called

Data Objects a term. A term is a constant, a variable (terms) or a compound term.

Simple data object is not

Simple Structured

decomposable; e.g. atoms, numbers, constants, variables.

Constants Variables

Syntax distinguishes the data objects,
hence no need for declaring them.

Atoms Numbers

Structured data object are made of
several components.

All these data components are explained in next slide.

55

fo

.in

rs

de

KR – Logic Programming

ea

(a) Data Objects :

yr

.m

w The data objects of any kind is called a term.

w

,w

ty

◇ Term : Examples

or

ab

kr

‡ Constants:

ha

C

C

R

Denote elements such as integers, floating point, atoms.

‡ Variables:

Denote a single but unspecified element; symbols for variables
begin with an uppercase letter or an underscore.

‡ Compound terms:

Comprise a functor and sequence of one or more compound
terms called arguments.

► Functor: is characterized by its name and number of
arguments; name is an atom, and number of arguments is
arity.

$f/n = f(t_1, t_2, \dots, t_n)$

where f is name of the functor and is of arity n

t_i 's are the argument

f/n denotes functor f of arity n

Functors with same name but different arities are distinct.

‡ Ground and non-ground:

Terms are ground if they contain no variables (only constant signs); otherwise they are non-ground.

Goals are atoms or compound terms, and are generally non-ground.

56

fo

.in

rs

de

KR – Logic Programming

ea

(b) Simple Data Objects : Atoms, Numbers, Variables

yr

.m

w

w

◇ Atoms

,w

ty

or

‡ a lower-case letter, possibly followed by other letters of either

ab

kr

case, digits, and underscore character.

ha

C

C

e.g. a greaterThan two_B_or_not_2_b

R

‡ a string of special characters such as: + - * / \ = ^ < > : ~ # \$ &

e.g. <> ##&& ::=

‡ a string of any characters enclosed within single quotes.

e.g. 'ABC' '1234' 'a<>b'

‡ following are also atoms ! ; [] { }

◇ Numbers

‡ applications involving heavy numerical calculations are rarely written in Prolog.

‡ integer representation: e.g. 0 -16 33 +100

‡ real numbers written in standard or scientific notation, e.g. 0.5 -3.1416 6.23e+23 11.0e-3 -2.6e-2

◇ Variables

‡ begins by a capital letter, possibly followed by other letters of either case, digits, and underscore character.

e.g. X25 List Noun_Phrase

57

fo

.in

rs

de

KR – Logic Programming

ea

(c) Structured Data Objects : General Structures , Special Structures

yr

.m

w

w

◇ General Structures

,w

ty

or

‡ a structured term is syntactically formed by a functor and a list of

ab

kr

arguments.

ha

C

C

‡ functor is an atom.

R

‡ list of arguments appear between parentheses.

‡ arguments are separated by a comma.

‡ each argument is a term (i.e., any Prolog data object).

‡ the number of arguments of a structured term is called its arity.

‡ e.g. `greaterThan(9, 6) f(a, g(b, c), h(d)) plus(2, 3, 5)`

Note : a structure in Prolog is a mechanism for combining terms together, like integers 2, 3, 5 are combined with the functor `plus`.

◇ Special Structures

‡ In Prolog an ordered collection of terms is called a list .

‡ Lists are structured terms and Prolog offers a convenient notation to represent them:

* Empty list is denoted by the atom `[]`.

* Non-empty list carries element(s) between square brackets, separating elements by comma.

e.g. `[bach, bee]` `[apples, oranges, grapes]`

58

fo

.in

rs

de

KR – Logic Programming

ea

(B) Program Components

yr

.m

w A Prolog program is a collection of predicates or rules.

w

,w

ty

A predicate establishes a relationships between objects.

or

ab

kr

ha

(a) Clause, Predicate, Sentence, Subject

C

C

R

‡ Clause is a collection of grammatically-related words .

‡ Predicate is composed of one or more clauses.

‡ Clauses are the building blocks of sentences;

every sentence contains one or more clauses.

‡ A Complete Sentence has two parts: subject and predicate.

o subject is what (or whom) the sentence is about.

o predicate tells something about the subject.

‡ Example 1 : "cows eat grass".

It is a clause, because it contains

the subject "cows" and

the predicate "eat grass."

‡ Example 2 : "cows eating grass are visible from highway"

This is a complete clause.

the subject "cows eating grass" and

the predicate "are visible from the highway" makes complete thought.

59

fo

.in

rs

de

KR – Logic Programming

ea

(b) Predicates & Clause

yr

.m

w

w

Syntactically a predicate is composed of one or more clauses.

,w

ty

or

ab

‡ The general form of clauses is

kr

ha

<left-hand-side> :- <right-hand-side>.

C

C

R

where LHS is a single goal called "goal" and RHS is composed of one or more goals, separated by commas, called "sub-goals" of the goal on left-hand side.

The symbol " :- " is pronounced as "it is the case" or "such that"

‡ The structure of a clause in logic program

head body

pred (functor(var1, var2)) :- pred(var1) , pred(var2)

literal literal

clause

Literals represent the possible choices in primitive types the particular language. Some of the choices of types of literals are often integers, floating point, Booleans and character strings.

‡ Example : grand_parent (X, Z) :- parent(X, Y), parent(Y, Z).

parent (X, Y) :- mother(X, Y).

parent (X, Y) :- father(X, Y).

Read as if x is mother of y then x is parent of y

[Continued in next slide]

60

fo

.in

rs

de

KR – Logic Programming

ea

[Continued from previous slide]

yr

.m

w

w

‡ Interpretation:

,w

ty

* A clause specifies the conditional truth of the goal on the LHS;

or

ab

kr

goal on LHS is assumed to be true if the sub-goals on RHS are all

ha

C

C

true. A predicate is true if at least one of its clauses is true.

R

* An individual "X" is the grand-parent of "Z" if a parent of that

same "X" is "Y" and "Y" is the parent of that "Z".

(X is parent of Y) (Y is parent of Z)

X Y Z

(X is grand parent of Z)

* An individual "X" is a parent of "Y" if "Y" is the mother of "X"

(X is parent of Y)

X Y

(X is mother of Y)

* An individual "X" is a parent of "Y" if "Y" is the father of "X".

(X is parent of Y)

X Y

(X is father of Y)

61

fo

.in

rs

de

KR – Logic Programming

ea

(c) Unit Clause - a special Case

yr

.m

w

w

Unlike the previous example of conditional truth, one often encounters

,w

ty

or

unconditional relationships that hold.

ab

kr

‡ In Prolog the clauses that are unconditionally true are called

ha

C

C

unit clause or fact .

R

‡ Example : Unconditionally relationships say

'X' is the father of 'Y' is unconditionally true.

This relationship as a Prolog clause is

father(X, Y) :- true.

Interpreted as relationship of father between X and Y is always true; or simply stated as X is father of Y .

‡ Goal true is built-in in Prolog and always holds.

‡ Prolog offers a simpler syntax to express unit clause or fact

father(X, Y)

ie the " :- true " part is simply omitted.

62

fo

.in

rs

de

KR – Logic Programming

ea

(d) Queries

yr

.m

w

w

In Prolog the queries are statements called directive.

,w

ty

or

A special case of directives, are called queries.

ab

kr

ha

C

‡ Syntactically, directives are clauses with an empty left-hand side.

C

R

Example : ? - grandparent(Q, Z).

This query Q is interpreted as : Who is a grandparent of Z ?

By issuing queries Q, Prolog tries to establish the validity of specific relationships.

The answer from previous slides is (X is grand parent of Z)

‡ The result of executing a query is either success or failure

Success, means the goals specified in the query holds according to

the facts and rules of the program.

Failure, means the goals specified in the query does not hold according to the facts and rules of the program.

63

fo

.in

rs

de

KR – Logic - models of computation

ea

- Programming Paradigms : Models of Computation

yr

.m

w

w

A complete description of a programming language includes the

,w

ty

or

computational model, syntax, semantics, and pragmatic considerations

ab

kr

that shape the language.

ha

C

C

R

Models of Computation :

A computational model is a collection of values and operations, while computation is the application of a sequence of operations to a value to yield another value.

There are three basic computational models :

(a) Imperative, (b) Functional, and (c) Logic.

In addition to these, there are two programming paradigms :

(a) concurrent (b) object-oriented programming .

While, these two are not models of computation, but they rank in importance with computational models.

64

fo

.in

rs

de

KR - Logic - models of computation

ea

(a) Imperative Model

yr

.m

w

w

The Imperative model of computation, consists of a state and an

,w

ty

or

operation of assignment which is used to modify the state.

ab

kr

□ Programs consist of sequences of commands.

ha

C

C

□ Computations are changes in the state.

R

Example : Linear function

A linear function $y = 2x + 3$ can be written as

$Y := 2 * X + 3$

The implementation requires to determine the value of X in the state and then creates a new state which differs from the old state.

New State: $X = 3, Y = 9$,

The imperative model is closest to the hardware model on which programs are executed, that makes it most efficient model in terms of execution time.

65

fo

.in

rs

de

KR - Logic - models of computation

ea

(b) Functional model

yr

.m

w

w

The Functional model of computation, consists of a set of values,

,w

ty

or

functions, and the operation of functions. The functions may be named

ab

kr

and composed with other functions. It can take other functions as

ha

C

C

arguments and return results.

R

□ Programs consist of definitions of functions.

□ Computations are application of functions to values.

‡ Example 1 : Linear function

A linear function $y = 2x + 3$ can be defined as :

$f(x) = 2 * x + 3$

‡ Example 2 : Determine a value for Circumference.

Assign a value to Radius, that determines a value for Circumference.

Circumference = $2 \times \pi \times \text{radius}$, where $\pi = 3.14$

Generalize Circumference with the variable "radius" ie

Circumference(radius) = $2 \times \pi \times \text{radius}$, where $\pi = 3.14$

Functional models are developed over many years. The notations and methods form the base upon which problem solving methodologies rest.

66

fo

.in

rs

de

KR - Logic - models of computation

ea

(c) Logic Model

yr

.m

w

w

The logic model of computation is based on relations and logical

,w

ty

or

inference.

ab

kr

□ Programs consist of definitions of relations.

ha

C

C

□ Computations are inferences (is a proof).

R

‡ Example 1 : Linear function

A linear function $y = 2x + 3$ can be represented as :

$f(X, Y)$ if Y is $2 * X + 3$.

Here the function represents the relation between X and Y .

‡ Example 2: Determine a value for Circumference.

The circumference computation can be represented as:

Circle (R, C) if $Pi = 3.14$ and $C = 2 * pi * R$.

Here the function is represented as the relation between radius R and circumference C .

‡ Example 3: Determine the mortality of Socrates and Penelope.

The program is to determine the mortality of Socrates and Penelope.

The fact given that Socrates and Penelope are human.

The rule is that all humans are mortal, that is

for all X , if X is human then X is mortal.

To determine the mortality of Socrates or Penelope, make the assumption that there are no mortals, that is $\neg \text{mortal}(Y)$

[logic model continued in the next slide]

67

fo

.in

rs

de

KR - Logic - models of computation

ea

[logic model continued in the previous slide]

yr

.m

w ‡ The equivalent form of the facts and rules stated before are

w

,w

ty

human (Socrates)

or

ab

mortal (X) if human (X)

kr

ha

‡ To determine the mortality of Socrates and Penelope, we made the

C

C

R

assumption that there are no mortals i.e. $\neg$ mortal (Y)

‡ Computation (proof) that Socrates is mortal

1. (a) human(Socrates) Fact

2. mortal(X) if human(X) Rule

3 $\neg$ mortal(Y) assumption

4.(a) $X=Y$

from 2 & 3 by unification

4.(b) $\neg$ human(Y)

and modus tollens

5. $Y = \text{Socrates}$ from 1 and 4 by unification

6. Contradiction 5, 4b, and 1

‡ Explanation :

* The 1st line is the statement "Socrates is a man."

* The 2nd line is a phrase "all human are mortal"

into the equivalent "for all X, if X is a man then X is mortal".

* The 3rd line is added to the set to determine the mortality of Socrates.

* The 4th line is the deduction from lines 2 and 3. It is justified by the inference rule modus tollens which states that if the conclusion of a rule is known to be false, then so is the hypothesis.

* Variables X and Y are unified because they have same value.

* By unification, Lines 5, 4b, and 1 produce contradictions and identify Socrates as mortal.

* Note that, resolution is an inference rule which looks for a contradiction

and it is facilitated by unification which determines if there is a substitution which makes two terms the same.

Logic model formalizes the reasoning process. It is related to relational data bases and expert systems.

68

fo

.in

rs

de

KR – forward-backward reasoning

ea

3.3 Forward versus Backward Reasoning

yr

.m

w

w

Rule-Based system architecture consists a set of rules, a set of facts, and

,w

ty

or

an inference engine. The need is to find what new facts can be derived.

ab

kr

ha

C

Given a set of rules, there are essentially two ways to generate new

C

R

knowledge: one, forward chaining and the other, backward chaining.

■ Forward chaining : also called data driven.

It starts with the facts, and sees what rules apply.

■ Backward chaining : also called goal driven.

It starts with something to find out, and looks for rules that will help in answering it.

69

fo

.in

rs

de

KR – forward-backward reasoning

ea

■ Example 1

yr

.m

w Rule R1 : IF hot AND smoky THEN fire

w

,w

Rule R2 : IF alarm_beeps THEN smoky

ty

or

ab

Rule R3 : IF fire THEN switch_on_sprinklers

kr

ha

Fact F1 : alarm_beeps [Given]

C

C

R

Fact F2 : hot [Given]

■ Example 2

Rule R1 : IF hot AND smoky THEN ADD fire

Rule R2 : IF alarm_beeps THEN ADD smoky

Rule R3 : IF fire THEN ADD switch_on_sprinklers

Fact F1 : alarm_beeps [Given]

Fact F2 : hot [Given]

70

fo

.in

rs

de

KR – forward-backward reasoning

ea

■ Example 3 : A typical Forward Chaining

yr

.m

w

w

Rule R1 : IF hot AND smoky THEN ADD fire

,w

ty

Rule R2 : IF alarm_beeps THEN ADD smoky

or

ab

kr

Rule R3 : If fire THEN ADD switch_on_sprinklers

ha

C

Fact F1 : alarm_beeps [Given]

C

R

Fact F2 : hot [Given]

Fact F4 : smoky [from F1 by R2]

Fact F2 : fire [from F2, F4 by R1]

Fact F6 : switch_on_sprinklers [from F2 by R3]

■ Example 4 : A typical Backward Chaining

Rule R1 : IF hot AND smoky THEN fire

Rule R2 : IF alarm_beeps THEN smoky

Rule R3 : If _fire THEN switch_on_sprinklers

Fact F1 : hot [Given]

Fact F2 : alarm_beeps [Given]

Goal : Should I switch sprinklers on?

71

fo

.in

rs

de

KR – forward chaining

ea

• Forward Chaining

yr

.m

w

w

The Forward chaining system, properties , algorithms, and conflict

,w

ty

or

resolution strategy are illustrated.

ab

kr

ha

C

■ Forward chaining system

C

R

facts

Working Inference

Memory Engine

facts

facts rules

Rule

User

Base

‡ facts are held in a working memory

‡ condition-action rules represent actions to be taken when specified facts occur in working memory.

‡ typically, actions involve adding or deleting facts from the working memory.

■ Properties of Forward Chaining

‡ all rules which can fire do fire.

‡ can be inefficient - lead to spurious rules firing, unfocused problem solving

‡ set of rules that can fire known as conflict set.

‡ decision about which rule to fire is conflict resolution.

72

fo

.in

rs

de

KR – forward chaining

ea

■ Forward chaining algorithm - I

yr

.m

w Repeat

w

,w

ty

‡ Collect the rule whose condition matches a fact in WM.

or

ab

‡ Do actions indicated by the rule.

kr

ha

(add facts to WM or delete facts from WM)

C

C

R

Until problem is solved or no condition match

Apply on the Example 2 extended (adding 2 more rules and 1 fact)

Rule R1 : IF hot AND smoky THEN ADD fire

Rule R2 : IF alarm_beeps THEN ADD smoky

Rule R3 : If fire THEN ADD switch_on_sprinklers

Rule R4 : IF dry THEN ADD switch_on_humidifier

Rule R5 : IF sprinklers_on THEN DELETE dry

Fact F1 : alarm_beeps [Given]

Fact F2 : hot [Given]

Fact F2 : Dry [Given]

Now, two rules can fire (R2 and R4)

Rule R4 ADD humidifier is on [from F2]

Rule R2 ADD smoky [from F1]

[followed by ADD fire [from F2 by R1]

sequence of

actions] ADD switch_on_sprinklers [by R3]

DELETE dry, ie [by R5]

humidifier is off a conflict !

■ Forward chaining algorithm - II (applied to example 2 above)

Repeat

‡ Collect the rules whose conditions match facts in WM.

‡ If more than one rule matches as stated above then

◇ Use conflict resolution strategy to eliminate all but one

‡ Do actions indicated by the rules

(add facts to WM or delete facts from WM)

Until problem is solved or no condition match

73

fo

.in

rs

de

KR – forward chaining

ea

■ Conflict Resolution Strategy

yr

.m

w

w

Conflict set is the set of rules that have their conditions satisfied by

,w

ty

or

working memory elements.

ab

kr

Conflict resolution normally selects a single rule to fire.

ha

C

C

The popular conflict resolution mechanisms are :

R

Refractory, Recency, Specificity.

◇ Refractory

‡ a rule should not be allowed to fire more than once on the same data.

‡ discard executed rules from the conflict set.

‡ prevents undesired loops.

◇ Recency

‡ rank instantiations in terms of the recency of the elements in the premise of the rule.

‡ rules which use more recent data are preferred.

‡ working memory elements are time-tagged indicating at what cycle each fact was added to working memory.

◇ Specificity

‡ rules which have a greater number of conditions and are

therefore more difficult to satisfy, are preferred to more general

rules with fewer conditions.

‡ more specific rules are 'better' because they take more of the data into account.

74

fo

.in

rs

de

KR – forward chaining

ea

■ Alternative to Conflict Resolution – Use Meta Knowledge

yr

.m

w

w

Instead of conflict resolution strategies, sometimes we want to use

,w

ty

or

knowledge in deciding which rules to fire. Meta-rules reason about

ab

kr

which rules should be considered for firing. They direct reasoning rather

ha

C

C

than actually performing reasoning.

R

‡ Meta-knowledge : knowledge about knowledge to guide search.

‡ Example of meta-knowledge

IF conflict set contains any rule (c , a) such that

a = "animal is mammal"

THEN fire (c , a)

‡ This example says meta-knowledge encodes knowledge about how to guide search for solution.

‡ Meta-knowledge, explicitly coded in the form of rules with "object level" knowledge.

75

fo

.in

rs

de

KR – backward chaining

ea

• Backward Chaining

yr

.m

w

w

Backward chaining system and the algorithm are illustrated.

,w

ty

or

ab

■ Backward chaining system

kr

ha

C

‡ Backward chaining means reasoning from goals back to facts.

C

R

The idea is to focus on the search.

‡ Rules and facts are processed using backward chaining interpreter.

‡ Checks hypothesis, e.g. "should I switch the sprinklers on?"

■ Backward chaining algorithm

‡ Prove goal G

If G is in the initial facts , it is proven.

Otherwise, find a rule which can be used to conclude G, and

try to prove each of that rule's conditions.

alarm_beeps

Smoky hot

fire

switch_on_sprinklers

Encoding of rules

Rule R1 : IF hot AND smoky THEN fire

Rule R2 : IF alarm_beeps THEN smoky

Rule R3 : If fire THEN switch_on_sprinklers

Fact F1 : hot [Given]

Fact F2 : alarm_beeeps [Given]

Goal : Should I switch sprinklers on?

76

fo

.in

rs

de

KR – backward chaining

ea

- Forward vs Backward Chaining

yr

.m

w

w

‡ Depends on problem, and on properties of rule set.

,w

ty

or

‡ Backward chaining is likely to be better if there is clear hypotheses.

ab

kr

ha

Examples : Diagnostic problems or classification problems, Medical

C

C

R

expert systems

‡ Forward chaining may be better if there is less clear hypothesis and

want to see what can be concluded from current situation;

Examples : Synthesis systems - design / configuration.

77

fo

.in

rs

de

KR – control knowledge

ea

3.4 Control Knowledge

yr

.m

w

w

An algorithm consists of : logic component, that specifies the knowledge

,w

ty

or

to be used in solving problems, and control component, that determines

ab

kr

the problem-solving strategies by means of which that knowledge is used.

ha

C

C

Thus Logic + Control .

R

Algorithm =

The logic component determines the meaning of the algorithm whereas the control component only affects its efficiency.

An algorithm may be formulated in different ways, producing same behavior. One formulation, may have a clear statement in logic component but employ a sophisticated problem solving strategy in the control component. The other formulation, may have a complicated logic component but employ a simple problem-solving strategy.

The efficiency of an algorithm can often be improved by improving the control component without changing the logic of the algorithm and therefore without changing the meaning of the algorithm.

The trend in databases is towards the separation of logic and control. The programming languages today do not distinguish between them. The programmer specifies both logic and control in a single language. The execution mechanism exercises only the most rudimentary problem-solving capabilities.

Computer programs will be more often correct, more easily improved, and more readily adapted to new problems when programming languages separate logic and control, and when execution mechanisms provide more powerful problem-solving facilities of the kind provided by intelligent theorem-proving systems.

78

fo

.in

rs

de

ea

yr

.m

AI- Knowledge Representation - References

w

w

4. References : Textbooks

,w

ty

or

1. "Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill

ab

kr

companies Inc., Chapter 4-6, page 105-192.

ha

C

C

R

2. "Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig, (2002), Prentice Hall, Chapter 7-10, page 194-369.

3. "Computational Intelligence: A Logical Approach", by David Poole, Alan Mackworth, and Randy Goebel, (1998), Oxford University Press, Chapter 5-7, page 169-278.

4. "Artificial Intelligence: Structures and Strategies for Complex Problem Solving", by George F. Luger, (2002), Addison-Wesley, Chapter 2, 7, page 35-77, 227-273.

5. "AI: A New Synthesis", by Nils J. Nilsson, (1998), Morgan Kaufmann Inc., Chapter 13, Page 217-229.

6. "Artificial Intelligence: Theory and Practice", by Thomas Dean, (1994), Addison-Wesley, Chapter 3, 6, Page 71-130, 255-296.

7. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

79

fo

.in

rs

de

ea

yr

Reasoning System - Symbolic , Statistical : AI Course Lecture 23 – 28, notes, slides

.m

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

www.myreaders.info

kr

ha

C

C

R

Reasoning System

Symbolic , Statistical

Artificial Intelligence

Reasoning system, topics : Definitions, formal and informal logic, uncertainty, monotonic logic, non-monotonic Logic; Methods of reasoning and examples - deductive, inductive, abductive, analogy; Sources of uncertainty; Reasoning and knowledge representation; Approaches to reasoning - symbolic, statistical and fuzzy. Symbolic reasoning : Non-monotonic reasoning - default reasoning, circumscription, truth maintenance systems; Implementation issues. Statistical Reasoning : Glossary of terms; Probability and Bayes' theorem; Certainty factors in rule based systems; Bayesian networks, Dempster Shafer theory - model, belief and plausibility, calculus, and combining beliefs; Fuzzy logic - description, membership function.

fo

.in

rs

de

ea

yr

Reasoning System

.m

w

w

,w

Symbolic , Statistical

ty

or

ab

Artificial Intelligence

kr

ha

C

C

R

Topics

(Lectures 23, 24, 25, 26, 27, 28, 6 hours) Slides

1. Reasoning: 03-22

Definitions : Reasoning, formal logic and informal logic, uncertainty, monotonic logic, non-monotonic Logic; Methods of reasoning and examples - deductive, inductive, abductive, analogy; Sources of uncertainty; Reasoning and KR ; Approaches to reasoning - symbolic, statistical and fuzzy.

2. Symbolic Reasoning: 23-32

Non-monotonic reasoning - Default Reasoning, Circumscription, Truth Maintenance Systems; Implementation issues.

3. Statistical Reasoning : 33-71

Glossary of terms; Probability and Bayes' theorem - probability, Bayes' theorem, examples ; Certainty factors rule-based systems; Bayesian networks and certainty factors - Bayesian networks; Dempster Shafer theory - model , belief and plausibility, calculus, combining beliefs; Fuzzy logic - description, membership.

4. References : 72

02

fo

.in

rs

de

ea

yr

Reasoning System

.m

w

w

,w

Symbolic , Statistical

ty

or

ab

kr

What is Reasoning ?

ha

C

C

R

- Reasoning is the act of deriving a conclusion from certain premises using a given methodology.
- Reasoning is a process of thinking; reasoning is logically arguing; reasoning is drawing inference.
- When a system is required to do something, that it has not been explicitly told how to do, it must reason. It must figure out what it needs to know from what it already knows.
- Many types of Reasoning have long been identified and recognized, but many questions regarding their logical and computational properties still remain controversial.
- The popular methods of Reasoning include abduction, induction, model-based, explanation and confirmation. All of them are intimately related to problems of belief revision and theory development, knowledge assimilation, discovery and learning.

03

fo

.in

rs

de

AI - Reasoning

ea

1. Reasoning

yr

.m

w

w

Any knowledge system to do something, if it has not been explicitly told

,w

ty

or

how to do it then it must reason.

ab

kr

The system must figure out what it needs to know from what it already knows.

ha

C

C

Example

R

If we know : Robins are birds. All birds have wings.

Then if we ask : Do robins have wings?

Some reasoning (although very simple) has to go on answering the question.

1.1 Definitions :

- Reasoning is the act of deriving a conclusion from certain premises using a given methodology.

■ Any knowledge system must reason, if it is required to do something which has not been told explicitly .

■ For reasoning, the system must find out what it needs to know from what it already knows.

■ Example :

If we know : Robins are birds.

All birds have wings

Then if we ask: Do robins have wings?

To answer this question - some reasoning must go.

04

fo

.in

rs

de

AI - Reasoning

ea

■ Human reasoning capabilities are divided into three areas:

yr

.m

w

w

‡ Mathematical Reasoning – axioms, definitions, theorems, proofs

,w

ty

or

ab

‡ Logical Reasoning – deductive, inductive, abductive

kr

ha

C

‡ Non-Logical Reasoning – linguistic , language

C

R

These three areas of reasoning, are in every human being, but the ability level depends on education, environment and genetics.

The IQ (Intelligence quotient) is the summation of mathematical reasoning skill and the logical reasoning.

The EQ (Emotional Quotient) depends mostly on non-logical reasoning capabilities.

Note : The Logical Reasoning is of our concern in AI

05

fo

.in

rs

de

AI - Reasoning

ea

• Logical Reasoning

yr

.m

w

w

Logic is a language for reasoning. It is a collection of rules called

,w

ty

or

Logic arguments, we use when doing logical reasoning.

ab

kr

ha

C

Logic reasoning is the process of drawing conclusions from premises

C

R

using rules of inference.

The study of logic is divided into formal and informal logic.

The formal logic is sometimes called symbolic logic.

Symbolic logic is the study of symbolic abstractions (construct) that capture the formal features of logical inference by a formal system.

Formal system consists of two components, a formal language plus a set of inference rules. The formal system has axioms.

Axiom is a sentence that is always true within the system.

Sentences are derived using the system's axioms and rules of derivation are called theorems.

06

fo

.in

rs

de

AI - Reasoning

ea

■ Formal Logic

yr

.m

w

w

The Formal logic is the study of inference with purely formal content,

,w

ty

or

ie. where content is made explicit.

ab

kr

ha

C

Examples - Propositional logic and Predicate logic.

C

R

‡ Here the logical arguments are a set of rules for manipulating symbols. The rules are of two types

◊ Syntax rules : say how to build meaningful expressions.

◊ Inference rules : say how to obtain true formulas from other true formulas.

‡ Logic also needs semantics, which says how to assign meaning to expressions.

07

fo

.in

rs

de

AI - Reasoning

ea

■ Informal Logic

yr

.m

w

w

The Informal logic is the study of natural language arguments.

,w

ty

or

ab

‡ The analysis of the argument structures in ordinary language is

kr

ha

part of informal logic.

C

C

R

‡ The focus lies in distinguishing good arguments (valid) from bad arguments or fallacies (invalid).

08

fo

.in

rs

de

AI - Reasoning

ea

■ Formal Systems

yr

.m

w

w

Formal systems can have following three properties :

,w

ty

or

ab

‡ Consistency : System's theorems do not contradict.

kr

ha

C

‡ Soundness : System's rules of derivation will never infer

C

R

anything false, so long as start is with only true premises.

‡ Completeness : There are no true sentences in the system that cannot be proved using the derivation rules of the system.

System Elements

Formal systems consist of following elements :

‡ A finite set of symbols for constructing formulae.

‡ A grammar, is a way of constructing well-formed formulae (wff).

‡ A set of axioms; each axiom has to be a wff.

‡ A set of inference rules.

‡ A set of theorems.

A well-formed formulae, wff, is any string generated by a grammar.

e.g., the sequence of symbols $((\alpha \rightarrow \beta) \rightarrow (\neg \beta \rightarrow \neg \alpha))$ is a WFF

because it is grammatically correct in propositional logic.

09

fo

.in

rs

de

AI - Reasoning

ea

■ Formal Language

yr

.m

w

w

A formal language may be viewed as being analogous to a collection

,w

ty

or

of words or a collection of sentences.

ab

kr

ha

C

‡ In computer science, a formal language is defined by precise

C

R

mathematical or machine process able formulas.

‡ A formal language L is characterized as a set F of finite-length sequences of elements drawn from a specified finite set A of symbols.

‡ Mathematically, it is an unordered pair $L = \{ A, F \}$

‡ If A is words

then the set A is called alphabet of L , and
the elements of F are called words.

‡ If A is sentence

then the set A is called the lexicon or vocabulary of F , and
the elements of F are then called sentences.

‡ The mathematical theory that treats formal languages in general
is known as formal language theory.

10

fo

.in

rs

de

AI - Reasoning

ea

- Uncertainty in Reasoning

yr

.m

w

w

■ The world is an uncertain place; often the Knowledge is imperfect

,w

ty

or

which causes uncertainty. Therefore reasoning must be able to

ab

kr

operate under uncertainty.

ha

C

C

R

■ AI systems must have ability to reason under conditions of uncertainty.

Uncertainties Desired action

‡ Incompleteness Knowledge : Compensate for lack of knowledge

‡ Inconsistencies Knowledge : Resolve ambiguities and contradictions

‡ Changing Knowledge : Update the knowledge base over time

11

fo

.in

rs

de

AI - Reasoning

ea

• Monotonic Logic

yr

.m

w

w

Formal logic is a set of rules for making deductions that seem

,w

ty

or

self evident. A Mathematical logic formalizes such deductions with rules

ab

kr

precise enough to program a computer to decide if an argument is

ha

C

C

valid, representing objects and relationships symbolically.

R

Examples

□ Predicate logic and the inferences we perform on it.

□ All humans are mortal. Socrates is a human.

Therefore Socrates is mortal.

In monotonic reasoning if we enlarge at set of axioms we cannot retract any existing assertions or axioms.

‡ Most formal logics have a monotonic consequence relation, meaning that adding a formula to a theory never produces a reduction of its set of consequences. In other words, a logic is monotonic if the truth of a proposition does not change when new information (axioms) are added. The traditional logic is monotonic.

‡ In mid 1970s, Marvin Minsky and John McCarthy pointed out that pure classical logic is not adequate to represent the commonsense nature of human reasoning. The reason is, the human reasoning is non-monotonic in nature. This means, we reach to conclusions from certain premises that we would not reach if certain other sentences are included in our premises.

‡ The non-monotonic human reasoning is caused by the fact that our knowledge about the world is always incomplete and therefore we are forced to reason in the absence of complete information. Therefore we often revise our conclusions, when new information becomes available.

‡ Thus, the need for non-monotonic reasoning in AI was recognized, and several formalizations of non-monotonic reasoning.

Only the non-monotonic logic reasoning is presented in next few slides.

12

fo

.in

rs

de

AI - Reasoning

ea

- Non-Monotonic Logic

yr

.m

w

w

Inadequacy of monotonic logic for reasoning is said in the previous slide.

,w

ty

or

A monotonic logic cannot handle :

ab

kr

□ Reasoning by default : because consequences may be derived only

ha

C

C

because of lack of evidence of the contrary.

R

□ Abductive reasoning : because consequences are only deduced as most likely explanations.

□ Belief revision : because new knowledge may contradict old beliefs.

A non-monotonic logic is a formal logic whose consequence relation is not monotonic. A logic is non-monotonic if the truth of a proposition may change when new information (axioms) are added.

‡ Allows a statement to be retracted.

‡ Used to formalize plausible (believable) reasoning.

Example 1 :

Birds typically fly.

Tweety is a bird.

Tweety (presumably) flies.

‡ Conclusion of non-monotonic argument may not be correct.

Example-2 : (Ref. Example-1)

If Tweety is a penguin, it is incorrect to conclude that Tweety flies.

(Incorrect because, in example-1, default rules were applied when case-specific information was not available.)

‡ All non-monotonic reasoning are concerned with consistency.

Inconsistency is resolved, by removing the relevant conclusion(s) derived by default rules, as shown in the example below.

Example -3 :

The truth value (true or false), of propositions such as "Tweety is a bird"

accepts default that is normally true, such as "Birds typically fly".
 Conclusions derived was "Tweety flies". When an inconsistency is
 recognized, only the truth value of the last type is changed.

13

fo

.in

rs

de

AI - Reasoning

ea

1.2 Different Methods of Reasoning

yr

.m

w

w

Mostly three kinds of logical reasoning: Deduction, Induction, Abduction.

,w

ty

or

ab

■ Deduction

kr

ha

‡ Example: "When it rains, the grass gets wet. It rains. Thus, the

C

C

R

grass is wet."

This means in determining the conclusion; it is using rule and its
 precondition to make a conclusion.

‡ Applying a general principle to a special case.

‡ Using theory to make predictions

‡ Usage: Inference engines, Theorem provers, Planning.

■ Induction

‡ Example: "The grass has been wet every time it has rained. Thus,
 when it rains, the grass gets wet."

This means in determining the rule; it is learning the rule after
 numerous examples of conclusion following the precondition.

‡ Deriving a general principle from special cases

‡ From observations to generalizations to knowledge

‡ Usage: Neural nets, Bayesian nets, Pattern recognition

14

fo

.in

rs

de

AI - Reasoning

ea

■ Abduction

yr

.m

w ‡ Example: "When it rains, the grass gets wet. The grass is wet, it

w

,w

ty

must have rained."

or

ab

Means determining the precondition; it is using the conclusion and

kr

ha

C

the rule to support that the precondition could explain the conclusion.

C

R

‡ Guessing that some general principle can relate a given pattern of cases

‡ Extract hypotheses to form a tentative theory

‡ Usage: Knowledge discovery, Statistical methods, Data mining.

■ Analogy

‡ Example: "An atom, with its nucleus and electrons, is like the solar system, with its sun and planets."

Means analogous; it is illustration of an idea by means of a more familiar idea that is similar to it in some significant features. and thus said to be analogous to it.

‡ finding a common pattern in different cases

‡ usage: Matching labels, Matching sub-graphs, Matching transformations.

Note: Deductive reasoning and Inductive reasoning are the two most commonly used explicit methods of reasoning to reach a conclusion.

15

fo

.in

rs

de

AI - Reasoning

ea

- More about different methods of Reasoning

yr

.m

w

w

■ Deduction Example

,w

ty

or

Reason from facts and general principles to other facts.

ab

kr

Guarantees that the conclusion is true.

ha

C

C

‡ Modus Ponens : a valid form of argument affirming the antecedent.

R

◇ If it is rainy, John carries an umbrella

It is rainy

----- (doted line read as "therefore")

John carries an umbrella.

◇ If p then q

p

q

‡ Modus Tollens : a valid form of argument denying the consequent.

◇ If it is rainy, John carries an umbrella

John does not carry an umbrella

----- (doted line read as "because")

It is not rainy

◇ If p then q

not q

not p

16

fo

.in

rs

de

AI - Reasoning

ea

■ Induction Example

yr

.m

w Reasoning from many instances to all instances.

w

,w

ty

or

‡ Good Movie

ab

kr

Fact You have liked all movies starring Mery.

ha

C

C

Inference You will like her next movie.

R

‡ Birds

Facts: Woodpeckers, swifts, eagles, finches have four toes on each foot.

Inductive Inference All birds have 4 toes on each foot.

(Note: partridges have only 3).

‡ Objects

Facts Cars, bottles, blocks fall if not held up.

Inductive Inference If not supported, an object will fall.

(Note: an unsupported helium balloon will rise.)

‡ Medicine

Noted People who had cowpox did not get smallpox.

Induction: Cowpox prevents smallpox.

Problem : Sometime inference is correct, sometimes not correct.

Advantage : Inductive inference may be useful even if not correct.

It generates a proposition which may be validated deductively.

17

fo

.in

rs

de

AI - Reasoning

ea

■ Abduction Example

yr

.m

w Common form of human reasoning- "Inference to the best explanation".

w

,w

ty

In Abductive reasoning you make an assumption which, if true,

or

ab

together with your general knowledge, will explain the facts.

kr

ha

C

C

R

‡ Dating

Fact: Mary asks John to a party.

Abductive Inferences Mary likes John.

John is Mary's last choice.

Mary wants to make someone else jealous.

‡ Smoking house

Fact: A large amount of black smoke is coming from a home.

Abduction1: the house is on fire.

Abduction2: bad cook.

‡ Diagnosis

Facts: A thirteen year-old boy has a sharp pain in his right side, a fever, and a high white

blood count.

Abductive inference Appendicitis.

Problem: Not always correct; many explanations possible.

Advantage : Understandable conclusions.

18

fo

.in

rs

de

AI - Reasoning

ea

■ Analogy Example

yr

.m

w Analogical Reasoning yields conjectures, possibilities.

w

,w

ty

If A is like B in some ways, then infer A is like B in other ways.

or

ab

kr

ha

‡ Atom and Solar System

C

C

R

Statements: An atom, with its nucleus and electrons, is like the solar system, with its sun and planets.

Inferences: Electrons travel around the nucleus.

Orbits are circular.

? Orbits are all in one plane.

? Electrons have little people living on them.

Idea: Transfer information from known (source) to unknown (target).

‡ Sun and Girl

Statement: She is like the sun to me.

Inferences: She lights up my life.

She gives me warmth.

? She is gaseous.

? She is spherical.

‡ Sale man Logic

Statement: John has a fancy car and a pretty girlfriend.

Inferences: If Peter buys a fancy car,

Then Peter will have a pretty girlfriend.

Problems : Few analogical inferences are correct

Advantage : Suggests novel possibilities. Helps to organize information.

19

fo

.in

rs

de

AI - Reasoning

ea

1.3 Sources of Uncertainty in Reasoning

yr

.m

w

w

In many problem domains it is not possible to create complete, consistent

,w

ty

or

models of the world. Therefore agents (and people) must act in uncertain

ab

kr

worlds (which the real world is). We want an agent to make rational

ha

C

C

decisions even when there is not enough information to prove that an action

R

will work.

■ Uncertainty is omnipresent because of

‡ Incompleteness

‡ Incorrectness

■ Uncertainty in Data or Expert Knowledge

‡ Data derived from defaults/assumptions

‡ Inconsistency between knowledge from different experts.

‡ “Best Guesses”

■ Uncertainty in Knowledge Representation

‡ Restricted model of the real system.

‡ Limited expressiveness of the representation mechanism.

■ Uncertainty in Rules or Inference Process

‡ Incomplete because too many conditions to be explicitly enumerated

‡ Incomplete because some conditions are unknown

‡ Conflict Resolution

20

fo

.in

rs

de

AI - Reasoning

ea

1.4 Reasoning and KR

yr

.m

w

w

To certain extent, the reasoning depends on the way the knowledge is

,w

ty

or

represented or chosen.

ab

kr

ha

C

■ A good knowledge representation scheme allows easy, natural and

C

R

plausible (credible) reasoning.

■ Reasoning methods are broadly identified as :

‡ Formal reasoning : Using basic rules of inference with logic

knowledge representations.

‡ Procedural reasoning : Uses procedures that specify how to perhaps solve sub problems.

‡ Reasoning by analogy : This is as Human do, but more difficult for AI systems.

‡ Generalization and : This is also as Human do; are basically abstraction

learning and understanding methods.

‡ Meta-level reasoning : Uses knowledge about what we know and ordering them as per importance.

■ Note : What ever may be the reasoning method, the AI model must be able to reason under conditions of uncertainty mentioned before.

21

fo

.in

rs

de

AI - Reasoning

ea

1.5 Approaches to Reasoning

yr

.m

w

w

There are three different approaches to reasoning under uncertainties.

,w

ty

or

ab

‡ Symbolic reasoning

kr

ha

C

‡ Statistical reasoning

C

R

‡ Fuzzy logic reasoning

The first two approaches are presented in the subsequent slides.

22

fo

.in

rs

de

AI - Symbolic Reasoning

ea

2. Symbolic Reasoning

yr

.m

w

w

The basis for intelligent mathematical software is the integration of the "power

,w

ty

or

of symbolic mathematical tools" with the suitable "proof technology".

ab

kr

ha

C

Mathematical reasoning enjoys a property called monotonicity, that says,

C

R

"If a conclusion follows from given premises A, B, C, ...

then it also follows from any larger set of premises, as long as the

original premises A, B, C, ... are included."

Human reasoning is not monotonic.

People arrive to conclusions only tentatively, based on partial or incomplete information, reserve the right to retract those conclusions while they learn new facts. Such reasoning is non-monotonic, precisely because the set of accepted conclusions have become smaller when the set of premises is expanded.

23

fo

.in

rs

de

AI - Symbolic Reasoning

ea

2.1 Non-Monotonic Reasoning

yr

.m

w

w

Non-Monotonic reasoning is a generic name to a class or a specific theory of

,w

ty

or

reasoning. Non-monotonic reasoning attempts to formalize reasoning with

ab

kr

incomplete information by classical logic systems.

ha

C

C

R

The Non-Monotonic reasoning are of the type

■ Default reasoning

■ Circumscription

■ Truth Maintenance Systems

24

fo

.in

rs

de

AI - Symbolic Reasoning

ea

• Default Reasoning

yr

.m

w

w

This is a very common form of non-monotonic reasoning. The conclusions

,w

ty

or

are drawn based on what is most likely to be true.

ab

kr

There are two approaches, both are logic type, to Default reasoning :

ha

C

C

one is Non-monotonic logic and the other is Default logic.

R

■ Non-monotonic logic

It has already been defined. It says, "the truth of a proposition may change when new information (axioms) are added and a logic may be build to allows the statement to be retracted."

Non-monotonic logic is predicate logic with one extension called modal operator M which means "consistent with everything we know". The purpose of M is to allow consistency.

A way to define consistency with PROLOG notation is :

To show that fact P is true, we attempt to prove $\neg P$.

If we fail we may say that P is consistent since $\neg P$ is false.

Example :

$\forall x : \text{plays_instrument}(x) \wedge M \text{ manage}(x) \rightarrow \text{jazz_musician}(x)$

States that for all x, the x plays an instrument and if the fact that x can manage is consistent with all other knowledge then we can conclude that x is a jazz musician.

25

fo

.in

rs

de

AI - Symbolic Reasoning

ea

■ Default Logic

yr

.m

A:B

w Default logic initiates a new inference rule: where

w

C

,w

ty

or

A is known as the prerequisite,

ab

kr

ha

B as the justification, and

C

C

R

C as the consequent.

‡ Read the above inference rule as:

" if A, and if it is consistent with the rest of what is known to assume that B, then conclude that C ".

‡ The rule says that given the prerequisite, the consequent can be inferred, provided it is consistent with the rest of the data.

‡ Example : Rule that "birds typically fly" would be represented as

bird(x) : flies(x)

which says

flies (x)

" If x is a bird and the claim that x flies is consistent with what we know, then infer that x flies".

‡ Note : Since, all we know about Tweety is that :

Tweety is a bird, we therefore inferred that Tweety flies.

‡ The idea behind non-monotonic reasoning is to reason with first order logic, and if an inference can not be obtained then use the set of default rules available within the first order formulation.

26

fo

.in

rs

de

AI - Symbolic Reasoning

ea

[continuing default logic]

yr

.m

‡ Applying Default Rules :

w

w

,w

While applying default rules, it is necessary to check their

ty

or

ab

justifications for consistency, not only with initial data, but also with

kr

ha

the consequents of any other default rules that may be applied. The

C

C

R

application of one rule may thus block the application of another.

To solve this problem, the concept of default theory was extended.

‡ Default Theory

It consists of a set of premises W and a set of default rules D .

An extension for a default theory is a set of sentences E which can be derived from W by applying as many rules of D as possible (together with the rules of deductive inference) without generating inconsistency.

Note : D the set of default rules has a unique syntax of the form

$\alpha(x) : E \beta(x)$ where

$\gamma(x)$

$\alpha(x)$ is the prerequisite of the default rule

$E \beta(x)$ is the consistency test of the default rule

$\gamma(x)$ is the consequent of the default rule

The rule can be read as

For all individual $x_1 \dots x_m$

If $\alpha(x)$ is believed and

If each of $\beta(x)$ is consistent with our beliefs,

Then $\gamma(x)$ may be believed.

27

fo

.in

rs

de

AI - Symbolic Reasoning

ea

[continuing default logic]

yr

.m

w Example :

w

,w

ty

A Default Rule says " Typically an American adult owns a car ".

or

ab

kr

ha

$American(x) \wedge Adult(x) : M((\exists y) . car(y) \wedge owns(x,y))$

C

C

R

$((\exists y) . car(y) \wedge owns(x,y))$

The rule is explained below :

The rule is only accessed if we wish to know whether or not John owns a car then an answer can not be deduced from our current beliefs.

This default rule is applicable if we can prove from our beliefs that John is an American and an adult, and believing that there is some car that is owned by John does not lead to an inconsistency.

If these two sets of premises are satisfied, then the rule states that we can conclude that John owns a car.

28

fo

.in

rs

de

AI - Symbolic Reasoning

ea

- Circumscription

yr

.m

w

w

Circumscription is a non-monotonic logic to formalize the common sense

,w

ty

or

assumption. Circumscription is a formalized rule of conjecture (guess) that

ab

kr

can be used along with the rules of inference of first order logic.

ha

C

C

R

Circumscription involves formulating rules of thumb with "abnormality" predicates and then restricting the extension of these predicates, circumscribing them, so that they apply to only those things to which they are currently known.

■ Example : Take the case of Bird Tweety

The rule of thumb is that "birds typically fly" is conditional. The predicate "Abnormal" signifies abnormality with respect to flying ability.

Observe that the rule $\forall x(\text{Bird}(x) \ \& \ \neg \text{Abnormal}(x) \rightarrow \text{Flies})$ does not allow us to infer that "Tweety flies", since we do not know that he is abnormal with respect to flying ability.

But if we add axioms which circumscribe the abnormality predicate to which they are currently known say "Bird Tweety" then the inference can be drawn. This inference is non-monotonic.

29

fo

.in

rs

de

AI - Symbolic Reasoning

ea

- Truth Maintenance Systems

yr

.m

w

w

Reasoning Maintenance System (RMS) is a critical part of a reasoning

,w

ty

or

system. Its purpose is to assure that inferences made by the reasoning

ab

kr

system (RS) are valid.

ha

C

C

R

The RS provides the RMS with information about each inference it performs, and in return the RMS provides the RS with information about the whole set of inferences.

Several implementations of RMS have been proposed for non-monotonic reasoning. The important ones are the :

- Truth Maintenance Systems (TMS) and
- Assumption-based Truth Maintenance Systems (ATMS).

The TMS maintains the consistency of a knowledge base as soon as new knowledge is added. It considers only one state at a time so it is not possible to manipulate environment.

The ATMS is intended to maintain multiple environments.

The typical functions of TMS are presented in the next slide.

30

fo

.in

rs

de

AI - Symbolic Reasoning

ea

[continuing Truth Maintenance Systems]

yr

.m

w

w

Truth Maintenance Systems (TMS)

,w

ty

A truth maintenance system maintains consistency in knowledge

or

ab

kr

representation of a knowledge base.

ha

C

C

The functions of TMS are to :

R

■ Provide justifications for conclusions

When a problem solving system gives an answer to a user's query, an explanation of that answer is required;

Example : An advice to a stockbroker is supported by an explanation of the reasons for that advice. This is constructed by the Inference Engine (IE) by tracing the justification of the assertion.

■ Recognize inconsistencies

The Inference Engine (IE) may tell the TMS that some sentences are contradictory. Then, TMS may find that all those sentences are believed true, and reports to the IE which can eliminate the inconsistencies by determining the assumptions used and changing them appropriately.

Example : A statement that either Abbott, or Babbitt, or Cabot is guilty together with other statements that Abbott is not guilty, Babbitt is not guilty, and Cabot is not guilty, form a contradiction.

■ Support default reasoning

In the absence of any firm knowledge, in many situations we want to reason from default assumptions.

Example : If "Tweety is a bird", then until told otherwise, assume that "Tweety flies" and for justification use the fact that "Tweety is a bird" and the assumption that "birds fly".

31

fo

.in

rs

de

AI - Symbolic Reasoning

ea

2.2 Implementation Issues

yr

.m

w

w

The issues and weaknesses related to implementation of non-monotonic

,w

ty

or

reasoning in problem solving are :

ab

kr

ha

C

■ How to derive exactly those non-monotonic conclusion that are relevant

C

R

to solving the problem at hand while not wasting time on those that are not necessary.

■ How to update our knowledge incrementally as problem solving progresses.

■ How to over come the problem where more than one interpretation of the known facts is qualified or approved by the available inference rules.

■ In general the theories are not computationally effective, decidable or semi decidable.

The solutions offered, considering the reasoning processes into two parts :

□ one, a problem solver that uses whatever mechanism it happens to have to draw conclusions as necessary, and

□ second, a truth maintenance system whose job is to maintain consistency in knowledge representation of a knowledge base.

32

fo

.in

rs

de

AI - Statistical Reasoning

ea

3. Statistical Reasoning :

yr

.m

w

w

In the logic based approaches described, we have assumed that everything is

,w

ty

or

either believed false or believed true.

ab

kr

ha

C

However, it is often useful to represent the fact that we believe such that

C

R

something is probably true, or true with probability (say) 0.65.

This is useful for dealing with problems where there is randomness and unpredictability (such as in games of chance) and also for dealing with problems where we could, if we had sufficient information, work out exactly what is true.

To do all this in a principled way requires techniques for probabilistic reasoning.

In this section, the Bayesian Probability Theory is first described and then discussed how uncertainties are treated.

33

fo

.in

rs

de

AI - Statistical Reasoning

ea

- Recall glossary of terms

yr

.m

w

w

■ Probabilities :

,w

ty

or

Usually, are descriptions of the likelihood of some event occurring

ab

kr

(ranging from 0 to 1).

ha

C

C

R

■ Event :

One or more outcomes of a probability experiment .

■ Probability Experiment :

Process which leads to well-defined results call outcomes.

■ Sample Space :

Set of all possible outcomes of a probability experiment.

■ Independent Events :

Two events, E1 and E2, are independent if the fact that E1 occurs does not affect the probability of E2 occurring.

■ Mutually Exclusive Events :

Events E1, E2, ..., En are said to be mutually exclusive if the occurrence of any one of them automatically implies the non-occurrence of the remaining $n - 1$ events.

■ Disjoint Events :

Another name for mutually exclusive events.

34

fo

.in

rs

de

AI - Statistical Reasoning

ea

■ Classical Probability :

yr

.m

w Also called a priori theory of probability.

w

,w

ty

The probability of event A = no of possible outcomes f divided by the

or

ab

total no of possible outcomes n ; ie., $P(A) = f / n$.

kr

ha

C

Assumption: All possible outcomes are equal likely.

C

R

■ Empirical Probability :

Determined analytically, using knowledge about the nature of the experiment rather than through actual experimentation.

■ Conditional Probability :

The probability of some event A, given the occurrence of some other

event B. Conditional probability is written $P(A|B)$, and read as "the probability of A, given B".

■ Joint probability :

The probability of two events in conjunction. It is the probability of both events together. The joint probability of A and B is written $P(A \cap B)$; also written as $P(A, B)$.

■ Marginal Probability :

The probability of one event, regardless of the other event. The marginal probability of A is written $P(A)$, and the marginal probability of B is written $P(B)$.

35

fo

.in

rs

de

AI - Statistical Reasoning

ea

• Examples

yr

.m

w

w

■ Example 1

,w

ty

or

Sample Space - Rolling two dice

ab

kr

The sums can be { 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 }.

ha

C

C

Note that each of these are not equally likely. The only way to get a sum 2 is to roll a 1 on both dice, but can get a sum 4 by rolling out comes as (1,3), (2,2), or (3,1).

Table below illustrates a sample space for the sum obtain.

Second Dice

Second Dice

First dice 1 2 3 4 5 6

1 2 3 4 5 6 7

2 3 4 5 6 7 8

3 4 5 6 7 8 9

4 5 6 7 8 9 10

5 6 7 8 9 10 11

6 7 8 9 10 11 12

Classical Probability

Table below illustrates frequency and distribution for the above sums.

Sum 2 3 4 5 6 7 8 9 10 11 12

Frequency 1 2 3 4 5 6 5 4 3 2 1

Relative frequency $1/36$ $2/36$ $3/36$ $4/36$ $5/36$ $6/36$ $5/36$ $4/36$ $3/36$ $1/36$ $1/36$

The classical probability is the relative frequency of each event.

Classical probability $P(E) = n(E) / n(S)$; $P(6) = 5 / 36$, $P(8) = 5 / 36$

Empirical Probability

The empirical probability of an event is the relative frequency of a frequency distribution based upon observation $P(E) = f / n$

36

fo

.in

rs

de

AI - Statistical Reasoning

ea

■ Example 2

yr

.m

w Mutually Exclusive Events (disjoint) : means nothing in common

w

,w

ty

Two events are mutually exclusive if they cannot occur at the

or

ab

same time.

kr

ha

C

(a) If two events are mutually exclusive,

C

R

then probability of both occurring at same time is $P(A \text{ and } B) = 0$

(b) If two events are mutually exclusive ,

then the probability of either occurring is $P(A \text{ or } B) = P(A) + P(B)$

Given $P(A) = 0.20$, $P(B) = 0.70$, where A and B are disjoint

then $P(A \text{ and } B) = 0$

The table below indicates intersections ie "and" of each pair of events. "Marginal" means total; the values in bold means given; the rest of the values are obtained by addition and subtraction.

B B' Marginal

A 0.00 0.20 0.20

A' 0.70 0.10 0.80

Marginal 0.70 0.30 1.00

Non-Mutually Exclusive Events

The non-mutually exclusive events have some overlap.

When $P(A)$ and $P(B)$ are added, the probability of the intersection (ie. "and") is added twice, so subtract once.

$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$

Given : $P(A) = 0.20$, $P(B) = 0.70$, $P(A \text{ and } B) = 0.15$

B B' Marginal

A 0.15 0.05 0.20

A' 0.55 0.25 0.80

Marginal 0.70 0.30 1.00

37

fo

.in

rs

de

AI - Statistical Reasoning

ea

■ Example 3

yr

.m

w

w

Factorial , Permutations and Combinations

,w

ty

or

ab

Factorial

kr

ha

C

The factorial of an integer $n \geq 0$ is written as $n!$.

C

R

$n! = n \times n-1 \times \dots \times 2 \times 1$. and in particular, $0! = 1$.

It is, the number of permutations of n distinct objects;

e.g., no of ways to arrange 5 letters A, B, C, D and E into a word is $5!$

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

$N! = (N) \times (N-1) \times (N-2) \times \dots \times (1)$

$n! = n (n - 1)! , 0! = 1$

[continuing next slide]

38

fo

.in

rs

de

AI - Statistical Reasoning

ea

[continuing example 3]

yr

.m

w Permutation

w

,w

ty

The permutation is arranging elements (objects or symbols) into

or

ab

distinguishable sequences. The ordering of the elements is important.

kr

ha

C

Each unique ordering is a permutation.

C

R

Number of permutations of 'n' different things taken 'r' at a time is

$n!$

given by .

$P_r = (n - r)!$

(for convenience in writing, here after the symbol P_{nr} is written as

nPr or $P(n, r)$)

Example 1

Consider a total of 10 elements, say integers {1, 2, ..., 10}.

A permutation of 3 elements from this set is (5, 3, 4).

Here $n = 10$ and $r = 3$.

The number of such unique sequences are calculated as $P(10, 3) = 720$.

Example 2

Find the number of ways to arrange the three letters in the word

CAT in to two-letter groups like CA or AC and no repeated letters.

This means permutations are of size $r = 2$ taken from a set of

size $n = 3$. so $P(n, r) = P(3, 2) = 6$.

The ways are listed as CA CT AC AT TC TA.

Similarly, permutations of size $r = 4$, taken from a set of size $n = 10$,

$10! 10! 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$

$P(n, r) = P(10, 4) = = =$

$(10 - 4)! 6! 6 \times 5 \times 4 \times 3 \times 2 \times 1$

[continuing next slide]

39

fo

.in

rs

de

AI - Statistical Reasoning

ea

[continuing example 3]

yr

.m

w Combinations

w

,w

ty

Combination means selection of elements (objects or symbols).

or

ab

The ordering of the elements has no importance.

kr

ha

C

Number of Combination of ' n ' different things, taken ' r ' at a time is

C

R

$n \times n \times n!$

$C_r = r = W$

$r!(n-r)!$ here

r is the size of each combination of elements,

n is the total size of elements from which elements are permuted,

! is the factorial operator.

(for convenience in writing, here after the symbol C_{nr} is written as

nC_r or $C(n,r)$)

Example

Find the number of combinations of size 2 without repeated letters that can be made from the three letters in the word CAT, order doesn't matter; AT is the same as TA.

This means combinations of size $r=2$ taken from a set of size $n=3$, so $C(n, r) = C(3, 2) = 3$. The ways are listed as CA CT CA .

Using the formula for finding the number of combinations of r objects from a set of n objects is:

$n! / 3! \times 2! \times 1! = 6$

$C(n, r) = C(3,2) = = = = 3$

$r! (n-r)! = 2! \times 1! \times 1! = 2$

If n is large then finding n! becomes difficult. The alternate way is given below

Find combinations of size $r=4$, taken from a set of size $n=10$,

$P(10,4) = 10! / 6!$

$C(n, r) = C(10,4) = = =$

$4! \times 6! = 4! \times (10-4)!$

$10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$

$4 \times 3 \times 2 \times 1 (6 \times 5 \times 4 \times 3 \times 2 \times 1)$

40

fo

.in

rs

de

AI - Statistical Reasoning

ea

3.1 Probability and Bayes' Theorem

yr

.m

w In probability theory, Bayes' theorem relates the conditional and

w

,w

ty

marginal probabilities of two random events.

or

ab

kr

ha

- Probability : The Probabilities are numeric values between 0 and 1

C

C

R

(both inclusive) that represent ideal uncertainties (not beliefs).

■ Probability of event A is $P(A)$

instances of the event A

$P(A) =$

total instances

$P(A) = 0$ indicates total uncertainty in A,

$P(A) = 1$ indicates total certainty and

$0 < P(A) < 1$ values in between tells degree of uncertainty

Probability Rules :

‡ All probabilities are between 0 and 1 inclusive $0 \leq P(E) \leq 1$.

‡ The sum of all the probabilities in the sample space is 1.

‡ The probability of an event which must occur is 1.

‡ The probability of the sample space is 1.

‡ The probability of any event which is not in the sample space is zero.

‡ The probability of an event not occurring is $P(E') = 1 - P(E)$

Example 1 : A single 6-sided die is rolled.

What is the probability of each outcome?

What is the probability of rolling an even number?

What is the probability of rolling an odd number?

The possible outcomes of this experiment are 1, 2, 3, 4, 5, 6.

The Probabilities are :

$P(1) = \text{No of ways to roll 1} / \text{total no of sides} = 1/6$

$P(2) = \text{No of ways to roll 2} / \text{total no of sides} = 1/6$

$P(3) = \text{No of ways to roll 3} / \text{total no of sides} = 1/6$

$P(4) = \text{No of ways to roll 4} / \text{total no of sides} = 1/6$

$P(5) = \text{No of ways to roll 5} / \text{total no of sides} = 1/6$

$P(6) = \text{No of ways to roll 6} / \text{total no of sides} = 1/6$

$P(\text{even}) = \text{ways to roll even no} / \text{total no of sides} = 3/6 = 1/2$

$P(\text{odd}) = \text{ways to roll odd no} / \text{total no of sides} = 3/6 = 1/2$

41

fo

.in

rs

de

AI - Statistical Reasoning

ea

Example 2 : Roll two dices

yr

.m

w Each dice shows one of 6 possible numbers;

w

,w

ty

Total unique rolls is $6 \times 6 = 36$;

or

ab

kr

List of the joint possibilities for the two dices are:

ha

C

(1, 1) (1, 2) (1, 3) (1, 4) (1, 5) (1, 6)

C

R

(2, 1) (2, 2) (2, 3) (2, 4) (2, 5) (2, 6)

(3, 1) (3, 2) (3, 3) (3, 4) (3, 5) (3, 6)

(4, 1) (4, 2) (4, 3) (4, 4) (4, 5) (4, 6)

(5, 1) (5, 2) (5, 3) (5, 4) (5, 5) (5, 6)

(6, 1) (6, 2) (6, 3) (6, 4) (6, 5) (6, 6)

Roll two dices;

The rolls that add up to 4 are ((1,3), (2,2), (3,1)).

The probability of rolling dices such that total of 4 is $3/36 = 1/12$
and the chance of it being true is $(1/12) \times 100 = 8.3\%$.

42

fo

.in

rs

de

AI - Statistical Reasoning

ea

■ Conditional probability $P(A|B)$

yr

.m

w

w

A conditional probability is the probability of an event given that

,w

ty

another event has occurred.

or

ab

kr

ha

Example : Roll two dices.

C

C

R

What is the probability that the total of two dice will be greater
than 8 given that the first die is a 6 ?

First List of the joint possibilities for the two dices are:

(1, 1) (1, 2) (1, 3) (1, 4) (1, 5) (1, 6)

(2, 1) (2, 2) (2, 3) (2, 4) (2, 5) (2, 6)

(3, 1) (3, 2) (3, 3) (3, 4) (3, 5) (3, 6)

(4, 1) (4, 2) (4, 3) (4, 4) (4, 5) (4, 6)

(5, 1) (5, 2) (5, 3) (5, 4) (5, 5) (5, 6)

(6, 1) (6, 2) (6, 3) (6, 4) (6, 5) (6, 6)

There are 6 outcomes for which the first die is a 6, and of these, there
are 4 outcomes that total more than 8 are (6,3; 6,4; 6,5; 6,6).

The probability of a total > 8 given that first die is 6 is

therefore $4/6 = 2/3$.

This probability is written as: $P(\text{total} > 8 \mid \text{1st die} = 6) = 2/3$

event condition

Read as "The probability that the total is > 8 given that die one is 6 is $2/3$."

Written as $P(A|B)$, is the probability of event A given that the event B has occurred.

43

fo

.in

rs

de

AI - Statistical Reasoning

ea

■ Probability of A and B is $P(A \text{ and } B)$

yr

.m

w

w

The probability that events A and B both occur.

,w

ty

Note : Two events are independent if the occurrence of one is

or

ab

kr

unrelated to the probability of the occurrence of the other.

ha

C

C

R

‡ If A and B are independent

then probability that events A and B both occur is:

$$P(A \text{ and } B) = P(A) \times P(B)$$

ie product of probability of A and probability of B.

‡ If A and B are not independent

then probability that events A and B both occur is:

$$P(A \text{ and } B) = P(A) \times P(B|A) \text{ where}$$

$P(B|A)$ is conditional probability of B given A

Example 1: $P(A \text{ and } B)$ if events A and B are independent

□ Draw a card from a deck, then replace it, draw another card.

□ Find probability that 1st card is Ace of clubs (event A) and 2nd card is any Club (event B).

□ Since there is only one Ace of Clubs, therefore probability $P(A) = 1/52$.

□ Since there are 13 Clubs, the probability $P(B) = 13/52 = 1/4$.

□ Therefore, $P(A \text{ and } B) = p(A) \times p(B) = 1/52 \times 1/4 = 1/208$.

Example 2: $P(A \text{ and } B)$ if events A and B are not independent

□ Draw a card from a deck, not replacing it, draw another card.

□ Find probability that both cards are Aces ie the 1st card is Ace (event A) and the 2nd card is also Ace (event B).

□ Since 4 of 52 cards are Aces, therefore probability $P(A) = 4/52 = 1/13$.

□ Of the 51 remaining cards, 3 are aces. so, probability of 2nd card is also Ace (event B) is $P(B|A) = 3/51 = 1/17$.

□ Therefore, $P(A \text{ and } B) = p(A) \times p(B|A) = 1/13 \times 1/17 = 1/221$

44

fo

.in

rs

de

AI - Statistical Reasoning

ea

■ Probability of A or B is $P(A \text{ or } B)$

yr

.m

w The probability of either event A or event B occur.

w

,w

ty

Two events are mutually exclusive if they cannot occur at same time.

or

ab

‡ If A and B are mutually exclusive

kr

ha

C

then probability that events A or B occur is:

C

R

$$P(A \text{ or } B) = p(A) + p(B)$$

ie sum of probability of A and probability of B

‡ If A and B are not mutually exclusive

then probability that events A and B both occur is:

$$P(A \text{ or } B) = P(A) + P(B|A) - P(A \text{ and } B) \text{ where}$$

$P(A \text{ and } B)$ is probability that events A and B both occur while events A and B are independent and $P(B|A)$ is conditional probability of B given A.

Example 1: $P(A \text{ or } B)$ if events A or B are mutually exclusive

□ Rolling a die.

□ Find probability of getting either, event A as 1 or event B as 6?

□ Since it is impossible to get both, the event A as 1 and event B as 6 in same roll, these two events are mutually exclusive.

□ The probability $P(A) = P(1) = 1/6$ and $P(B) = P(6) = 1/6$

□ Hence probability of either event A or event B is :

$$P(A \text{ or } B) = p(A) + p(B) = 1/6 + 1/6 = 1/3$$

Example 2: $P(A \text{ or } B)$ if events A or B are not mutually exclusive

□ Find probability that a card from a deck will be either an Ace or a Spade?

□ probability $P(A)$ is $P(\text{Ace}) = 4/52$ and $P(B)$ is $P(\text{spade}) = 13/52$.

□ Only way in a single draw to be Ace and Spade is Ace of Spade; which is only one, so probability $P(A \text{ and } B)$ is $P(\text{Ace and Spade}) = 1/52$.

□ Therefore, the probability of event A or B is :

$$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$$

$$= P(\text{ace}) + P(\text{spade}) - P(\text{Ace and Spade})$$

$$= 4/52 + 13/52 - 1/52 = 16/52 = 4/13$$

45

fo

.in

rs

de

AI - Statistical Reasoning

ea

Summary of symbols & notations

yr

.m

w

w

$A \cup B$ (A union B) 'Either A or B occurs or both occur'

,w

ty

or

$A \cap B$ (A intersection B) 'Both A and B occur'

ab

kr

$A \subseteq B$ (A is a subset of B) 'If A occurs, so does B'

ha

C

C

A' $\bar{A}$ 'Event A does not occur'

R

Φ (the empty set) An impossible event

S (the sample space) An event that is certain to occur

$A \cap B = \Phi$ Mutually exclusive Events

$P(A)$ Probability that event A occurs

$P(B)$ Probability that event B occurs

$P(A \cup B)$ Probability that event A or event B occurs

$P(A \cap B)$ Probability that event A and event B occur

$P(A \cap B) = P(A) \cdot P(B)$ Independent events

$P(A \cap B) = 0$ Mutually exclusive Events

$P(A \cup B) = P(A) + P(B) - P(A \cap B)$ Addition rule;

$P(A \cup B) = P(A) + P(B) - P(A) \cdot P(B)$ Addition rule; independent events

$P(A \cup B) = P(A) + P(B) - P(A \cap B)$

$P(A \cup B) = P(A) + P(B) - P(B|A) \cdot P(A)$

$P(A \cup B) = P(A) + P(B)$ Addition rule; mutually exclusive Events

$A|B$ (A given B) "Event A will occur given that event B has occurred"

$P(A|B)$ Conditional probability that event A will

occur given that event B has occurred

already

$P(B|A)$ Conditional probability that event B will

occur given that event A has occurred

already

$P(A \cap B) = P(A|B) \cdot P(B)$ or Multiplication rule

$P(A \cap B) = P(B|A) \cdot P(A)$

$P(A \cap B) = P(A) \cdot P(B)$ Multiplication rule; independent events;

ie probability of joint events A and B

$P(A|B) = P(A \cap B) / P(B)$ Rule to determine a conditional probability from unconditional probabilities.

46

fo

.in

rs

de

AI - Statistical Reasoning

ea

• Bayes' Theorem

yr

.m

w

w

Bayesian view of probability is related to degree of belief.

,w

ty

or

It is a measure of the plausibility of an event given incomplete knowledge.

ab

kr

ha

C

Bayes' theorem is also known as Bayes' rule or Bayes' law, or called

C

R

Bayesian reasoning.

The probability of an event A conditional on another event B ie $P(A|B)$ is generally different from probability of B conditional on A ie $P(B|A)$.

□ There is a definite relationship between the two, $P(A|B)$ and $P(B|A)$, and Bayes' theorem is the statement of that relationship.

□ Bayes theorem is a way to calculate $P(A|B)$ from a knowledge of $P(B|A)$.

□ Bayes' Theorem is a result that allows new information to be used to update the conditional probability of an event.

[Continued in next slide]

47

fo

.in

rs

de

AI - Statistical Reasoning

ea

[Continued from previous slide]

yr

.m

w ■ Bayes' Theorem

w

,w

ty

or

Let S be a sample space.

ab

kr

Let $A_1, A_2, \dots, A_n$ be a set of mutually exclusive events from S .

ha

C

C

Let B be any event from the same S , such that $P(B) > 0$.

R

Then Bayes' Theorem describes following two probabilities :

$P(A_k \cap B)$

$P(A_k|B)$ = and

$P(A_1 \cap B) + P(A_2 \cap B) + \dots + P(A_n \cap B)$

by invoking the fact $P(A_k \cap B) = P(A_k).P(B|A_k)$ the probability

$P(A_k).P(B|A_k)$

$P(A_k|B) =$

$P(A_1).P(B|A_1) + P(A_2).P(B|A_2) + \dots + P(A_n).P(B|A_n)$

Applying Bayes' Theorem :

Bayes' theorem is applied while following conditions exist.

‡ the sample space S is partitioned into a set of mutually exclusive events $\{A_1, A_2, \dots, A_n\}$.

‡ within S , there exists an event B , for which $P(B) > 0$.

‡ the goal is to compute a conditional probability of the form :

$P(A_k|B)$.

‡ you know at least one of the two sets of probabilities

described below

◇ $P(A_k \cap B)$ for each A_k

◇ $P(A_k)$ and $P(B|A_k)$ for each A_k

The Bayes' theorem is best understood through an example below.

48

fo

.in

rs

de

AI - Statistical Reasoning

ea

Example 1: Applying Bayes' Theorem

yr

.m

w

w

Problem : Marie's marriage is tomorrow.

,w

ty

or

□ in recent years, each year it has rained only 5 days.

ab

kr

□ the weatherman has predicted rain for tomorrow.

ha

C

C

□ when it actually rains, the weatherman correctly forecasts rain 90% of

R

the time.

□ when it doesn't rain, the weatherman incorrectly forecasts rain 10% of the time.

The question : What is the probability that it will rain on the day of Marie's wedding?

Solution : The sample space is defined by two mutually exclusive events – "it rains" or "it does not rain". Additionally, a third event occurs when the "weatherman predicts rain".

The events and probabilities are stated below.

◇ Event A1 : rains on Marie's wedding.

◇ Event A2 : does not rain on Marie's wedding

◇ Event B : weatherman predicts rain.

◇ $P(A1) = 5/365 = 0.0136985$ [Rains 5 days in a year.]

◇ $P(A2) = 360/365 = 0.9863014$ [Does not rain 360 days in a year.]

◇ $P(B|A1) = 0.9$ [When it rains, the weatherman predicts rain 90% time.]

◇ $P(B|A2) = 0.1$ [When it does not rain, weatherman predicts rain 10% time.]

We want to know $P(A1|B)$, the probability that it will rain on the day of Marie's wedding, given a forecast for rain by the weatherman.

The answer can be determined from Bayes' theorem, shown below.

$$P(A1).P(B|A1) (0.014)(0.9)$$

$$P(A1|B) = =$$

$$P(A1).P(B|A1)+P(A2).P(B|A2) [(0.014)(0.9)+(0.986)(0.1)]$$

$$= 0.111$$

So, despite the weatherman's prediction, there is a good chance that Marie will not get rain on at her wedding.

Thus Bayes theorem is used to calculate conditional probabilities.

49

fo

.in

rs

de

AI - Statistical Reasoning

ea

Example 2: Applying Bayes' Theorem

yr

.m

w ‡ Let S be a sample space.

w

,w

ty

‡ Let E1 and E2 be two mutually exclusive events forming a partition

or

ab

of the sample space S

kr

ha

C

‡ Let E be any event of the sample space such that $P(E) \neq 0$.

C

R

Recall from Conditional Probability

The notation $P(E1 | E)$ means "the probability of the event E1 given that E has already occurred".

‡ The sample space S is described as "the integers 1 to 15" and is

partitioned into :

E_1 = "the integers 1 to 8" and

E_2 = "the integers 9 to 15".

‡ If E is the event "even number" then the probabilities for the situation described by Baye's Theorem can be calculated in two ways, both giving same results.

$$P(E_1 \cap E) = 4 / 15$$

$$P(E_1|E) = 4/8$$

$$P(E_1 \cap E) + P(E_2 \cap E) = (4 / 15) + (3 / 15)$$

$$P(E_1).P(E|E_1) = 8 / 15 \times 4 / 8$$

$$P(E_1|E) = 4/8$$

$$P(E_1).P(E|E_1) + P(E_2).P(E|E_2) = (8/15 \times 4/8) + (7/15 \times 3/15)$$

Thus Bayes' Theorem can be extended for Mutually Exclusive Events as :

$$P(E_i \cap E)$$

$$P(E_i | E) =$$

$$P(E_1 \cap E) + P(E_2 \cap E) + \dots + P(E_k \cap E)$$

50

fo

.in

rs

de

AI - Statistical Reasoning

ea

Example 3 : Clinic Trial

yr

.m

w In a clinic, the probability of the patients having HIV virus is 0.15.

w

,w

ty

A blood test done on patients :

or

ab

If patient has virus, then the test is +ve with probability 0.95.

kr

ha

If the patient does not have the virus, then the test is +ve with probability 0.02.

C

C

R

Assign labels to events : H = patient has virus; P = test +ve

Given : $P(H) = 0.15$; $P(P|H) = 0.95$; $P(P|\neg H) = 0.02$

Find :

If the test is +ve what are the probabilities that the patient

i) has the virus ie $P(H|P)$; ii) does not have virus ie $P(\neg H|P)$;

If the test is -ve what are the probabilities that the patient

iii) has the virus ie $P(H|\neg P)$; iv) does not have virus ie $P(\neg H|\neg P)$;

Calculations :

i) For $P(H|P)$ we can write down Bayes Theorem as

$$P(H|P) = [P(P|H) P(H)] / P(P)$$

We know $P(P|H)$ and $P(H)$ but not $P(P)$ which is probability of a +ve result.

There are two cases, that a patient could have a +ve result, stated below :

1. Patient has virus and gets a +ve result : $H \cap P$

2. Patient does not have virus and gets a +ve result: $\neg H \cap P$

Find probabilities for the above two cases and then add

ie $P(P) = P(H \cap P) + P(\neg H \cap P)$.

But from the second axiom of probability we have :

$$P(H \cap P) = P(P|H) P(H) \text{ and } P(\neg H \cap P) = P(P|\neg H) P(\neg H).$$

Therefore putting these we get :

$$P(P) = P(P|H) P(H) + P(P|\neg H) P(\neg H) = 0.95 \times 0.15 + 0.02 \times 0.85 = 0.1595$$

Now substitute this into Bayes Theorem and obtain $P(H|P)$

$$P(P|H) P(H)$$

$$P(H|P) = 0.95 \times 0.15 / 0.1595 = 0.8934$$

$$P(P|H) P(H) + P(P|\neg H) P(\neg H)$$

ii) Next is to work out $P(\neg H|P)$

$$P(\neg H|P) = 1 - P(H|P) = 1 - 0.8934 = 0.1066$$

iii) Next is to work out $P(H|\neg P)$; again we write down Bayes Theorem

$$P(\neg P|H) P(H)$$

$$P(H|\neg P) = \text{here we need } P(\neg P) \text{ which is } 1 - P(P)$$

$$P(\neg P)$$

$$= (0.05 \times 0.15) / (1 - 0.1595) = 0.008923$$

iv) Finally, work out $P(\neg H|\neg P)$

$$\text{It is just } 1 - P(H|\neg P) = 1 - 0.008923 = 0.99107$$

51

fo

.in

rs

de

AI - Statistical Reasoning

ea

3.2 Certainty Factors in Rule-Based Systems

yr

.m

w

w

The certainty-factor model was one of the most popular model for

,w

ty

or

the representation and manipulation of uncertain knowledge in the

ab

kr

early (1980s) Rule-based expert systems.

ha

C

C

R

The model was criticized by researchers in artificial intelligence and statistics being ad-hoc-in nature. Researchers and developers have stopped using the model.

Its place has been taken by more expressive formalisms of Bayesian belief networks for the representation and manipulation of uncertain knowledge.

The manipulation of uncertain knowledge in the Rule-based expert systems is illustrated in the next three slide before moving to Bayesian Networks.

52

fo

.in

rs

de

AI - Statistical Reasoning

ea

- Rule Based Systems

yr

.m

w

w

Rule based systems have been discussed in previous lectures.

,w

ty

or

Here it is recalled to explain uncertainty.

ab

kr

■ A rule is an expression of the form

ha

"if A then B"

C

C

where A is an assertion and B can be either an action or

R

another assertion.

Example : Trouble shooting of water pumps

1. If pump failure then the pressure is low

2. If pump failure then check oil level

3. If power failure then pump failure

■ Rule based system consists of a library of such rules.

■ Rules reflect essential relationships within the domain.

■ Rules reflect ways to reason about the domain.

■ Rules draw conclusions and points to actions, when specific information about the domain comes in. This is called inference.

■ The inference is a kind of chain reaction like :

If there is a power failure then (see rules 1, 2, 3 mentioned above)

Rule 3 states that there is a pump failure, and

Rule 1 tells that the pressure is low, and

Rule 2 gives a (useless) recommendation to check the oil level.

■ It is very difficult to control such a mixture of inference back and forth in the same session and resolve such uncertainties.

How to deal such uncertainties ?

[continued in the next slide]

53

fo

.in

rs

de

AI - Statistical Reasoning

ea

[continued from the previous slide]

yr

.m

w

w

How to deal uncertainties in rule based system?

,w

ty

or

ab

A problem with rule-based systems is that often the connections

kr

ha

reflected by the rules are not absolutely certain (i.e. deterministic),

C

C

R

and the gathered information is often subject to uncertainty.

In such cases, a certainty measure is added to the premises as well as the conclusions in the rules of the system.

A rule then provides a function that describes : how much a change in the certainty of the premise will change the certainty of the conclusion.

In its simplest form, this looks like :

If A (with certainty x) then B (with certainty $f(x)$)

This is a new rule, say rule 4, added to earlier three rules.

[continued in the next slide]

54

fo

.in

rs

de

AI - Statistical Reasoning

ea

[continued from the previous slide]

yr

.m

w

w

■ There are many schemes for treating uncertainty in rule based systems.

,w

ty

or

ab

The most common are :

kr

ha

C

‡ Adding certainty factors.

C

R

‡ Adoptions of Dempster-Shafer belief functions.

‡ Inclusion of fuzzy logic.

In these schemes, uncertainty is treated locally, means action is connected directly to incoming rules and uncertainty of their elements.

Example : In addition to rule 4 , in previous slide, we have the rule

If C (with certainty x) then B (with certainty $g(x)$)

Now If the information is that A holds with certainty a and C holds with certainty c , Then what is the certainty of B ?

Note : Depending on the scheme, there are different algebras for such a combination of uncertainty. But all these algebras in many cases come to incorrect conclusions because combination of uncertainty is not a local phenomenon, but it is strongly dependent on the entire situation (in principle a global matter).

55

fo

.in

rs

de

AI - Statistical Reasoning

ea

3.3 Bayesian Networks and Certainty Factors

yr

.m

w

w

A Bayesian network (or a belief network) is a probabilistic graphical model
,w
ty
or

that represents a set of variables and their probabilistic independencies.
ab
kr

For example, a Bayesian network could represent the probabilistic
ha
C
C

relationships between diseases and symptoms. Given symptoms, the
R

network can be used to compute the probabilities of the presence of various
diseases.

Bayesian Networks are also called : Bayes nets, Bayesian Belief Networks
(BBNs) or simply Belief Networks. Causal Probabilistic Networks (CPNs).

A Bayesian network consists of :

- a set of nodes and a set of directed edges between nodes.
- the edges reflect cause-effect relations within the domain.
- The effects are not completely deterministic (e.g. disease -> symptom).
- the strength of an effect is modeled as a probability.

56

fo

.in

rs

de

AI - Statistical Reasoning

ea

- Bayesian Networks

yr

.m

w

w

We have applied Bayesian probability theory, in earlier three examples

,w

ty

or

(example 1, 2, and 3) , to relate two or more events. But this can be

ab

kr

used to relate many events by tying them together in a network.

ha

C

C

R

Consider the previous example 3 - Clinic trial

The trial says, the probability of the patients having HIV virus is 0.15.

A blood test done on patients :

If patient has virus, the test is +ve with probability 0.95.

If the patient does not have the virus, the test is +ve with probability 0.02.

This means given : $P(H) = 0.15$; $P(P|H) = 0.95$; $P(P|\neg H) = 0.02$

Imagine, the patient is given a second test independently of the first; means the second test is done at a later date by a different person using different equipment. So, the error on the first test does not affect the probability of an error on the second test.

In other words the two tests are independent. This is depicted using the diagram below :

A simple example of a Bayesian Network.

Event H is the cause of the two events P1 and P2.

P1 The arrows represent the fact that H is driving

H

P1 and P2.

P2

The network contained 3 nodes.

If both P1 and P2 are +ve
then find the probability that patient has the virus ?

In other words asked to find $P(H|P1 \wedge P2)$.

How to find ?

[continued in the next slide]

57

fo

.in

rs

de

AI - Statistical Reasoning

ea

■ Bayes Theorem (Ref. previous previous slide example 3)

yr

.m

w

w

$P(P1 \cap P2 | H) \cdot P(H)$

,w

ty

$P(H | P1 \cap P2) =$

or

$P(P1 \cap P2)$

ab

kr

ha

Here there are two quantities which we do not know.

C

C

The first is $P(P1 \cap P2 | H)$ and the second is $P(P1 \cap P2)$

R

‡ Find $P(P1 \cap P2 | H)$

Since the two tests are independent, so

$P(P1 \cap P2 | H) = (P1 | H)P(P2 | H)$

‡ Find $P(P1 \cap P2)$

As worked before for $P(P)$ which is the probability of a +ve result, here again break this into two separate cases:

◇ patient has virus and both tests are +ve

◇ patient not having virus and both tests are +ve

‡ As before use the second axiom of probability

$P(P1 \cap P2) = P(P1 \cap P2 | H) P(H) + P(P1 \cap P2 | \neg H) P(\neg H)$

‡ Because the two tests are independent given H we can write :

$P(P1 \cap P2) = P(P1 | H) P(P2 | H) P(H) + P(P1 | \neg H) P(P2 | \neg H) P(\neg H)$

$= 0.95 \times 0.95 \times 0.15 + 0.02 \times 0.02 \times 0.85$

$= 0.135715$

‡ Substitute this into Bayes Theorem above and obtain

$P(P1 \cap P2 | H) \cdot P(H)$

$P(H | P1 \cap P2) =$

$P(P1 \cap P2)$

$= (0.95 \times 0.95 \times 0.15) / 0.135715 = 0.99749$

‡ Note : The results while two independent HIV tests performed

□ Previously we calculated the probability, that the patient had HIV given one +ve test, as 0.8934.

□ Later second HIV test was performed. After two +ve tests, we see that the probability has gone up to 0.99749.

□ So after two +ve tests it is more certain that the patient does have the HIV virus.

The next slide : a case where one tests is +ve and other is -ve.

58

fo

.in

rs

de

AI - Statistical Reasoning

ea

Case where one tests is +ve and other is -ve.

yr

.m

w This means, an error on one of the tests but we don't know which one;

w

,w

ty

it may be any one.

or

ab

kr

The issue is - whether the patient has HIV virus or not ?

ha

C

C

R

‡ We need to calculate $P(H | P1 \cap \neg P2)$.

Following same steps for the case of two +ve tests,

write Bayes Theorem

$$P(P1 \cap \neg P2 | H) P(H)$$

$$P(H | P1 \cap \neg P2) =$$

$$P(P1 \cap \neg P2)$$

‡ Now work out $P(P1 \cap \neg P2 | H)$ and $P(P1 \cap \neg P2)$ using the fact that

$P1$ and $P2$ are independent given H ,

$$P(P1 \cap \neg P2 | H) = P(P1 | H) P(\neg P2 | H) \text{ and}$$

$$P(P1 \cap \neg P2) = P(P1 \cap \neg P2 | H) P(H) + P(P1 \cap \neg P2 | \neg H) P(\neg H)$$

$$= P(P1 | H) P(\neg P2 | H) P(H) + P(P1 | \neg H) P(\neg P2 | \neg H) P(\neg H)$$

$$= 0.95 \times 0.05 \times 0.15 + 0.02 \times 0.98 \times 0.85$$

$$= 0.023785$$

‡ Substitute these values into Bayes Theorem, we obtain

$$0.95 \times 0.05 \times 0.15$$

$$P(H|P1 \wedge \neg P2) = 0.299$$

$$0.023785$$

‡ Note :

□ Belief in H, the event that the patient has virus, has increased.

□ Prior belief was 0.15 but it has now gone up to 0.299.

□ This appears strange because we have been given two contradictory pieces of data. But looking closely we see that probability of an error in each case is not equal.

‡ The probability of a +ve test when patient is actually -ve is 0.02.

The probability of a -ve test when patient is actually +ve is 0.05.

Therefore we are more inclined to believe an error on the second test and this slightly increases our belief that the patient is +ve.

59

fo

.in

rs

de

AI - Statistical Reasoning

ea

• More Complicated Bayesian Networks

yr

.m

w

w

,w

The previous network was simple contained three nodes. Let us look at a

ty

or

slightly more complicated one in the context of heart disease.

ab

kr

ha

C

Given the following facts about heart disease.

C

R

■ Either smoking or bad diet or both can make heart disease more likely.

■ Heart disease can produce either or both of the following two symptoms:

‡ high blood pressure

‡ an abnormal electrocardiogram

■ Here smoking and bad diet are regarded as causes of heart disease.

The heart disease in turn is a cause of high blood pressure and an abnormal electrocardiogram.

[continued in the next slide]

60

fo

.in

rs

de

AI - Statistical Reasoning

ea

[continued from the previous slide]

yr

.m

w

w

■ An appropriate network for heart disease is represented as

,w

ty

or

The symbols define :

ab

kr

S B

ha

S = smoking,

C

C

R

D = bad diet,

H H = heart disease,

B = high blood pressure,

D E E = abnormal electrocardiogram

Here H has two causes S and D.

Find probability of H, given each of the four possible combinations of S and D.

A medical survey gives us the following data :

$$P(S) = 0.3 \quad P(D) = 0.4$$

$$P(H|S \cap D) = 0.8$$

$$P(H|\neg S \cap D) = 0.5$$

$$P(H|S \cap \neg D) = 0.4$$

$$P(H|\neg S \cap \neg D) = 0.1$$

$$P(B|H) = 0.7 \quad P(B|\neg H) = 0.1$$

$$P(E|H) = 0.8 \quad P(E|\neg H) = 0.1$$

Given these information, an answer to the question concerning this network :

what is the probability of heart disease ?

[Note : The interested students may try to find the answer.]

61

fo

.in

rs

de

AI - Statistical Reasoning

ea

3.4 Dempster – Shafer Theory (DST)

yr

.m

w

w

DST is a mathematical theory of evidence based on belief functions and

,w

ty

or

plausible reasoning. It is used to combine separate pieces of information

ab

kr

(evidence) to calculate the probability of an event.

ha

C

C

R

DST offers an alternative to traditional probabilistic theory for the

mathematical representation of uncertainty.

DST can be regarded as, a more general approach to represent uncertainty than the Bayesian approach.

Bayesian methods are sometimes inappropriate

Example :

Let A represent the proposition "Moore is attractive".

Then the axioms of probability insist that $P(A) + P(\neg A) = 1$.

Now suppose that Andrew does not even know who "Moore" is, then

‡ We cannot say that Andrew believes the proposition if he has no idea what it means.

‡ Also, it is not fair to say that he disbelieves the proposition.

‡ It would therefore be meaningful to denote Andrew's belief B of B(A) and B($\neg A$) as both being 0.

‡ Certainty factors do not allow this.

62

fo

.in

rs

de

AI - Statistical Reasoning

ea

- Dempster-Shafer Model

yr

.m

w

w

The idea is to allocate a number between 0 and 1 to indicate a

,w

ty

or

degree of belief on a proposal as in the probability framework.

ab

kr

However, it is not considered a probability but a belief mass.

ha

C

C

The distribution of masses is called basic belief assignment.

R

In other words, in this formalism a degree of belief (referred as mass) is represented as a belief function rather than a Bayesian probability distribution.

Example: Belief assignment (continued from previous slide)

Suppose a system has five members, say five independent states, and exactly one of which is actual. If the original set is called S , $|S| = 5$, then the set of all subsets (the power set) is called $2S$.

□ If each possible subset is represented as a binary vector (describing any member is present or not by writing 1 or 0), then 2 subsets are possible,

ranging from the empty subset $(0, 0, 0, 0, 0)$ to the "everything" subset $(1, 1, 1, 1, 1)$.

□ The "empty" subset represents a "contradiction", which is not true in any state, and is thus assigned a mass of one;

□ The remaining masses are normalized so that their total is 1.

□ The "everything" subset is labeled as "unknown"; it represents the state where all elements are present one, in the sense that you cannot tell which is actual.

S

Note : Given a set S , the power set of S , written 2^S , is the set of all subsets of S , including the empty set and S .

63

fo

.in

rs

de

AI - Statistical Reasoning

ea

• Belief and Plausibility

yr

.m

w

w

Shafer's framework allows for belief about propositions to be represented

,w

ty

or

as intervals, bounded by two values, belief (or support) and plausibility:

ab

kr

belief $\leq$ plausibility

ha

C

C

R

Belief in a hypothesis is constituted by the sum of the masses of all sets enclosed by it (i.e. the sum of the masses of all subsets of the hypothesis). It is the amount of belief that directly supports a given hypothesis at least in part, forming a lower bound.

Plausibility is 1 minus the sum of the masses of all sets whose intersection with the hypothesis is empty. It is an upper bound on the possibility that the hypothesis could possibly happen, up to that value, because there is only so much evidence that contradicts that hypothesis.

Example :

A proposition say "the cat in the box is dead."

Suppose we have belief of 0.5 and plausibility of 0.8 for the proposition.

[continued in the next slide]

64

fo

.in

rs

de

AI - Statistical Reasoning

ea

[continued in the previous slide]

yr

.m

w

w

Example :

,w

A proposition say "the cat in the box is dead."

ty

or

ab

Suppose we have belief of 0.5 and plausibility of 0.8 for the proposition.

kr

ha

C

C

□ Evidence to state strongly, that proposition is true with confidence 0.5.

R

□ Evidence contrary to hypothesis ("the cat is alive") has confidence 0.2.

□ Remaining mass of 0.3 (the gap between the 0.5 supporting evidence and the 0.2 contrary evidence) is "indeterminate," meaning that the cat could either be dead or alive. This interval represents the level of uncertainty based on the evidence in the system.

Hypothesis Mass belief plausibility

Null (neither alive nor dead) 0 0 0

Alive 0.2 0.2 0.5

Dead 0.5 0.5 0.8

Either (alive or dead) 0.3 1.0 1.0

□ Null hypothesis is set to zero by definition, corresponds to "no solution".

□ Orthogonal hypotheses "Alive" and "Dead" have probabilities of 0.2 and 0.5, respectively. This could correspond to "Live/Dead Cat Detector" signals, which have respective reliabilities of 0.2 and 0.5.

□ All-encompassing "Either" hypothesis (simply acknowledges there is a cat in the box) picks up the slack so that the sum of the masses is 1.

□ Belief for the "Alive" and "Dead" hypotheses matches their corresponding masses because they have no subsets;

□ Belief for "Either" consists of the sum of all three masses (Either, Alive, and Dead) because "Alive" and "Dead" are each subsets of "Either".

□ "Alive" plausibility is $1 - m(\text{Death})$ and "Dead" plausibility is $1 - m(\text{Alive})$.

□ "Either" plausibility sums $m(\text{Alive}) + m(\text{Dead}) + m(\text{Either})$.

□ Universal hypothesis ("Either") will always have 100% belief and plausibility; it acts as a checksum of sorts.

65

fo

.in

rs

de

AI - Statistical Reasoning

ea

• Dempster-Shafer Calculus

yr

.m

w

w

,w

In the previous slides, two specific examples of Belief and plausibility have

ty

or

been stated. It would now be easy to understand their generalization.

ab

kr

ha

C

The Dempster-Shafer (DS) Theory, requires a Universe of Discourse U

C

R

(or Frame of Judgment) consisting of mutually exclusive alternatives, corresponding to an attribute value domain. For instance, in satellite image classification the set U may consist of all possible classes of interest.

Each subset $S \subseteq U$ is assigned a basic probability $m(S)$, a belief $Bel(S)$, and a plausibility $Pls(S)$ so that

$m(S), Bel(S), Pls(S) \in [0, 1]$ and $Pls(S) \geq Bel(S)$ where

- m represents the strength of an evidence, is the basic probability; e.g., a group of pixels belong to certain class, may be assigned value m .
- $Bel(S)$ summarizes all the reasons to believe S .
- $Pls(S)$ expresses how much one should believe in S if all currently unknown facts were to support S .

The true belief in S is somewhere in the belief interval $[Bel(S), Pls(S)]$.

The basic probability assignment m is defined as function

U

$m : 2^U \rightarrow [0,1]$, where $m(\emptyset) = 0$ and sum of m over all subsets of

U is 1 (i.e., $\sum_{S \subseteq U} m(S) = 1$).

For a given basic probability assignment m , the belief Bel of a subset A of U is the sum of $m(B)$ for all subsets B of A , and the plausibility Pls of a subset A of U is $Pls(A) = 1 - Bel(A')$ (5) where A' is complement of A in U .

[continued in the next slide]

66

fo

.in

rs

de

AI - Statistical Reasoning

ea

[continued in the next slide]

yr

.m

w Summarize :

w

,w

ty

The confidence interval is that interval of probabilities within which

or

ab

kr

the true probability lies with a certain confidence based on the

ha

C

belief "B" and plausibility "PL" provided by some evidence "E" for a

C

R

proposition "P".

The belief brings together all the evidence that would lead us to

believe in the proposition P with some certainty.

The plausibility brings together the evidence that is compatible with

the proposition P and is not inconsistent with it.

If " Ω " is the set of possible outcomes, then a mass probability "M"

Ω

is defined for each member of the set 2 and takes values in

Ω

the range $[0,1]$. The Null set, " ϕ ", is also a member of 2 .

Example

If Ω is the set { Flu (F), Cold (C), Pneumonia (P) }

Ω

Then 2 is the set $\{\phi, \{F\}, \{C\}, \{P\}, \{F, C\}, \{F, P\}, \{C, P\}, \{F, C, P\}\}$

Confidence interval is then defined as $[B(E), PL(E)]$ where

$B(E) = \sum A M$, where $A \subseteq E$ i.e., all evidence that makes us believe in the correctness of P, and

$PL(E) = 1 - B(\neg E) = \sum \neg A M$, where $\neg A \subseteq \neg E$ i.e., all the evidence that contradicts P.

67

fo

.in

rs

de

AI - Statistical Reasoning

ea

- Combining Beliefs

yr

.m

w

w

The Dempster-Shafer calculus combines the available evidences

,w

ty

or

resulting in a belief and a plausibility in the combined evidence that

ab

kr

represents a consensus on the correspondence. The model maximizes

ha

C

C

the belief in the combined evidences.

R

The rule of combination states that two basic probability assignments

M1 and M2 are combined into a third basic probability assignment

by the normalized orthogonal sum $m1 \oplus m2$ stated below.

Suppose M1 and M2 are two belief functions.

Let X be the set of subsets of Ω to which M1 assigns a nonzero

value and let Y be a similar set for M2 ,

then a new belief function M3 from the combination of beliefs in M1

and M2 is obtained as

$$\sum_{x \cap Y = Z} M1(X) M2(Y)$$

$$M3(Z) =$$

$$1-K$$

where $\sum_{x \cap Y = \phi} M1(X) M2(Y)$, for $Z = \phi$

M3(ϕ) is defined to be 0 so that the orthogonal sum remains

a basic probability assignment.

68

fo

.in

rs

de

AI - Statistical Reasoning

ea

3.5 Fuzzy Logic

yr

.m

w

w

We have discussed only binary valued logic and classical set theory like :

,w

ty

or

ab

A person belongs to a set of all human beings, and if given a

kr

ha

specific subset, say all males, then one can say whether or not

C

C

R

the particular person belongs to this set.

This is ok since it is the way human reason. e.g.,

IF person is male AND a parent THEN person is a father.

The rules are formed using operators.

Here, it is intersection operator "AND" which manipulates the sets.

However, not everything can be described using binary valued sets.

□ The grouping of persons into "male" or "female" is easy,

but as "tall" or "not tall" is problematic.

□ A set of "tall" people is difficult to define, because there is no distinct cut-off point at which tall begins.

Fuzzy logic was suggested by Zadeh as a method for mimicking the ability of human reasoning using a small number of rules and still producing a smooth output via a process of interpolation.

69

fo

.in

rs

de

AI - Statistical Reasoning

ea

- Description of Fuzzy Logic

yr

.m

w

w

With fuzzy logic an element could partially belong to a set represented by

,w

ty

or

the set membership. Example, a person of height 1.79 m would belong

ab

kr

to both tall and not tall sets with a particular degree of membership.

ha

C

C

R

Difference between binary logic and fuzzy logic

Grade of thruth Grade of thruth

Not tall Tall Not tall Tall

1 1

0 0

1.8 M height x 1.8 M height x

Binary valued logic {0, 1} Fuzzy logic [0, 1]

A fuzzy logic system is one that has at least one system component that uses fuzzy logic for its internal knowledge representation.

Fuzzy system communicate information using fuzzy sets.

Fuzzy logic is used purely for internal knowledge representation and externally it can be considered as any other system component.

70

fo

.in

rs

de

AI - Statistical Reasoning

ea

- Fuzzy Membership

yr

.m

w

w

,w

Example : Five tumblers

ty

or

ab

□ Consider two sets: F and E.

kr

ha

C

□ F is set of all tumblers belong to the class full, and

C

R

□ E is set of all tumblers belong to the class empty.

Definition of the set F and E

Tumblers

Grade of membership to set F 100% 75% 50% 25% 0%

Grade of membership to set E 0% 25% 50% 75% 100%

Graphical representation of set F and E

Grade of

Membership

Set F

1.0

Set E

0.5

Tumblers

The sets F and E have some elements, having partial membership.

Such kind of non-crisp sets are called fuzzy sets.

The set "all tumblers" here is the basis of the fuzzy sets F and E, is called the base set.

71

fo

.in

rs

de

ea

AI – Reasoning - References

yr

.m

4. References : Textbooks

w

w

,w

ty

1. "Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill

or

companies Inc., Chapter 7-8, page 195-248.

ab

kr

ha

C

2. "Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig,

C

R

(2002), Prentice Hall, Chapter 7-10, page 195-369.

3. "Computational Intelligence: A Logical Approach", by David Poole, Alan Mackworth, and Randy Goebel, (1998), Oxford University Press, Chapter 2-3, page 23-104.

4. "Artificial Intelligence: Structures and Strategies for Complex Problem Solving", by George F. Luger, (2002), Addison-Wesley, Chapter 9, page 333-380.

5. "AI: A New Synthesis", by Nils J. Nilsson, (1998), Morgan Kaufmann Inc., Chapter 13-20, Page 217-358.

6. "Artificial Intelligence: Theory and Practice", by Thomas Dean, (1994), Addison-Wesley, Chapter 2-8, Page 23-355.

7. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

72

fo

.in

rs

de

ea

yr

Game Playing : AI Course Lecture 29 – 30, notes, slides

.m

w

w

www.myreaders.info/ , RC Chakraborty, e-mail rcchak@gmail.com , June 01, 2010

,w

ty

www.myreaders.info/html/artificial_intelligence.html

or

ab

www.myreaders.info

kr

ha

C

C

R

Game Playing

Artificial Intelligence

Game Playing, topics : Overview, definition of game, game theory, relevance of game theory and game plying, Glossary of terms – game, player, strategy, zero-Sum game, constant-sum game, nonzero-sum game, Prisoner's dilemma, N-Person game, utility function, mixed strategies, expected payoff, Mini-Max theorem, saddle point; taxonomy of games. Mini-Max search procedure : formalizing game - general and a Tic-Tac-Toe game, evaluation function ; MINI-MAX technique : game trees, Mini-Max algorithm. Game Playing with Mini-Max : example of Tic-Tac-Toe - moves, static evaluation, back-up the evaluations and evaluation obtained. Alpha-Beta Pruning : Alpha-cutoff, Beta-cutoff.

fo

.in

rs

de

ea

Game Playing

yr

.m

w

w

,w

Artificial Intelligence

ty

or

ab

kr

ha

Topics

C

C

R

(Lectures 29, 30, 2 hours) Slides

1. Overview 03-18

Definition of Game, Game theory, Relevance of Game theory and Game playing, Glossary of terms – Game, Player, Strategy, Zero-Sum game, Constant-Sum game, Nonzero-Sum game, Prisoner's dilemma, N-Person Game, Utility function, Mixed strategies, Expected payoff, Mini-Max theorem, Saddle point; Taxonomy of games.

2. Mini-Max Search Procedure 19-25

Formalizing game : General and a Tic-Tac-Toe game, Evaluation function ; MINI-MAX Technique : Game Trees, Mini-Max algorithm.

3. Game Playing with Mini-Max 26-32

Example : Tic-Tac-Toe - Moves, Static evaluation, Back-up the evaluations, Evaluation obtained.

4. Alpha-Beta Pruning 33-35

Alpha-cutoff, Beta-cutoff

5. References 36

02

fo

.in

rs

de

ea

yr

Game Playing

.m

w

w

,w

ty

What is Game ?

or

ab

kr

ha

- The term Game means a sort of conflict in which n individuals or

C

C

R

groups (known as players) participate.

- Game theory denotes games of strategy.
- John von Neumann is acknowledged as father of game theory. Neumann defined Game theory in 1928 and 1937 and established the mathematical framework for all subsequent theoretical developments.
- Game theory allows decision-makers (players) to cope with other decision-makers (players) who have different purposes in mind. In other words, players determine their own strategies in terms of the strategies and goals of their opponent.
- Games are integral attribute of human beings.
Games engage the intellectual faculties of humans.
- If computers are to mimic people they should be able to play games.

03

fo

.in

rs

de

Overview

ea

1. Over View

yr

.m

w

w

Game playing, besides the topic of attraction to the people, has close

,w

ty

or

relation to "intelligence", and its well-defined states and rules.

ab

kr

ha

The most commonly used AI technique in game is "Search".

C

C

R

A "Two-person zero-sum game" is most studied game where the two players have exactly opposite goals. Besides there are "Perfect information games" (such as chess and Go) and "Imperfect information games" (such as bridge and games where a dice is used).

Given sufficient time and space, usually an optimum solution can be obtained for the former by exhaustive search, though not for the latter. However, for many interesting games, such a solution is usually too inefficient to be practically used.

Applications of game theory are wide-ranging. Von Neumann and Morgenstern indicated the utility of game theory by linking with economic behavior.

□ Economic models : For markets of various commodities with differing numbers of buyers and sellers, fluctuating values of supply and demand, seasonal and cyclical variations, analysis of conflicts of interest in maximizing profits and promoting the widest distribution of goods and services.

□ Social sciences : The n-person game theory has interesting uses in studying the distribution of power in legislative procedures, problems of majority rule, individual and group decision making.

□ Epidemiologists : Make use of game theory, with respect to immunization procedures and methods of testing a vaccine or other medication.

□ Military strategists : Turn to game theory to study conflicts of interest resolved through "battles" where the outcome or payoff of a war game is either victory or defeat.

04

fo

.in

rs

de

Overview

ea

1.1 Definition of Game

yr

.m

w

w

- A game has at least two players.

,w

ty

or

Solitaire is not considered a game by game theory.

ab

kr

The term 'solitaire' is used for single-player games of concentration.

ha

C

C

- An instance of a game begins with a player choosing from a set of specified

R

(game rules) alternatives. This choice is called a move.

- After first move, the new situation determines which player to make next move and alternatives available to that player.

□ In many board games, the next move is by other player.

□ In many multi-player card games, the player making next move depends on who dealt, who took last trick, won last hand, etc.

- The moves made by a player may or may not be known to other players.

Games in which all moves of all players are known to everyone are called games of perfect information.

□ Most board games are games of perfect information.

□ Most card games are not games of perfect information.

- Every instance of the game must end.

- When an instance of a game ends, each player receives a payoff.

A payoff is a value associated with each player's final situation.

A zero-sum game is one in which elements of payoff matrix sum to zero.

In a typical zero-sum game :

□ win = 1 point,

□ draw = 0 points, and

□ loss = -1 points.

05

fo

.in

rs

de

Overview

ea

1.2 Game Theory

yr

.m

w

w

Game theory does not prescribe a way or say how to play a game.

,w

ty

or

Game theory is a set of ideas and techniques for analyzing conflict

ab

kr

situations between two or more parties. The outcomes are determined

ha

C

C

by their decisions.

R

General game theorem : In every two player, zero sum, non-random, perfect knowledge game, there exists a perfect strategy guaranteed to at least result in a tie game.

The frequently used terms :

- The term "game" means a sort of conflict in which n individuals or groups (known as players) participate.
- A list of "rules" stipulates the conditions under which the game begins.
- A game is said to have "perfect information" if all moves are known to each of the players involved.
- A "strategy" is a list of the optimal choices for each player at every stage of a given game.
- A "move" is the way in which game progresses from one stage to another, beginning with an initial state of the game to the final state.
- The total number of moves constitute the entirety of the game.
- The payoff or outcome, refers to what happens at the end of a game.
- Minimax - The least good of all good outcomes.
- Maximin - The least bad of all bad outcomes.

The primary game theory is the Mini-Max Theorem. This theorem says :

"If a Minimax of one player corresponds to a Maximin of the other player, then that outcome is the best both players can hope for."

06

fo

.in

rs

de

Overview

ea

1.3 Relevance Game Theory and Game Plying

yr

.m

w

w

How relevant the Game theory is to Mathematics, Computer science

,w

ty

or

and Economics is shown in the Fig below.

ab

kr

ha

C

C

R

Game Playing

□ Games can be Deterministic or non-deterministic.

□ Games can have perfect information or imperfect information.

Games Deterministic Non- Deterministic

Perfect Chess, Checkers, Go, Backgammon,

information Othello, Tic-tac-toe Monopoly

Imperfect Navigating Bridge, Poker,

information a maze Scrabble

07

fo

.in

rs

de

Overview

ea

1.4 Glossary of terms in the context of Game Theory

yr

.m

w

w

■ Game

,w

ty

or

Denotes games of strategy. It allows decision-makers (players)

ab

kr

to cope with other decision-makers (players) who have different

ha

C

C

purposes in mind. In other words, players determine their own

R

strategies in terms of the strategies and goals of their opponent.

■ Player

Could be one person, two persons or a group of people who share identical interests with respect to the game.

■ Strategy

A player's strategy in a game is a complete plan of action for whatever situation might arise. It is the complete description of how one will behave under every possible circumstance. You need to analyze the game mathematically and create a table with "outcomes" listed for each strategy.

A two player strategy table

Players Player A Player A Player A etc

Strategies Strategy 1 Strategy 2 Strategy 3

Player B

Strategy 1 Tie A wins B wins ...

Player B

Strategy 2 B wins Tie A wins ...

Player B

Strategy 3 A wins B wins Tie ...

etc

08

fo

.in

rs

de

Overview

ea

■ Zero-Sum Game

yr

.m

w It is the game where the interests of the players are diametrically

w

,w

ty

opposed. Regardless of the outcome of the game, the winnings of the

or

ab

player(s) are exactly balanced by the losses of the other(s).

kr

ha

C

No wealth is created or destroyed.

C

R

There are two types of zero-sum games:

□ Perfect information zero-sum games

□ General zero-sum games

The difference is the amount of information available to the players.

Perfect Information Games :

Here all moves of all players are known to everyone.

e.g., Chess and Go;

General Zero-Sum Games :

Players must choose their strategies simultaneously, neither knowing what the other player is going to do.

e.g., If you play a single game of chess with someone, one person will lose and one person will win. The win (+1) added to the loss (-1) equals zero.

09

fo

.in

rs

de

Overview

ea

■ Constant-Sum Game

yr

.m

w Here the algebraic sum of the outcomes are always constant,

w

,w

ty

though not necessarily zero.

or

ab

It is strategically equivalent to zero-sum games.

kr

ha

C

C

R

■ Nonzero-Sum Game

Here the algebraic sum of the outcomes are not constant. In these games, the sum of the pay offs are not the same for all outcomes.

They are not always completely solvable but provide insights into important areas of inter-dependent choice.

In these games, one player's losses do not always equal another player's gains.

The nonzero-sum games are of two types:

Negative Sum Games (Competitive) : Here nobody really wins, rather, everybody loses. Example - a war or a strike.

Positive Sum Games (Cooperative) : Here all players have one goal that they contribute together. Example - an educational game, building blocks, or a science exhibit.

10

fo

.in

rs

de

Overview

ea

■ Prisoner's Dilemma

yr

.m

w

w

It is a two-person nonzero-sum game. It is a non cooperative game

,w
ty
or

because the players can not communicate their intentions.

ab
kr

Example : The two players are partners in a crime who have been captured by

ha
C
C

the police. Each suspect is placed in a separate cell and offered the

R

opportunity to confess to the crime.

Now set up the payoff matrix. The entries in the matrix are two numbers representing the payoff to the first and second player respectively.

2nd player 2nd player

Players Not Confess Confess

1st player

Not Confess 5 , 5 0 , 10

1st player

Confess 10 , 0 1 , 1

□ If neither suspect confesses, they go free, and split the proceeds of their crime, represented by 5 units of payoff for each suspect.

□ If one prisoner confesses and the other does not, the prisoner who confesses testifies against the other in exchange for going free and gets the entire 10 units of payoff, while the prisoner who did not confess goes to prison and gets nothing.

□ If both prisoners confess, then both are convicted but given a reduced term, represented by 1 unit of payoff : it is better than having just the other prisoner confess, but not so good as going free.

This game represents many important aspects of game theory.

No matter what a suspect believes his partner is going to do, it is always best to confess.

□ If the partner in the other cell is not confessing, it is possible to get 10 instead of 5 as a payoff.

□ If the partner in the other cell is confessing, it is possible to get 1 instead of 0 as a payoff.

□ Thus the pursuit of individually sensible behavior results in each player getting only 1 as a payoff, much less than the 5 which they would get if neither confessed.

This conflict between the pursuit of individual goals and the common good is at the heart of many game theoretic problems.

11

fo

.in

rs

de

Overview

ea

■ N-Person Game

yr

.m

w Involve more than two players.

w

,w

ty

Analysis of such games is more complex than zero-sum games.

or

ab

Conflicts of interest are less obvious.

kr

ha

C

C

R

Here, what is good for player-1 may be bad for player-2 but good for player-3. In such a situation coalitions may form and change a game radically. The obvious questions are :

□ how would the coalition form ?

□ who will form coalitions ?

□ would the weak gang up against the strong ? or

would weak players make an alliance with a strong ?

In either case it could be looked at as two coalitions, and so effectively a two person game.

■ Utility Function

It is quantification of person's preferences respect to certain objects. In any game, utility represents the motivations of players. A utility function for a given player assigns a number for every possible outcome of the game with the property that a higher number implies that the outcome is more preferred.

12

fo

.in

rs

de

Overview

ea

■ Mixed Strategies

yr

.m

w A player's strategy in a game is a complete plan of action for

w

,w

ty

whatever situation might arise. It is a complete algorithm for playing

or

ab

kr

the game, telling a player what to do for every possible situation

ha

C

C

throughout the game.

R

A Pure strategy provides a complete definition of how a player will play a game. In particular, it determines the move a player will make for any situation they could face. A player's strategy set is the set of pure strategies available to that player.

A Mixed strategy is an assignment of a probability to each pure strategy. This allows for a player to randomly select a pure strategy. Since probabilities are continuous, there are infinitely many mixed strategies available to a player, even if their strategy set is finite.

A mixed strategy for a player is a probability distribution on the set of his pure strategies.

Suppose a player has only a finite number, m , of pure strategies, then a mixed strategy reduces to an m -vector, $X = (x_1, \dots, x_m)$, satisfying

m

$$x_i \geq 0, \sum_{i=1}^m x_i = 1$$

i=1

Now denote the set of all mixed strategies for player-1 by X , and the set of all mixed strategies for player-2 by Y .

m

$$X = \{ x = (x_1, \dots, x_m) : x_i \geq 0, \sum_{i=1}^m x_i = 1 \}$$

$i=1$

n

$$Y = \{ y = (y_1, \dots, y_n) : y_i \geq 0, \sum_{i=1}^n y_i = 1 \}$$

$i=1$

[continued in the next slide]

13

fo

.in

rs

de

Overview

ea

[continuing from previous slide- mixed strategy]

yr

.m

w Expected Payoff

w

,w

ty

Suppose that player1 and player2 are playing the matrix game A .

or

ab

If player1 chooses the mixed strategy x_i , and

k

ha

C

player2 chooses the mixed strategy y_j ,

C

R

then the expected payoff a_{ij} will be computed by

$$y_1 \dots y_n \quad y_1 \dots y_n$$

$$x_1 a_{11} \dots a_{1n} \quad x_1 a_{11} y_1 \dots x_1 a_{1n} y_n$$

$$\dots \dots \dots$$

$$\dots \dots \dots$$

$$x_m a_{m1} \dots a_{mn} \quad x_m a_{m1} y_1 \dots x_m a_{mn} y_n$$

$m \times n$

That is $A(x, y) = \sum_{i=1}^m \sum_{j=1}^n x_i a_{ij} y_j$

$i=1 \quad j=1$

T

or in matrix form $A(x, y) = x A y$

This can be thought as a weighted average of the expected payoffs.

Player1's Maximin Strategy :

Assume that player1 uses x , and player2 chooses y to minimize $A(x, y)$;

player1's expected gain will be

$V(x) = \min_j x A_j$ where A_j is j th column of the matrix A

j

Player1 should choose x so as to maximize $V(x)$:

$V_1 = \max_{x \in X} \min_j x A_j$

$x \in X$

Such a strategy x is player1's maximin strategy.

Player2's Minimax Strategy :

Player2's expected loss ceiling will be

T

$V(y) = \max_i A_i y$ where A_i is i th row of the matrix A

i

Player2 should choose y so as to minimize $V(y)$:

T

$V_2 = \min_{y \in Y} \max_i A_i y$

$y \in Y$

Such a strategy y is player2's minimax strategy.

Thus we obtain the two numbers V_1 and V_2 . These numbers are called the values of the game to player1 and player2, respectively.

14

fo

.in

rs

de

Overview

ea

■ Mini-Max Theorem [Ref. previous slide]

yr

.m

w

w

It is a concept of Games theory.

,w

ty

or

Players adopt those strategies which will maximize their gains, while

ab

kr

minimizing their losses. Therefore the solution is, the best each

ha

C

C

player can do for him/herself in the face of opposition of the other

R

player.

In the previous slide, the two numbers V_1 and V_2 are the values of the game to player1 and player2, respectively.

Mini-Max Theorem : $V_1 = V_2$

The minimax theorem states that for every two-person, zero-sum game, there always exists a mixed strategy for each player such that the expected payoff for one player is same as the expected cost for the other.

This theorem is most important in game theory. It says that every two-person zero-sum game will have optimal strategies.

15

fo

.in

rs

de

Overview

ea

■ Saddle Point

yr

.m

w An element a_{ij} of a matrix is described as a saddle point if it

w

,w

ty

equals both the minimum of row i and the maximum of column j .

or

ab

kr

ha

A Saddle Point is a payoff that is simultaneously a row minimum

C

C

R

and a column maximum. In a game matrix, if the element a_{ij} corresponds to a saddle point then it is the largest in its column and the smallest in its row (LCSR).

The value a_{ij} is called the optimal payoff.

Examples : Three game payoff matrixes

A j B j C j

5 1 3 4 3 5 -1 1

i

i 3 2 4 i 0 1 0 1 -1

-3 0 1 6 3 9

one saddle point two saddle points no saddle point

$a_{22} = 2$ $a_{12} = 3$, $a_{32} = 3$

□ The game payoff matrix B shows that saddle point may not be unique, but the optimal payoff is unique.

□ A payoff matrix needn't have a saddle point, but if it does, then the usually minimax theorem is easily shown to hold true.

The Zero Sum Games modeled in matrices are solved by finding the saddle point solution.

[continued in the next slide]

16

fo

.in

rs

de

Overview

ea

[continuing from the previous slide – Saddle point]

yr

.m

How to find Saddle Point ?

w

w

,w

Below shown two examples : A and B

ty

or

ab

A j B j

kr

min min

ha

C

5 1 3 1 4 3 5 3

C

R

i 3 2 4 2 i 0 1 0 0

-3 0 1 -3 6 3 9 3

Max 5 2 4 Max 6 3 9

one saddle point two saddle points

a22 = 2

1. Find your min payoff : Label each row at its end with its minimum payoff. This way you'll define your worst case scenarios when choosing a strategy.
2. Find your opponent's min payoff : Label each column at its bottom with its maximum payoff. This will show the worst case scenarios for your opponent.
3. Find out which is the highest value in the series of minimum values. It is at one place as 2 in matrix A and at two places as 3 in matrix B.
4. Then find out which is the lowest value in the series of maximum values. It is as 2 in matrix A and as 3 in matrix B.
5. Find out if there is a minimax solution : If these two values match, then you have found the saddle point cell.
6. If there is a minimax solution, then it is possible that both agents choose the corresponding strategies. You and your opponent are maximizing the gain that the worst possible scenario can drive.

17

fo

.in

rs

de

Overview

ea

1.5 Taxonomy of Games

yr

.m

w

w

All that are explained in previous section are summarized below.

,w

ty

or

ab

kr

Game Theory

ha

C

C

R

Games of Skill Games of Chance Games of Strategy

Games involving risk

Games involving uncertainty Two-persons Multi-persons

Cooperative Mixed-motive Zero-sum Coalitions not permitted

Purely Minimal Infinite Finite Non- Cooperative cooperative social cooperative situation

saddle Non-saddle Essential Non-essential

Perfect info

Coalitions Coalitions

Imperfect info

Symmetric games Mixed strategy

Have optimal Have no optimal equilibrium points equilibrium points

18

fo

.in

rs

de

Mini-Max Search

ea

2. The Mini-Max Search Procedure

yr

.m

w

w

Consider two players, zero sum, non-random, perfect knowledge games.

,w

ty

or

Examples: Tic-Tac-Toe, Checkers, Chess, Go, Nim, and Othello.

ab

kr

ha

2.1 Formalizing Game

C

C

R

A general and a Tic-Tac-Toe game in particular.

■ Consider 2-Person, Zero-Sum, Perfect Information

‡ Both players have access to complete information about the state of the game.

‡ No information is hidden from either player.

‡ Players alternately move.

■ Apply Iterative methods

‡ Required because search space may be large for the games to search for a solution.

‡ Do search before each move to select the next best move.

19

fo

.in

rs

de

Mini-Max Search

ea

■ Adversary Methods

yr

.m

w

w

‡ Required because alternate moves are made by an opponent ,

,w

ty

or

who is trying to win, are not controllable.

ab

kr

ha

C

■ Static Evaluation Function $f(n)$

C

R

‡ Used to evaluate the "goodness" of a configuration of the game.

‡ It estimates board quality leading to win for one player.

‡ Example: Let the board associated with node n then

◇ If $f(n)$ = large +ve value

means the board is good for me and bad for opponent.

◇ If $f(n)$ = large -ve value

means the board is bad for me and good for opponent.

◇ If $f(n)$ near 0

Means the board is a neutral position.

◇ If $f(n)$ = +infinity

means a winning position for me.

◇ If $f(n)$ = -infinity

means a winning position for opponent.

20

fo

.in

rs

de

Mini-Max Search

ea

■ Zero-Sum Assumption

yr

.m

w

w

‡ One player's loss is the other player's gain.

,w

ty

or

ab

‡ Do not know how our opponent plays ?;

kr

ha

So use a single evaluation function to describe the goodness of a

C

C

R

board with respect to both players.

‡ Example : Evaluation Function for the game Tic-Tac-Toe :

$f(n) = [\text{number of 3-lengths open for me}] -$

$[\text{number of 3-lengths open for opponent}]$

where a 3-length is a complete row, column, or diagonal.

21

fo

.in

rs

de

Mini-Max Search

ea

2.2 MINI-MAX Technique

yr

.m

w

w

For Two-Agent , Zero-Sum , Perfect-Information Game.

,w

ty

or

The Mini-Max procedure can solve the problem if sufficient computational

ab

kr

resources are available.

ha

C

C

R

■ Elements of Mini-Max technique

‡ Game tree (search tree)

‡ Static evaluation,

e.g., +ve for a win, -ve for a lose and 0 for a draw or neutral.

‡ Backing up the evaluations, level by level, on the basis of opponent's turn.

22

fo

.in

rs

de

Mini-Max Search

ea

■ Game Trees : description

yr

.m

w

w

‡ Root node represents board configuration and decision required

,w

ty

or

as to what is the best single next move.

ab

kr

ha

C

If my turn to move, then the root is labeled a MAX node

C

R

indicating it is my turn;

otherwise it is labeled a MIN node to indicate it is my opponent's turn.

‡ Arcs represent the possible legal moves for the player that the arcs emanate from.

‡ At each level, the tree has nodes that are all MAX or all MIN;

‡ Since moves alternate, the nodes at level i are of opposite kind from those at level $i+1$.

23

fo

.in

rs

de

Mini-Max Search

ea

■ Mini-Max Algorithm

yr

.m

w

w

‡ Searching Game Tree using the Mini-Max Algorithm

,w

ty

or

ab

‡ Steps used in picking the next move:

kr

ha

C

◇ Since it's my turn to move, the start node is MAX node with

C

R

current board configuration.

◇ Expand nodes down (play) to some depth of look-ahead in the game.

◇ Apply evaluation function at each of the leaf nodes

◇ "Back up" values for each non-leaf nodes until computed for the root node.

◇ At MIN nodes, the backed up value is the minimum of the values associated with its children.

◇ At MAX nodes, the backed up value is the maximum of the values associated with its children.

Note: The process of "backing up" values gives the optimal strategy, that is, both players assuming that your opponent is using the same static evaluation function as you are.

24

fo

.in

rs

de

Mini-Max Search

ea

■ Example : Mini-Max Algorithm

yr

.m

w

w

■ The MAX player considers all

,w

5 S max

ty

or

three possible moves.

ab

kr

2 C min ■ The opponent MIN player also

ha

-1 A 5B

C

C

considers all possible moves.

R

D E F G H I J ■ The evaluation function is applied

9 3 -1 6 5 2 7 to leaf level only.

■ Apply Evaluation function :

‡ Apply static evaluation function at leaf nodes & begin backing up.

‡ First compute backed-up values at the parents of the leaves.

◇ Node A is a MIN node ie it is the opponent's turn to move.

◇ A's backed-up value is -1 ie min of (9, 3, -1), meaning

if opponent ever reaches this node,

then it will pick the move associated with the arc from A to F.

◇ Similarly, B's backed-up value is 5 and

C's backed-up value is 2.

‡ Next, backup values to next higher level,

◇ Node S is a MAX node ie it's our turn to move.

◇ look best on backed-up values at each of S's children.

◇ the best child is B since value is 5 ie max of (-1, 5, 2).

◇ So the minimax value for the root node S is 5, and the move selected is associated with the arc from S to B.

25

fo

.in

rs

de

Game playing with Mini-Max

ea

3. Game Playing with Mini-Max - Tic-Tac-Toe

yr

.m

w

w

Here, Minimax Game Tree is used that can program computers to play games.

,w

ty

or

There are two players taking turns to play moves.

ab

kr

Physically, it is just a tree of all possible moves.

ha

C

C

R

3.1 Moves

■ Start: X's Moves

26

fo

.in

rs

de

Game playing with Mini-Max

ea

■ Next: O's Moves

yr

.m

w

w

,w

ty

or

ab

kr

ha

C

C

R

27

fo

.in

rs

de

Game playing with Mini-Max

ea

■ Again: X's moves

yr

.m

w

w

,w

ty

or

ab

kr

ha

C

C

R

28

fo

.in

rs

de

Game playing with Mini-Max

ea

3.2 Static Evaluation:

yr

.m

w

w

'+1' for a win, '0' for a draw

,w

ty

or

ab

■ Criteria '+1' for a Win, '0' for a Draw

kr

ha

C

C

R

29

fo

.in

rs

de

Game playing with Mini-Max

ea

3.3 Back-up the Evaluations:

yr

.m

w

w

Level by level, on the basis of opponent's turn

,w

ty

or

ab

■ Up : One Level

kr

ha

C

C

R

30

fo

.in

rs

de

Game playing with Mini-Max

ea

■ Up : Two Levels

yr

.m

w

w

,w

ty

or

ab

kr

ha

C

C

R

31

fo

.in

rs

de

Game playing with Mini-Max

ea

3.4 Evaluation obtained :

yr

.m

w

w

Choose best move which is maximum

,w

ty

or

ab

kr

ha

Best move

C

C

R

32

fo

.in

rs

de

Alpha-beta pruning

ea

4. Alpha-Beta Pruning

yr

.m

w

w

The problem with Mini-Max algorithm is that the number of game states

,w

ty

or

it has to examine is exponential in the number of Moves.

ab

kr

ha

C

The Alpha-Beta Pruning helps to arrive at correct Min-Max algorithm decision

C

R

without looking at every node of the game tree.

While using Mini-Max, some situations may arise when search of a particular branch can be safely terminated. So, while doing search, figure out those nodes that do not require to be expanded. The method is explained below :

■ Max-player cuts off search when he knows Min-player can force a provably bad outcome.

■ Min player cuts of search when he knows Max-player can force provably good (for max) outcome

■ Applying an alpha-cutoff means we stop search of a particular branch because we see that we already have a better opportunity elsewhere.

■ Applying a beta-cutoff means we stop search of a particular branch because we see that the opponent already has a better opportunity elsewhere.

■ Applying both forms is alpha-beta pruning.

33

fo

.in

rs

de

Alpha-cutoff pruning

ea

5.1 Alpha-Cutoff

yr

.m

w

w

It may be found that, in the current branch, the opponent can achieve a

,w

ty

or

state with a lower value for us than one achievable in another branch. So

ab

kr

the current branch is one that we will certainly not move the game to.

ha

C

C

Search of this branch can be safely terminated.

R

34

fo

.in

rs

de

Beta-cutoff

ea

5.2 Beta-Cutoff

yr

.m

w

w

It is just the reverse of Alpha-Cutoff.

,w

ty

or

It may also be found, that in the current branch, we would be able to

ab

kr

achieve a state which has a higher value for us than one the opponent can

ha

C

C

hold us to in another branch. The current branch can be identified as one

R

that the opponent will certainly not move the game to. Search in this branch can be safely terminated.

35

fo

.in

rs

de

ea

yr

.m

AI- Game Playing

w

w

6. References : Textbooks

,w

ty

or

1. "Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill

ab

kr

companies Inc., Chapter 12, page 305-326.

ha

C

C

2. "Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig,

R

(2002), Prentice Hall, Chapter 6, page 161-189.

3. "Computational Intelligence: A Logical Approach", by David Poole, Alan Mackworth, and Randy Goebel, (1998), Oxford University Press, Chapter 4, page 113-163.

4. "AI: A New Synthesis", by Nils J. Nilsson, (1998), Morgan Kaufmann Inc., Chapter

12, Page 195-213.

5. Related documents from open source, mainly internet. An exhaustive list is being prepared for inclusion at a later date.

36